

ETAS RTA-BSW v12.7.0



RTA-Sec Stack Reference Guide

Copyright

The data in this document may not be altered or amended without special notification from ETAS GmbH. ETAS GmbH undertakes no further obligation in relation to this document. The software described in it can only be used if the customer is in possession of a general license agreement or single license. Using and copying is only allowed in concurrence with the specifications stipulated in the contract.

Under no circumstances may any part of this document be copied, reproduced, transmitted, stored in a retrieval system or translated into another language without the express written permission of ETAS GmbH.

© Copyright 2025 ETAS GmbH, Stuttgart

The names and designations used in this document are trademarks or brands belonging to the respective owners.

ETAS RTA-BSW 12.7.0 - RTA-Sec Stack Reference Guide R01 EN – 05.2025

Contents

1	Crypto Interface.....	7
1.1	Introduction	7
1.2	Supported Features	8
1.3	Extensions.....	8
1.4	Limitations	11
1.5	Exclusions.....	11
1.6	Deviations.....	11
1.7	API Definition	11
1.8	Configuration Advice	12
1.9	Integration Advice	13
1.10	Hardware Requirements	13
2	Crypto	15
2.1	Introduction	15
2.2	Supported Features	16
2.3	Extensions.....	16
2.4	Limitations	39
2.5	Exclusions.....	39
2.6	Deviations.....	40
2.7	API Definition	45
2.8	Configuration Advice	46
2.9	Integration Advice.....	50
2.10	Hardware Requirements	51
3	Crypto Service Manager	52
3.1	Introduction	52
3.2	Supported Features	53
3.3	Extensions	63
3.4	Limitations.....	81
3.5	Exclusions.....	83
3.6	Deviations	87
3.7	API Definition.....	101
3.8	Configuration Advice	103
3.9	Integration Advice	120
3.10	Hardware Requirements.....	124

4	Key Manager	125
4.1	Introduction	125
4.2	Supported Features	126
4.3	Extensions	127
4.4	Limitations	138
4.5	Exclusions	140
4.6	Deviations	140
4.7	API Definition	146
4.8	Configuration Advice	147
4.9	Integration Advice	147
4.10	Hardware Requirements	149
5	Intrusion Detection System Manager	150
5.1	Introduction	150
5.2	Supported Features	152
5.3	Extensions	173
5.4	Limitations	190
5.5	Exclusions	190
5.6	Deviations	190
5.7	API Definition	208
5.8	Configuration Advice	210
5.9	Integration Advice	211
5.10	Hardware Requirements	212
6	rba_CryptoAuCSC	213
6.1	Introduction	213
6.2	Supported Features	214
6.3	Extensions	214
6.4	Limitations	214
6.5	Exclusions	217
6.6	Deviations	217
6.7	API Definition	217
6.8	Configuration Advice	218
6.9	Integration Advice	219
6.10	Hardware Requirements	219
7	rba_CryptoCCL	220

7.1	Introduction.....	220
7.2	Supported Features	221
7.3	Extensions	222
7.4	Limitations	223
7.5	Exclusions	224
7.6	Deviations.....	224
7.7	API Definition.....	224
7.8	Configuration Advice.....	226
7.9	Integration Advice	229
7.10	Hardware Requirements	230
8	rba_CryptoHSM	231
8.1	Introduction	231
8.2	Supported Features.....	232
8.3	Extensions	238
8.4	Limitations	239
8.5	Exclusions.....	241
8.6	Deviations	241
8.7	API Definition	241
8.8	Configuration Advice.....	243
8.9	Integration Advice.....	257
8.10	Hardware Requirements	263
9	rba_CryptoAuHSM3.....	264
9.1	Introduction.....	264
9.2	Supported Features.....	265
9.3	Extensions.....	266
9.4	Limitations	267
9.5	Exclusions	267
9.6	Deviations.....	267
9.7	API Definition.....	267
9.8	Configuration Advice.....	268
9.9	Integration Advice	268
9.10	Hardware Requirements	269
10	SecOC.....	270
10.1	Introduction.....	270

10.2	Supported Features	271
10.3	Extensions.....	271
10.4	Limitations	291
10.5	Exclusions	292
10.6	Deviations	292
10.7	API Definition.....	309
10.8	Configuration Advice.....	311
10.9	Integration Advice	319
10.10	Hardware Requirements	320
11	Glossary	321

1 Crypto Interface

Module name	Crylf
Package	RTA_SEC
AUTOSAR Module ID	112
AUTOSAR Version	22-11

1.1 Introduction

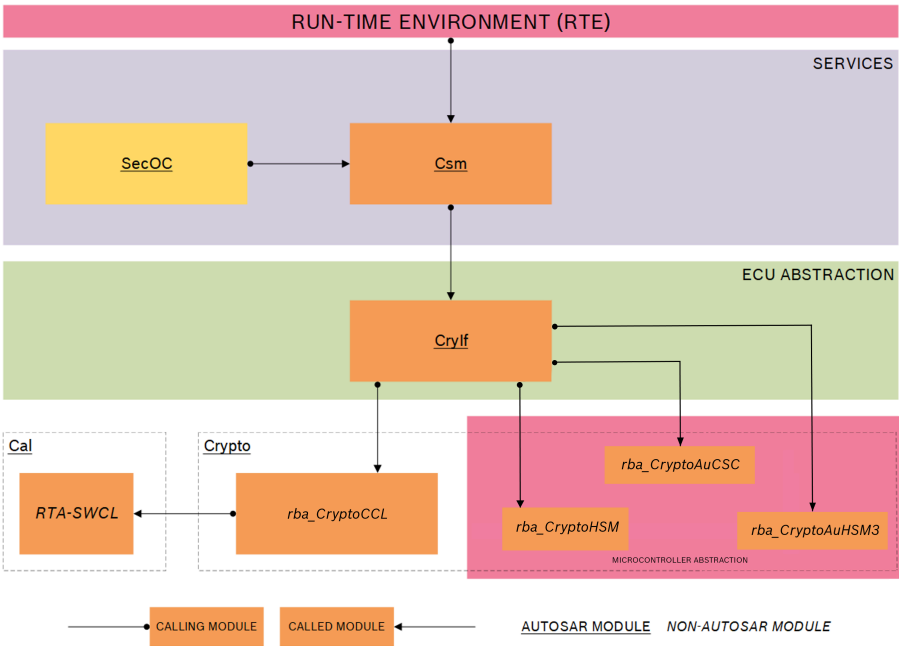


Figure 1.1.RTA-SEC Stack Architecture

The Crypto Interface module (Crylf) is located in the ECU abstraction layer, between the upper services layer and the underlying driver layer. The diagram above shows the position of Crylf in the AUTOSAR layered architecture.

Crylf provides a generic interface between the Crypto Service Manager (Csm) and the drivers encapsulated by the Crypto Driver library module (Crypto).

Requests for cryptographic services are issued from Csm in the form of job requests, which Crylf interprets and passes to Crypto for processing by the relevant driver.

Crylf itself has no knowledge of which driver will perform the requested service, as this will be determined by Crypto. Similarly, Crylf relies on the configuration information provided by Csm for each job request, including the operation mode being requested, the assigned priority of the job, and the cryptographic algorithm to be used.

Crylf also provides a similar interface (i.e. between the Csm and Crypto modules)

for processing of Csm's Key Management and Certificate Verification functions.

Crylf functions are re-entrant and can process multiple concurrent tasks from Csm.

Module Configuration

This module is not currently supported by the RTA-BSW Configuration Generator (ConfGen) and must be manually configured.

1.2 Supported Features

1.2.1 General

The Crylf module is responsible for providing unique interfaces for Csm to access all the functionality described in this chapter, which includes the following:

- Key Management (Key) - copy, generate, derive, get status.
- Key Management (Key Element) - get, copy, copy all, update, set valid.
- Certificate parsing and validation.
- Random number generation.
- Shared secret and public key computation.

It also handles the processing (and cancellation) of jobs passed to it from Csm.

1.2.2 Callback

A callback function (Crylf_CallbackNotification) is provided, which is used by the driver object to notify Crylf that a request has been processed.

1.2.3 Cryptographic Algorithms

The cryptographic service and algorithms supported through Crylf are dependent on those supported by the Csm module, and which are implemented in the Crypto Driver library modules rba_CryptoCCL and rba_CryptoHSM. See the Csm Supported Features section for a full list of the cryptographic services and algorithms that are currently supported by Csm.

Crylf passes the configured parameters in the driver objects directly to rba_CryptoCCL and rba_CryptoHSM (via Crypto) and thus no cryptographic method selection is required within Crylf itself.

1.3 Extensions

1.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

1.3.1.1 Crylf/CrylfGeneral/CrylfRbInterCryptoCopyKeyElementMaxSize

Short Name	CrylfRbInterCryptoCopyKeyElementMaxSize
------------	---

Description	This parameter should be used to limit the size of the buffers used in CryIf to perform key elements copy operation between different crypto drivers. (If the value is larger than the largest cryptoKeyElement in the configuration, then the largest cryptoKeyElement's size will be used for the buffer size. Size 0 deactivates the feature.)
Multiplicity	0..1
Value Range	0..65535
Default Value	65535
Type	ECUC-INTEGER-PARAM-DEF

1.3.1.2 CryIf/CryIfGeneral/CryIfRbSupportAR4_4_0

Short Name	CryIfRbSupportAR4_4_0
Description	If this parameter is set to True, the CryIf_CallbackNotification API is updated to the AR4.4.0 specification.
Introduction	True: AR4.4.0 crypto driver support enabled. False: AR4.4.0 crypto driver support disabled.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

1.3.2.1 CryIf_CertificateParse

Syntax	Std_ReturnType CryIf_CertificateParse(uint32 cryIfKeyId)
Parameter In	cryIfKeyId: The identifier of the key which shall be parsed.
Return Value	E_OK: Request successful.
	E_NOT_OK: Request Failed.
	CRYPTO_E_BUSY: Request Failed, Crypto Driver Object is Busy.
	CRYPTO_E_KEY_EMPTY: Request failed because of uninitialized source key element.
Description	Dispatches the certificate parse function to the configured crypto driver object.

1.3.2.2 CryIf_CertificateVerify

Syntax	Std_ReturnType CryIf_CertificateVerify(uint32 cryIfKeyId, uint32 verifyCryIfKeyId, Crypto_VerifyResultType* verifyPtr)
Parameters In	cryIfKeyId: The identifier of the key which shall be used to validate the certificate.
	verifyCryIfKeyId: The identifier of the key containing the certificate to be verified.
Parameter Out	verifyPtr: Pointer to the memory location which will contain the result of the certificate verification.
Return Value	E_OK: Request successful.
	E_NOT_OK: Request Failed.
	CRYPTO_E_KEY_EMPTY: Request failed because of uninitialized source key element.
Description	Verifies the certificate stored in the key referenced by verifyCryIfKeyId with the certificate stored in the key referenced by cryIfKeyId.

1.3.2.3 CryIf_KeyGetStatus

Syntax	Std_ReturnType CryIf_KeyGetStatus (uint32 cryIfKeyId, Crypto_KeyStatusType* keyStatusPtr)
Parameter In	cryIfKeyId: The identifier of the key for which the key state shall be returned.
Parameter Out	keyStatusPtr: Pointer to the data where the status of the key shall be stored.
Return Value	E_OK: Request successful.
	E_NOT_OK: Request Failed.
Description	Returns the key state of the key identified by cryIfKeyId.

1.3.2.4 CryIf_Rb_StorePermanentData

Syntax	CryIf_Rb_StorePermanentData()
Parameters	NONE
Return Value	E_OK: Request successful.
	E_NOT_OK: Request Failed.
	CRYPTO_E_PENDING: Request in progress.
Description	Function to trigger permanent data store and poll its status. All permanent data will be saved. On first call the store operation is automatically started, on subsequent calls the status is polled.
	This function dispatches the command to ALL configured Crypto driver objects.

1.4 Limitations

- There are no known limitations.

1.5 Exclusions

The following features from AR v22-11 are not supported in version v12.7.0.

API function not currently implemented:

- CryIf_CustomSync

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- There are no known exclusions in this version.

1.6 Deviations

1.6.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

1.6.1.1 CryIf/CryIfChannel/CryIfChannelId

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

1.6.1.2 CryIf/CryIfKey/CryIfKeyId

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

1.7 API Definition

List of supported API functions

CryIf_Init	Initializes the module.
CryIf_GetVersionInfo	Returns the version information of this module.
CryIf_ProcessJob	Dispatches the specified job to the configured crypto driver object.
CryIf_CancelJob	Sends a request to cancel the specified job to the configured crypto driver object.
CryIf_KeyElementSet	Requests an update of a key element.
CryIf_KeySetValid	Requests the setting a key element as valid.
CryIf_KeySetInvalid	Sets invalid for the status of the key identified.

CryIf_KeyGetStatus	Returns the key state of the key identified.
CryIf_KeyElementGet	Requests retrieval of a key element.
CryIf_KeyElementCopy	Copies a key element from the source key to a target key.
CryIf_KeyElementCopyPartial	Uses keyElementOffsets and keyElementCopyLength to copy partial key element to another key element.
CryIf_KeyCopy	Copies all key elements from the source key to a target key.
CryIf_RandomSeed	Requests generation of a random number.
CryIf_KeyGenerate	Requests a generated key value for the specified key.
CryIf_KeyDerive	Request to derive a key from another source key.
CryIf_KeyExchangeCalcPubVal	Requests a key exchange public value calculation.
CryIf_KeyExchangeCalcSecret	Requests a common shared secret calculation.
CryIf_CallbackNotification	Notifies CryIf on the completion of the request with the result of the cryptographic operation.

For Further information please refer to AUTOSAR_SWS_CryptoInterface.pdf

1.8 Configuration Advice



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

1.8.1 Automatic Configuration (Via Csm)

Some of the configuration of the Crypto stack modules, including CryIf, can be achieved automatically. This is done through the Csm module using the "Flexible" configuration method, which is enabled via the CsmRbAutoFillConfig parameter (in Csm/CsmGeneral). See the Configuration Advice in the Csm section for more details.

1.8.2 AUTOSAR Compliant Crypto Driver Configuration

To configure the link between CryIf and Crypto, configure the Crypto driver objects and select them in the CryIfDriverObjectRef parameter (in CryIf/CryIfChannels/).

Do the same for any keys by setting the CryIfKeyRef parameter (in CryIf/CryIfKeys/) to reference the CryptoKeys configured in the Crypto Driver.

1.8.3 Data Length

All CryIf functions that have a length parameter describing the input data length will have a data type of uint32. In theory, the full input data length of $2^{32}-1$ bytes

could be used for the parameter value.

If the time slot for the calculation is not restricted, the Crylf functions should work properly with such extremely long input data streams, but in combination with a restricted time slot for processing, long input data streams may cause a trap induced by a time-out. This means in practice that you should restrict the input data length with respect to the reserved processing time.

1.9 **Integration Advice**

1.9.1 **Init Functions**

Crylf is initialized via Crylf_Init.

1.9.2 **Delnit Functions**

No Delnit is required.

1.9.3 **Scheduled Functions**

Crylf does not have a scheduled function.

3.9.4 **Module Compatibility**

The modules Csm, Crylf, Crypto, rba_CryptoCCL and rba_CryptoHSM form a functional composition. Only modules from the same package version should be used together.

The integration of the library Crypto and either rba_CryptoCCL or rba_CryptoHSM is a prerequisite for using Crylf.

1.9.5 **AUTOSAR Compliant Crypto Driver Integration**

If CrylfDriverObjectRef points to a Crypto Object with the shortname "Crypto", then the mapping must be defined from each AUTOSAR named function to the Crypto driver's equivalent in integration file (Crylf_Cfg_ApiRouting.h).

1.9.6 **OS Resources**

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The Crylf module does not use any exclusive OS resources.

1.10 **Hardware Requirements**

Table 1.11.Crylf Hardware Requirements

Support	Usage
FLASH	2218 bytes
RAM	257 bytes

Usage calculated based on a sample configuration.

2 Crypto

Module name	Crypto
Package	RTA-SEC
AUTOSAR Module ID	114
AUTOSAR Version	22-11

2.1 Introduction

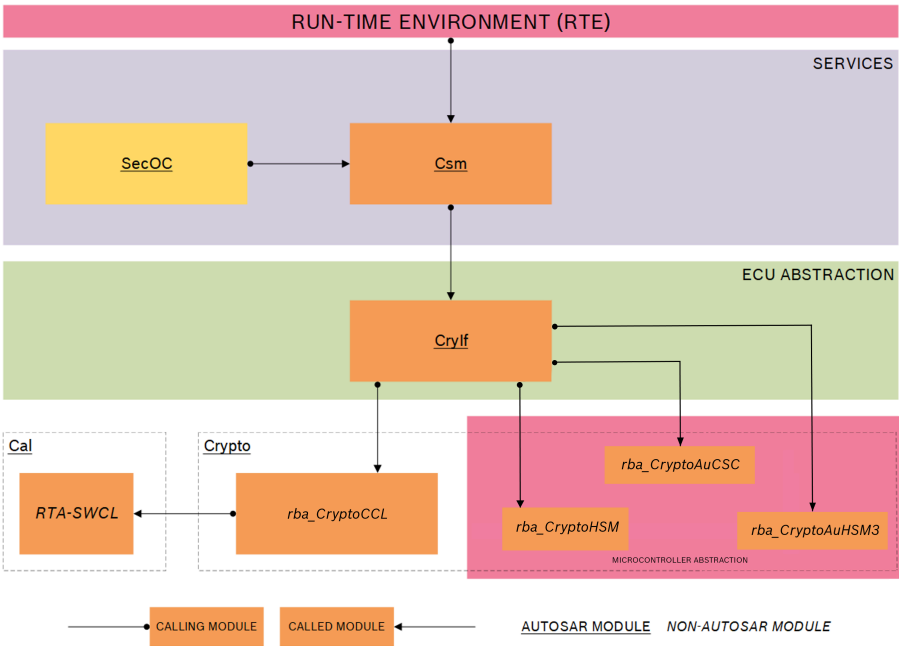


Figure 2.1.RTA-SEC Stack Architecture

The Crypto Driver module (Crypto) is located in the Microcontroller Hardware Abstraction layer (below the ECU Abstraction layer). The diagram above shows the position of Crypto in the AUTOSAR layered architecture. In the RTA-BSW implementation, Crypto also controls access to software-based cryptographic libraries which are considered outside of the layered architecture, but which generally function in the same way as the hardware-based drivers.

Requests for cryptographic services are issued to Crylf from the Crypto Service Manager module (Csm) in the form of job requests, which Crylf then interprets and passes to the driver object defined in the Crypto module for the specified cryptographic operation.

Crypto provides a configuration mechanism which defines which driver object will be called by the Crypto Interface module (Crylf) in order to select the correct cryptographic primitive for a requested operation.

Crypto can define multiple driver objects (either hardware or software based), each of which takes the form of a particular instance which becomes the end

point of a logical channel. In this context, a channel is the path from Csm, via Crylf and Crypto, to a configured Crypto object.

Module Configuration

This module is not currently supported by the RTA-BSW Configuration Generator (ConfGen) and must be manually configured.

2.2 Supported Features

2.2.1 General

In the RTA-BSW implementation, Crypto does not implement any APIs. Calls to Crypto from Crylf are made directly to the driver modules, which in RTA-BSW are rba_CryptoCCL (software) and rba_CryptoHSM, rba_CryptoAuCSC and rba_CryptoAuHSM3 (hardware).

Crypto supports the configuration parameters required to define unique driver object instances for cryptographic primitives defined in rba_CryptoCCL, rba_CryptoHSM, rba_CryptoAuCSC and rba_CryptoAuHSM3.

Crypto is a structural component which also provides common code generation/build functionalities used by other components in the package. It provides the configuration for all derived crypto drivers and performs forwarding operation on those configurations.

2.3 Extensions

2.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

2.3.1.1 Crypto/CryptoDriverObjects/CryptoDriverObject/CryptoDriverObjectMainFunctionRef

Short Name	CryptoDriverObjectMainFunctionRef
Description	This parameter refers to the used CryptoMainFunction.
Introduction	Reference to CryptoMainFunction, where the according CryptoDriverObject is assigned to.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

2.3.1.2 Crypto/CryptoDriverObjects/CryptoDriverObject/CryptoRbAuCscAccelerationEngineld

Short Name	CryptoRbAuCscAccelerationEngineld
------------	-----------------------------------

Description	Id of the cryptographic acceleration engine that shall be used for jobs processed by this driver object. This directly corresponds to the CycurSoC symbolic names. So if you want to use "cycursoc_Crypto-Engine_One", you need to configure "1" here. Not configuring this parameter will result in the usage of CycurSoCs default engine for the respective platform (same as configuring "0"). Consult the platform specific addendum to the CycurSoC user guide to get an idea of what to configure here.
Multiplicity	0..1
Value Range	0..2
Default Value	0
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.3 Crypto/CryptoDriverObjects/CryptoDriverObject/CryptoRbBulkProcessingSize

Short Name	CryptoRbBulkProcessingSize
Description	Number of jobs can be processed in a bulk.
Multiplicity	0..1
Value Range	1..4294967295
Default Value	1
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.4 Crypto/CryptoDriverObjects/CryptoDriverObject/CryptoRbDriverObjectPriority

Short Name	CryptoRbDriverObjectPriority
Description	Determines the execution priority for all jobs processed by this driver object. This parameter is not related to the CryptoStack-internal queueing priority configured in the job itself (CsmJobPriority). Instead, it applies to the actual primitive execution on the underlying hardware/software (if a priority concept is in place). This is currently only supported by rba_CryptoAuCSC.
Multiplicity	0..1
Default Value	NORMAL
Enumeration Literals	HIGH, HIGHEST, LOW, LOWEST, NORMAL

Type	ECUC-ENUMERATION-PARAM-DEF
------	----------------------------

2.3.1.5 Crypto/CryptoDriverObjects/CryptoDriverObject/CryptoRbHsmUseHwCsp

Short Name	CryptoRbHsmUseHwCsp
Description	This parameter will select the cryptoDriverObject to use an AES Engine provided by Hardware Crypto Service Provider (HwCsp) in HSM to perform the service.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.6 Crypto/CryptoGeneral/CryptoRbAuCscKeyProvisioning

Short Name	CryptoRbAuCscKeyProvisioning
Description	Switches the key provisioning support for CycurSoc on or off. If enabled, then the Crypto component will try to use cycursoc_ProvisionKey api to provision keys via Crypto_KeyElementSet request else only creation of ephemeral keys is possible.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.7 Crypto/CryptoGeneral/CryptoRbAuCscOutputBufferAlignment

Short Name	CryptoRbAuCscOutputBufferAlignment
Description	Memory alignment for CycurSoC output buffers in bytes. This should be your platforms cache line size.
Multiplicity	0..1
Value Range	1..4294967295
Default Value	32
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.8 Crypto/CryptoGeneral/CryptoRbCclCipherAsymLoopCounter

Short Name	CryptoRbCclCipherAsymLoopCounter
Description	Holds the number of iterations to be performed internally by the cipher asymmetric primitive. This parameter can be used to increase or reduce the runtime of a single cipher (finish) call.
Multiplicity	0..1

Value Range	1..4294967295
Default Value	4294967295
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.9 Crypto/CryptoGeneral/CryptoRbCclKeyExchangeLoopCounter

Short Name	CryptoRbCclKeyExchangeLoopCounter
Description	Holds the number of iterations to be performed internally by the key exchange primitive. This parameter can be used to increase or reduce the runtime of a single key exchange call.
Multiplicity	0..1
Value Range	1..4294967295
Default Value	4294967295
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.10 Crypto/CryptoGeneral/CryptoRbCclKeyGenerateLoopCounter

Short Name	CryptoRbCclKeyGenerateLoopCounter
Description	Holds the number of iterations to be performed internally by the key Generate primitive. This parameter can be used to increase or reduce the runtime of a single key Generate call.
Multiplicity	0..1
Value Range	1..4294967295
Default Value	4294967295
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.11 Crypto/CryptoGeneral/CryptoRbCclPreCalculateK1_K2

Short Name	CryptoRbCclPreCalculateK1_K2
Description	True: Enable the precalculation of K1 and K2 for CMAC generation. This option results in a faster generation but safety is threatened because the keys are stored within global variables and will not be cleared. False: Disable the precalculation of K1 and K2 for CMAC generation. The keys are calculated for each CMAC generation and are cleared when CMAC generation was completed (safety is enhanced = default).
Multiplicity	0..1
Default Value	false

Type	ECUC-BOOLEAN-PARAM-DEF
------	------------------------

2.3.1.12 Crypto/CryptoGeneral/CryptoRbCclSignatureLoopCounter

Short Name	CryptoRbCclSignatureLoopCounter
Description	Holds the number of iterations to be performed internally by the signature primitive (update). This parameter can be used to increase or reduce the runtime of a single signature (update) call.
Multiplicity	0..1
Value Range	1..4294967295
Default Value	4294967295
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.13 Crypto/CryptoGeneral/CryptoRbHsmAEADSessionPriority

Short Name	CryptoRbHsmAEADSessionPriority
Description	Determines the session priority for all jobs with this primitive type
Multiplicity	0..1
Default Value	PRIO_MEDIUM
Enumeration Literals	PRIO_BACKGROUND, PRIO_HIGH, PRIO_LOW, PRIO_MEDIUM
Type	ECUC-ENUMERATION-PARAM-DEF

2.3.1.14 Crypto/CryptoGeneral/CryptoRbHsmBulkMacGenerateSessionPriority

Short Name	CryptoRbHsmBulkMacGenerateSessionPriority
Description	Determines the session priority for all jobs with this primitive type
Multiplicity	0..1
Default Value	PRIO_HIGH
Enumeration Literals	PRIO_BACKGROUND, PRIO_HIGH, PRIO_LOW, PRIO_MEDIUM
Type	ECUC-ENUMERATION-PARAM-DEF

2.3.1.15 Crypto/CryptoGeneral/CryptoRbHsmBulkMacVerifySessionPriority

Short Name	CryptoRbHsmBulkMacVerifySessionPriority
Description	Determines the session priority for all jobs with this primitive type
Multiplicity	0..1
Default Value	PRIO_HIGH
Enumeration Literals	PRIO_BACKGROUND, PRIO_HIGH, PRIO_LOW, PRIO_MEDIUM
Type	ECUC-ENUMERATION-PARAM-DEF

2.3.1.16 Crypto/CryptoGeneral/CryptoRbHsmCancelTimeout

Short Name	CryptoRbHsmCancelTimeout
Description	Maximum timeout duration to wait for a HSM timeout monitoring cancel operation to complete in ticks. This parameter is only used when CryptoRbHsm TimeoutMonitoringEnabled is enabled. The tick duration is set by the integrators timer configuration for timeout monitoring.
Multiplicity	0..1
Value Range	1..4294967295
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.17 Crypto/CryptoGeneral/CryptoRbHsmCipherSessionPriority

Short Name	CryptoRbHsmCipherSessionPriority
Description	Determines the session priority for all jobs with this primitive type
Multiplicity	0..1
Default Value	PRIO_MEDIUM
Enumeration Literals	PRIO_BACKGROUND, PRIO_HIGH, PRIO_LOW, PRIO_MEDIUM
Type	ECUC-ENUMERATION-PARAM-DEF

2.3.1.18 Crypto/CryptoGeneral/CryptoRbHsmCipherTimeout

Short Name	CryptoRbHsmCipherTimeout
Description	Maximum timeout for the synchronous HSM Cipher primitive operations in ms. This parameter is only used when CryptoRbHsmTimeoutMonitoringEnabled is set to FALSE
Multiplicity	0..1
Value Range	1..4294967295
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.19 Crypto/CryptoGeneral/CryptoRbHsmCsaiErrorPropagationEnabled

Short Name	CryptoRbHsmCsaiErrorPropagationEnabled
------------	--

Description	TRUE: Enables the propagation of CSAI error codes to an external module for specific cryptographic primitives executed on HSM. This is only used by the rba_CryptoHSM CryptoDriver. The propagation is performed only for specific primitives (MAC Generate/Verify, Encrypt/Decrypt with ECB/CBC, Random Generate/Seed, SHE Load Key/Load Plain Key, SHE Export RAM key). FALSE: The error propagation is disabled and the CSAI error propagation call is not performed.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.20 Crypto/CryptoGeneral/CryptoRbHsmHashSessionPriority

Short Name	CryptoRbHsmHashSessionPriority
Description	Determines the session priority for all jobs with this primitive type
Multiplicity	0..1
Default Value	PRIO_MEDIUM
Enumeration Literals	PRIO_BACKGROUND, PRIO_HIGH, PRIO_LOW, PRIO_MEDIUM
Type	ECUC-ENUMERATION-PARAM-DEF

2.3.1.21 Crypto/CryptoGeneral/CryptoRbHsmHwCspNumberOfAESEngines

Short Name	CryptoRbHsmHwCspNumberOfAESEngines
Description	Holds a number of AES Engines available in the hardware which will limit the number of configured cryptoDriverObjects which uses HwCsp. Only applicable in case of any cryptoDriver uses HwCsp.
Multiplicity	0..1
Value Range	1..4294967295
Default Value	4
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.22 Crypto/CryptoGeneral/CryptoRbHsmInitializeKeyElements

Short Name	CryptoRbHsmInitializeKeyElements
Description	True: Enable the initialization of HSM key elements with configured initialization data on startup and if the persistent storage was never used before by the HSM. False: Disable the initialization of HSM key elements on startup (True = default).

Multiplicity	0..1
Default Value	true
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.23 Crypto/CryptoGeneral/CryptoRbHsmKeyElementCopyWithoutNVStorage

Short Name	CryptoRbHsmKeyElementCopyWithoutNVStorage
Description	If enabled, when a KeyElementCopy operation is performed in the rba_CryptoHSM, the involved keys will not be written in the non-volatile memory (NV).
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.24 Crypto/CryptoGeneral/CryptoRbHsmKeyExchangeCalcPubValResultMaxLength

Short Name	CryptoRbHsmKeyExchangeCalcPubValResultMaxLength
Description	Maximum size of the calculated public value in bytes, only needed when HSM output buffering is enabled.
Multiplicity	0..1
Value Range	1..4294967295
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.25 Crypto/CryptoGeneral/CryptoRbHsmKeyExchangeCalcSecretDataMaxLength

Short Name	CryptoRbHsmKeyExchangeCalcSecretDataMaxLength
Description	Maximum input size of the key exchange calculate secret primitive in bytes, only needed when HSM input buffering is enabled.
Multiplicity	0..1
Value Range	1..4294967295
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.26 Crypto/CryptoGeneral/CryptoRbHsmKeyExchangeTimeout

Short Name	CryptoRbHsmKeyExchangeTimeout
------------	-------------------------------

Description	Maximum timeout for the synchronous KEYEXCHANGE operations in ms. This parameter is only used when CryptoRbHsmTimeoutMonitoringEnabled is set to FALSE
Multiplicity	0..1
Value Range	1..4294967295
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.27 Crypto/CryptoGeneral/CryptoRbHsmKeyExchangeWaitForCalcSecretTimeout

Short Name	CryptoRbHsmKeyExchangeWaitForCalcSecretTimeout
Description	Maximum timeout duration to wait for the corresponding Job KeyExchange CalcSecret request after a successful Job KeyExchange CalcPubVal call in ticks. The tick duration is set by the integrators timer configuration for timeout monitoring.
Multiplicity	0..1
Value Range	1..4294967295
Default Value	30000
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.28 Crypto/CryptoGeneral/CryptoRbHsmKeyExchangeWithHandle

Short Name	CryptoRbHsmKeyExchangeWithHandle
Description	True: Enable the key exchange mode to use the key handle of an existing Priv key element for calculating the shared secret. False: The Priv key element for calculating the shared secret is a generated random number (FALSE = default).
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.29 Crypto/CryptoGeneral/CryptoRbHsmKeyManagementSessionPriority

Short Name	CryptoRbHsmKeyManagementSessionPriority
Description	Determines the session priority for all jobs with this primitive type
Multiplicity	0..1
Default Value	PRIO_LOW

Enumeration Literals	PRIO_BACKGROUND, PRIO_HIGH, PRIO_LOW, PRIO_MEDIUM
Type	ECUC-ENUMERATION-PARAM-DEF

2.3.1.30 Crypto/CryptoGeneral/CryptoRbHsmKeyManagementTimeout

Short Name	CryptoRbHsmKeyManagementTimeout
Description	Maximum timeout for the synchronous KEYGENERATE and key management operations in ms. This parameter is only used when CryptoRbHsmTimeout-MonitoringEnabled is set to FALSE
Multiplicity	0..1
Value Range	1..4294967295
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.31 Crypto/CryptoGeneral/CryptoRbHsmMacBulkGenerateMaxInput

Short Name	CryptoRbHsmMacBulkGenerateMaxInput
Description	Holds the maximum number of input bytes summed up from all MAC Generate jobs to be performed by one HSM bulk job. If this value is not configured, then no restriction will be applied to the total byte count in the HSM bulk job.
Multiplicity	0..1
Value Range	1..4294967295
Default Value	4294967295
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.32 Crypto/CryptoGeneral/CryptoRbHsmMacBulkMaxGenerateJobs

Short Name	CryptoRbHsmMacBulkMaxGenerateJobs
Description	Holds the maximum number of mac generate jobs to be performed my one HSM bulk job. The total number of all configured mac generate jobs will be used automatically if this configuration value is unset.
Multiplicity	0..1
Value Range	2..4294967295
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.33 Crypto/CryptoGeneral/CryptoRbHsmMacBulkMaxVerifyJobs

Short Name	CryptoRbHsmMacBulkMaxVerifyJobs
Description	Holds the maximum number of mac verify jobs to be performed by one HSM bulk job. The total number of all configured mac verify jobs will be used automatically if this configuration value is unset.
Multiplicity	0..1
Value Range	2..4294967295
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.34 Crypto/CryptoGeneral/CryptoRbHsmMacBulkVerifyMaxInput

Short Name	CryptoRbHsmMacBulkVerifyMaxInput
Description	Holds the maximum number of input bytes summed up from all MAC Verify jobs to be performed by one HSM bulk job. If this value is not configured, then no restriction will be applied to the total byte count in the HSM bulk job.
Multiplicity	0..1
Value Range	1..4294967295
Default Value	4294967295
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.35 Crypto/CryptoGeneral/CryptoRbHsmMacSingleShotSessionPriority

Short Name	CryptoRbHsmMacSingleShotSessionPriority
Description	Determines the session priority for all jobs with this primitive type
Multiplicity	0..1
Default Value	PRIO_HIGH
Enumeration Literals	PRIO_BACKGROUND, PRIO_HIGH, PRIO_LOW, PRIO_MEDIUM
Type	ECUC-ENUMERATION-PARAM-DEF

2.3.1.36 Crypto/CryptoGeneral/CryptoRbHsmMacTimeout

Short Name	CryptoRbHsmMacTimeout
Description	Maximum timeout for the synchronous HSM MAC primitive operations in ms. This parameter is only used when CryptoRbHsmTimeoutMonitoringEnabled is set to FALSE

Multiplicity	0..1
Value Range	1..4294967295
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.37 Crypto/CryptoGeneral/CryptoRbHsmRandomGenerateResult-MaxLength

Short Name	CryptoRbHsmRandomGenerateResultMaxLength
Description	Max size of the random generate in bytes, only needed when HSM output buffering is enabled.
Multiplicity	0..1
Value Range	1..4294967295
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.38 Crypto/CryptoGeneral/CryptoRbHsmRandomSeedDataMaxLength

Short Name	CryptoRbHsmRandomSeedDataMaxLength
Description	Maximum input size of random seed primitive in bytes, only needed when HSM input buffering is enabled.
Multiplicity	0..1
Value Range	1..4294967295
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.39 Crypto/CryptoGeneral/CryptoRbHsmRandomSessionPriority

Short Name	CryptoRbHsmRandomSessionPriority
Description	Determines the session priority for all jobs with this primitive type
Multiplicity	0..1
Default Value	PRIO_MEDIUM
Enumeration Literals	PRIO_BACKGROUND, PRIO_HIGH, PRIO_LOW, PRIO_MEDIUM
Type	ECUC-ENUMERATION-PARAM-DEF

2.3.1.40 Crypto/CryptoGeneral/CryptoRbHsmRandomTimeout

Short Name	CryptoRbHsmRandomTimeout
Description	Maximum timeout for synchronous HSM random primitive operations in ms. This parameter is only used when CryptoRbHsmTimeoutMonitoringEnabled is set to FALSE

Multiplicity	0..1
Value Range	1..4294967295
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.41 Crypto/CryptoGeneral/CryptoRbHsmSHESessionPriority

Short Name	CryptoRbHsmSHESessionPriority
Description	Determines the session priority for all jobs with this primitive type
Multiplicity	0..1
Default Value	PRIO_MEDIUM
Enumeration Literals	PRIO_BACKGROUND, PRIO_HIGH, PRIO_LOW, PRIO_MEDIUM
Type	ECUC-ENUMERATION-PARAM-DEF

2.3.1.42 Crypto/CryptoGeneral/CryptoRbHsmSafeMac

Short Name	CryptoRbHsmSafeMac
Description	True: Enable the safe CMac mode of the HSM for any mac generation/verification (only available if bulk mode is activated also). False: Disable the safe CMac mode of the HSM for any mac generation/verification (FALSE = default).
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.43 Crypto/CryptoGeneral/CryptoRbHsmSheTimeout

Short Name	CryptoRbHsmSheTimeout
Description	Maximum timeout for synchronous HSM SHE primitive operations in ms. This parameter is only used when CryptoRbHsmTimeoutMonitoringEnabled is set to FALSE
Multiplicity	0..1
Value Range	1..4294967295
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.44 Crypto/CryptoGeneral/CryptoRbHsmSignatureSessionPriority

Short Name	CryptoRbHsmSignatureSessionPriority
Description	Determines the session priority for all jobs with this primitive type

Multiplicity	0..1
Default Value	PRIO_LOW
Enumeration Literals	PRIO_BACKGROUND, PRIO_HIGH, PRIO_LOW, PRIO_MEDIUM
Type	ECUC-ENUMERATION-PARAM-DEF

2.3.1.45 Crypto/CryptoGeneral/CryptoRbHsmSignatureTimeout

Short Name	CryptoRbHsmSignatureTimeout
Description	Maximum timeout for the synchronous HSM signature primitive operations in ms. This parameter is only used when CryptoRbHsmTimeoutMonitoringEnabled is set to FALSE
Multiplicity	0..1
Value Range	1..4294967295
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.46 Crypto/CryptoGeneral/CryptoRbHsmTimeout

Short Name	CryptoRbHsmTimeout
Description	Maximum timeout for the synchronous HSM primitive operations in ms. This value is used for all primitives which do not have a dedicated timeout parameter defined. This parameter is only used when CryptoRbHsmTimeoutMonitoringEnabled is set to FALSE .
Multiplicity	0..1
Value Range	1..4294967295
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.47 Crypto/CryptoGeneral/CryptoRbHsmTimeoutMonitoringEnabled

Short Name	CryptoRbHsmTimeoutMonitoringEnabled
Description	TRUE: Enables timeout monitoring for HSM operations, whereby jobs are cancelled if the timeout duration for a given primitive is exceeded.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.48 Crypto/CryptoGeneral/CryptoRbHsmVKMSSessionPriority

Short Name	CryptoRbHsmVKMSSessionPriority
------------	--------------------------------

Description	Determines the session priority for all jobs with this primitive type
Multiplicity	0..1
Default Value	PRIOR_MEDIUM
Enumeration Literals	PRIOR_BACKGROUND, PRIOR_HIGH, PRIOR_LOW, PRIOR_MEDIUM
Type	ECUC-ENUMERATION-PARAM-DEF

2.3.1.49 Crypto/CryptoGeneral/CryptoRbHsmVkmsTimeout

Short Name	CryptoRbHsmVkmsTimeout
Description	Maximum timeout for the synchronous HSM Vkms primitive operations in ms. This parameter is only used when CryptoRbHsmTimeoutMonitoringEnabled is set to FALSE
Multiplicity	0..1
Value Range	1..4294967295
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.50 Crypto/CryptoGeneral/CryptoRbInputBufferDefaultSize

Short Name	CryptoRbInputBufferDefaultSize
Description	Default size used for the allocation of input buffers if the optimal size can not be calculated from related Csm job configurations. Only relevant if CryptoRbInputBuffering is TRUE.
Multiplicity	0..1
Value Range	0..4294967295
Default Value	0
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.51 Crypto/CryptoGeneral/CryptoRbInputBuffering

Short Name	CryptoRbInputBuffering
Description	TRUE: The Crypto driver allocates and uses extra buffers for inputs read by the underlying software/hardware (e.g. in case the HSM can not access the memory areas provided by the caller of the CryptoStack). FALSE: The Crypto driver uses the input buffers provided by the caller of the CryptoStack directly.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.52 Crypto/CryptoGeneral/CryptoRbKeyDynamicCfgPrefixCheck

Short Name	CryptoRbKeyDynamicCfgPrefixCheck
Description	Pre-processor switch to enable and disable the DYNAMIC_CFG_CRYPT0 prefix checking for crypto keys.
Introduction	True: DYNAMIC_CFG_CRYPT0 prefix check is enabled. False: DYNAMIC_CFG_CRYPT0 prefix check is disabled.
Multiplicity	0..1
Default Value	true
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.53 Crypto/CryptoGeneral/CryptoRbKeySetValidPersistingEnabled

Short Name	CryptoRbKeySetValidPersistingEnabled
Description	Enables the usage of rba_CryptoHSM_KeySetValid API to trigger the persisting of crypto keys to NvM. Currently only supported for rba_CryptoHSM.
Introduction	True: rba_CryptoHSM_KeySetValid will be used to trigger persisting. False: rba_CryptoHSM_KeySetValid will not be used to trigger persisting.
Multiplicity	0..1
Default Value	true
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.54 Crypto/CryptoGeneral/CryptoRbMainFunctionLoopCounter

Short Name	CryptoRbMainFunctionLoopCounter
Description	Holds the number of iterations to be performed internally by the main function. This parameter can be used to increase or reduce the runtime of a single main function call.
Multiplicity	0..1
Value Range	1..4294967295
Default Value	4294967295
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.55 Crypto/CryptoGeneral/CryptoRbOutputBufferDefaultSize

Short Name	CryptoRbOutputBufferDefaultSize
Description	Default size used for the allocation of output buffers if the optimal size can not be calculated from related Csm job configurations. Only relevant if CryptoRbOutputBuffering is TRUE.
Multiplicity	0..1

Value Range	0..4294967295
Default Value	0
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.56 Crypto/CryptoGeneral/CryptoRbOutputBuffering

Short Name	CryptoRbOutputBuffering
Description	TRUE: The Crypto driver allocates and uses extra buffers for outputs written by the underlying software/hardware (e.g. in case the HSM can not access the output memory areas provided by the caller of the CryptoStack). FALSE: The Crypto driver uses the output buffers provided by the caller of the CryptoStack directly.
Multiplicity	0..1
Default Value	true
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.57 Crypto/CryptoGeneral/CryptoRbWaitForNvMRead

Short Name	CryptoRbWaitForNvMRead
Description	True: In Crypto Init wait for NvM Read to be completed for required NvM block. (Calls Memstack Mainfunctions intern). False: Cyclically check NvM Read status in Crypto Mainfunction if read not already finished before Crypto Init. Note: Used only by rba_CryptoCCL init function.
Multiplicity	0..1
Default Value	true
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.58 Crypto/CryptoKeyElements/CryptoKeyElement/CryptoRbAuCscKeyId

Short Name	CryptoRbAuCscKeyId
Description	Maps the key element to the specified key id inside CysurSoC. If left empty, an available key id is automatically assigned.
Multiplicity	0..1
Value Range	0..65535
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.59 Crypto/CryptoKeyElements/CryptoKeyElement/CryptoRbHsmKeySlotId

Short Name	CryptoRbHsmKeySlotId
Description	This parameter explicitly maps the given key element into a specified key slot inside the HSM secure flash. As value, either the name of the key slot (e.g. "DYNAMIC_CFG_CRYPT_AES_KEY_BASE_0") or its enumerator representation (e.g. "ecy_hsm_CSAI_KEYID_AES_KEY_DYNAMIC_CFG_CRYPT_AES_KEY_BASE_0") can be provided. If left empty, an available key slot is automatically assigned.
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

2.3.1.60 Crypto/CryptoKeyElements/CryptoKeyElement/CryptoRbHsmUserService

Short Name	CryptoRbHsmUserService
Description	This parameter determines the session to be used in the KeyDerive service based on the selected crypto service.
Multiplicity	0..1
Default Value	CRYPTO_KEYEXCHANGEALCSECRET
Enumeration Literals	CRYPTO_AEADENCRYPT, CRYPTO_ENCRYPT, CRYPTO_HASH, CRYPTO_KEYELEMENTGET, CRYPTO_KEYEXCHANGEALCSECRET, CRYPTO_MACGENERATE, CRYPTO_RANDOMGENERATE, CRYPTO_RANDOMSEED, CRYPTO_SIGNATUREGENERATE
Type	ECUC-ENUMERATION-PARAM-DEF

2.3.1.61 Crypto/CryptoKeyElements/CryptoKeyElement/CryptoRbKeyElementRteSize

Short Name	CryptoRbKeyElementRteSize
Description	Maximum size of a CRYPTO key element in bytes for RTE generation. Can be used for ASN.1 encoded keys to avoid RTE generation errors.
Multiplicity	0..1
Value Range	1..4294967295
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.62 Crypto/CryptoKeyElements/CryptoKeyElement/CryptoRbKeyElementType

Short Name	CryptoRbKeyElementType
Multiplicity	0..1
Default Value	-
Type	ECUC-ENUMERATION-PARAM-DEF

Description: Define the type of a key element. This parameter is optional and can be used to overrule the process to discover the type automatically.

CRYPTO_ELEMENT_TYPE_GENERIC - generic type for elements that do not need special identification (default)

CRYPTO_ELEMENT_TYPE_SYMMETRIC_KEY - symmetric key (e.g. AES key)

CRYPTO_ELEMENT_TYPE_CERT_X509 - Certificate (X.509) element

CRYPTO_ELEMENT_TYPE_EDDSA_ED25519_PRIV_KEY - element to be used for ED25519 signature generation (32 bytes)

CRYPTO_ELEMENT_TYPE_EDDSA_ED25519_PUB_KEY - element to be used for ED25519 signature verification (32 bytes)

CRYPTO_ELEMENT_TYPE_EDDSA_ED25519_KEYPAIR - element to be used for ED25519 signature generation and verification (64 bytes)

CRYPTO_ELEMENT_TYPE_EDDSA_ED448_PRIV_KEY: element to be used for ED448 signature generation (57 bytes)

CRYPTO_ELEMENT_TYPE_EDDSA_ED448_PUB_KEY - element to be used for ED448 signature verification (57 bytes)

CRYPTO_ELEMENT_TYPE_EDDSA_ED448_KEYPAIR - element to be used for ED448 signature generation and verification (114 bytes)

CRYPTO_ELEMENT_TYPE_ECDSA_SECP256R1_PRIV_KEY - element to be used for SECP256R1 signature generation (32 bytes)

CRYPTO_ELEMENT_TYPE_ECDSA_SECP256R1_PUB_KEY - element to be used for SECP256R1 signature verification (64 bytes)

CRYPTO_ELEMENT_TYPE_ECDSA_SECP256R1_KEYPAIR - element to be used for SECP256R1 signature generation and verification (96 bytes)

CRYPTO_ELEMENT_TYPE_ECDSA_SECP521R1_KEYPAIR - element to be used for SECP521R1 both signature generation and verification (198 bytes)

CRYPTO_ELEMENT_TYPE_ECDSA_SECP521R1_PRIV_KEY - element to be used for SECP521R1 signature generation (66 bytes)

CRYPTO_ELEMENT_TYPE_ECDSA_SECP521R1_PUB_KEY - element to be used for SECP521R1 signature verification (132 bytes)

CRYPTO_ELEMENT_TYPE_ECDSA_SECP384R1_PRIV_KEY - element to be used for SECP384R1 signature generation (48 bytes)

CRYPTO_ELEMENT_TYPE_ECDSA_SECP384R1_PUB_KEY - element to be used

for SECP384R1 signature verification (96 bytes)

CRYPTO_ELEMENT_TYPE_ECDSA_SECP384R1_KEYPAIR - element to be used for SECP384R1 signature generation and verification (144 bytes)

CRYPTO_ELEMENT_TYPE_ECDSA_SECP224R1_PRIV_KEY - element to be used for SECP224R1 signature generation (28 bytes)

CRYPTO_ELEMENT_TYPE_ECDSA_SECP224R1_PUB_KEY - element to be used for SECP224R1 signature verification (58 bytes)

CRYPTO_ELEMENT_TYPE_ECDSA_SECP224R1_KEYPAIR - element to be used for SECP224R1 signature generation and verification (84 bytes)

CRYPTO_ELEMENT_TYPE_RSA2048_PRIV_KEY - 2048 bit RSA private key (512 bytes)

CRYPTO_ELEMENT_TYPE_RSA2048_PUB_KEY - 2048 bit RSA public key (260 bytes)

CRYPTO_ELEMENT_TYPE_RSA3072_PRIV_KEY - 3072 bit RSA private key (768 bytes)

CRYPTO_ELEMENT_TYPE_RSA3072_PUB_KEY - 3072 bit RSA public key (388 bytes)

CRYPTO_ELEMENT_TYPE_RSA4096_PRIV_KEY - 4096 bit RSA private key (1024 bytes)

CRYPTO_ELEMENT_TYPE_RSA4096_PUB_KEY - 4096 bit RSA public key (516 bytes)

CRYPTO_ELEMENT_TYPE_VKMS_GET_KEY - element to be used for VKMS key retrieval

CRYPTO_ELEMENT_TYPE_VKMS_GET_METADATA - element to be used for VKMS key meta data retrieval

CRYPTO_ELEMENT_TYPE_VKMS_GET_STATUS - element to be used for VKMS status retrieval (8 bytes)

CRYPTO_ELEMENT_TYPE_VKMS_GET_TRAININGCOUNTER - element to be used for VKMS training counter retrieval (3 bytes)

CRYPTO_ELEMENT_TYPE_VKMS_GET_VIN - element to be used for VKMS VIN retrieval (17 bytes)

CRYPTO_ELEMENT_TYPE_SHARED_VALUE - key element is a key exchange shared value

Enumeration Literals: CRYPTO_ELEMENT_TYPE_CERT_X509, CRYPTO_ELEMENT_TYPE_ECDSA_SECP224R1_KEYPAIR, CRYPTO_ELEMENT_TYPE_ECDSA_SECP224R1_PRIV_KEY, CRYPTO_ELEMENT_TYPE_ECDSA_SECP224R1_PUB_KEY, CRYPTO_ELEMENT_TYPE_ECDSA_SECP256R1_KEYPAIR, CRYPTO_ELEMENT_TYPE_ECDSA_SECP256R1_PRIV_KEY, CRYPTO_ELEMENT_TYPE_ECDSA_SECP256R1_PUB_KEY, CRYPTO_ELEMENT_TYPE_ECDSA_SECP384R1_KEYPAIR, CRYPTO_ELEMENT_TYPE_ECDSA_SECP384R1_PRIV_KEY, CRYPTO_ELEMENT_TYPE_ECDSA_SECP384R1_PUB_KEY, CRYPTO_ELEMENT_TYPE_ECDSA_SECP521R1_KEYPAIR, CRYPTO_ELEMENT_TYPE_ECD-

SA_SECP521R1_PRIV_KEY, CRYPTO_ELEMENT_TYPE_ECDSA_SECP521R1_PUB_KEY, CRYPTO_ELEMENT_TYPE_EDDSA_ED25519_KEYPAIR, CRYPTO_ELEMENT_TYPE_EDDSA_ED25519_PRIV_KEY, CRYPTO_ELEMENT_TYPE_EDDSA_ED25519_PUB_KEY, CRYPTO_ELEMENT_TYPE_EDDSA_ED448_KEYPAIR, CRYPTO_ELEMENT_TYPE_EDDSA_ED448_PRIV_KEY, CRYPTO_ELEMENT_TYPE_EDDSA_ED448_PUB_KEY, CRYPTO_ELEMENT_TYPE_GENERIC, CRYPTO_ELEMENT_TYPE_RSA2048_PRIV_KEY, CRYPTO_ELEMENT_TYPE_RSA2048_PUB_KEY, CRYPTO_ELEMENT_TYPE_RSA3072_PRIV_KEY, CRYPTO_ELEMENT_TYPE_RSA3072_PUB_KEY, CRYPTO_ELEMENT_TYPE_RSA4096_PRIV_KEY, CRYPTO_ELEMENT_TYPE_RSA4096_PUB_KEY, CRYPTO_ELEMENT_TYPE_SHARED_VALUE, CRYPTO_ELEMENT_TYPE_SYMMETRIC_KEY, CRYPTO_ELEMENT_TYPE_VKMS_GET_KEY, CRYPTO_ELEMENT_TYPE_VKMS_GET_METADATA, CRYPTO_ELEMENT_TYPE_VKMS_GET_STATUS, CRYPTO_ELEMENT_TYPE_VKMS_GET_TRAININGCOUNTER, CRYPTO_ELEMENT_TYPE_VKMS_GET_VIN

2.3.1.63 Crypto/CryptoKeys/CryptoKey/CryptoRbHsmVkmsKeyTypeId

Short Name	CryptoRbHsmVkmsKeyTypeId
Description	This parameter maps the given cryptokey into a specified VKMS KeyTypeId specified inside HSM VKMS Core (to be used in VKMS GetKey and GetMetadata requests).
Multiplicity	0..1
Value Range	0..65534
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.64 Crypto/CryptoMainFunction

Short Name	CryptoMainFunction
Description	Container for configuration of a Crypto mainfunction. The container name serves as a symbolic name for the mainfunction configuration.
Multiplicity	0..*
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.65 Crypto/CryptoMainFunction/CryptoMainFunctionPartitionRef

Short Name	CryptoMainFunctionPartitionRef
Description	Reference to EcucPartition, where the according CryptoMainFunction instance is assigned to.
Multiplicity	1..1
Default Value	-
Type	ECUC-REFERENCE-DEF

2.3.1.66 Crypto/CryptoMainFunction/CryptoRbMainFunctionLoopCounter

Short Name	CryptoRbMainFunctionLoopCounter
Description	Holds the number of iterations to be performed internally by the main function. This parameter can be used to increase or reduce the runtime of a single main function call.
Multiplicity	0..1
Value Range	1..4294967295
Default Value	4294967295
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.67 Crypto/CryptoPrimitives/CryptoPrimitive/CryptoRbHsmBulkProcessingModeEnabled

Short Name	CryptoRbHsmBulkProcessingModeEnabled
Description	This parameter can be filled out to switch between Mac processing modes.
Introduction	True: Enables the bulk processing mode for any mac generation/verification. False: Disables the bulk processing mode for any mac generation/verification (FALSE = default).
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.68 Crypto/CryptoRbPersistentDataMappings

Short Name	CryptoRbPersistentDataMappings
Description	Container for CryptoRbPersistentDataMapping Objects
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.69 Crypto/CryptoRbPersistentDataMappings/CryptoRbPersistentDataMapping

Short Name	CryptoRbPersistentDataMapping
Description	Container for CryptoRbPersistentDataMapping configuraton parameters
Multiplicity	0..*
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.70 Crypto/CryptoRbPersistentDataMappings/CryptoRbPersistent-

DataMapping/CryptoRbMappingTag

Short Name	CryptoRbMappingTag
Multiplicity	1..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

Description: Unique string tag of data mapping. This links key valid flags and key element data to offsets in the NvM data array they will be stored in or, in the case of secured keys stored in HSM, it links the keys to ecy_hsm Key ID's.

There is a strict correlation between the CryptoRbMappingTag and the short name of a key or key element it is linked to. The CryptoRbMappingTag is constructed as follows:

[Key ShortName]__[Persistent Block Id]__[Length]__[Key Element ID]

- The Key Element ID will only be present for mappings linked to Key Elements.

Example 1: There is a key with short name 'CryptoKey_AesEncrypt' and block ID 0. To link this key to this mapping, the CryptoRbMappingTag must be 'CryptoKey_AesEncrypt_0_1'. CryptoKeys by themselves contain no data except their valid flag, hence the length is always 1.

Example 2: There is a key element with size 32, key element id 1 and short name 'CryptoKeyElement_AesEncrypt' and it is linked through a CryptoKeyType to CryptoKey 'CryptoKey_AesEncrypt' that is in persistent block 1. To link this key element to this mapping, the CryptoRbMappingTag must be 'CryptoKey_AesEncrypt_1_32_1'.

2.3.1.71 Crypto/CryptoRbPersistentDataMappings/CryptoRbPersistentDataMapping/CryptoRbPersistentHsmKeySlot

Short Name	CryptoRbPersistentHsmKeySlot
Description	String representation of ecy_hsm key ID to use for this key element. This field is valid only for data mappings linked to rba_CryptoHSM key elements that will be stored in the HSM. The name of the key slot (DYNAMIC_CFG_CRYPT_AES_KEY_BASE_0) can be provided, or the enumerator representation of the key slot (ecy_hsm_CSAI_KEYID_AES_KEY_DYNAMIC_CFG_CRYPT_AES_KEY_BASE_0) can be provided.
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

2.3.1.72 Crypto/CryptoRbPersistentDataMappings/CryptoRbPersistentDataMapping/CryptoRbPersistentOffset

Short Name	CryptoRbPersistentOffset
Description	Offset (in bytes) of key data or key valid flag into its corresponding NvM Block data array.
Multiplicity	0..1
Value Range	0..4294967295
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

Crypto is a structural component of the RTA-SEC stack, but does not itself provide any API functions.

Please see the sections for rba_CryptoCCL, rba_CryptoHSM, rba_CryptoAuCSC and rba_CryptoAuHSM3 for details of their module-specific extensions to the standard AUTOSAR specification.

2.4 Limitations

Crypto does not have specific limitations (see Supported Features section). Please refer to the Limitations sections of the rba_CryptoCCL, rba_CryptoHSM, rba_CryptoAuCSC and rba_CryptoAuHSM3 modules.

2.5 Exclusions

Crypto is a structural component of the RTA-SEC stack, but does not itself provide any API functions.

Please see the sections for rba_CryptoCCL, rba_CryptoHSM, rba_CryptoAuCSC and rba_CryptoAuHSM3 for details of module-specific exclusions of the following generic AUTOSAR-defined APIs.

- Crypto_KeyDerive
- Crypto_NvBlock_Init_<NvBlock>
- Crypto_NvBlock_ReadFrom_<NvBlock>
- Crypto_NvBlock_WriteTo_<NvBlock>
- Crypto_NvBlock_Callback_<NvBlock>

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- Crypto/CryptoDriverObjects/CryptoDriverObject/CryptoDriverObjectEcuc-PartitionRef

- Crypto/CryptoGeneral/CryptoEcucPartitionRef

2.6 Deviations

2.6.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

2.6.1.1 Crypto/CryptoDriverObjects/CryptoDriverObject/CryptoDriverObjectId

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

2.6.1.2 Crypto/CryptoGeneral/CryptoInstancelId

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

2.6.1.3 Crypto/CryptoGeneral/CryptoVersionInfoApi

[RTA-BSW] Introduction	True: API Crypto_GetVersionInfo() is available False: API Cryptosm_GetVersionInfo() is not available.
[AUTOSAR] Introduction	True: API Crypto_GetVersionInfo() is available False: API Crypto_GetVersionInfo() is not available.

2.6.1.4 Crypto/CryptoKeyElements/CryptoKeyElement/CryptoKeyElementFormat

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1
[RTA-BSW] Enumeration Literals	CRYPTO_KE_FORMAT_BIN_CERT_CVC, CRYPTO_KE_FORMAT_BIN_CERT_X509_V3, CRYPTO_KE_FORMAT_BIN_IDENT_PRIVATEKEY_PKCS8, CRYPTO_KE_FORMAT_BIN_IDENT_PUBLICKEY, CRYPTO_KE_FORMAT_BIN_OCTET, CRYPTO_KE_FORMAT_BIN_RSA_PRIVATEKEY, CRYPTO_KE_FORMAT_BIN_RSA_PUBLICKEY, CRYPTO_KE_FORMAT_BIN_SHEKEYS
[AUTOSAR] Enumeration Literals	CRYPTO_KE_FORMAT_BIN_IDENT_PRIVATEKEY_PKCS8, CRYPTO_KE_FORMAT_BIN_IDENT_PUBLICKEY, CRYPTO_KE_FORMAT_BIN_OCTET, CRYPTO_KE_FORMAT_BIN_RSA_PRIVATEKEY, CRYPTO_KE_FORMAT_BIN_RSA_PUBLICKEY, CRYPTO_KE_FORMAT_BIN_SHEKEYS

[ADDED] Default Value	CRYPTO_KE_FORMAT_BIN_OCTET
-----------------------	----------------------------

2.6.1.5 Crypto/CryptoKeyElements/CryptoKeyElement/CryptoKeyElementReadAccess

[RTA-BSW] Description	Define the reading access rights of the key element.
[AUTOSAR] Description	Define the reading access rights of the key element through external API.

2.6.1.6 Crypto/CryptoKeyElements/CryptoKeyElement/CryptoKeyElementSize

[RTA-BSW] Description	Maximum size of a CRYPTO key element in bytes
[AUTOSAR] Description	Maximum Size size of a CRYPTO key element in bytes

2.6.1.7 Crypto/CryptoKeyElements/CryptoKeyElement/CryptoKeyElementWriteAccess

[RTA-BSW] Description	Define the writing access rights of the key element
[AUTOSAR] Description	Define the writing access rights of the key element through external API.

2.6.1.8 Crypto/CryptoKeys/CryptoKey/CryptoKeyId

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

2.6.1.9 Crypto/CryptoKeys/CryptoKey/CryptoKeyTypeRef

[RTA-BSW] Description	Refers to a pointer in the CRYPTO to a CryptoKeyType. The CryptoKeyType provides the information which key elements are contained in a CryptoKey.
[AUTOSAR] Description	Refers to a pointer in the CRYPTO to a CryptoKeyType. The CryptoKeyType provides the information which key elements are contained in a CryptoKey.

2.6.1.10 Crypto/CryptoMainFunction/CryptoMainFunctionPeriod

[RTA-BSW] Parent Container	Crypto/CryptoMainFunction/CryptoMainFunctionPeriod
[AUTOSAR] Parent Container	Crypto/CryptoGeneral/CryptoMainFunctionPeriod

[RTA-BSW] Description	Specifies the period of main function Crypto_MainFunction in seconds (applicable only for the default MainFunction instance).
[AUTOSAR] Description	Specifies the period of main function Crypto_MainFunction in seconds.

2.6.1.11 Crypto/CryptoNvStorage/CryptoNvBlock/CryptoNvBlockFailedRetries

[RTA-BSW] Minimum Value	0
[AUTOSAR] Minimum Value	1

2.6.1.12 Crypto/CryptoPrimitives/CryptoPrimitive/CryptoPrimitiveAlgorithmFamily

[RTA-BSW] Enumeration Literals: CRYPTO_ALGOFAM_3DES, CRYPTO_ALGOFAM_AES, CRYPTO_ALGOFAM_BLAKE_1_256, CRYPTO_ALGOFAM_BLAKE_1_512, CRYPTO_ALGOFAM_BLAKE_2s_256, CRYPTO_ALGOFAM_BLAKE_2s_512, CRYPTO_ALGOFAM_BRAINPOOL, CRYPTO_ALGOFAM_CHACHA, CRYPTO_ALGOFAM_CUSTOM, CRYPTO_ALGOFAM_DH, CRYPTO_ALGOFAM_DRBG, CRYPTO_ALGOFAM_ECCANSI, CRYPTO_ALGOFAM_ECCNIST, CRYPTO_ALGOFAM_ECCSEC, CRYPTO_ALGOFAM_ECDH, CRYPTO_ALGOFAM_ECDSA, CRYPTO_ALGOFAM_ED25519, CRYPTO_ALGOFAM_EEA3, CRYPTO_ALGOFAM_EIA3, CRYPTO_ALGOFAM_FIPS186, CRYPTO_ALGOFAM_HKDF, CRYPTO_ALGOFAM_KDFX963, CRYPTO_ALGOFAM_NOT_SET, CRYPTO_ALGOFAM_PADDING_ONETHZEROS, CRYPTO_ALGOFAM_PADDING_PKCS7, CRYPTO_ALGOFAM_PBKDF2, CRYPTO_ALGOFAM_POLY1305, CRYPTO_ALGOFAM_RIPEMD160, CRYPTO_ALGOFAM_RNG, CRYPTO_ALGOFAM_RSA, CRYPTO_ALGOFAM_SHA1, CRYPTO_ALGOFAM_SHA2_224, CRYPTO_ALGOFAM_SHA2_256, CRYPTO_ALGOFAM_SHA2_384, CRYPTO_ALGOFAM_SHA2_512, CRYPTO_ALGOFAM_SHA2_512_224, CRYPTO_ALGOFAM_SHA2_512_256, CRYPTO_ALGOFAM_SHA3_224, CRYPTO_ALGOFAM_SHA3_256, CRYPTO_ALGOFAM_SHA3_384, CRYPTO_ALGOFAM_SHA3_512, CRYPTO_ALGOFAM_SHAKE128, CRYPTO_ALGOFAM_SHAKE256, CRYPTO_ALGOFAM_SIPHASH, CRYPTO_ALGOFAM_SM2, CRYPTO_ALGOFAM_SM3, CRYPTO_ALGOFAM_VKMS_GET_IDENTITY_HASH, CRYPTO_ALGOFAM_VKMS_GET_PSS_HASH, CRYPTO_ALGOFAM_VKMS_GET_VERIFICATION_HASH, CRYPTO_ALGOFAM_VKMS_HANDLE_DLC_FINISH, CRYPTO_ALGOFAM_VKMS_HANDLE_DLC_START, CRYPTO_ALGOFAM_X25519

[AUTOSAR] Enumeration Literals: CRYPTO_ALGOFAM_3DES, CRYPTO_ALGOFAM_AES, CRYPTO_ALGOFAM_BLAKE_1_256, CRYPTO_ALGOFAM_BLAKE_1_512, CRYPTO_ALGOFAM_BLAKE_2s_256, CRYPTO_ALGOFAM_BLAKE_2s_512, CRYPTO_ALGOFAM_BRAINPOOL, CRYPTO_ALGOFAM_CHACHA, CRYPTO_ALGOFAM_CUSTOM, CRYPTO_ALGOFAM_DH, CRYPTO_ALGOFAM_DRBG, CRYPTO_ALGOFAM_ECCANSI, CRYPTO_ALGOFAM_ECCNIST, CRYPTO_ALGOFAM_ECCSEC, CRYPTO_ALGOFAM_ECDH, CRYPTO_ALGOFAM_ECDSA, CRYPTO_ALGOFAM_ED25519, CRYPTO_ALGOFAM_EEA3, CRYPTO_ALGOFAM_EIA3, CRYPTO_ALGOFAM_FIPS186, CRYPTO_ALGOFAM_HKDF, CRYPTO_ALGOFAM_KDFX963, CRYPTO_ALGOFAM_NOT_SET, CRYPTO_ALGOFAM_PADDING_ONETHZEROS, CRYPTO_ALGOFAM_PADDING_PKCS7, CRYPTO_ALGOFAM_PBKDF2, CRYPTO_ALGOFAM_POLY1305, CRYPTO_ALGOFAM_RIPEMD160, CRYPTO_ALGOFAM_RNG, CRYPTO_ALGOFAM_RSA, CRYPTO_ALGOFAM_SHA1, CRYPTO_ALGOFAM_SHA2_224, CRYPTO_ALGOFAM_SHA2_256, CRYPTO_ALGOFAM_SHA2_384, CRYPTO_ALGOFAM_SHA2_512, CRYPTO_ALGOFAM_SHA2_512_224, CRYPTO_ALGOFAM_SHA2_512_256, CRYPTO_ALGOFAM_SHA3_224, CRYPTO_ALGOFAM_SHA3_256, CRYPTO_ALGOFAM_SHA3_384, CRYPTO_ALGOFAM_SHA3_512, CRYPTO_ALGOFAM_SHAKE128, CRYPTO_ALGOFAM_SHAKE256, CRYPTO_ALGOFAM_SIPHASH, CRYPTO_ALGOFAM_SM2, CRYPTO_ALGOFAM_SM3, CRYPTO_ALGOFAM_VKMS_GET_IDENTITY_HASH, CRYPTO_ALGOFAM_VKMS_GET_PSS_HASH, CRYPTO_ALGOFAM_VKMS_GET_VERIFICATION_HASH, CRYPTO_ALGOFAM_VKMS_HANDLE_DLC_FINISH, CRYPTO_ALGOFAM_VKMS_HANDLE_DLC_START, CRYPTO_ALGOFAM_X25519

TO_ALGOFAM_EIA3, CRYPTO_ALGOFAM_FIPS186, CRYPTO_ALGOFAM_HKDF, CRYPTO_ALGOFAM_KDFX963, CRYPTO_ALGOFAM_NOT_SET, CRYPTO_ALGOFAM_PADDING_ONETHZEROS, CRYPTO_ALGOFAM_PADDING_PKCS7, CRYPTO_ALGOFAM_PBKDF2, CRYPTO_ALGOFAM_POLY1305, CRYPTO_ALGOFAM_RIPEMD160, CRYPTO_ALGOFAM_RNG, CRYPTO_ALGOFAM_RSA, CRYPTO_ALGOFAM_SHA1, CRYPTO_ALGOFAM_SHA2_224, CRYPTO_ALGOFAM_SHA2_256, CRYPTO_ALGOFAM_SHA2_384, CRYPTO_ALGOFAM_SHA2_512, CRYPTO_ALGOFAM_SHA2_512_224, CRYPTO_ALGOFAM_SHA2_512_256, CRYPTO_ALGOFAM_SHA3_224, CRYPTO_ALGOFAM_SHA3_256, CRYPTO_ALGOFAM_SHA3_384, CRYPTO_ALGOFAM_SHA3_512, CRYPTO_ALGOFAM_SHAKE128, CRYPTO_ALGOFAM_SHAKE256, CRYPTO_ALGOFAM_SIPHASH, CRYPTO_ALGOFAM_SM2, CRYPTO_ALGOFAM_SM3, CRYPTO_ALGOFAM_X25519

2.6.1.13 Crypto/CryptoPrimitives/CryptoPrimitive/CryptoPrimitiveAlgorithmFamilyCustomRef

[RTA-BSW] Description	Reference to custom family container
[AUTOSAR] Description	Reference to a customer specific algorithm family custom container

2.6.1.14 Crypto/CryptoPrimitives/CryptoPrimitive/CryptoPrimitiveAlgorithmMode

[RTA-BSW] Enumeration Literals: CRYPTO_ALGOMODE_12ROUNDS, CRYPTO_ALGOMODE_20ROUNDS, CRYPTO_ALGOMODE_8ROUNDS, CRYPTO_ALGOMODE_AESKEYWRAP, CRYPTO_ALGOMODE_CBC, CRYPTO_ALGOMODE_CFB, CRYPTO_ALGOMODE_CMAC, CRYPTO_ALGOMODE_CTR, CRYPTO_ALGOMODE_CTRDRBG, CRYPTO_ALGOMODE_CUSTOM, CRYPTO_ALGOMODE_ECB, CRYPTO_ALGOMODE_GCM, CRYPTO_ALGOMODE_GMAC, CRYPTO_ALGOMODE_HMAC, CRYPTO_ALGOMODE_NOT_SET, CRYPTO_ALGOMODE_OFB, CRYPTO_ALGOMODE_PXXXR, CRYPTO_ALGOMODE_RSAES_OAEP, CRYPTO_ALGOMODE_RSAES_PKCS1_v1_5, CRYPTO_ALGOMODE_RSASSA_PKCS1_v1_5, CRYPTO_ALGOMODE_RSASSA_PSS, CRYPTO_ALGOMODE_SIPHASH_2_4, CRYPTO_ALGOMODE_SIPHASH_4_8, CRYPTO_ALGOMODE_XTS

[AUTOSAR] Enumeration Literals: CRYPTO_ALGOMODE_12ROUNDS, CRYPTO_ALGOMODE_20ROUNDS, CRYPTO_ALGOMODE_8ROUNDS, CRYPTO_ALGOMODE_CBC, CRYPTO_ALGOMODE_CFB, CRYPTO_ALGOMODE_CMAC, CRYPTO_ALGOMODE_CTR, CRYPTO_ALGOMODE_CTRDRBG, CRYPTO_ALGOMODE_CUSTOM, CRYPTO_ALGOMODE_ECB, CRYPTO_ALGOMODE_GCM, CRYPTO_ALGOMODE_GMAC, CRYPTO_ALGOMODE_HMAC, CRYPTO_ALGOMODE_NOT_SET, CRYPTO_ALGOMODE_OFB, CRYPTO_ALGOMODE_PXXXR, CRYPTO_ALGOMODE_RSAES_OAEP, CRYPTO_ALGOMODE_RSAES_PKCS1_v1_5, CRYPTO_ALGOMODE_RSASSA_PKCS1_v1_5, CRYPTO_ALGOMODE_RSASSA_PSS, CRYPTO_ALGOMODE_SIPHASH_2_4, CRYPTO_ALGOMODE_SIPHASH_4_8, CRYPTO_ALGOMODE_XTS

2.6.1.15 Crypto/CryptoPrimitives/CryptoPrimitive/CryptoPrimitiveAlgorithmModeCustomRef

[RTA-BSW] Description	Reference to custom mode container
[AUTOSAR] Description	Reference to a customer specific algorithm mode custom container

2.6.1.16 Crypto/CryptoPrimitives/CryptoPrimitive/CryptoPrimitiveAlgorithmSecondaryFamilyCustomRef

[RTA-BSW] Description	Reference to custom secondary family container
[AUTOSAR] Description	Reference to a customer specific algorithm family custom container

2.6.1.17 Crypto/CryptoPrimitives/CryptoPrimitive/CryptoPrimitiveService

[RTA-BSW] Enumeration Literals: CRYPTO_AEADDECRYPT, CRYPTO_AEADENCRYPT, CRYPTO_CERTIFICATEPARSE, CRYPTO_CERTIFICATEVERIFY, CRYPTO_CUSTOM_SYNC, CRYPTO_DECRYPT, CRYPTO_ENCRYPT, CRYPTO_HASH, CRYPTO_KEYDERIVE, CRYPTO_KEYELEMENTCOPY, CRYPTO_KEYELEMENTGET, CRYPTO_KEYELEMENTSET, CRYPTO_KEYEXCHANGEALCPUBVAL, CRYPTO_KEYEXCHANGEALCSECRET, CRYPTO_KEYGENERATE, CRYPTO_KEYPERSIST, CRYPTO_KEYSETINVALID, CRYPTO_KEYSETVALID, CRYPTO_MACGENERATE, CRYPTO_MACVERIFY, CRYPTO_RANDOMGENERATE, CRYPTO_RANDOMSEED, CRYPTO_SHEEXPORTRAMKEY, CRYPTO_SHEGETID, CRYPTO_SHELOADKEY, CRYPTO_SHELOADKEYSLOT, CRYPTO_SHELOADPLAINKEY, CRYPTO_SIGNATUREGENERATE, CRYPTO_SIGNATUREVERIFY, CUSTOM_SERVICE

[AUTOSAR] Enumeration Literals: CRYPTO_AEADDECRYPT, CRYPTO_AEADENCRYPT, CRYPTO_DECRYPT, CRYPTO_ENCRYPT, CRYPTO_HASH, CRYPTO_KEYDERIVE, CRYPTO_KEYEXCHANGEALCPUBVAL, CRYPTO_KEYEXCHANGEALCSECRET, CRYPTO_KEYGENERATE, CRYPTO_KEYSETINVALID, CRYPTO_KEYSETVALID, CRYPTO_MACGENERATE, CRYPTO_MACVERIFY, CRYPTO_RANDOMGENERATE, CRYPTO_RANDOMSEED, CRYPTO_SIGNATUREGENERATE, CRYPTO_SIGNATUREVERIFY, CUSTOM_SERVICE

2.6.1.18 Crypto/CryptoPrimitives/CryptoPrimitive/CryptoPrimitiveSupportContext

[RTA-BSW] Description	Configures if the crypto primitive supports to store or restore context data of the workspace. Since this option is vulnerable to security, it should be set to TRUE if absolutely needed
[AUTOSAR] Description	Configures if the crypto primitive supports to store or restore context data of the workspace. Since this option is vulnerable to security, it shall only set to TRUE if absolutely needed.

[ADDED] Introduction	True: Enables crypto primitive supports to store or restore context data. False: Disables crypto primitive supports to store or restore context data.
----------------------	---

2.6.1.19 Crypto/CryptoPrimitives/CryptoPrimitiveAlgorithmFamilyCustom

[RTA-BSW] Description	Container for custom algorithm families
[AUTOSAR] Description	Container of custom algorithm family values.
[REMOVED] Introduction	The container name serves as a symbolic name for the identifier of the custom algorithm family type.

2.6.1.20 Crypto/CryptoPrimitives/CryptoPrimitiveAlgorithmFamilyCustom/ CryptoPrimitiveAlgorithmFamilyCustomId

[RTA-BSW] Description	Custom algorithm family id
[AUTOSAR] Description	The custom value of this algorithm family

2.6.1.21 Crypto/CryptoPrimitives/CryptoPrimitiveAlgorithmModeCustom

[RTA-BSW] Description	Container for custom algorithm modes
[AUTOSAR] Description	Container of custom algorithm family values.
[REMOVED] Introduction	The container name serves as a symbolic name for the identifier of the custom algorithm family type.

2.6.1.22 Crypto/CryptoPrimitives/CryptoPrimitiveAlgorithmModeCustom/ CryptoPrimitiveAlgorithmModeCustomId

[RTA-BSW] Description	Custom mode mode id
[AUTOSAR] Description	The custom value of this algorithm mode

2.7 API Definition

Crypto is a structural component of the RTA-SEC stack, but does not itself provide any API functions.

Please see the sections for rba_CryptoCCL (Section 7.7), rba_CryptoHSM (Section 8.7), rba_CryptoAuCSC (Section 6.7) and rba_CryptoAuHSM3 (Section 9.7) for more details of the module-specific implementations of the following AUTOSAR-defined APIs:

Crypto_Init
Crypto_GetVersionInfo
Crypto_ProcessJob
Crypto_CancelJob
Crypto_KeyElementSet
Crypto_KeySetValid

Crypto_KeyElementGet
Crypto_KeyElementCopy
Crypto_KeyElementCopyPartial
Crypto_KeyCopy
Crypto_KeyElementIdsGet
Crypto_RandomSeed
Crypto_KeyGenerate
Crypto_KeyDerive
Crypto_KeyExchangeCalcPubVal
Crypto_KeyExchangeCalcSecret
Crypto_MainFunction
Crypto_KeySetInvalid
Crypto_KeyGetStatus
Crypto_CustomSync

2.8 Configuration Advice



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

2.8.1 Configuration of Package Paths

Since the Crypto module's *rba_Crypto_Prot_EcucValues.arxml* file is generated by RTA-BSW CodeGen with the package path */RB/UBK/Project/EcucModuleConfigurationValues*, it is necessary beforehand to change the package paths of the configuration files of the modules (e.g. *rba_CryptoHSM*, *rba_CryptoAuCSC*, *rba_CryptoAuHSM3*) that also contain Crypto configuration values.

This is because the latter configurations are, by default, generated with the RTA-CAR-standard package path */ETAS_Project/EcucModuleConfigurationValues*. With such a path mismatch, however, the Crypto configuration values will not be merged (at CodeGen) and an error will result.

Therefore, before proceeding with the initial ConfGen to generate Crypto, it is necessary to change the package path in Project settings from the RTA-CAR default to */RB/UBK/Project/EcucModuleConfigurationValues*:

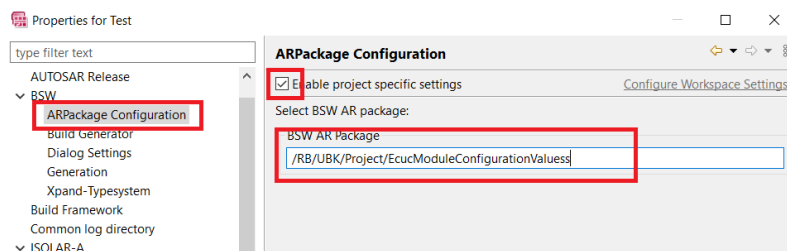


Figure 2.2.Update package path

If the Crypto module is being configured manually, the User can manually update Crypto's EcucValues configuration to be under the */RB/UBK/Project/EcucModuleConfigurationValues* path.

2.8.2 Automatic Configuration (Via Csm)

Some of the configuration of the Crypto stack modules, including Crypto, can be achieved automatically. This is done through the Csm module using the "Flexible" configuration method, which is enabled via the CsmRbAutoFillConfig parameter (in Csm/CsmGeneral). See the Configuration Advice in the Csm section for more details.

2.8.3 Configuration of Crypto Drivers

As described in the Supported Features, Crypto provides the AUTOSAR-defined configuration parameters required to define unique driver object instances for cryptographic primitives defined in *rba_CryptoCCL*, *rba_CryptoHSM*, *rba_CryptoAuCSC* and *rba_CryptoAuHSM3*. Please refer to the AUTOSAR documentation on the Crypto driver (AUTOSAR_SWS_CryptoDriver.pdf) for full details of these standard configuration parameters.

Configuration advice that is specific to *rba_CryptoCCL*, *rba_CryptoHSM*, *rba_CryptoAuCSC* and *rba_CryptoAuHSM3* can be found in the relevant section of this Reference Guide.

Configuration advice that is relevant to *rba_CryptoCCL*, *rba_CryptoHSM*, *rba_CryptoAuCSC* and *rba_CryptoAuHSM3* can be found below.

To support 3rd party CryptoDriver, the parameter *cryptoPrimitiveRef* must have multiplicity [1:1] in paramdef, otherwise CodeGen throws error `Couldn't find operation 'Csm_Priv_getParameterOrDefault(List,String,String)' for ECUC::Autosar`

2.8.4 CryptoRbWaitForNvMRead

The *rba_CryptoCCL*, *rba_CryptoHSM*, *rba_CryptoAuCSC* and *rba_CryptoAuHSM3* modules may have (based on configuration) variables which need to be persistent stored via NvM, and during startup these values have to be loaded. Crypto components provide configuration options how to handle the loading from NvM and how to assign keys/keyElements to different NvM blocks.

- True: In Crypto Init wait for NvM Read to be completed for required NvM

block (Calls Memstack Mainfunctions intern).

- False: Cyclically check NvM Read status in Crypto mainfunction if read not already finished before Crypto Init.

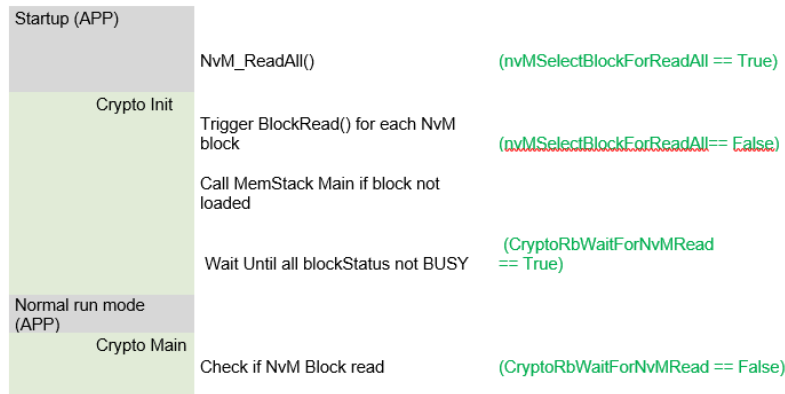


Figure 2.3.NvM Init Handling

2.8.5 NvMSelectBlockForReadAll

The parameter found in the referenced NvMBlockDescriptors, when automatically created via Forwarding mechanism, will then always have a False value configured. To change to ReadAll True, NvMBlockDescriptors must be explicitly configured and in all blocks needs to be set to True.

2.8.6 CryptoKeyNvBlockRef

CryptoKeyNvBlockRef (CsmRbAutoConfigCryptoNvBlockRef in Csm) is an optional parameter found in each CryptoKey (CsmKey), this is used to explicitly link a CryptoKey with its referenced CryptoKeyElements material to a specific NvM Block Descriptor. This way the Crypto will know which NvM block to use to store or load a persistent block.

If not set then all such persistent keys will be automatically mapped to a single NvM block (the first one found in the configuration). Based on the mapping for all NvM blocks the minimum required size is automatically calculated and filled out. KeyElements are grouped together based on the parent key persistent block reference.

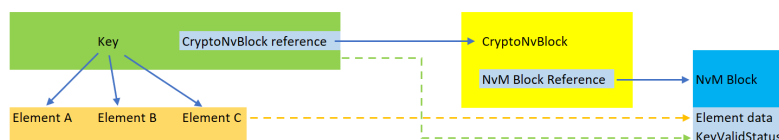


Figure 2.4.NvM Mapping

2.8.7 CryptoRbPersistentDataMapping

Each persistent group containing key and key elements is stored in its own NvM block. The valid status flag for each CryptoKey and the data content of each CryptoKeyElement is stored in these blocks. A fixed position is assigned to each data and valid status during code generation. With this fixed position and group mapping, Crypto always knows from where to load and where to store the persistent data elements.

The fixed position of each key status and each key element data is configurable via the CryptoRbPersistentDataMappings container, or are automatically positioned by the Crypto Drivers if a key is not linked to a specific mapping.

In addition, CryptoKeyElements stored in the HSM have a fixed HSM Slot ID, which can also be configured through the CryptoRbPersistentDataMappings container (CryptoRbPersistentHsmKeySlot).

Each CryptoDriver generator will generate an arxml file (rba_Crypto*_PDM_E-cucValues.arxml) that contains all CryptoRbPersistentDataMappings for all crypto keys and crypto key elements configured in the project. In order to preserve the positions and data of key and key elements between project software versions, this output shall be used as input for the next version of project software.

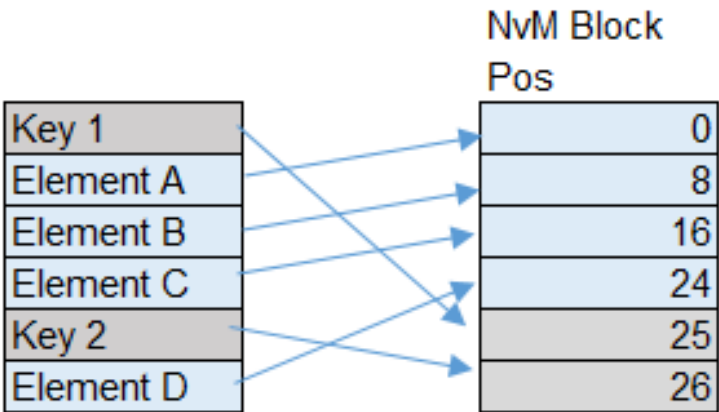


Figure 2.5.NvM Block Content

Manual configuration is not usually required for the data mapping configuration, and the output of the previous software is directly used as input for the next software. However, it is possible to manually configure the positions to link the CryptoRbPersistentDataMapping container to a key or key element so that the key or key element’s data can be manually specified.

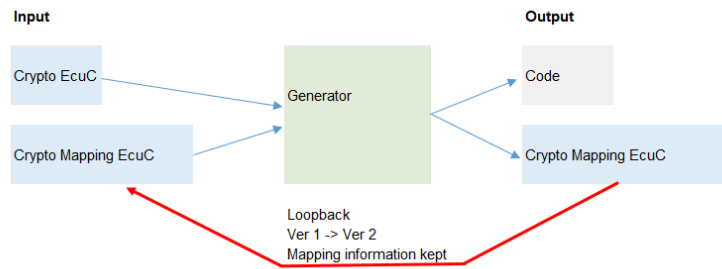


Figure 2.6.NvM Data Mappings

2.8.8 Deprecated Parameters

CryptoRbNvmDeviceld and CryptoRbNvmBlockReadMode are now deprecated.

If CryptoRbNvmDeviceld was configured with a value of 0, then it can be removed and no other changes are needed. If it was configured with a non-default value, then after its removal, an NvM block needs to be configured with the NvMN-vramDeviceld parameter set to the old CryptoRbNvmDeviceld value. The other parameters in the NvM block can be left empty.

If CryptoRbNvmBlockReadMode was configured with the value SINGLE_BLOCK, then it can be removed and no other changes are needed. If it was configured as READ_ALL, then after its removal, an NvM block needs to be configured with the NvMSelectBlockForReadAll parameter set to True.

In both of the above cases, the configured NvM block needs to be referenced by a CryptoRbNvmMapping with CryptoRbPersistentGroupld set to 0 (to map it to the default grouping).

2.9 Integration Advice

2.9.1 Init Functions

No Init is required for Crypto, but see rba_CryptoCCL_Init, rba_CryptoHSM_Init, rba_CryptoAuCSC_Startup and rba_CryptoAuHSM3_Startup in their respective sections.

2.9.2 DelInit Functions

No DelInit is required for Crypto, but see rba_CryptoCCL_DelInit, rba_CryptoHSM_DelInit, rba_CryptoAuCSC_Shutdown and rba_CryptoAuHSM3_Shutdown in their respective sections.

2.9.3 Scheduled Functions

Crypto does not have a scheduled function, but see rba_CryptoCCL_MainFunction, rba_CryptoHSM_MainFunction, rba_CryptoAuCSC_Process and rba_CryptoAuHSM3_Process in their respective sections.

To avoid Maintask cycles being wasted, Maintasks should be called in top-down

order (e.g. SecOC → Csm → CryptoDriver).

2.9.4 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The Crypto module does not use any exclusive OS resources.

2.10 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

3 Crypto Service Manager

Module name	Csm
Package	RTA-SEC
AUTOSAR Module ID	110
AUTOSAR Version	22-11

3.1 Introduction

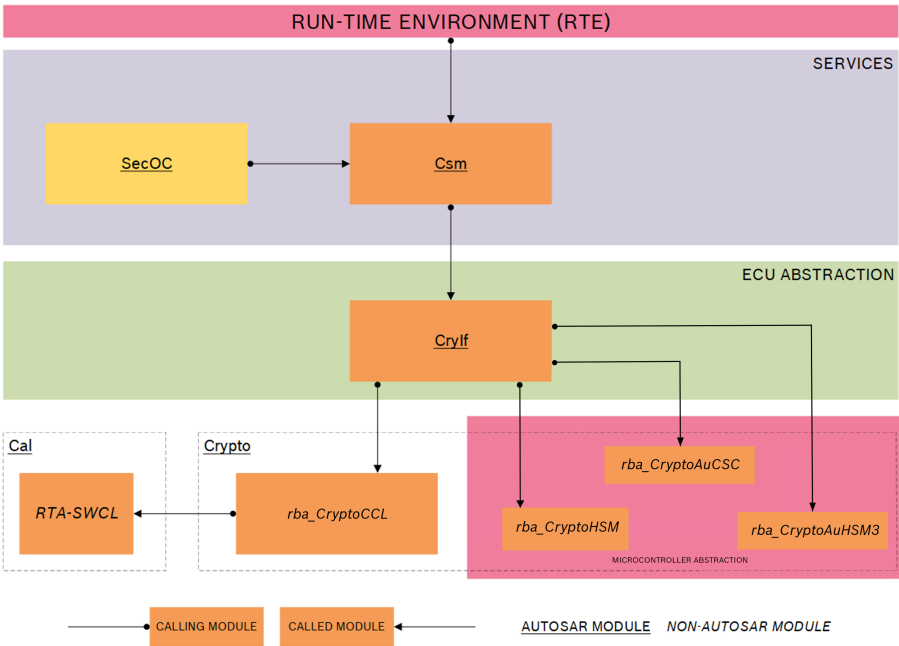


Figure 3.1.RTA-SEC Stack Architecture

The Crypto Service Manager module (Csm) is located in the system services layer of the AUTOSAR layered architecture as shown in the diagram above.

Csm provides both synchronous and asynchronous access via a standard interface (CryIf) to low-level cryptographic primitives for other RTA-BSW modules, typically SecOC.

Csm may be configured and initialised on a per-module basis, depending on the specific requirements of each module. It may also be called directly by application software through the RTE.

Csm supports multi-core, re-entrant services. This means that jobs requested by multiple applications, which may reside on the same or a separate processor core, may be processed simultaneously and in parallel, regardless of type.

Module Configuration

This module is not currently supported by the RTA-BSW Configuration Generator (ConfGen) and must be manually configured.

3.2 Supported Features

Csm provides the following:

- Synchronous and asynchronous job processing by creating a unique instance of a configured cryptographic primitive for each job.
- A state machine for each job request, which defines the operation mode of the job.
- Multiple priority-based queues for job processing, where each queue is mapped to a logical channel. In this context, a channel is defined as the path from Csm, through Crylf and Crypto, to a Crypto object.
- Key Management services for cryptographic operations, including access to the Persistent Storage of key elements.
- Certificate handling, via `Csm_CertificateVerify` and `Csm_CertificateParse` APIs (jobs are not supported).
- External Callout Support (for the `RandomGenerate` and `RandomSeed` operations).
- VKMS Functionalities (Job Operations and Key Operations).
- Job Context (Save Operation and Restore Operation).

3.2.1 Csm Queues

Csm passes asynchronous jobs to the Crylf module periodically by maintaining priority queues for all pending jobs. Performance is optimized for small queues, as this requires a simpler implementation. Queues are local to `Csm_MainFunction` and are not shared with other processes. `Csm_MainFunction` also handles job cancellation, both internally and (if possible) for the corresponding Crylf job.

3.2.2 Callback Interface

Csm provides the `Csm_CallBackNotification` callback, which is called by Crylf to signal that a job has completed.

3.2.3 Job States

In addition to the AUTOSAR-specified job states of *Idle* and *Active*, the RTA-BSW implementation of Csm includes two additional states of *Waiting* and *Running* to facilitate asynchronous processing. The 4 states are as follows:

- **Idle** - The job is not referenced in any queues and is not currently being processed by a Crypto driver.
- **Waiting** - The job is waiting in the Csm queue or Crypto queue for processing.
- **Running** - The job is currently being processed by a Crypto driver.
- **Active** - The job has finished all operations and is available for update calls.

These states control how job requests and cancellations are handled. A job can

be accepted or updated only in the *Idle* or *Active* states, while only jobs in the *Active* or *Waiting* states can be cancelled.

3.2.4 Job Processing

The following extra functions have been implemented which allow certain key management functions to be processed as jobs.

- Csm_JobRandomSeed (rba_CryptoHSM)
- Csm_JobKeyExchangeCalcPubVal (rba_CryptoHSM and rba_CryptoCCL)
- Csm_JobKeyExchangeCalcSecret (rba_CryptoHSM and rba_CryptoCCL)

These services run only synchronously or asynchronously in Singlecall mode and do not support the streaming approach.

3.2.5 Persistent Key Storage

Csm supports persistent storage of keys and certificates in non-volatile memory. Persistent storage is available to all RTA-SEC modules.

3.2.6 Cryptographic Services

Csm supports the following cryptographic services and algorithms:

Cryptographic Service	Algorithm Mode	Algorithm Family	Secondary Family	Crypto Driver Support
AEADENCRYPT	GCM	AES	N/A	rba_CryptoHSM and rba_CryptoCCL
AEADENCRYPT	20ROUNDS	CHACHA	POLY1305	rba_CryptoCCL
AEADDECRYPT	CCM	AES	N/A	rba_CryptoCCL
AEADDECRYPT	GCM	AES	N/A	rba_CryptoHSM and rba_CryptoCCL
AEADDECRYPT	20ROUNDS	CHACHA	POLY1305	rba_CryptoCCL
AEADDECRYPT	CCM	AES	N/A	rba_CryptoCCL
DECRYPT	CBC	AES	N/A	rba_CryptoHSM and rba_CryptoCCL
DECRYPT	CBC_NONE	AES	N/A	rba_CryptoHSM and rba_CryptoCCL
DECRYPT	CBC_PKCS7_V1_5	AES	N/A	rba_CryptoHSM and rba_CryptoCCL
DECRYPT	CBC_NIST_SP800_38A	AES	N/A	rba_CryptoHSM
DECRYPT	ECB	AES	N/A	rba_CryptoHSM
DECRYPT	EBC_NONE	AES	N/A	rba_CryptoHSM
DECRYPT	ECB_PKCS7_V1_5	AES	N/A	rba_CryptoHSM
DECRYPT	RS_AES_OAEP	RSA	N/A	rba_CryptoHSM
DECRYPT	ECB	SM4	N/A	rba_CryptoHSM
DECRYPT	ECB_NONE	SM4	N/A	rba_CryptoHSM
ENCRYPT	CBC	AES	N/A	rba_CryptoHSM and rba_CryptoCCL

ENCRYPT	CBC_NONE	AES	N/A	rba_CryptoHSM and rba_CryptoCCL
ENCRYPT	CBC_PKCS7_V1_5	AES	N/A	rba_CryptoHSM and rba_CryptoCCL
ENCRYPT	CBC_NIST_SP800_38A	AES	N/A	rba_CryptoHSM and rba_CryptoCCL
ENCRYPT	ECB	AES	N/A	rba_CryptoHSM
ENCRYPT	ECB_NONE	AES	N/A	rba_CryptoHSM
ENCRYPT	ECB_PKCS7_V1_5	AES	N/A	rba_CryptoHSM
ENCRYPT	RSAES_OAEP	RSA	N/A	rba_CryptoHSM
ENCRYPT	ECB	SM4	N/A	rba_CryptoHSM
ENCRYPT	ECB_NONE	SM4	N/A	rba_CryptoHSM
HASH	N/A	SHA2_512	N/A	rba_CryptoHSM and rba_CryptoCCL
HASH	N/A	SHA2_256	N/A	rba_CryptoHSM and rba_CryptoCCL
HASH	N/A	SHA2_384	N/A	rba_CryptoHSM and rba_CryptoCCL
HASH	N/A	SHA1	N/A	rba_CryptoHSM and rba_CryptoCCL
HASH	N/A	SM3	N/A	rba_CryptoHSM
HASH	N/A	SHA3_384	N/A	rba_CryptoHSM
HASH	N/A	SHA3_512	N/A	rba_CryptoHSM
KEYEXCHANGEALCPUBVAL	N/A	ECDH	EC_CURVE25519	rba_CryptoHSM and rba_CryptoCCL

KEYEXCHANGEALCPUBVAL	N/A	ECDH	EC_SECP256R1	rba_CryptoHSM and rba_CryptoCCL
KEYEXCHANGEALCPUBVAL	N/A	ECDH	EC_SECP384R1	rba_CryptoHSM and rba_CryptoCCL
KEYEXCHANGEALCPUBVAL	N/A	ECDH	EC_SECP521R1	rba_CryptoHSM and rba_CryptoCCL
KEYEXCHANGEALCPUBVAL	N/A	ECDH	EC_CURVE448	rba_CryptoHSM
KEYEXCHANGEALCPUBVAL	N/A	ECDH	EC_SECP224R1	rba_CryptoHSM
KEYEXCHANGEALCPUBVAL	N/A	ECDH	EC_SECP521R1	rba_CryptoCCL
KEYEXCHANGEALCPUBVAL	N/A	ECBD	EC_SECP224R1	rba_CryptoHSM
KEYEXCHANGEALCSECRET	N/A	ECDH	EC_CURVE25519	rba_CryptoHSM and rba_CryptoCCL
KEYEXCHANGEALCSECRET	N/A	ECDH	EC_SECP256R1	rba_CryptoHSM and rba_CryptoCCL
KEYEXCHANGEALCSECRET	N/A	ECDH	EC_SECP384R1	rba_CryptoHSM and rba_CryptoCCL
KEYEXCHANGEALCSECRET	N/A	ECDH	EC_SECP521R1	rba_CryptoHSM and rba_CryptoCCL
KEYEXCHANGEALCSECRET	N/A	ECDH	EC_CURVE448	rba_CryptoHSM
KEYEXCHANGEALCSECRET	N/A	ECDH	EC_SECP224R1	rba_CryptoHSM
KEYEXCHANGEALCSECRET	N/A	ECDH	EC_SECP521R1	rba_CryptoCCL
KEYEXCHANGEALCSECRET	N/A	ECBD	EC_SECP224R1	rba_CryptoHSM
KEYDERIVE	N/A	KDFX963	SHA2_512	rba_CryptoCCL
KEYDERIVE	N/A	HKDF	SHA256	rba_CryptoCCL
KEYDERIVE	N/A	KDFX963	EC_SECP224R1	rba_CryptoHSM

KEYDERIVE	CTR_CMAC	KDF_CMAC_NIST_SP800_108	RB_AES	rba_CryptoHSM
KEYGENERATE	N/A	N/A	N/A	rba_CryptoHSM and rba_CryptoCCL
KEYELEMENTGET	N/A	N/A	N/A	rba_CryptoHSM and rba_CryptoCCL
KEYELEMENTSET	N/A	N/A	N/A	rba_CryptoHSM and rba_CryptoCCL
KEYSETVALID	N/A	N/A	N/A	rba_CryptoHSM and rba_CryptoCCL
KEYSETINVALID	N/A	N/A	N/A	rba_CryptoHSM and rba_CryptoCCL
MACGENERATE	CMAC	AES	N/A	rba_CryptoHSM and rba_CryptoCCL
MACGENERATE	SIPHASH_2_4	SIPHASH	N/A	rba_CryptoHSM and rba_CryptoCCL
MACGENERATE	N/A	POLY1305	N/A	rba_CryptoCCL
MACGENERATE	HMAC	SHA1	N/A	rba_CryptoCCL
MACGENERATE	HMAC	SHA2_256	N/A	rba_CryptoHSM and rba_CryptoCCL
MACGENERATE	HMAC	SHA2_384	N/A	rba_CryptoHSM and rba_CryptoCCL
MACGENERATE	HMAC	SHA2_512	N/A	rba_CryptoHSM and rba_CryptoCCL
MACVERIFY	CMAC	AES	N/A	rba_CryptoHSM and rba_CryptoCCL

MACVERIFY	SIPHASH_2_4	SIPHASH	N/A	rba_CryptoHSM and rba_CryptoCCL
MACVERIFY	N/A	POLY1305	N/A	rba_CryptoCCL
MACVERIFY	HMAC	SHA1	N/A	rba_CryptoCCL
MACVERIFY	HMAC	SHA2_256	N/A	rba_CryptoHSM and rba_CryptoCCL
MACVERIFY	HMAC	SHA2_384	N/A	rba_CryptoHSM and rba_CryptoCCL
MACVERIFY	HMAC	SHA2_512	N/A	rba_CryptoHSM and rba_CryptoCCL
RANDOMGENERATE	N/A	RNG	N/A	rba_CryptoHSM
RANDOMSEED	N/A	RNG	N/A	rba_CryptoHSM
SIGNATUREGENERATE	PREHASHED	ED25519	SHA2_512	rba_CryptoHSM and rba_CryptoCCL
SIGNATUREGENERATE	PURE	ED25519	SHA2_512	rba_CryptoHSM and rba_CryptoCCL
SIGNATUREGENERATE	PREHASHED	ED448	SHAKE256	rba_CryptoHSM and rba_CryptoCCL
SIGNATUREGENERATE	PURE	ED448	SHAKE256	rba_CryptoHSM and rba_CryptoCCL
SIGNATUREGENERATE	N/A	ECDSA	SHA2_512	rba_CryptoHSM
SIGNATUREGENERATE	N/A	ECDSA	SHA2_384	rba_CryptoHSM
SIGNATUREGENERATE	N/A	ECDSA	SHA2_256	rba_CryptoHSM
SIGNATUREGENERATE	N/A	ECDSA_SECP384R1	SHA2_512	rba_CryptoHSM and rba_CryptoCCL

SIGNATUREGENERATE	N/A	ECDSA_SECP384R1	SHA2_384	rba_CryptoHSM and rba_CryptoCCL
SIGNATUREGENERATE	N/A	ECDSA_SECP256R1	SHA2_256	rba_CryptoHSM and rba_CryptoCCL
SIGNATUREGENERATE	N/A	ECDSA_SECP256R1	SHA2_512	rba_CryptoHSM and rba_CryptoCCL
SIGNATUREGENERATE	N/A	ECDSA_SECP521R1	SHA2_512	rba_CryptoHSM
SIGNATUREGENERATE	RSASSA_PSS	RSA	SHA2_256	rba_CryptoHSM and rba_CryptoCCL
SIGNATUREGENERATE	RSASSA_PKCS1_V1_5	RSA	SHA2_256	rba_CryptoCCL
SIGNATUREGENERATE	N/A	DETERMINISTIC_ECDSA_SECP384R1	SHA2_512	rba_CryptoCCL
SIGNATUREGENERATE	N/A	DETERMINISTIC_ECDSA_SECP521R1	SHA2_512	rba_CryptoCCL
SIGNATUREGENERATE	N/A	SM2	RB_SM3	rba_CryptoHSM
SIGNATUREVERIFY	PREHASHED	ED25519	SHA2_512	rba_CryptoHSM and rba_CryptoCCL
SIGNATUREVERIFY	PURE	ED25519	SHA2_512	rba_CryptoHSM and rba_CryptoCCL
SIGNATUREVERIFY	PREHASHED	ED448	SHAKE256	rba_CryptoHSM and rba_CryptoCCL
SIGNATUREVERIFY	PURE	ED448	SHAKE256	rba_CryptoHSM and rba_CryptoCCL
SIGNATUREVERIFY	N/A	ECDSA	SHA2_512	rba_CryptoHSM
SIGNATUREVERIFY	N/A	ECDSA	SHA2_384	rba_CryptoHSM
SIGNATUREVERIFY	N/A	ECDSA	SHA2_256	rba_CryptoHSM

SIGNATUREVERIFY	N/A	ECDSA_SECP384R1	SHA2_512	rba_CryptoHSM and rba_CryptoCCL
SIGNATUREVERIFY	N/A	ECDSA_SECP384R1	SHA2_384	rba_CryptoHSM and rba_CryptoCCL
SIGNATUREVERIFY	N/A	ECDSA_SECP256R1	SHA2_256	rba_CryptoHSM and rba_CryptoCCL
SIGNATUREVERIFY	N/A	ECDSA_SECP256R1	SHA2_512	rba_CryptoHSM and rba_CryptoCCL
SIGNATUREVERIFY	N/A	ECDSA_SECP521R1	SHA2_512	rba_CryptoHSM
SIGNATUREVERIFY	N/A	DETERMINISTIC_ECDSA	SHA2_512	rba_CryptoCCL
SIGNATUREVERIFY	N/A	DETERMINISTIC_ECDSA_SECP384R1	SHA2_512	rba_CryptoCCL
SIGNATUREVERIFY	N/A	DETERMINISTIC_ECDSA_SECP521R1	SHA2_512	rba_CryptoCCL
SIGNATUREVERIFY	RSASSA_PSS	RSA	SHA2_256	rba_CryptoHSM and rba_CryptoCCL
SIGNATUREVERIFY	RSASSA_PKCS1_V1_5	RSA	SHA2_256	rba_CryptoCCL
SIGNATUREVERIFY	N/A	SM2	RB_SM3	rba_CryptoHSM

1. Security algorithms to ISO15118-20.
2. Supports 284bit key length.

3.2.7 Key Management Services

Csm supports the following key management services:

Service	Crypto Driver Support
CertificateVerify (X509 v1)	rba_CryptoHSM and rba_CryptoCCL
CertificateParse (X509 v1)	rba_CryptoHSM and rba_CryptoCCL
KeyCopy	rba_CryptoHSM and rba_CryptoCCL
KeyDerive	rba_CryptoHSM and rba_CryptoCCL
KeyElementCopy	rba_CryptoHSM and rba_CryptoCCL
KeyElementGet	rba_CryptoHSM and rba_CryptoCCL
KeyElementIdsGet	rba_CryptoHSM and rba_CryptoCCL
KeyElementSet	rba_CryptoHSM and rba_CryptoCCL
KeyExchangeCalcPubVal	rba_CryptoHSM and rba_CryptoCCL
KeyExchangeCalcSecret	rba_CryptoHSM and rba_CryptoCCL
KeyGenerate	rba_CryptoHSM
KeySetValid	rba_CryptoHSM and rba_CryptoCCL
KeySetInvalid	rba_CryptoHSM and rba_CryptoCCL
SHE Key Operations	rba_CryptoHSM

- Support for KeyDerive with HMAC SHA2-384 (HKDF)
- Support for AES encryption/decryption in CBC mode with 256bit key length
- Support for KeyDerive with AES-CMAC (KDF in Counter mode NIST_SP800-108 and key length 128 as per IEEE 802.1X-2020)

3.2.8 SHE Keys Loading

Csm is extended beyond the AUTOSAR specification to support loading SHE keys, as described in Chapter 9 ("Memory Update protocol") from the SHE v1.1 specification. Basic SHE keys (up to RAM_KEY slot) can be loaded through the function *Csm_Rb_SheLoadKey*. Up to 90 SHE keys can be loaded, which is more than the keys defined in the SHE specification. This is achieved through the function: *Csm_Rb_SheLoadKeySlot*. All successful SHE key load operations will automatically set the corresponding Csm key state to valid.

The SHE LoadKey and SHE LoadKeySlot operations are also supported via the AUTOSAR defined *rba_CryptoHSM_KeyElementSet* synchronous interface.

3.2.9 SHE Get Identity

Csm is extended beyond the AUTOSAR specification to support the SHE Get

Identity function (for retrieving the SHE UID) as described in the SHE functional specification v1.1. This is achieved through the function: `Csm_Rb_SheGetId`

3.2.10 SHE Export Ram Key

Csm is extended beyond the AUTOSAR specification to support the export of the RAM key into a SHE-wrapped key format protected by the secret key, as described in the SHE functional specification v1.1. This is achieved through the function: `Csm_Rb_SheExportRamKey`

The SHE Export Ram Key operation is also supported via the AUTOSAR defined `rba_CryptoHSM_KeyElementGet` synchronous interface.

3.2.11 SHE Plain Keys Loading

Csm is extended beyond the AUTOSAR specification to support loading plaintext keys to the RAM_KEY slot, as described in the SHE specification v1.1. This is achieved through the function: `Csm_Rb_SheLoadPlainKey`

The SHE Load Plain Key Loading operation is also supported via the AUTOSAR defined `rba_CryptoHSM_KeyElementSet` synchronous interface.

3.2.12 External Callout Support

External callout support is available for the `RandomGenerate` and `RandomSeed` operations. See the Configuration Advice for more details.

3.2.13 VKMS Functionalities

Csm does not provide dedicated function calls to access VKMS functionalities within `rba_CryptoHSM`, so a specific configuration must be done from Csm to `rba_CryptoHSM`. See the Configuration Advice section for more details.

3.2.14 MainFunction Partitioning

RTA-SEC supports MainFunction partitioning of resources. For Csm, this involves the generation of multiple `Csm_MainFunction` instances and mapping of `CsmQueues` to these instances where they should be handled. See the Configuration Advice for more details.

3.2.15 Job Context Interface

The Job Context Interface allows saving/restoring the workspace's context data for a specific service from the Crypto driver. It is only supported in CCL (please see Section 7.2.4 for more details).

3.3 Extensions

3.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

3.3.1.1 Csm/CsmGeneral/CsmRbAutoFillConfig

Short Name	CsmRbAutoFillConfig
Description	If this parameter is set to True, then Csm generator try to automatically fill missing configuration parameters and connection in Csm, Crylf and Crypto components.
Multiplicity	1..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.2 Csm/CsmGeneral/CsmRbMainFunctionPeriod

Short Name	CsmRbMainFunctionPeriod
Description	Specifies the period of main function Csm_MainFunction in seconds (applicable only for the default MainFunction instance).
Multiplicity	0..1
Value Range	0..INF
Default Value	0.01
Type	ECUC-FLOAT-PARAM-DEF

3.3.1.3 Csm/CsmGeneral/CsmRbSupportAR4_4_0

Short Name	CsmRbSupportAR4_4_0
Description	If this parameter is set to True, then type definitions are extended to support AR4.4.0 functionality. This adds Crypto_JobInfoType to Crypto_JobType, resultLength to Crypto_PrimitiveInfoType and input64 and output64Ptr to Crypto_JobPrimitive InputOutputType. The removed Std_ReturnType extension values CRYPTO_E_QUEUE_FULL and CRYPTO_E_SMALL_BUFFER are added.
Introduction	True: AR4.4.0 crypto driver support enabled. False: AR4.4.0 crypto driver support disabled.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.4 Csm/CsmJobs/CsmJob/CsmRbJobRteQLength

Short Name	CsmRbJobRteQLength
------------	--------------------

Description	This parameter is the depth of the q-length, the history buffer, provided by the Rte. Usage might be needed if: Rte is used (AND) CsmJobUsePort == TRUE (AND) CsmJobPrimitiveRef is referencing an operation of a Client-Server-Interface e.g. MacGenerate (AND) Port provision and request are asynchronous and running on different OsApplications e.g. 1ms 10ms (AND) a history of all values is needed on the requesting side.
Multiplicity	0..1
Value Range	0..4294967295
Default Value	1
Type	ECUC-INTEGER-PARAM-DEF

3.3.1.5 Csm/CsmKeys/CsmKey/CsmRbAutoConfigCryptoNvBlockRef

Short Name	CsmRbAutoConfigCryptoNvBlockRef
Description	Reference to the Crypto NV block where the persistent key elements of this key shall be stored to.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

3.3.1.6 Csm/CsmKeys/CsmKey/CsmRbAutoConfigCryptoSelect

Short Name	CsmRbAutoConfigCryptoSelect
Description	When automatic configuration completion used for the key, then this parameter needs to be filled out to select the crypto target for the key.
Multiplicity	0..1
Default Value	-
Enumeration Literals	rba_CryptoCCL, rba_CryptoHSM
Type	ECUC-ENUMERATION-PARAM-DEF

3.3.1.7 Csm/CsmKeys/CsmKey/CsmRbAutoConfigHsmKeySlotId

Short Name	CsmRbAutoConfigHsmKeySlotId
------------	-----------------------------

Description	This parameter explicitly maps the secure cryptokey element into a specified key slot inside the HSM secure flash. If left empty, then an available key slot automatically assigned to it. The name (e.g. DYNAMIC_CFG_CRYPT_AES_KEY_BASE_0) of the key slot must be provided. This autoconfiguration parameter will only apply to key element which are stored in HSM. Note: When this parameter is configured with a SHE key slot ID, the CryptoKeyElementFormat will be set to CRYPTO_KE_FORMAT_BIN_SHEKEYS for the key elements with CryptoKeyElementId = 1 and CryptoKeyElementId = 2, which it is expected that they are configured via the CsmRbAutoConfigKeyElement parameter.
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

3.3.1.8 Csm/CsmKeys/CsmKey/CsmRbAutoConfigKeyElement

Short Name	CsmRbAutoConfigKeyElement
Description	When automatic configuration completion is used for the key, then this parameter needs to be filled out to define keyElements. The configuration needs to be defined in the following format: [elementId]_[P/p(Persistent/Volatile)][(RIEIC)/(rieIc)((Read allowed Encrypted Internal copy) / Read denied)][(WIEIC)/(wieIc)((Write allowed Encrypted Internal copy) / Write denied)][P/p(Partial access allowed / Partial access denied)]_[Size in bytes]_(optional) [Initvalue as hex string] Example: 1_PRWp_16_0011 or 1_PRWp_16 (KeyElementId=1, persistent stored with read/write access, partial access denied, Length=16bytes, (optional) Init Value 0x00, 0x11)
Multiplicity	0..*
Default Value	-
Type	ECUC-STRING-PARAM-DEF

3.3.1.9 Csm/CsmPrimitives/CsmAEADDecrypt/CsmAEADDecryptConfig/CsmRbAEADDecryptTagMaxLength

Short Name	CsmRbAEADDecryptTagMaxLength
Description	Max size of the input Tag length in BITS

Multiplicity	0..1
Value Range	1..4294967295
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

3.3.1.10 Csm/CsmPrimitives/CsmJobKeyElementGet

Short Name	CsmJobKeyElementGet
Description	Configurations of KeyElementGet primitives
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

3.3.1.11 Csm/CsmPrimitives/CsmJobKeyElementGet/CsmJobKeyElementGetConfig

Short Name	CsmJobKeyElementGetConfig
Description	Container for configuration of a CSM key element get operation. The container name serves as a symbolic name for the identifier of a key configuration.
Multiplicity	1..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

3.3.1.12 Csm/CsmPrimitives/CsmJobKeyElementGet/CsmJobKeyElementGetConfig/CsmJobKeyElementGetAlgorithmFamily

Short Name	CsmJobKeyElementGetAlgorithmFamily
Description	Determines the algorithm family used for the crypto service. This parameter defines the most significant part of the algorithm.
Multiplicity	1..1
Default Value	-
Enumeration Literals	CRYPTO_ALGOFAM_CUSTOM, CRYPTO_ALGOFAM_NOT_SET
Type	ECUC-ENUMERATION-PARAM-DEF

3.3.1.13 Csm/CsmPrimitives/CsmJobKeyElementGet/CsmJobKeyElementGetConfig/CsmJobKeyElementGetAlgorithmFamilyCustomRef

Short Name	CsmJobKeyElementGetAlgorithmFamilyCustomRef
Description	Reference to a customer specific algorithm family custom container.

Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

3.3.1.14 Csm/CsmPrimitives/CsmJobKeyElementGet/CsmJobKeyElement-GetConfig/CsmJobKeyElementGetAlgorithmMode

Short Name	CsmJobKeyElementGetAlgorithmMode
Description	Determines the algorithm mode used for the crypto service. This parameter defines the most significant part of the algorithm.
Multiplicity	1..1
Default Value	-
Enumeration Literals	CRYPTO_ALGOMODE_CUSTOM, CRYPTO_ALGOMODE_NOT_SET
Type	ECUC-ENUMERATION-PARAM-DEF

3.3.1.15 Csm/CsmPrimitives/CsmJobKeyElementGet/CsmJobKeyElement-GetConfig/CsmJobKeyElementGetAlgorithmModeCustomRef

Short Name	CsmJobKeyElementGetAlgorithmModeCustomRef
Description	Reference to a customer specific algorithm mode custom container.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

3.3.1.16 Csm/CsmPrimitives/CsmJobKeyElementGet/CsmJobKeyElement-GetConfig/CsmJobKeyElementGetAlgorithmSecondaryFamily

Short Name	CsmJobKeyElementGetAlgorithmSecondaryFamily
Description	Determines the algorithm family used for the crypto service. This parameter defines the most significant part of the algorithm.
Multiplicity	1..1
Default Value	-
Enumeration Literals	CRYPTO_ALGOFAM_CUSTOM, CRYPTO_ALGOFAM_NOT_SET
Type	ECUC-ENUMERATION-PARAM-DEF

3.3.1.17 Csm/CsmPrimitives/CsmJobKeyElementGet/CsmJobKeyElement-GetConfig/CsmJobKeyElementGetAlgorithmSecondaryFamilyCustomRef

Short Name	CsmJobKeyElementGetAlgorithmSecondaryFamilyCustomRef
------------	--

Description	Reference to a customer specific algorithm family container in the Crypto Driver.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

3.3.1.18 Csm/CsmPrimitives/CsmJobKeyElementSet

Short Name	CsmJobKeyElementSet
Description	Configurations of KeyElementSet primitives
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

3.3.1.19 Csm/CsmPrimitives/CsmJobKeyElementSet/CsmJobKeyElementSetConfig

Short Name	CsmJobKeyElementSetConfig
Description	Container for configuration of a CSM key element set operation. The container name serves as a symbolic name for the identifier of a key configuration.
Multiplicity	1..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

3.3.1.20 Csm/CsmPrimitives/CsmJobKeyElementSet/CsmJobKeyElementSetConfig/CsmJobKeyElementSetAlgorithmFamily

Short Name	CsmJobKeyElementSetAlgorithmFamily
Description	Determines the algorithm family used for the crypto service. This parameter defines the most significant part of the algorithm.
Multiplicity	1..1
Default Value	-
Enumeration Literals	CRYPTO_ALGOFAM_CUSTOM, CRYPTO_ALGOFAM_NOT_SET
Type	ECUC-ENUMERATION-PARAM-DEF

3.3.1.21 Csm/CsmPrimitives/CsmJobKeyElementSet/CsmJobKeyElementSetConfig/CsmJobKeyElementSetAlgorithmFamilyCustomRef

Short Name	CsmJobKeyElementSetAlgorithmFamilyCustomRef
Description	Reference to a customer specific algorithm family custom container.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

3.3.1.22 Csm/CsmPrimitives/CsmJobKeyElementSet/CsmJobKeyElementSetConfig/CsmJobKeyElementSetAlgorithmMode

Short Name	CsmJobKeyElementSetAlgorithmMode
Description	Determines the algorithm mode used for the crypto service. This parameter defines the most significant part of the algorithm.
Multiplicity	1..1
Default Value	-
Enumeration Literals	CRYPTO_ALGOMODE_CUSTOM, CRYPTO_ALGOMODE_NOT_SET
Type	ECUC-ENUMERATION-PARAM-DEF

3.3.1.23 Csm/CsmPrimitives/CsmJobKeyElementSet/CsmJobKeyElementSetConfig/CsmJobKeyElementSetAlgorithmModeCustomRef

Short Name	CsmJobKeyElementSetAlgorithmModeCustomRef
Description	Reference to a customer specific algorithm mode custom container.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

3.3.1.24 Csm/CsmPrimitives/CsmJobKeyElementSet/CsmJobKeyElementSetConfig/CsmJobKeyElementSetAlgorithmSecondaryFamily

Short Name	CsmJobKeyElementSetAlgorithmSecondaryFamily
Description	Determines the algorithm family used for the crypto service. This parameter defines the most significant part of the algorithm.
Multiplicity	1..1
Default Value	-
Enumeration Literals	CRYPTO_ALGOFAM_CUSTOM, CRYPTO_ALGOFAM_NOT_SET

Type	ECUC-ENUMERATION-PARAM-DEF
------	----------------------------

3.3.1.25 Csm/CsmPrimitives/CsmJobKeyElementSet/CsmJobKeyElementSetConfig/CsmJobKeyElementSetAlgorithmSecondaryFamilyCustomRef

Short Name	CsmJobKeyElementSetAlgorithmSecondaryFamilyCustomRef
Description	Reference to a customer specific algorithm family container in the Crypto Driver.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

3.3.1.26 Csm/CsmPrimitives/CsmMacGenerate/CsmMacGenerateConfig/CsmRbAutoConfigBulkProcessingModeEnabled

Short Name	CsmRbAutoConfigBulkProcessingModeEnabled
Description	When automatic configuration completion is used for a Mac primitive, then this parameter needs to be filled out to switch between Mac behaviors.
Introduction	True: Enables the bulk processing mode for any mac generation. False: Disables the bulk processing mode for any mac generation (FALSE = default).
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.27 Csm/CsmPrimitives/CsmMacVerify/CsmMacVerifyConfig/CsmRbAutoConfigBulkProcessingModeEnabled

Short Name	CsmRbAutoConfigBulkProcessingModeEnabled
Description	When automatic configuration completion is used for a Mac primitive, then this parameter needs to be filled out to switch between Mac behaviors.
Introduction	True: Enables the bulk processing mode for any mac verification. False: Disables the bulk processing mode for any mac verification (FALSE = default).
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.28 Csm/CsmPrimitives/CsmRbAutoConfigCryptoSelect

Short Name	CsmRbAutoConfigCryptoSelect
------------	-----------------------------

Description	When automatic configuration completion used for a job-primitive pair, then this parameter needs to be filled out to select the crypto target for the key.
Multiplicity	0..1
Default Value	-
Enumeration Literals	rba_CryptoCCL, rba_CryptoHSM
Type	ECUC-ENUMERATION-PARAM-DEF

3.3.1.29 Csm/CsmPrimitives/CsmRbSheExportRamKey

Short Name	CsmRbSheExportRamKey
Description	Configuration of She Export Ram Key primitives
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

3.3.1.30 Csm/CsmPrimitives/CsmRbSheGetID

Short Name	CsmRbSheGetID
Description	Configuration of She Get ID primitives
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

3.3.1.31 Csm/CsmPrimitives/CsmRbSheLoadKey

Short Name	CsmRbSheLoadKey
Description	Configuration of She Load Key primitives - which is applicable only for basic SHE keys (up to RAM_KEY slot)
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

3.3.1.32 Csm/CsmPrimitives/CsmRbSheLoadKeySlot

Short Name	CsmRbSheLoadKeySlot
Description	Configuration of She Load Key Slot primitives - for updating up to 50 general purpose SHE key slots
Multiplicity	0..1
Default Value	-

Type	ECUC-PARAM-CONF-CONTAINER-DEF
------	-------------------------------

3.3.1.33 Csm/CsmPrimitives/CsmRbSheLoadPlainKey

Short Name	CsmRbSheLoadPlainKey
Description	Configuration of She Load Plain Key primitives
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

3.3.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

3.3.2.1 Csm_CertificateParse

Syntax	Std_ReturnType Csm_CertificateParse(uint32 keyId)
Parameter In	KeyId: The identifier of the key to be used for the certificate parsing.
Return Value	E_OK: Request successful.
	E_NOT_OK: Request Failed.
	CRYPTO_E_BUSY: Request Failed, Crypto Driver Object is Busy.
	CRYPTO_E_KEY_NOT_VALID: Request Failed, the key's state is "invalid".
	CRYPTO_E_KEY_EMPTY: Request failed because of uninitialized source key element.
Description	Dispatches the certificate parse function to Crylf.

3.3.2.2 Csm_CertificateVerify

Syntax	Std_ReturnType Csm_CertificateVerify(uint32 keyId, uint32 verifyKeyId, Crypto_VerifyResultType* verifyPtr)
Parameters In	KeyId: The identifier of the key which shall be used to validate the certificate.
	verifyKeyId: The identifier of the key containing the certificate to be verified.
Parameter Out	verifyPtr: Pointer to the memory location which will contain the result of the certificate verification.
Return Value	E_OK: Request successful.
	E_NOT_OK: Request Failed.
	CRYPTO_E_BUSY: Request Failed, Crypto Driver Object is Busy.
	CRYPTO_E_KEY_NOT_VALID: Request Failed, the key's state is "invalid".

	CRYPTO_E_KEY_EMPTY: Request failed because of uninitialized source key element.
Description	Verifies the certificate stored in the key referenced by verifyKeyId with the certificate stored in the key referenced by KeyId.

3.3.2.3 Csm_JobKeyElementGet

Syntax	Std_ReturnType Csm_JobKeyElementGet(uint32 jobId, uint32 keyId, uint32 keyElementId, uint8* keyPtr, uint32* keyLengthPtr)
Parameters In	jobId: The identifier of the job using the CSM service.
	keyId: The identifier of the key from which a key element shall be extracted.
	keyElementId: The identifier of the key element to be extracted.
Parameter Out	keyPtr: Pointer to the memory location where the key shall be copied to.
Parameter InOut	keyLengthPtr: Pointer to the memory location in which the output buffer length in bytes is stored.
Return Value	E_OK: Request successful.
	E_NOT_OK: Request Failed.
	CRYPTO_E_BUSY: Request Failed, Crypto Driver Object is Busy.
	CRYPTO_E_KEY_NOT_AVAILABLE: Request failed, the requested key element is not available.
	CRYPTO_E_KEY_READ_FAIL: Request failed because read access was denied.
	CRYPTO_E_KEY_EMPTY: Request failed because of uninitialized source key element.
Description	Retrieves key element bytes from specific key element of key identified by keyId and stores key element in memory location pointed by key pointer.

3.3.2.4 Csm_JobKeyElementSet

Syntax	Std_ReturnType Csm_JobKeyElementSet(uint32 jobId, uint32 keyId, uint32 keyElementId, const uint8* keyPtr, uint32 keyLength)
Parameters In	jobId: The identifier of the job using the CSM service.
	keyId: The identifier of the key for which a new material shall be set.
	keyElementId: The identifier of the key element to be written.
	keyPtr: Pointer to the key element bytes to be processed.
	keyLength: The number of key element bytes.
Return Value	E_OK: Request successful.
	E_NOT_OK: Request Failed.
	CRYPTO_E_BUSY: Request Failed, Crypto Driver Object is Busy.

	CRYPTO_E_KEY_NOT_AVAILABLE: Request failed because the key is not available.
	CRYPTO_E_KEY_WRITE_FAIL: Request failed because write access was denied.
	CRYPTO_E_KEY_SIZE_MISMATCH: Request failed, key element size does not match size of provided data.
Description	Sets the given key element bytes to the key identified by keyId.

3.3.2.5 Csm_KeyGetStatus

Syntax	Std_ReturnType Csm_KeyGetStatus (uint32 keyId, Crypto_KeyStatusType* keyStatusPtr)
Parameter In	keyId: The identifier of the key for which the key state shall be returned.
Parameter Out	keyStatusPtr: The pointer to the data where the status of the key shall be stored.
Return Value	E_OK: Request successful.
	E_NOT_OK: Request Failed.
Description	Returns the key state of the key identified by keyId.

3.3.2.6 Csm_Rb_Deinit

Syntax	Csm_Rb_Deinit()
Parameters	NONE
Return Value	NONE
Description	Disables main function processing and prevents further API calls until the module is reinitialised.

3.3.2.7 Csm_Rb_StorePermanentData

Syntax	Csm_Rb_StorePermanentData()
Parameters	NONE
Return Value	E_OK: Request successful.
	E_NOT_OK: Request Failed.
	CRYPTO_E_PENDING: Request in progress.
Description	Function to trigger permanent data store and poll its status. All permanent data will be saved. On first call the store operation is automatically started, on subsequent calls the status is polled.

3.3.2.8 Csm_Rb_SheLoadKeySlot

Syntax	Csm_Rb_SheLoadKeySlot(uint32 jobId, const uint8* KeySlotM1M2M3Ptr, uint32 KeySlotM1M2M3Length, uint8* M4M5Ptr, uint32* M4M5LengthPtr)
Parameters	(IN) <i>jobId</i> Holds the identifier of the job using the CSM service.
	(IN) <i>KeySlotM1M2M3Ptr</i> Pointer to common input buffer for all input parameters for this API: - KeySlot - Key slot number - M1 - UIDIIDIAuthID - M2 - ENC_CBC,K_1,IV=0(C_IDIF_IDI"0...0"95IK_ID) - M3 - CMAC_K_2(M_1IM_2)
	(IN) <i>KeySlotM1M2M3Length</i> Length of the common input buffer containing KeySlot, M1,M2,M3 in bytes
	(OUT) <i>M4M5Ptr</i> Pointer to common output buffer for all output parameters for this API: - M4 - UIDIIDIAuthIDIM_4 - M5 - CMAC_K_4(M_4)
	(INOUT) <i>M4M5LengthPtr</i> Holds a pointer to the memory location in which the length in bytes of the output buffer M4M5Ptr is stored. On calling this function, this parameter shall contain the size of the buffer provided by M4M5Ptr. When the request has finished, the actual length of the returned M4,M5 blocks shall be stored.
Return Value	E_NOT_OK: Request Failed.
	CRYPTO_E_QUEUE_FULL: request failed, the queue is full
	RBA_CRYPTOE_SHE_NO_ERROR: (value same as E_OK): request successful
	RBA_CRYPTOE_SHE_BUSY: (value same as CRYPTO_E_BUSY) request failed, service is still busy
	RBA_CRYPTOE_SHE_GENERAL_ERROR
	RBA_CRYPTOE_SHE_KEY_EMPTY
	RBA_CRYPTOE_SHE_KEY_INVALID
	RBA_CRYPTOE_SHE_KEY_NOT_AVAILABLE
	RBA_CRYPTOE_SHE_KEY_UPDATE_ERROR
	RBA_CRYPTOE_SHE_KEY_WRITE_PROTECTED
	RBA_CRYPTOE_SHE_MEMORY_FAILURE
	RBA_CRYPTOE_SHE_SEQUENCE_ERROR
	CRYPTO_E_SMALL_BUFFER: the provided buffer is too small to store the result

Description	The function updates an internal key of SHE with the protocol described in Chapter 9 - Memory Update protocol from the SHE specification v1.1. This interface can be used to update, apart from the basic SHE key set, an additional 90 general purpose SHE key slots (total range: from 0x1 to 0x69). In addition to the generic M1,M2,M3 blocks expected for key loading, this interface expects 1 extra byte in the input buffer which represents the key slot number provided by the user in order to indicate the actual memory slot in HSM to be updated (the KeyID field from M1 alone does not provide sufficient information). If the RAM_KEY slot is updated via this interface, the user must ensure that the plain key status flag in the encrypted block M2 is reset. If this function is called while another SHE key management operation is in progress, the SHE error code CRYPTOE_BUSY will be set in the return value. In asynchronous operation SHE_ERC_BUSY never returned to the user since it is internally polled and the user only receives a callback when operation is completed. When more SHE jobs started parallel then the CryptoStack automatically detects the collision and reports RYPTOE_BUSY. All input parameters are checked for NULL_PTR and correct size at crypto driver level. Errors are reported via DET reporting interface. This primitive supports only sync/async SINGLECALL job execution.
-------------	---

3.3.2.9 Csm_Rb_SheLoadKey

Syntax	<pre>Csm_Rb_SheLoadKey(uint32 jobId, const uint8* KeySlotM1M2M3Ptr, uint32 KeySlotM1M2M3Length, uint8* M4M5Ptr, uint32* M4M5LengthPtr)</pre>
Parameters	(IN) <i>jobId</i> Holds the identifier of the job using the CSM service.
	(IN) <i>KeySlotM1M2M3Ptr</i> Pointer to common input buffer for all input parameters for this API: - KeySlot - Key slot number - M1 - UIDIIDIAuthID - M2 - ENC_CBC,K_1,IV=0(C_IDIF_IDI*0...0*95IK_ID) - M3 - CMAC_K_2(M_1IM_2)
	(IN) <i>KeySlotM1M2M3Length</i> Length of the common input buffer containing KeySlot, M1,M2,M3 in bytes
	(OUT) <i>M4M5Ptr</i> Pointer to common output buffer for all output parameters for this API: - M4 - UIDIIDIAuthIDIM_4 - M5 - CMAC_K_4(M_4)
	(INOUT) <i>M4M5LengthPtr</i> Holds a pointer to the memory location in which the length in bytes of the output buffer M4M5Ptr is stored. On calling this function, this parameter shall contain the size of the buffer provided by M4M5Ptr. When the request has finished, the actual length of the returned M4,M5 blocks shall be stored.

Return Value	E_NOT_OK: Request Failed.
	CRYPTO_E_QUEUE_FULL: request failed, the queue is full
	RBA_CRYPTOE_SHE_NO_ERROR: (value same as E_OK): request successful
	RBA_CRYPTOE_SHE_BUSY: (value same as CRYPTO_E_BUSY) request failed, service is still busy
	RBA_CRYPTOE_SHE_GENERAL_ERROR
	RBA_CRYPTOE_SHE_KEY_EMPTY
	RBA_CRYPTOE_SHE_KEY_INVALID
	RBA_CRYPTOE_SHE_KEY_NOT_AVAILABLE
	RBA_CRYPTOE_SHE_KEY_UPDATE_ERROR
	RBA_CRYPTOE_SHE_KEY_WRITE_PROTECTED
	RBA_CRYPTOE_SHE_MEMORY_FAILURE
	RBA_CRYPTOE_SHE_SEQUENCE_ERROR
	CRYPTO_E_SMALL_BUFFER: the provided buffer is too small to store the result
Description	The function updates an internal key of SHE with the protocol described in Chapter 9 - Memory Update protocol from the SHE specification v1.1. This interface can be used to update only the first basic set of SHE key slots (from 0x1 to 0xE). If the RAM_KEY slot is updated via this interface, the user must ensure that the plain key status flag in the encrypted block M2 is reset. If this function is called while another SHE key management operation is in progress, the SHE error code CRYPTO_E_BUSY will be set in the return value. In asynchronous operation SHE_ERC_BUSY never returned to the user since it is internally polled and the user only receives a callback when operation is completed. When more SHE jobs started parallel then the CryptoStack automatically detects the collision and reports CRYPTO_E_BUSY. All input parameters are checked for NULL_PTR and correct size at crypto driver level. Errors are reported via DET reporting interface. This primitive supports only sync/async SINGLECALL job execution.

3.3.2.10 Csm_Rb_SheGetId

Syntax	Csm_Rb_SheGetId(uint32 jobId, const uint8* challengePtr, uint32 challengeLength, uint8* uidInfoPtr, uint32* uidInfoLengthPtr)
Parameters	(IN) <i>jobId</i> Holds the identifier of the job using the CSM service.
	(IN) <i>challengePtr</i> Pointer to the challenge data buffer.
	(IN) <i>challengeLength</i> Length of the challenge in bytes.

	(OUT) <i>uidInfoPtr</i> Pointer to common output buffer for all output parameters for this API as defined by SHE: - SHE UID - Fifteen byte buffer to receive the SHE module UID - SREG - One byte containing the value of the SHE status register - MAC - Sixteen bytes containing the CMAC of the challenge, UID and status value
	(INOUT) <i>uidInfoLengthPtr</i> Holds a pointer to the memory location in which the length in bytes of the output buffer <i>uidInfoPtr</i> is stored. On calling this function, this parameter shall contain the size of the buffer provided by <i>uidInfoPtr</i> . When the request has finished, the actual length of the returned UID info shall be stored.
Return Value	E_NOT_OK: Request Failed.
	CRYPTO_E_QUEUE_FULL: request failed, the queue is full
	RBA_CRYPTOE_SHE_NO_ERROR: (value same as E_OK): request successful
	RBA_CRYPTOE_SHE_BUSY: (value same as CRYPTO_E_BUSY) request failed, service is still busy
	RBA_CRYPTOE_SHE_GENERAL_ERROR
	RBA_CRYPTOE_SHE_KEY_NOT_AVAILABLE
	RBA_CRYPTOE_SHE_MEMORY_FAILURE
	RBA_CRYPTOE_SHE_SEQUENCE_ERROR
Description	This function returns the identity (UID) and the value of the status register protected by a MAC over a challenge and the data. All output values from the SHE specification are grouped into a single output parameter named <i>uidInfoPtr</i> . If MASTER_ECU_KEY is empty, then the returned MAC is always all zeroes. If this function is called while another SHE key management operation is in progress, the SHE error code CRYPTO_E_BUSY will be set in the return value. In asynchronous operation SHE_ERC_BUSY never returned to the user since it's internally polled, and user receives only callback when operation is done. When more SHE jobs started parallel then the Cryptostack automatically detect the collision and reports CRYPTO_E_BUSY. All input parameters are checked for NULL_PTR and correct size. Errors are reported via DET reporting interface. This primitive supports only sync/async SINGLECALL job execution.

3.3.2.11 Csm_Rb_SheExportRamKey

Syntax	Csm_Rb_SheExportRamKey(uint32 jobId, uint8* M1M2M3M4M5Ptr, uint32* M1M2M3M4M5LengthPtr)
Parameters	(IN) <i>jobId</i> Holds the identifier of the job using the CSM service.

	(OUT) <i>M1M2M3M4M5Ptr</i> Pointer to common output buffer for all output parameters for this API: - M1 - UIDIIDIAuthID - M2 - ENC_CBC,K_1,IV=0(C_IDIF_IDI"0...0"95IK_ID) - M3 - CMAC_K_2(M_1M_2) - M4 - UIDIIDIAuthIDIM_4 - M5 - CMAC_K_4(M_4)
	(INOUT) <i>M1M2M3M4M5LengthPtr</i> Holds a pointer to the memory location in which the length in bytes of the output buffer <i>M1M2M3M4M5Ptr</i> is stored. On calling this function, this parameter shall contain the size of the buffer provided by <i>M1M2M3M4M5Ptr</i> . When the request has finished, the actual length of the returned M1,M2,M3,M4,M5 blocks shall be stored.
Return Value	E_NOT_OK: Request Failed.
	CRYPTO_E_QUEUE_FULL: request failed, the queue is full
	RBA_CRYPTOE_SHE_NO_ERROR: (value same as E_OK): request successful
	RBA_CRYPTOE_SHE_BUSY: (value same as CRYPTO_E_BUSY) request failed, service is still busy
	RBA_CRYPTOE_SHE_GENERAL_ERROR
	RBA_CRYPTOE_SHE_KEY_EMPTY
	RBA_CRYPTOE_SHE_KEY_INVALID
	RBA_CRYPTOE_SHE_MEMORY_FAILURE
	RBA_CRYPTOE_SHE_SEQUENCE_ERROR
Description	The function exports the RAM key into a format protected by the secret key. The key can be imported again by using the SHE CMD_LOAD_KEY protocol. The RAM key can only be exported if it was previously written into SHE in plaintext using the CMD_LOAD_PLAIN_KEY protocol. If this function is called while another SHE key management operation is in progress, the SHE error code CRYPTO_E_BUSY will be set in the return value. In asynchronous operation SHE_ERC_BUSY never returned to the user since its internally polled, and user receives only callback when operation is done. When more SHE jobs started parallel then the Cryptostack automatically detect the collision and reports CRYPTO_E_BUSY. All input parameters are checked for NULL_PTR and correct size. Errors are reported via DET reporting interface. This primitive supports only sync/async SINGLECALL job execution.

3.3.2.12 Csm_Rb_SheLoadPlainKey

Syntax	Csm_Rb_SheLoadPlainKey(uint32 jobId, const uint8* KeyDataPtr, uint32 KeyDataLength)
Parameters	(IN) <i>jobId</i> Holds the identifier of the job using the CSM service.

	(IN) <i>KeyDataPtr</i> Pointer to common input buffer which must contain the 16-byte plaintext key data.
	(IN) <i>KeyDataLength</i> Length of the common input buffer containing the 16-byte plaintext key data.
Return Value	E_NOT_OK: Request Failed.
	CRYPTO_E_QUEUE_FULL: request failed, the queue is full
	RBA_CRYPTOE_SHE_NO_ERROR: (value same as E_OK): request successful
	RBA_CRYPTOE_SHE_BUSY: (value same as CRYPTO_E_BUSY) request failed, service is still busy
	RBA_CRYPTOE_SHE_GENERAL_ERROR
	RBA_CRYPTOE_SHE_KEY_EMPTY
	RBA_CRYPTOE_SHE_KEY_INVALID
	RBA_CRYPTOE_SHE_KEY_NOT_AVAILABLE
	RBA_CRYPTOE_SHE_KEY_UPDATE_ERROR
	RBA_CRYPTOE_SHE_KEY_WRITE_PROTECTED
	RBA_CRYPTOE_SHE_MEMORY_FAILURE
	RBA_CRYPTOE_SHE_SEQUENCE_ERROR
Description	The function loads a plaintext key into the RAM_KEY slot without encryption and verification, as described in SHE specification v1.1. If this function is called while another SHE key management operation is in progress, the SHE error code CRYPTO_E_BUSY will be set in the return value. In asynchronous operation SHE_ERC_BUSY never returned to the user since its internally polled, and user receives only callback when operation is done. When more SHE jobs started parallel then the Cryptostack automatically detect the collision and reports CRYPTO_E_BUSY. All input parameters are checked for NULL_PTR and correct size. Errors are reported via DET reporting interface. This primitive supports only sync/async SINGLECALL job execution.

3.3.3 Other Extensions

The API functions `Csm_KeyExchangeCalcPubVal` and `Csm_JobKeyExchangeCalcPubVal` have been extended in this release. The calculated public key value calculated is stored in the predefined `CRYPTO_KE_KEYEXCHANGE_OWN_PUBKEY` keyElement as well as in the buffer defined by the calling process.

3.4 Limitations

3.4.1 Random Generator

The `CRYPTO_E_ENTROPY_EXHAUSTED` return value is not implemented because `rba_CryptoCCL` doesn't support it and `rba_CryptoHSM` uses TRNG.

3.4.2 Single Call Operation

As Csm has been designed with the 'single call' model in mind (for better performance), there are no separate API functions for Start, Update or Finish operations in any of the cryptographic services that it supports.

These operations are deprecated in AUTOSAR and have been removed from the AR specification.

3.4.3 CsmQueueSize

AUTOSAR defines a minimum `csmQueueSize` of 1. However, if you only want queueing in Crypto, or if you want to save RAM/ROM resources, a queue would still be generated for every channel in Csm, so a `csmQueueSize` of 0 is allowed. This deactivates async job queueing for a given channel in Csm so that all async jobs are forwarded to `Crylf_ProcessJob` without queueing.

We strongly recommend that you use async job queueing only in Crypto components, and that you disable all Csm queueing (`CsmQueueSize = 0`) to minimize Cryptostack runtime. Csm queues always have less efficient job handling compared to Crypto queues..

3.4.4 Cancel Job

AUTOSAR defines that queued jobs should be removed from the queue synchronously, but the internal queueing implementation in RTA-BSW does not allow for this. If a job is asynchronous and is active, running or queued, then `Csm_CancelJob` will always cancel it asynchronously.

3.4.5 KeyExchangeCalcPubVal and KeyExchangeCalcSecret

The `KeyExchangeCalcPubVal` and `KeyExchangeCalcSecret` services (in HSM and CCL) are additional functions of the `KeyManagement` interface. These two services run only synchronously or asynchronously in Singlecall mode and do not support the streaming approach.

3.4.6 Csm_KeyCopy, Csm_KeyElementCopy and Csm_KeyElement-CopyPartial between Cryptos

Static buffers reserved for operations between crypto key copy operations are created based on the largest `keyElements` size found in the project configuration. If these features are not used, or if a smaller buffer size is enough, then its size can be controlled via `CrylfRbInterCryptoCopyKeyElementMaxSize`.

If a key or `keyElement` copy operation between Crypto components is performed, all those operations must be called sequentially because reentrancy is not supported for this kind of operation.

3.4.7 RSA Signature Keys

Due to the size of RSA keys, the available space in RAM could not be enough to host 2 RSA signature keys. This could cause a problem when injecting the keys into RAM memory.

3.5 Exclusions

The following API functions from AR v22-11 are not supported in version v12.7.0.

- Csm_SaveContextJob
- Csm_RestoreContextJob
- Csm_CustomService

The following configuration parameters are not supported in version v12.7.0.

- Csm/CsmPrimitives/CsmAEADDecrypt/CsmAEADDecryptConfig/CsmAEADDecryptAlgorithmFamilyCustomRef
- Csm/CsmPrimitives/CsmAEADDecrypt/CsmAEADDecryptConfig/CsmAEADDecryptAlgorithmSecondaryFamilyCustomRef
- Csm/CsmPrimitives/CsmAEADDecrypt/CsmAEADDecryptConfig/CsmAEADDecryptKeyRef
- Csm/CsmPrimitives/CsmAEADEncrypt/CsmAEADEncryptConfig/CsmAEADEncryptAlgorithmFamilyCustomRef
- Csm/CsmPrimitives/CsmAEADEncrypt/CsmAEADEncryptConfig/CsmAEADEncryptAlgorithmSecondaryFamilyCustomRef
- Csm/CsmPrimitives/CsmAEADEncrypt/CsmAEADEncryptConfig/CsmAEADEncryptKeyRef
- Csm/CsmPrimitives/CsmCustom/CsmCustomConfig
- Csm/CsmPrimitives/CsmCustom/CsmCustomConfig/CsmCustomAlgorithmFamily
- Csm/CsmPrimitives/CsmCustom/CsmCustomConfig/CsmCustomAlgorithmFamilyCustomRef
- Csm/CsmPrimitives/CsmCustom/CsmCustomConfig/CsmCustomAlgorithmMode
- Csm/CsmPrimitives/CsmCustom/CsmCustomConfig/CsmCustomAlgorithmModeCustomRef
- Csm/CsmPrimitives/CsmCustom/CsmCustomConfig/CsmCustomAlgorithmSecondaryFamily
- Csm/CsmPrimitives/CsmCustom/CsmCustomConfig/CsmCustomAlgorithmSecondaryFamilyCustomRef
- Csm/CsmPrimitives/CsmCustom/CsmCustomConfig/CsmCustomInputMaxLength
- Csm/CsmPrimitives/CsmCustom/CsmCustomConfig/CsmCustomOutputMaxLength
- Csm/CsmPrimitives/CsmCustom/CsmCustomConfig/CsmCustomSecondaryInputMaxLength
- Csm/CsmPrimitives/CsmCustom/CsmCustomConfig/CsmCustomSecondaryOutputMaxLength

- Csm/CsmPrimitives/CsmCustom/CsmCustomConfig/CsmCustomTargetKeyMaxLength
- Csm/CsmPrimitives/CsmCustom/CsmCustomConfig/CsmCustomTertiaryInputMaxLength
- Csm/CsmPrimitives/CsmDecrypt/CsmDecryptConfig/CsmDecryptAlgorithmFamilyCustomRef
- Csm/CsmPrimitives/CsmDecrypt/CsmDecryptConfig/CsmDecryptAlgorithmSecondaryFamily
- Csm/CsmPrimitives/CsmDecrypt/CsmDecryptConfig/CsmDecryptAlgorithmSecondaryFamilyCustomRef
- Csm/CsmPrimitives/CsmEncrypt/CsmEncryptConfig/CsmEncryptAlgorithmFamilyCustomRef
- Csm/CsmPrimitives/CsmEncrypt/CsmEncryptConfig/CsmEncryptAlgorithmSecondaryFamily
- Csm/CsmPrimitives/CsmEncrypt/CsmEncryptConfig/CsmEncryptAlgorithmSecondaryFamilyCustomRef
- Csm/CsmPrimitives/CsmHash/CsmHashConfig/CsmHashAlgorithmFamilyCustomRef
- Csm/CsmPrimitives/CsmHash/CsmHashConfig/CsmHashAlgorithmMode
- Csm/CsmPrimitives/CsmHash/CsmHashConfig/CsmHashAlgorithmModeCustomRef
- Csm/CsmPrimitives/CsmHash/CsmHashConfig/CsmHashAlgorithmSecondaryFamily
- Csm/CsmPrimitives/CsmHash/CsmHashConfig/CsmHashAlgorithmSecondaryFamilyCustomRef
- Csm/CsmPrimitives/CsmJobKeyDerive/CsmJobKeyDeriveConfig
- Csm/CsmPrimitives/CsmJobKeyDerive/CsmJobKeyDeriveConfig/CsmJobKeyDeriveAlgorithmFamily
- Csm/CsmPrimitives/CsmJobKeyDerive/CsmJobKeyDeriveConfig/CsmJobKeyDeriveAlgorithmFamilyCustomRef
- Csm/CsmPrimitives/CsmJobKeyDerive/CsmJobKeyDeriveConfig/CsmJobKeyDeriveAlgorithmMode
- Csm/CsmPrimitives/CsmJobKeyDerive/CsmJobKeyDeriveConfig/CsmJobKeyDeriveAlgorithmModeCustomRef
- Csm/CsmPrimitives/CsmJobKeyDerive/CsmJobKeyDeriveConfig/CsmJobKeyDeriveAlgorithmSecondaryFamily
- Csm/CsmPrimitives/CsmJobKeyDerive/CsmJobKeyDeriveConfig/CsmJobKeyDeriveAlgorithmSecondaryFamilyCustomRef
- Csm/CsmPrimitives/CsmJobKeyExchangeCalcPubVal/CsmJobKeyExchangeCalcPubValConfig/CsmJobKeyExchangeCalcPubValAlgorithmFamilyCustomRef

- Csm/CsmPrimitives/CsmJobKeyExchangeCalcPubVal/CsmJobKeyExchangeCalcPubValConfig/CsmJobKeyExchangeCalcPubValAlgorithmModeCustomRef
- Csm/CsmPrimitives/CsmJobKeyExchangeCalcSecret/CsmJobKeyExchangeCalcSecretConfig/CsmJobKeyExchangeCalcSecretAlgorithmFamilyCustomRef
- Csm/CsmPrimitives/CsmJobKeyExchangeCalcSecret/CsmJobKeyExchangeCalcSecretConfig/CsmJobKeyExchangeCalcSecretAlgorithmModeCustomRef
- Csm/CsmPrimitives/CsmJobKeyGenerate/CsmJobKeyGenerateConfig
- Csm/CsmPrimitives/CsmJobKeyGenerate/CsmJobKeyGenerateConfig/CsmJobKeyGenerateAlgorithmFamily
- Csm/CsmPrimitives/CsmJobKeyGenerate/CsmJobKeyGenerateConfig/CsmJobKeyGenerateAlgorithmFamilyCustomRef
- Csm/CsmPrimitives/CsmJobKeyGenerate/CsmJobKeyGenerateConfig/CsmJobKeyGenerateAlgorithmMode
- Csm/CsmPrimitives/CsmJobKeyGenerate/CsmJobKeyGenerateConfig/CsmJobKeyGenerateAlgorithmModeCustomRef
- Csm/CsmPrimitives/CsmJobKeyGenerate/CsmJobKeyGenerateConfig/CsmJobKeyGenerateAlgorithmSecondaryFamily
- Csm/CsmPrimitives/CsmJobKeyGenerate/CsmJobKeyGenerateConfig/CsmJobKeyGenerateAlgorithmSecondaryFamilyCustomRef
- Csm/CsmPrimitives/CsmJobKeySetInvalid/CsmJobKeySetInvalidConfig
- Csm/CsmPrimitives/CsmJobKeySetInvalid/CsmJobKeySetInvalidConfig/CsmJobKeySetInvalidAlgorithmFamily
- Csm/CsmPrimitives/CsmJobKeySetInvalid/CsmJobKeySetInvalidConfig/CsmJobKeySetInvalidAlgorithmFamilyCustomRef
- Csm/CsmPrimitives/CsmJobKeySetInvalid/CsmJobKeySetInvalidConfig/CsmJobKeySetInvalidAlgorithmMode
- Csm/CsmPrimitives/CsmJobKeySetInvalid/CsmJobKeySetInvalidConfig/CsmJobKeySetInvalidAlgorithmModeCustomRef
- Csm/CsmPrimitives/CsmJobKeySetInvalid/CsmJobKeySetInvalidConfig/CsmJobKeySetInvalidAlgorithmSecondaryFamily
- Csm/CsmPrimitives/CsmJobKeySetInvalid/CsmJobKeySetInvalidConfig/CsmJobKeySetInvalidAlgorithmSecondaryFamilyCustomRef
- Csm/CsmPrimitives/CsmJobKeySetValid/CsmJobKeySetValidConfig
- Csm/CsmPrimitives/CsmJobKeySetValid/CsmJobKeySetValidConfig/CsmJobKeySetValidAlgorithmFamily
- Csm/CsmPrimitives/CsmJobKeySetValid/CsmJobKeySetValidConfig/CsmJobKeySetValidAlgorithmFamilyCustomRef
- Csm/CsmPrimitives/CsmJobKeySetValid/CsmJobKeySetValidConfig

- CsmJobKeySetValidAlgorithmMode
- Csm/CsmPrimitives/CsmJobKeySetValid/CsmJobKeySetValidConfig/
CsmJobKeySetValidAlgorithmModeCustomRef
- Csm/CsmPrimitives/CsmJobKeySetValid/CsmJobKeySetValidConfig/
CsmJobKeySetValidAlgorithmSecondaryFamily
- Csm/CsmPrimitives/CsmJobKeySetValid/CsmJobKeySetValidConfig/
CsmJobKeySetValidAlgorithmSecondaryFamilyCustomRef
- Csm/CsmPrimitives/CsmJobRandomSeed/CsmJobRandomSeedConfig
- Csm/CsmPrimitives/CsmJobRandomSeed/CsmJobRandomSeedConfig/
CsmJobRandomSeedAlgorithmFamily
- Csm/CsmPrimitives/CsmJobRandomSeed/CsmJobRandomSeedConfig/
CsmJobRandomSeedAlgorithmFamilyCustomRef
- Csm/CsmPrimitives/CsmJobRandomSeed/CsmJobRandomSeedConfig/
CsmJobRandomSeedAlgorithmMode
- Csm/CsmPrimitives/CsmJobRandomSeed/CsmJobRandomSeedConfig/
CsmJobRandomSeedAlgorithmModeCustomRef
- Csm/CsmPrimitives/CsmJobRandomSeed/CsmJobRandomSeedConfig/
CsmJobRandomSeedAlgorithmSecondaryFamily
- Csm/CsmPrimitives/CsmJobRandomSeed/CsmJobRandomSeedConfig/
CsmJobRandomSeedAlgorithmSecondaryFamilyCustomRef
- Csm/CsmPrimitives/CsmMacGenerate/CsmMacGenerateConfig/CsmMac-
GenerateAlgorithmFamilyCustomRef
- Csm/CsmPrimitives/CsmMacGenerate/CsmMacGenerateConfig/CsmMac-
GenerateAlgorithmModeCustomRef
- Csm/CsmPrimitives/CsmMacGenerate/CsmMacGenerateConfig/CsmMac-
GenerateAlgorithmSecondaryFamilyCustomRef
- Csm/CsmPrimitives/CsmMacVerify/CsmMacVerifyConfig/CsmMacVerifyAl-
gorithmFamilyCustomRef
- Csm/CsmPrimitives/CsmMacVerify/CsmMacVerifyConfig/CsmMacVerifyAl-
gorithmModeCustomRef
- Csm/CsmPrimitives/CsmMacVerify/CsmMacVerifyConfig/CsmMacVerifyAl-
gorithmSecondaryFamilyCustomRef
- Csm/CsmPrimitives/CsmRandomGenerate/CsmRandomGenerateConfig/
CsmRandomGenerateAlgorithmFamilyCustomRef
- Csm/CsmPrimitives/CsmRandomGenerate/CsmRandomGenerateConfig/
CsmRandomGenerateAlgorithmModeCustomRef
- Csm/CsmPrimitives/CsmRandomGenerate/CsmRandomGenerateConfig/
CsmRandomGenerateAlgorithmSecondaryFamilyCustomRef
- Csm/CsmPrimitives/CsmSignatureGenerate/CsmSignatureGenerateConfig/
CsmSignatureGenerateAlgorithmSecondaryFamilyCustomRef
- Csm/CsmPrimitives/CsmSignatureVerify/CsmSignatureVerifyConfig/

CsmSignatureVerifyAlgorithmSecondaryFamilyCustomRef

3.6 Deviations

3.6.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

3.6.1.1 Csm/CsmCallbacks/CsmCallback/CsmCallbackFunc

[RTA-BSW] Description	Callback function to be called if an asynchronous operation has finished. The corresponding job has to be configured to be processed asynchronously. (If the linked CsmJob has the parameter CsmJobInterfaceUsePort equal to CRYPTO_USE_PORT_OPTIMIZED, then this parameter will not be used and the callback is handled through an Rte Client Server Port.)
[AUTOSAR] Description	Callback function to be called if an asynchronous operation has finished. The corresponding job has to be configured to be processed asynchronously.

3.6.1.2 Csm/CsmJobs/CsmJob/CsmJobId

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

3.6.1.3 Csm/CsmJobs/CsmJob/CsmJobKeyRef

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

3.6.1.4 Csm/CsmJobs/CsmJob/CsmJobQueueRef

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

3.6.1.5 Csm/CsmJobs/CsmJob/CsmJobServiceInterfaceContextUsePort

[REMOVED] Multiplicity	0..1
[RTA-BSW] Enumeration Literals	CRYPTO_USE_FNC, CRYPTO_USE_PORT
[AUTOSAR] Enumeration Literals	CRYPTO_USE_FUNC, CRYPTO_USE_PORT
[AUTOSAR] Default Value	CRYPTO_USE_FUNC
[RTA-BSW] Default Value	CRYPTO_USE_FNC

3.6.1.6 Csm/CsmKeys/CsmKey/CsmKeyId

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

3.6.1.7 Csm/CsmKeys/CsmKey/CsmKeyRef

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

3.6.1.8 Csm/CsmMainFunction

[RTA-BSW] Description	Each element of this container defines one instance of Csm_MainFunction. For each partition, where the Csm module shall be instantiated, at least one MainFunction instance needs to be configured.
[AUTOSAR] Description	Each element of this container defines one instance of Csm_MainFunction.
[REMOVED] Introduction	For each partition, where the Csm module shall be instantiated, at least one MainFunction instance needs to be configured.

3.6.1.9 Csm/CsmMainFunction/CsmMainFunctionPeriod

[RTA-BSW] Description	Specifies the period of the respective main function Csm_MainFunction instance in seconds.
[AUTOSAR] Description	Specifies the period of main function Csm_MainFunction in seconds.

3.6.1.10 Csm/CsmPrimitives/CsmAEADDecrypt/CsmAEADDecryptConfig/CsmAEADDecryptAlgorithmModeCustomRef

[RTA-BSW] Description	Reference to a customer specific algorithm mode custom container.
[AUTOSAR] Description	Reference to a customer specific algorithm mode custom container

3.6.1.11 Csm/CsmPrimitives/CsmAEADDecrypt/CsmAEADDecryptConfig/CsmAEADDecryptAssociatedDataMaxLength

[RTA-BSW] Description	Max size of the input associated data length in bytes
-----------------------	---

[AUTOSAR] Description	Size of the input associated data buffer for the synchronous RTE service interface if this primitive is referenced by a Csm job. It may also be possible that other BSW modules use the length parameter for buffer size calculation.
-----------------------	---

3.6.1.12 Csm/CsmPrimitives/CsmAEADDecrypt/CsmAEADDecryptConfig/CsmAEADDecryptCiphertextMaxLength

[RTA-BSW] Description	Max size of the input ciphertext in bytes
[AUTOSAR] Description	Size of the input ciphertext buffer for the synchronous RTE service interface if this primitive is referenced by a Csm job. It may also be possible that other BSW modules use the length parameter for buffer size calculation.

3.6.1.13 Csm/CsmPrimitives/CsmAEADDecrypt/CsmAEADDecryptConfig/CsmAEADDecryptPlaintextMaxLength

[RTA-BSW] Description	Size of the output plaintext length in bytes
[AUTOSAR] Description	Size of the output plaintext buffer for the synchronous RTE service interface if this primitive is referenced by a Csm job. It may also be possible that other BSW modules use the length parameter for buffer size calculation.

3.6.1.14 Csm/CsmPrimitives/CsmAEADDecrypt/CsmAEADDecryptConfig/CsmAEADDecryptTagLength

[RTA-BSW] Description	Size of the input Tag length in BITS
[AUTOSAR] Description	Size of the input tag buffer in BITS for the synchronous RTE service interface if this primitive is referenced by a Csm job. It may also be possible that other BSW modules use the length parameter for buffer size calculation."
[RTA-BSW] Lower Multiplicity	1
[AUTOSAR] Lower Multiplicity	0

3.6.1.15 Csm/CsmPrimitives/CsmAEADEncrypt/CsmAEADEncryptConfig/CsmAEADEncryptAlgorithmModeCustomRef

[RTA-BSW] Description	Reference to a customer specific algorithm mode custom container.
-----------------------	---

[AUTOSAR] Description	Reference to a customer specific algorithm mode custom container
-----------------------	--

3.6.1.16 Csm/CsmPrimitives/CsmAEADEncrypt/CsmAEADEncryptConfig/CsmAEADEncryptAssociatedDataMaxLength

[RTA-BSW] Description	Max size of the input associated data length in bytes
[AUTOSAR] Description	Size of the input associated data buffer for the synchronous RTE service interface if this primitive is referenced by a Csm job. It may also be possible that other BSW modules use the length parameter for buffer size calculation.

3.6.1.17 Csm/CsmPrimitives/CsmAEADEncrypt/CsmAEADEncryptConfig/CsmAEADEncryptCiphertextMaxLength

[RTA-BSW] Description	Max size of the output ciphertext length in bytes
[AUTOSAR] Description	Size of the output ciphertext buffer for the synchronous RTE service interface if this primitive is referenced by a Csm job. It may also be possible that other BSW modules use the length parameter for buffer size calculation.

3.6.1.18 Csm/CsmPrimitives/CsmAEADEncrypt/CsmAEADEncryptConfig/CsmAEADEncryptPlaintextMaxLength

[RTA-BSW] Description	Max size of the input plaintext length in bytes
[AUTOSAR] Description	Size of the input plaintext buffer for the synchronous RTE service interface if this primitive is referenced by a Csm job. It may also be possible that other BSW modules use the length parameter for buffer size calculation

3.6.1.19 Csm/CsmPrimitives/CsmAEADEncrypt/CsmAEADEncryptConfig/CsmAEADEncryptTagLength

[RTA-BSW] Description	Size of the output Tag length in bytes
[AUTOSAR] Description	Size of the output tag length buffer for the synchronous RTE service interface if this primitive is referenced by a Csm job. It may also be possible that other BSW modules use the length parameter for buffer size calculation.
[RTA-BSW] Lower Multiplicity	1

[AUTOSAR] Lower Multiplicity	0
------------------------------	---

3.6.1.20 Csm/CsmPrimitives/CsmDecrypt/CsmDecryptConfig/CsmDecryptAlgorithmMode

[RTA-BSW] Enumeration Literals	CRYPTO_ALGOMODE_12ROUNDS, CRYPTO_ALGOMODE_20ROUNDS, CRYPTO_ALGOMODE_8ROUNDS, CRYPTO_ALGOMODE_AESKEYWRAP, CRYPTO_ALGOMODE_CBC, CRYPTO_ALGOMODE_CFB, CRYPTO_ALGOMODE_CTR, CRYPTO_ALGOMODE_CUSTOM, CRYPTO_ALGOMODE_ECB, CRYPTO_ALGOMODE_OFB, CRYPTO_ALGOMODE_RSAES_OAEP, CRYPTO_ALGOMODE_RSAES_PKCS1_v1_5, CRYPTO_ALGOMODE_XTS
[AUTOSAR] Enumeration Literals	CRYPTO_ALGOMODE_12ROUNDS, CRYPTO_ALGOMODE_20ROUNDS, CRYPTO_ALGOMODE_8ROUNDS, CRYPTO_ALGOMODE_CBC, CRYPTO_ALGOMODE_CFB, CRYPTO_ALGOMODE_CTR, CRYPTO_ALGOMODE_CUSTOM, CRYPTO_ALGOMODE_ECB, CRYPTO_ALGOMODE_OFB, CRYPTO_ALGOMODE_RSAES_OAEP, CRYPTO_ALGOMODE_RSAES_PKCS1_v1_5, CRYPTO_ALGOMODE_XTS

3.6.1.21 Csm/CsmPrimitives/CsmDecrypt/CsmDecryptConfig/CsmDecryptAlgorithmModeCustomRef

[RTA-BSW] Description	Reference to a customer specific algorithm mode custom container.
[AUTOSAR] Description	Reference to a customer specific algorithm mode custom container

3.6.1.22 Csm/CsmPrimitives/CsmDecrypt/CsmDecryptConfig/CsmDecryptDataMaxLength

[RTA-BSW] Description	Max size of the input ciphertext length in bytes
[AUTOSAR] Description	Size of the input ciphertext buffer for the synchronous RTE service interface if this primitive is referenced by a Csm job. It may also be possible that other BSW modules use the length parameter for buffer size calculation.

3.6.1.23 Csm/CsmPrimitives/CsmDecrypt/CsmDecryptConfig/CsmDecryptResultMaxLength

[RTA-BSW] Description	Max size of the output plaintext length in bytes
-----------------------	--

[AUTOSAR] Description	Size of the result data buffer for the synchronous RTE service interface if this primitive is referenced by a Csm job. It may also be possible that other BSW modules use the length parameter for buffer size calculation.
-----------------------	---

3.6.1.24 Csm/CsmPrimitives/CsmEncrypt/CsmEncryptConfig/CsmEncryptAlgorithmMode

[RTA-BSW] Enumeration Literals	CRYPTO_ALGOMODE_12ROUNDS, CRYPTO_ALGOMODE_20ROUNDS, CRYPTO_ALGOMODE_8ROUNDS, CRYPTO_ALGOMODE_AESKEYWRAP, CRYPTO_ALGOMODE_CBC, CRYPTO_ALGOMODE_CFB, CRYPTO_ALGOMODE_CTR, CRYPTO_ALGOMODE_CUSTOM, CRYPTO_ALGOMODE_ECB, CRYPTO_ALGOMODE_NOT_SET, CRYPTO_ALGOMODE_OFB, CRYPTO_ALGOMODE_RSAES_OAEP, CRYPTO_ALGOMODE_RSAES_PKCS1_v1_5, CRYPTO_ALGOMODE_XTS
[AUTOSAR] Enumeration Literals	CRYPTO_ALGOMODE_12ROUNDS, CRYPTO_ALGOMODE_20ROUNDS, CRYPTO_ALGOMODE_8ROUNDS, CRYPTO_ALGOMODE_CBC, CRYPTO_ALGOMODE_CFB, CRYPTO_ALGOMODE_CTR, CRYPTO_ALGOMODE_CUSTOM, CRYPTO_ALGOMODE_ECB, CRYPTO_ALGOMODE_NOT_SET, CRYPTO_ALGOMODE_OFB, CRYPTO_ALGOMODE_RSAES_OAEP, CRYPTO_ALGOMODE_RSAES_PKCS1_v1_5, CRYPTO_ALGOMODE_XTS

3.6.1.25 Csm/CsmPrimitives/CsmEncrypt/CsmEncryptConfig/CsmEncryptAlgorithmModeCustomRef

[RTA-BSW] Description	Reference to a customer specific algorithm mode custom container.
[AUTOSAR] Description	Reference to a customer specific algorithm mode custom container

3.6.1.26 Csm/CsmPrimitives/CsmEncrypt/CsmEncryptConfig/CsmEncryptDataMaxLength

[RTA-BSW] Description	Max size of the input plaintext length in bytes
[AUTOSAR] Description	Size of the input plaintext buffer for the synchronous RTE service interface if this primitive is referenced by a Csm job. It may also be possible that other BSW modules use the length parameter for buffer size calculation.

3.6.1.27 Csm/CsmPrimitives/CsmEncrypt/CsmEncryptConfig/CsmEncryptResultMaxLength

[RTA-BSW] Description	Max size of the output cipher length in bytes
-----------------------	---

[AUTOSAR] Description	Size of the result data buffer for the synchronous RTE service interface if this primitive is referenced by a Csm job. It may also be possible that other BSW modules use the length parameter for buffer size calculation.
-----------------------	---

3.6.1.28 Csm/CsmPrimitives/CsmHash/CsmHashConfig/CsmHashData-MaxLength

[RTA-BSW] Description	Max size of the input data length in bytes
[AUTOSAR] Description	Size of the input data buffer for the synchronous RTE service interface if this primitive is referenced by a Csm job. It may also be possible that other BSW modules use the length parameter for buffer size calculation.

3.6.1.29 Csm/CsmPrimitives/CsmHash/CsmHashConfig/CsmHashResultLength

[RTA-BSW] Description	Size of the output hash length in bytes. Used only in Crypto_HSM if output buffering feature is enabled.
[AUTOSAR] Description	Size of the result data buffer for the synchronous RTE service interface if this primitive is referenced by a Csm job. It may also be possible that other BSW modules use the length parameter for buffer size calculation.
[RTA-BSW] Lower Multiplicity	1
[AUTOSAR] Lower Multiplicity	0

3.6.1.30 Csm/CsmPrimitives/CsmJobKeyExchangeCalcPubVal/CsmJobKeyExchangeCalcPubValConfig

[RTA-BSW] Description	Container for configuration of a CSM JobKeyExchangeCalcPubVal. The container name serves as a symbolic name for the identifier of JobKeyExchangeCalcPubVal interface.
[AUTOSAR] Description	Container for configuration of a CSM JobKeyExchangeCalcPubVal. The container name serves as a symbolic name for the identifier of a key configuration.

3.6.1.31 Csm/CsmPrimitives/CsmJobKeyExchangeCalcPubVal/CsmJobKeyExchangeCalcPubValConfig/CsmJobKeyExchangeCalcPubValAlgorithmSecondaryFamily

[ADDED] Default Value	CRYPTO_ALGOFAM_NOT_SET
-----------------------	------------------------

3.6.1.32 Csm/CsmPrimitives/CsmJobKeyExchangeCalcPubVal/ CsmJobKeyExchangeCalcPubValConfig/CsmJobKeyExchangeCal- cPubValAlgorithmSecondaryFamilyCustomRef

[RTA-BSW] Description	Reference to a customer specific algorithm family container in the Crypto Driver.
[AUTOSAR] Description	Reference to a customer specific algorithm family container in the Crypto Driver

3.6.1.33 Csm/CsmPrimitives/CsmJobKeyExchangeCalcSecret/ CsmJobKeyExchangeCalcSecretConfig/CsmJobKeyExchange- CalcSecretAlgorithmSecondaryFamilyCustomRef

[RTA-BSW] Description	Reference to a customer specific algorithm family container in the Crypto Driver.
[AUTOSAR] Description	Reference to a customer specific algorithm family container in the Crypto Driver

3.6.1.34 Csm/CsmPrimitives/CsmMacGenerate/CsmMacGenerateConfig/ CsmMacGenerateDataMaxLength

[RTA-BSW] Description	Max size of the input data length in bytes
[AUTOSAR] Description	Size of the input data buffer for the synchronous RTE service interface if this primitive is referenced by a Csm job. It may also be possible that other BSW modules use the length parameter for buffer size calculation.

3.6.1.35 Csm/CsmPrimitives/CsmMacGenerate/CsmMacGenerateConfig/ CsmMacGenerateResultLength

[RTA-BSW] Description	Size of the output MAC length in bytes. Used only in Crypto_HSM if output buffering feature is enabled.
[AUTOSAR] Description	Size of the result data buffer for the synchronous RTE service interface if this primitive is referenced by a Csm job. It may also be possible that other BSW modules use the length parameter for buffer size calculation.
[RTA-BSW] Lower Multiplicity	1
[AUTOSAR] Lower Multiplicity	0

3.6.1.36 Csm/CsmPrimitives/CsmMacVerify/CsmMacVerifyConfig/CsmMacVerifyAlgorithmFamily

[RTA-BSW] Enumeration Literals: CRYPTO_ALGOFAM_3DES, CRYPTO_ALGOFAM_AES, CRYPTO_ALGOFAM_BLAKE_1_256, CRYPTO_ALGOFAM_BLAKE_1_512, CRYPTO_ALGOFAM_BLAKE_2s_256, CRYPTO_ALGOFAM_BLAKE_2s_512, CRYPTO_ALGOFAM_CHACHA, CRYPTO_ALGOFAM_CUSTOM, CRYPTO_ALGOFAM_EEA3, CRYPTO_ALGOFAM_EIA3, CRYPTO_ALGOFAM_POLY1305, CRYPTO_ALGOFAM_RIPEMD160, CRYPTO_ALGOFAM_RNG, CRYPTO_ALGOFAM_SHA1, CRYPTO_ALGOFAM_SHA2_224, CRYPTO_ALGOFAM_SHA2_256, CRYPTO_ALGOFAM_SHA2_384, CRYPTO_ALGOFAM_SHA2_512, CRYPTO_ALGOFAM_SHA2_512_224, CRYPTO_ALGOFAM_SHA2_512_256, CRYPTO_ALGOFAM_SHA3_224, CRYPTO_ALGOFAM_SHA3_256, CRYPTO_ALGOFAM_SHA3_384, CRYPTO_ALGOFAM_SHA3_512, CRYPTO_ALGOFAM_SHAKE128, CRYPTO_ALGOFAM_SHAKE256, CRYPTO_ALGOFAM_SIPHASH, CRYPTO_ALGOFAM_SM3

[AUTOSAR] Enumeration Literals: CRYPTO_ALGOFAM_3DES, CRYPTO_ALGOFAM_AES, CRYPTO_ALGOFAM_BLAKE_1_256, CRYPTO_ALGOFAM_BLAKE_1_512, CRYPTO_ALGOFAM_BLAKE_2s_256, CRYPTO_ALGOFAM_BLAKE_2s_512, CRYPTO_ALGOFAM_CHACHA, CRYPTO_ALGOFAM_EEA3, CRYPTO_ALGOFAM_EIA3, CRYPTO_ALGOFAM_POLY1305, CRYPTO_ALGOFAM_RIPEMD160, CRYPTO_ALGOFAM_RNG, CRYPTO_ALGOFAM_SHA1, CRYPTO_ALGOFAM_SHA2_224, CRYPTO_ALGOFAM_SHA2_256, CRYPTO_ALGOFAM_SHA2_384, CRYPTO_ALGOFAM_SHA2_512, CRYPTO_ALGOFAM_SHA2_512_224, CRYPTO_ALGOFAM_SHA2_512_256, CRYPTO_ALGOFAM_SHA3_224, CRYPTO_ALGOFAM_SHA3_256, CRYPTO_ALGOFAM_SHA3_384, CRYPTO_ALGOFAM_SHA3_512, CRYPTO_ALGOFAM_SHAKE128, CRYPTO_ALGOFAM_SHAKE256, CRYPTO_ALGOFAM_SIPHASH, CRYPTO_ALGOFAM_SM3, CRYPTO_ALGOMODE_CUSTOM

3.6.1.37 Csm/CsmPrimitives/CsmMacVerify/CsmMacVerifyConfig/CsmMacVerifyCompareLength

[RTA-BSW] Description	Size of the input MAC length, that shall be verified, in BITS
[AUTOSAR] Description	Size of the input MAC buffer in BITS for the synchronous RTE service interface if this primitive is referenced by a Csm job. It may also be possible that other BSW modules use the length parameter for buffer size calculation.
[RTA-BSW] Lower Multiplicity	1
[AUTOSAR] Lower Multiplicity	0

3.6.1.38 Csm/CsmPrimitives/CsmMacVerify/CsmMacVerifyConfig/CsmMacVerifyDataMaxLength

[RTA-BSW] Description	Max size of the input data length, for whichs MAC shall be verified, in bytes
[AUTOSAR] Description	Size of the input data buffer for the synchronous RTE service interface if this primitive is referenced by a Csm job. It may also be possible that other BSW modules use the length parameter for buffer size calculation.

3.6.1.39 Csm/CsmPrimitives/CsmRandomGenerate/CsmRandomGenerateConfig/CsmRandomGenerateAlgorithmFamily

[RTA-BSW] Enumeration Literals: CRYPTO_ALGOFAM_3DES, CRYPTO_ALGOFAM_AES, CRYPTO_ALGOFAM_BLAKE_1_256, CRYPTO_ALGOFAM_BLAKE_1_512, CRYPTO_ALGOFAM_BLAKE_2s_256, CRYPTO_ALGOFAM_BLAKE_2s_512, CRYPTO_ALGOFAM_CHACHA, CRYPTO_ALGOFAM_CUSTOM, CRYPTO_ALGOFAM_DRBG, CRYPTO_ALGOFAM_RIPEMD160, CRYPTO_ALGOFAM_RNG, CRYPTO_ALGOFAM_SHA1, CRYPTO_ALGOFAM_SHA2_224, CRYPTO_ALGOFAM_SHA2_256, CRYPTO_ALGOFAM_SHA2_384, CRYPTO_ALGOFAM_SHA2_512, CRYPTO_ALGOFAM_SHA2_512_224, CRYPTO_ALGOFAM_SHA2_512_256, CRYPTO_ALGOFAM_SHA3_224, CRYPTO_ALGOFAM_SHA3_256, CRYPTO_ALGOFAM_SHA3_384, CRYPTO_ALGOFAM_SHA3_512, CRYPTO_ALGOFAM_SHAKE128, CRYPTO_ALGOFAM_SHAKE256, CRYPTO_ALGOFAM_SM3

[AUTOSAR] Enumeration Literals: CRYPTO_ALGOFAM_3DES, CRYPTO_ALGOFAM_AES, CRYPTO_ALGOFAM_BLAKE_1_256, CRYPTO_ALGOFAM_BLAKE_1_512, CRYPTO_ALGOFAM_BLAKE_2s_256, CRYPTO_ALGOFAM_BLAKE_2s_512, CRYPTO_ALGOFAM_CHACHA, CRYPTO_ALGOFAM_CUSTOM, CRYPTO_ALGOFAM_EEA3, CRYPTO_ALGOFAM_RIPEMD160, CRYPTO_ALGOFAM_RNG, CRYPTO_ALGOFAM_SHA1, CRYPTO_ALGOFAM_SHA2_224, CRYPTO_ALGOFAM_SHA2_256, CRYPTO_ALGOFAM_SHA2_384, CRYPTO_ALGOFAM_SHA2_512, CRYPTO_ALGOFAM_SHA2_512_224, CRYPTO_ALGOFAM_SHA2_512_256, CRYPTO_ALGOFAM_SHA3_224, CRYPTO_ALGOFAM_SHA3_256, CRYPTO_ALGOFAM_SHA3_384, CRYPTO_ALGOFAM_SHA3_512, CRYPTO_ALGOFAM_SHAKE128, CRYPTO_ALGOFAM_SHAKE256, CRYPTO_ALGOFAM_SM3

3.6.1.40 Csm/CsmPrimitives/CsmRandomGenerate/CsmRandomGenerateConfig/CsmRandomGenerateAlgorithmMode

[RTA-BSW] Enumeration Literals	CRYPTO_ALGOMODE_CMAC, CRYPTO_ALGOMODE_CTRDRBG, CRYPTO_ALGOMODE_CUSTOM, CRYPTO_ALGOMODE_GMAC, CRYPTO_ALGOMODE_HMAC, CRYPTO_ALGOMODE_NOT_SET, CRYPTO_ALGOMODE_SIPHASH_2_4, CRYPTO_ALGOMODE_SIPHASH_4_8
--------------------------------	--

[AUTOSAR] Enumeration Literals	CRYPTO_ALGOMODE_CTRDRBG, CRYPTO_ALGOMODE_CUSTOM, CRYPTO_ALGOMODE_GMAC, CRYPTO_ALGOMODE_HMAC, CRYPTO_ALGOMODE_NOT_SET, CRYPTO_ALGOMODE_SIPHASH_2_4, CRYPTO_ALGOMODE_SIPHASH_4_8
--------------------------------	--

3.6.1.41 Csm/CsmPrimitives/CsmRandomGenerate/CsmRandomGenerateConfig/CsmRandomGenerateAlgorithmSecondaryFamily

[RTA-BSW] Enumeration Literals	CRYPTO_ALGOFAM_CUSTOM, CRYPTO_ALGOFAM_NOT_SET, CRYPTO_ALGOFAM_SHA2_224, CRYPTO_ALGOFAM_SHA2_256, CRYPTO_ALGOFAM_SHA2_384, CRYPTO_ALGOFAM_SHA2_512
[AUTOSAR] Enumeration Literals	CRYPTO_ALGOFAM_CUSTOM, CRYPTO_ALGOFAM_NOT_SET

3.6.1.42 Csm/CsmPrimitives/CsmRandomGenerate/CsmRandomGenerateConfig/CsmRandomGenerateResultLength

[RTA-BSW] Description	Size of the random generate key in bytes. Used only in Crypto_HSM if output buffering feature is enabled.
[AUTOSAR] Description	Size of the result data buffer for the synchronous RTE service interface if this primitive is referenced by a Csm job. It may also be possible that other BSW modules use the length parameter for buffer size calculation
[RTA-BSW] Lower Multiplicity	1
[AUTOSAR] Lower Multiplicity	0

3.6.1.43 Csm/CsmPrimitives/CsmSignatureGenerate/CsmSignatureGenerateConfig/CsmSignatureGenerateAlgorithmFamilyCustomRef

[RTA-BSW] Description	Reference to a customer specific algorithm family custom container.
[AUTOSAR] Description	Reference to a customer specific algorithm family custom container

3.6.1.44 Csm/CsmPrimitives/CsmSignatureGenerate/CsmSignatureGenerateConfig/CsmSignatureGenerateAlgorithmModeCustomRef

[RTA-BSW] Description	Reference to a customer specific algorithm mode custom container.
[AUTOSAR] Description	Reference to a customer specific algorithm mode custom container

3.6.1.45 Csm/CsmPrimitives/CsmSignatureGenerate/CsmSignatureGenerateConfig/CsmSignatureGenerateAlgorithmSecondaryFamily

[RTA-BSW] Enumeration Literals: CRYPTO_ALGOFAM_AES, CRYPTO_ALGOFAM_BLAKE_1_256, CRYPTO_ALGOFAM_BLAKE_1_512, CRYPTO_ALGOFAM_BLAKE_2s_256, CRYPTO_ALGOFAM_BLAKE_2s_512, CRYPTO_ALGOFAM_CUSTOM, CRYPTO_ALGOFAM_NOT_SET, CRYPTO_ALGOFAM_RIPEMD160, CRYPTO_ALGOFAM_SHA1, CRYPTO_ALGOFAM_SHA2_224, CRYPTO_ALGOFAM_SHA2_256, CRYPTO_ALGOFAM_SHA2_384, CRYPTO_ALGOFAM_SHA2_512, CRYPTO_ALGOFAM_SHA2_512_224, CRYPTO_ALGOFAM_SHA2_512_256, CRYPTO_ALGOFAM_SHA3_224, CRYPTO_ALGOFAM_SHA3_256, CRYPTO_ALGOFAM_SHA3_384, CRYPTO_ALGOFAM_SHA3_512, CRYPTO_ALGOFAM_SHAKE128, CRYPTO_ALGOFAM_SHAKE256

[AUTOSAR] Enumeration Literals: CRYPTO_ALGOFAM_BLAKE_1_256, CRYPTO_ALGOFAM_BLAKE_1_512, CRYPTO_ALGOFAM_BLAKE_2s_256, CRYPTO_ALGOFAM_BLAKE_2s_512, CRYPTO_ALGOFAM_CUSTOM, CRYPTO_ALGOFAM_NOT_SET, CRYPTO_ALGOFAM_RIPEMD160, CRYPTO_ALGOFAM_SHA1, CRYPTO_ALGOFAM_SHA2_224, CRYPTO_ALGOFAM_SHA2_256, CRYPTO_ALGOFAM_SHA2_384, CRYPTO_ALGOFAM_SHA2_512, CRYPTO_ALGOFAM_SHA2_512_224, CRYPTO_ALGOFAM_SHA2_512_256, CRYPTO_ALGOFAM_SHA3_224, CRYPTO_ALGOFAM_SHA3_256, CRYPTO_ALGOFAM_SHA3_384, CRYPTO_ALGOFAM_SHA3_512, CRYPTO_ALGOFAM_SHAKE128, CRYPTO_ALGOFAM_SHAKE256

3.6.1.46 Csm/CsmPrimitives/CsmSignatureGenerate/CsmSignatureGenerateConfig/CsmSignatureGenerateDataMaxLength

[RTA-BSW] Description	Size of the input data length in bytes
[AUTOSAR] Description	Size of the input data buffer for the synchronous RTE service interface if this primitive is referenced by a Csm job. It may also be possible that other BSW modules use the length parameter for buffer size calculation.

3.6.1.47 Csm/CsmPrimitives/CsmSignatureGenerate/CsmSignatureGenerateConfig/CsmSignatureGenerateResultLength

[RTA-BSW] Description	Size of the output signature length in bytes. Used only in Crypto_HSM if output buffering feature is enabled.
[AUTOSAR] Description	Size of the result data buffer for the synchronous RTE service interface if this primitive is referenced by a Csm job. It may also be possible that other BSW modules use the length parameter for buffer size calculation.
[RTA-BSW] Lower Multiplicity	1

[AUTOSAR] Lower Multiplicity	0
------------------------------	---

3.6.1.48 Csm/CsmPrimitives/CsmSignatureVerify/CsmSignatureVerifyConfig/CsmSignatureVerifyAlgorithmFamilyCustomRef

[RTA-BSW] Description	Reference to a customer specific algorithm family custom container.
[AUTOSAR] Description	Reference to a customer specific algorithm family custom container

3.6.1.49 Csm/CsmPrimitives/CsmSignatureVerify/CsmSignatureVerifyConfig/CsmSignatureVerifyAlgorithmModeCustomRef

[RTA-BSW] Description	Reference to a customer specific algorithm mode custom container.
[AUTOSAR] Description	Reference to a customer specific algorithm mode custom container

3.6.1.50 Csm/CsmPrimitives/CsmSignatureVerify/CsmSignatureVerifyConfig/CsmSignatureVerifyAlgorithmSecondaryFamily

[RTA-BSW] Enumeration Literals: CRYPTO_ALGOFAM_AES, CRYPTO_ALGOFAM_BLAKE_1_256, CRYPTO_ALGOFAM_BLAKE_1_512, CRYPTO_ALGOFAM_BLAKE_2s_256, CRYPTO_ALGOFAM_BLAKE_2s_512, CRYPTO_ALGOFAM_CUSTOM, CRYPTO_ALGOFAM_NOT_SET, CRYPTO_ALGOFAM_RIPEMD160, CRYPTO_ALGOFAM_SHA1, CRYPTO_ALGOFAM_SHA2_224, CRYPTO_ALGOFAM_SHA2_256, CRYPTO_ALGOFAM_SHA2_384, CRYPTO_ALGOFAM_SHA2_512, CRYPTO_ALGOFAM_SHA2_512_224, CRYPTO_ALGOFAM_SHA2_512_256, CRYPTO_ALGOFAM_SHA3_224, CRYPTO_ALGOFAM_SHA3_256, CRYPTO_ALGOFAM_SHA3_384, CRYPTO_ALGOFAM_SHA3_512, CRYPTO_ALGOFAM_SHAKE128, CRYPTO_ALGOFAM_SHAKE256

[AUTOSAR] Enumeration Literals: CRYPTO_ALGOFAM_BLAKE_1_256, CRYPTO_ALGOFAM_BLAKE_1_512, CRYPTO_ALGOFAM_BLAKE_2s_256, CRYPTO_ALGOFAM_BLAKE_2s_512, CRYPTO_ALGOFAM_CUSTOM, CRYPTO_ALGOFAM_NOT_SET, CRYPTO_ALGOFAM_RIPEMD160, CRYPTO_ALGOFAM_SHA1, CRYPTO_ALGOFAM_SHA2_224, CRYPTO_ALGOFAM_SHA2_256, CRYPTO_ALGOFAM_SHA2_384, CRYPTO_ALGOFAM_SHA2_512, CRYPTO_ALGOFAM_SHA2_512_224, CRYPTO_ALGOFAM_SHA2_512_256, CRYPTO_ALGOFAM_SHA3_224, CRYPTO_ALGOFAM_SHA3_256, CRYPTO_ALGOFAM_SHA3_384, CRYPTO_ALGOFAM_SHA3_512, CRYPTO_ALGOFAM_SHAKE128, CRYPTO_ALGOFAM_SHAKE256

3.6.1.51 Csm/CsmPrimitives/CsmSignatureVerify/CsmSignatureVerifyConfig/CsmSignatureVerifyCompareLength

[RTA-BSW] Description	Number of the least significant bytes of the signature, for which the verification shall be calculated.
[AUTOSAR] Description	Size of the input signature buffer for the synchronous RTE service interface if this primitive is referenced by a Csm job. It may also be possible that other BSW modules use the length parameter for buffer size calculation.
[RTA-BSW] Lower Multiplicity	1
[AUTOSAR] Lower Multiplicity	0

3.6.1.52 Csm/CsmPrimitives/CsmSignatureVerify/CsmSignatureVerifyConfig/CsmSignatureVerifyDataMaxLength

[RTA-BSW] Description	Max size of the input data, for which the signature shall be verified, in bytes.
[AUTOSAR] Description	Size of the input data buffer for the synchronous RTE service interface if this primitive is referenced by a Csm job. It may also be possible that other BSW modules use the length parameter for buffer size calculation.

3.6.1.53 Csm/CsmQueues/CsmQueue/CsmChannelRef

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

3.6.1.54 Csm/CsmQueues/CsmQueue/CsmQueueMainFunctionRef

[RTA-BSW] Description	This parameter refers to the used CsmMainFunction.
[AUTOSAR] Description	Reference to CsmMainFunction, where the according CsmQueue is assigned to.
[ADDED] Introduction	Reference to CsmMainFunction, where the according CsmQueue is assigned to.
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

3.6.1.55 Csm/CsmQueues/CsmQueue/CsmQueueSize

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

[RTA-BSW] Minimum Value	0
[AUTOSAR] Minimum Value	1

3.7 API Definition

List of supported API functions. Functions marked as *obsolete* or *deprecated* in the AUTOSAR specification (version v22-11) for the Csm module are not supported.

Csm_Init	Initializes the Csm module.
Csm_MainFunction	Called cyclically to process the requested jobs.
Csm_GetVersionInfo	Returns the version information of this module.
Csm_AEADDecrypt	Performs AEAD decryption and stores the ciphertext and the MAC in the memory locations referenced by the ciphertext and Tag pointers.
Csm_AEADEncrypt	Performs AEAD encryption and stores the ciphertext and the MAC in the memory locations referenced by the ciphertext and Tag pointers.
Csm_Encrypt	Encrypts the given data and store the ciphertext in the memory location pointed to by the result pointer.
Csm_Decrypt	Decrypts the given encrypted data and stores the resulting plaintext in the memory location pointed to by the result pointer.
Csm_Hash	Uses the given data to perform the hash calculation and stores the hash.
Csm_JobKeyDerive	Derives a new key by using the key elements in the given key identified by the keyId.
Csm_JobKeyExchangeCalcPubVal	Calculates the public value of the current user for the key exchange and stores the public key in the memory location pointed by the public value pointer.
Csm_JobKeyExchangeCalcSecret	Calculates the shared secret key for the key exchange with the key material of the key identified by the keyId and the partner public key.
Csm_JobKeyGenerate	Generates new key material and stores it in the key identified by keyId.
Csm_JobKeySetValid	Stores the key if necessary and sets to valid the key state of the key identified by keyId.
Csm_JobRandomSeed	Dispatches the random seed function to the configured Crypto Driver object.
Csm_KeyCopy	Copies all key elements from the source key to a target key.
Csm_KeyDerive	Derives a new key by using the key elements in the given key identified by <i>keyId</i> .

Csm_KeyElementCopy	Copies a key element from one key to a target key.
Csm_KeyElementCopyPartial	Uses keyElementSourceOffset and keyElementCopyLength to copy partial key element to another key element in the same crypto driver.
Csm_KeyElementGet	Retrieves a specific key element of the key identified by <i>keyId</i> and stores it in the memory location pointed by the key pointer.
Csm_KeyElementSet	Sets the given key element bytes to the key identified by <i>keyId</i> .
Csm_KeyExchangeCalcPubVal	Calculates the public value of the current user for the key exchange and stores the public key in the memory location pointed by the public value pointer.
Csm_KeyExchangeCalcSecret	Calculates the shared secret key for the key exchange with the key material of the key identified by <i>keyId</i> and the partner public key. The shared secret key is stored as a key element in the same key.
Csm_KeyGenerate	Generates new key material and stores it in the key identified by <i>keyId</i> .
Csm_KeySetValid	Sets the key state of the key identified by <i>keyId</i> to valid.
Csm_MacGenerate	Uses the given data to perform a MAC generation and stores the MAC in the memory location pointed to by the MAC pointer.
Csm_MacVerify	Verifies the given MAC by comparing if the MAC is generated with the given data.
Csm_RandomGenerate	Starts the random number generation service.
Csm_RandomSeed	Dispatches the random seed function to the configured Crypto Driver object.
Csm_RestoreContextJob	Crypto Driver extracts context data from the context Buffer and restores the internal state so that a further operation will continue at same point.
Csm_SaveContextJob	The Crypto Driver stores the internal context of the respective Crypto operation to the context Buffer.
Csm_SignatureGenerate	Uses the given data to perform the signature calculation and stores the signature in the memory location pointed by the result pointer.
Csm_SignatureVerify	Verifies the given MAC by comparing if the signature is generated with the given data.
Csm_CallbackNotification	Used by the underlying layer (Crylf) to notify Csm that a job has finished.
Csm_KeyGetStatus	Returns the key state of the key identified by <i>keyId</i> .
Csm_JobKeySetInvalid	Stores the key if necessary and sets the key state of the key identified by <i>keyId</i> to invalid.

Csm_CancelJob	Cancels the job processing from asynchronous or streaming jobs.
---------------	---

For details on using these calls, please refer to AUTOSAR_SWS_CryptoServiceManager.pdf (version v22-11).

Performs AEAD encryption and stores the ciphertext and the MAC in the memory locations referenced by the ciphertext and Tag pointers.

3.8 Configuration Advice



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

3.8.1 Configuration Methods

Two methods are available for the configuration of the modules within the Crypto stack (Csm, Crylf, Crypto and rba_CryptoHSM or rba_CryptoCCL), as described in the two sub-sections below.

Whenever possible, the flexible method should be used because it is easier, quicker and less error-prone.

3.8.1.1 Fixed Configuration

The "fixed" configuration method involves the manual configuration of all the parameters for each of the Crypto stack modules. With this method, all the dependencies and relationships between each of the modules must be taken into consideration.

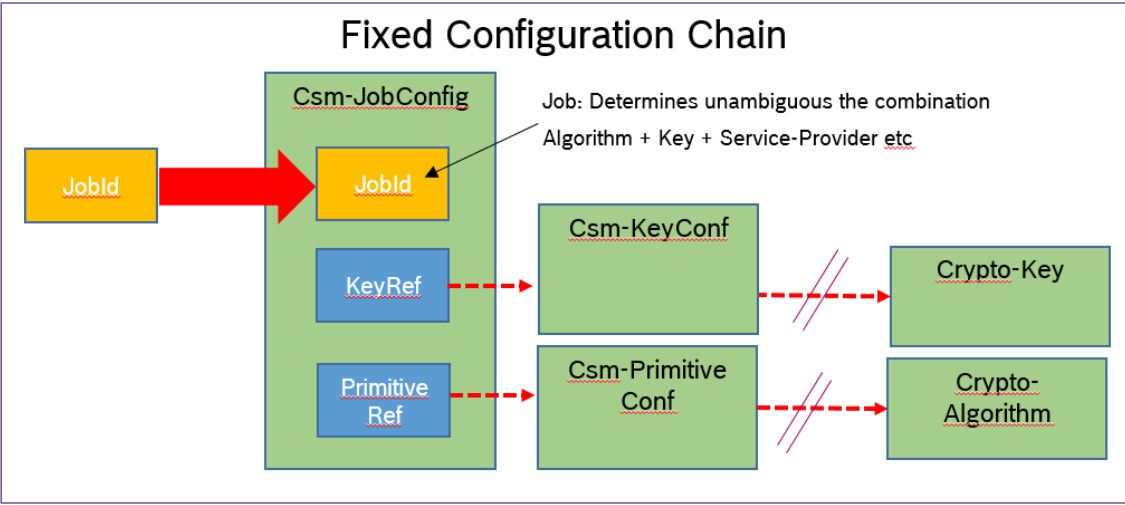


Figure 3.2.Fixed Configuration (Manual)

3.8.1.2 Flexible Configuration

The “flexible” configuration method involves the configuration of only the Csm module - the configuration of the other modules is done automatically.

If the CsmRbAutoFillConfig parameter (in Csm/CsmGeneral) is set to True, then the Csm generator will try to automatically fill-in the missing configuration parameters and connections in the Csm, Crylf and Crypto components.

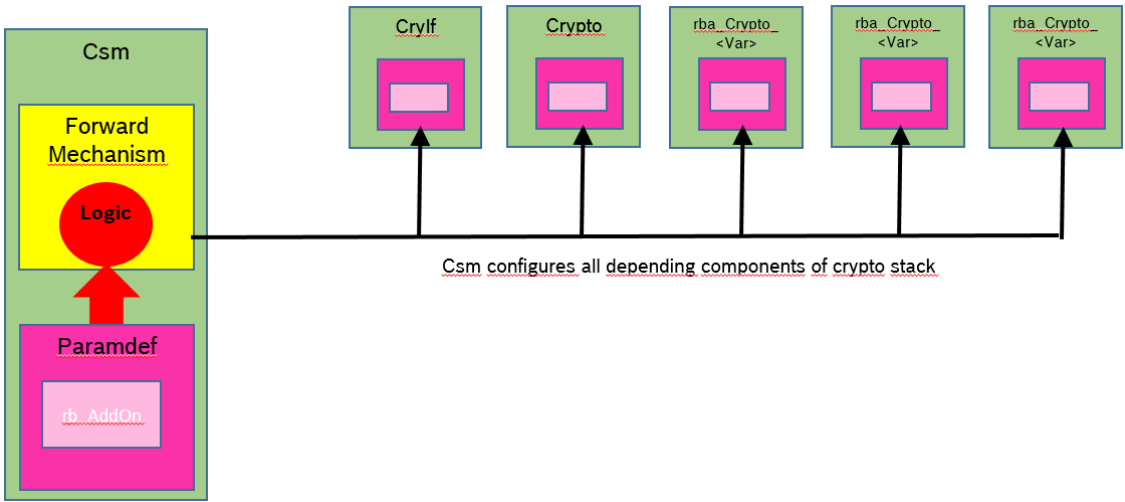


Figure 3.3.Flexible Configuration (Automatic)

- When automatic configuration completion is used for a key, CsmRbAutoConfigCryptoSelect (in Csm/CsmKeys/CsmKey) needs to select the crypto target for the key, and CsmRbAutoConfigKeyElement (also in Csm/CsmKeys/CsmKey) needs to define the keyElements.

- When automatic configuration completion used for a job/primitive pair, CsmRbAutoConfigCryptoSelect (in Csm/CsmPrimitives) needs to select the crypto target for the key.

3.8.2 Job Configuration

Various jobs can be created within Csm, and these jobs are always linked to one redirection of input/output parameters, one primitive, one queue and one key.

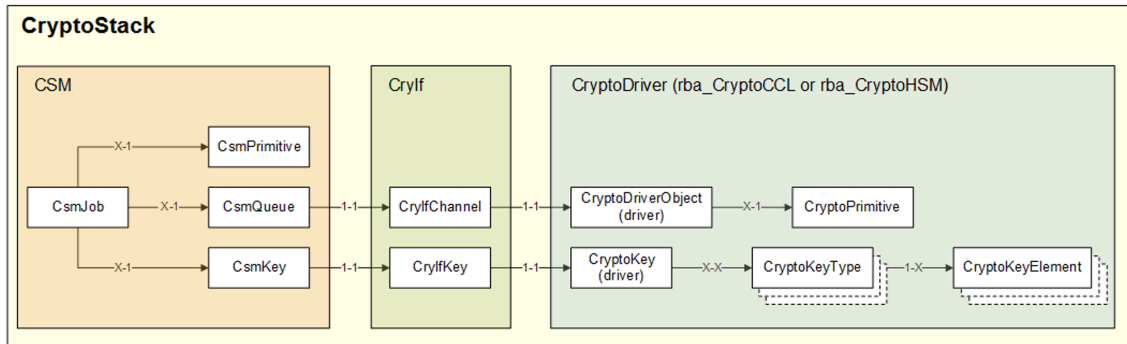


Figure 3.4. Csm Job Structure

If the flexible configuration method has been used, all that's required is to link a previously created key, set CsmRbAutoConfigCryptoSelect to assign the key to a CryptoDriver, and define at least one CsmRbAutoConfigKeyElement.

The CsmKeyId and CsmKeyRef are automatically filled in by Csm. One key can be used by several jobs.

The JobId is assigned automatically by Csm. All IDs in the configuration (e.g. CsmJobId, CsmKeyId) should be left blank whenever possible, allowing them to be set automatically by Csm.

The input and/or output parameters must be configured. A redirection can be reused by several jobs.

Primitives cannot be configured automatically by Csm, and so the configuration must be done manually. A primitive can be reused by several jobs.

If you use a blank queue reference in the job, Csm will automatically create and link a queue if needed. Csm will always try to use the same queue for jobs with the same primitive, and it will serialize all jobs within the same queue - parallel execution is not possible, regardless of whether the jobs are synchronous or asynchronous.

If parallel execution is required, you can create a queue manually and reference the job to it. This will create different CryptoDriverObjects within the corresponding CryptoDriver, and the jobs can be processed in parallel. Note that it may also be necessary to make further settings in the driver.

Queue sizes are configured via CryptoQueueSize and CsmQueueSize, although the actual size of the queues may differ because the code generator will automatically optimize them based on the job counts related to a queue. If the config-

uredqueue size is larger than the job count related to the queue, the maximal number of jobs will be used instead. The smaller number will always be used for the actual size.

If Crypto queues are enabled (`CryptoQueueSize>0`), then more jobs can be queued up for a single `DriverObject` within this component. The jobs will be executed one after another based on their priorities. This queueing method is more efficient compared to Csm job queueing.

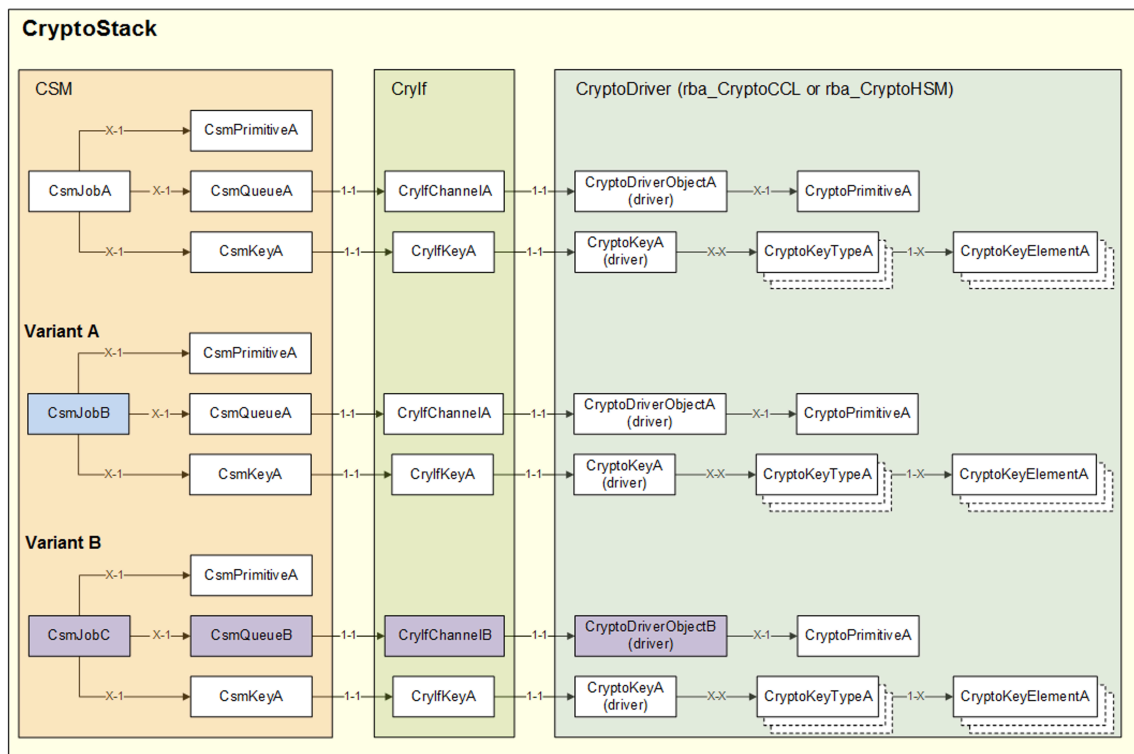


Figure 3.5.Csm Parallel Jobs

Variant A: Jobs A and B share a queue and are executed one after the other, since the Csm serializes all jobs within a queue.

Variant B: Jobs A and C have different queues and can therefore be parallelized if the CryptoDrivers have been configured accordingly.

Note that Callbacks should always be used when jobs are executed asynchronously. This ensures that a job has been successfully processed before the results are processed.

Callbacks cannot be configured automatically by Csm, and so the configuration must be done manually. A callback can be reused by several RTE jobs (with `CsmJobInterfaceUsePort` as `CRYPTO_USE_PORT_OPTIMIZED`) or by several normal jobs (with `CsmJobInterfaceUsePort` as `CRYPTO_USE_FNC`). A callback cannot be used by a RTE Csm job and by a normal Csm job at same time.

The `CsmCallbackFunc` parameter must contain exactly the same name as used in the C code, and the corresponding C function must exist in the application. `CsmCallbackFunc` must be left empty if the corresponding job has the RTE inter-

faceenabled (CsmJobInterfaceUsePort is set to CRYPTO_USE_PORT_OPTIMIZED).

3.8.2.1 Job Configuration Optimization

- If the Crypto driver is rba_CryptoHSM, note that the keys used by Csm can already be used in the initialization phase of the ECU by dummy calls to the corresponding jobs. The keys are pre loaded into a HSM session and are then immediately available during operation. Otherwise, the first call of a job always takes a little longer than the subsequent calls of the same job.
- To avoid watchdog triggering due to the available cycle time of the task being exceeded, time-consuming primitives, such as Signature and Cipher, should be handled by asynchronous jobs.
- Under normal circumstances, however, it makes no sense to execute primitives asynchronously because of the time loss due to the cycle times of the Csm and CryptoDriver main tasks. Synchronous jobs should be used as much as possible.
- As previously described, it is possible to assign separate queues to individual jobs. A separate ObjectId is created in the corresponding CryptoDriver for each queue. This allows Csm to submit several jobs to the driver in parallel.
- Normally the driver will process only one job per call of the maintask, but if CryptoRbMainFunctionLoopCounter is set to a value greater than 1, the driver tries to process several jobs per call of the maintask.
- In the case of HSM, this should only be used if CryptoRbHsmMainFunction<primitive>Mode for the primitive is set to blocking, otherwise the jobs will occasionally fail because the HSM on the lowest level can not process jobs in parallel. With blocking activated, the main task waits for the result of the first job (instead of just proceeding before the job is finished) and then processes the next job afterwards. The maintask of the CryptoDriver will serialize the jobs internally.

Processing of Job Queues

In the examples below there are two queues. Queue A accepts orders from 3 jobs (A, B and C) and Queue B from 2 jobs (E and F). Csm will serialize all jobs within a queue, which means it is waiting until one job is finished before the next one is transferred to the CryptoDriver.

CryptoRbMainFunctionLoopCounter is set to 2 so that a single maintask call cannot process more than 2 jobs at once. The execution of all 5 jobs therefore requires at least 3 maintask cycles.

In the case of HSM, CryptoRbHsmMainFunction<primitive>Mode is set to blocking, as previously described.

- In Maintask Cycle #1, one job is processed from each queue, so Job A (first loop) and Job E (second loop) are processed first. In the HSM example, the individual jobs are called synchronously, since the HSM cannot process several requests in parallel.

- In Maintask Cycle #2, Job B (first loop) and Job F (second loop) are processed.
- In Maintask Cycle #3, Queue B is empty so only Job C (first loop) is processed.

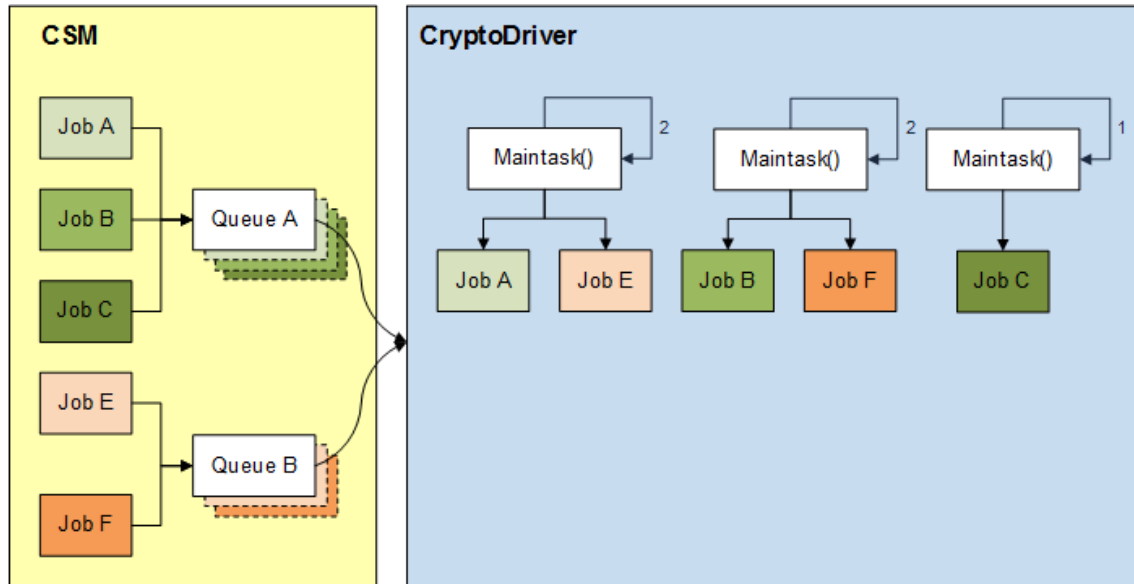


Figure 3.6.Csm Job Queue Processing (CCL)

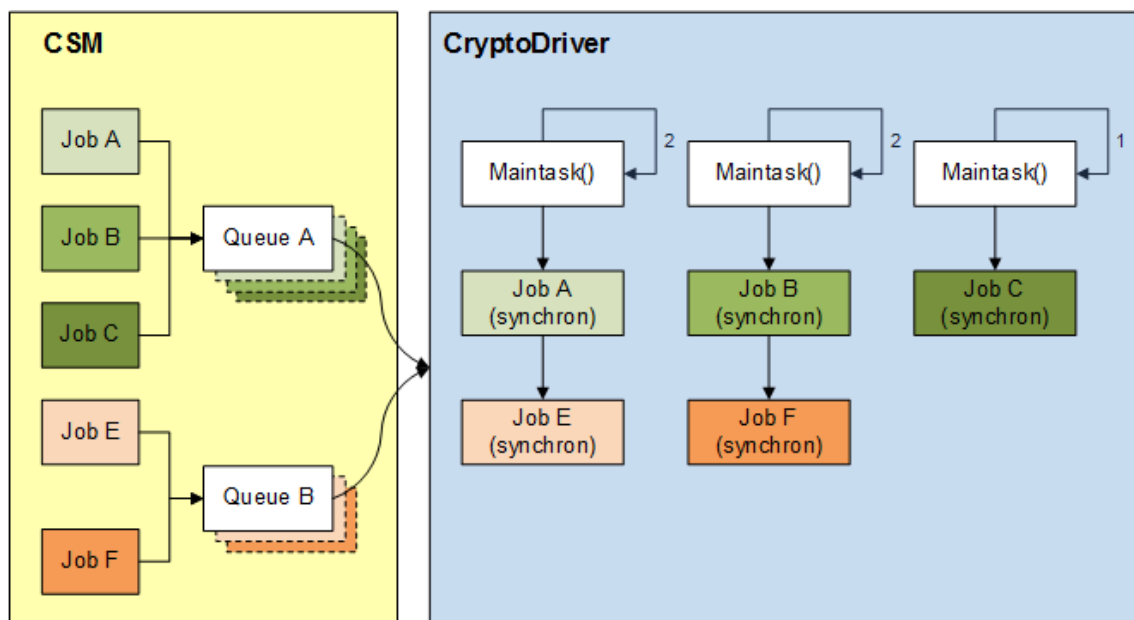


Figure 3.7.Csm Job Queue Processing (HSM)

3.8.2.2 Job Configuration Optimization (rba_CryptoCCL only)

CryptoRbCclSignatureLoopCounter

It is possible to slice any synchronous or asynchronous signature generation/verification into small pieces by using the CryptoRbCclSignatureLoopCounter parameter. This can help to prevent watchdog triggering.

3.8.2.3 Cycle Time Optimization (rba_CryptoHSM only)

The following examples show you how to identify and avoid possible bottlenecks under different configuration conditions.

Configuration 1

- Blocking disabled
- 1 Queue
- Mainloopcounter = 1

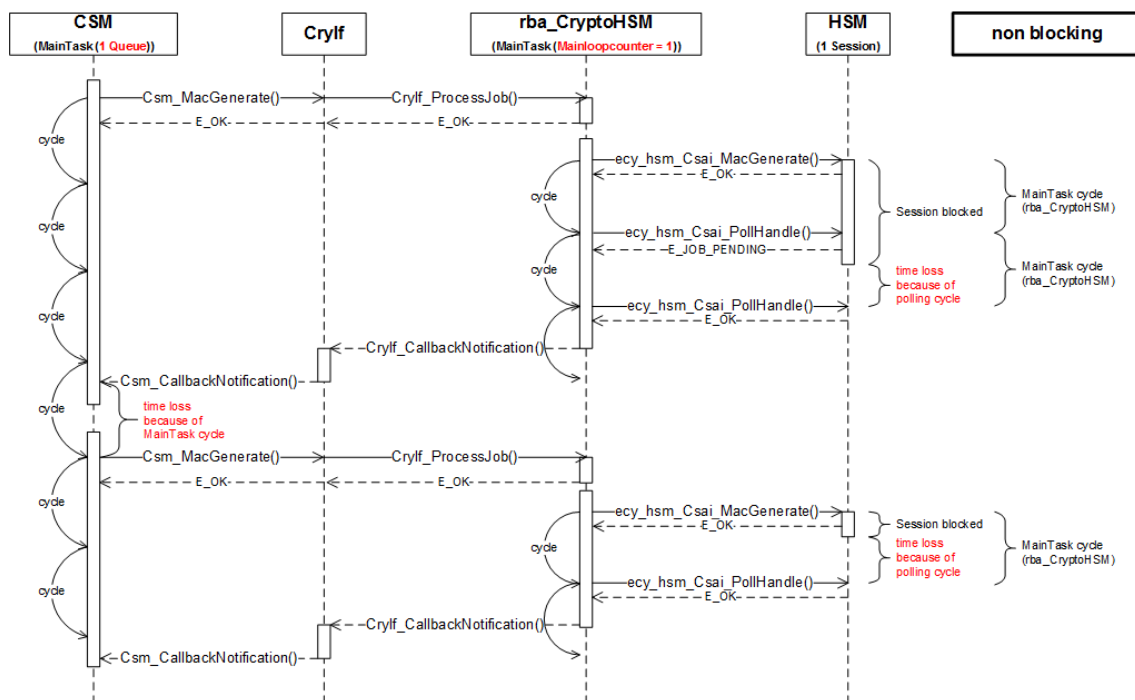


Figure 3.8.Configuration 1

The use of only one queue per primitive has a significant effect due to the asynchronous polling of the HSM, and further time losses will occur due to the cycle times of the Csm.

Configuration 2

- Blocking disabled
- 2 Queues

- Mainloopcounter = 2

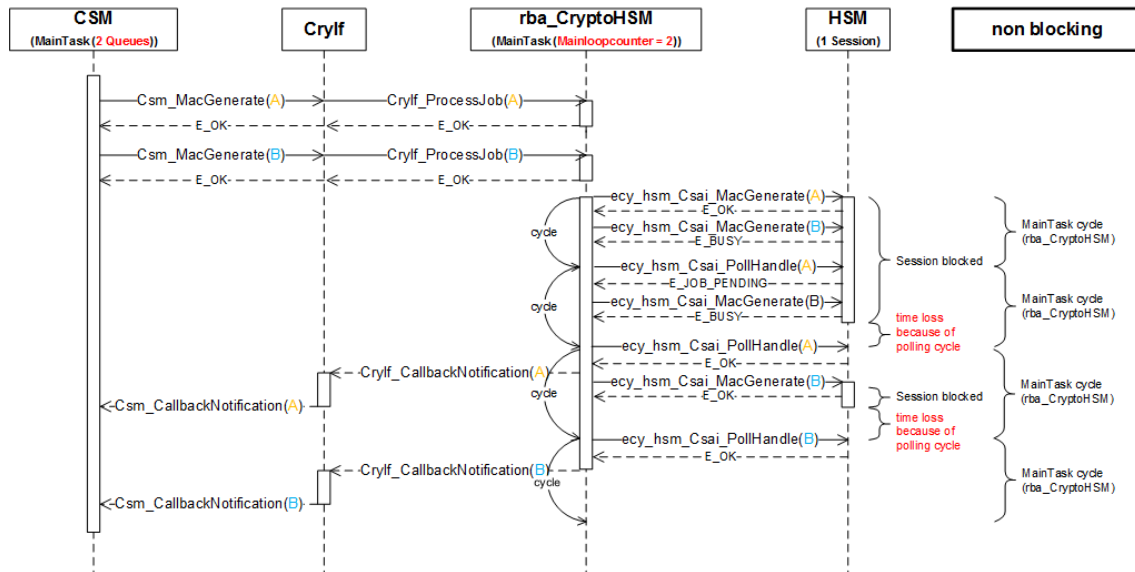


Figure 3.9.Configuration 2

This configuration will be faster than Configuration 1 because the time losses on the Csm level are avoided.

Configuration 3

- Blocking enabled
- 1 Queue
- Mainloopcounter = 1

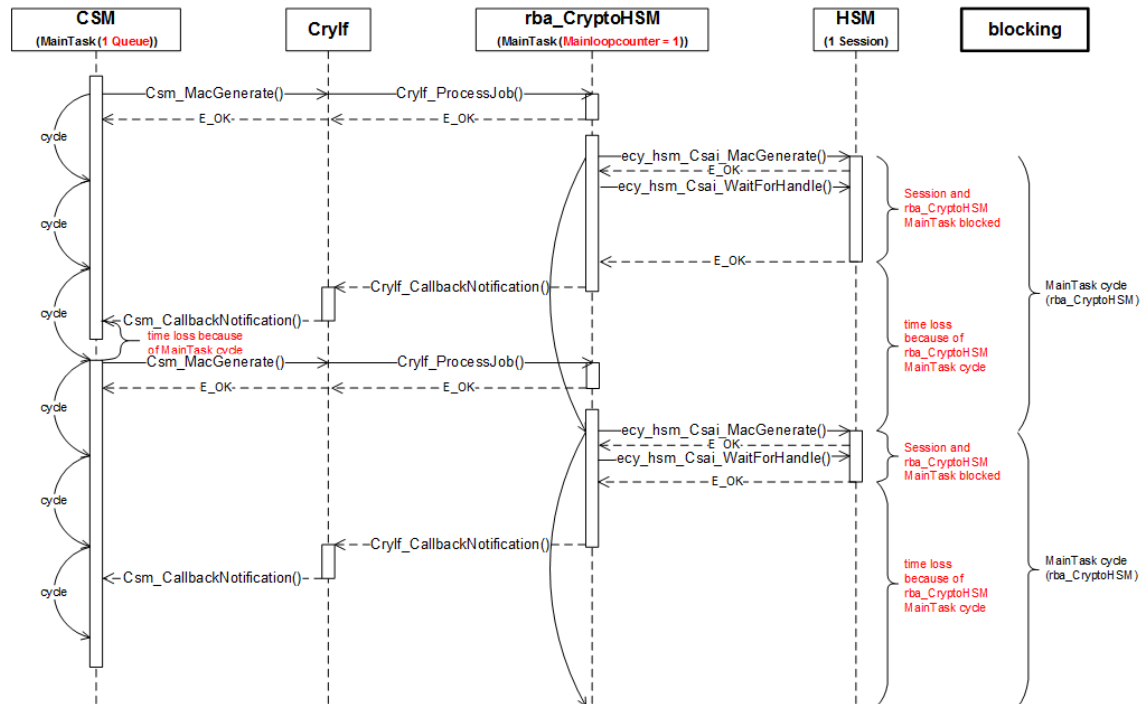


Figure 3.10.Configuration 3

With the blocking mode activated, the results can be made available to the application without loss of time. However, this has the disadvantage that the core is blocked during this time and cannot perform any other tasks.

Configuration 4

- Blocking enabled
- 2 Queues
- Mainloopcounter = 2

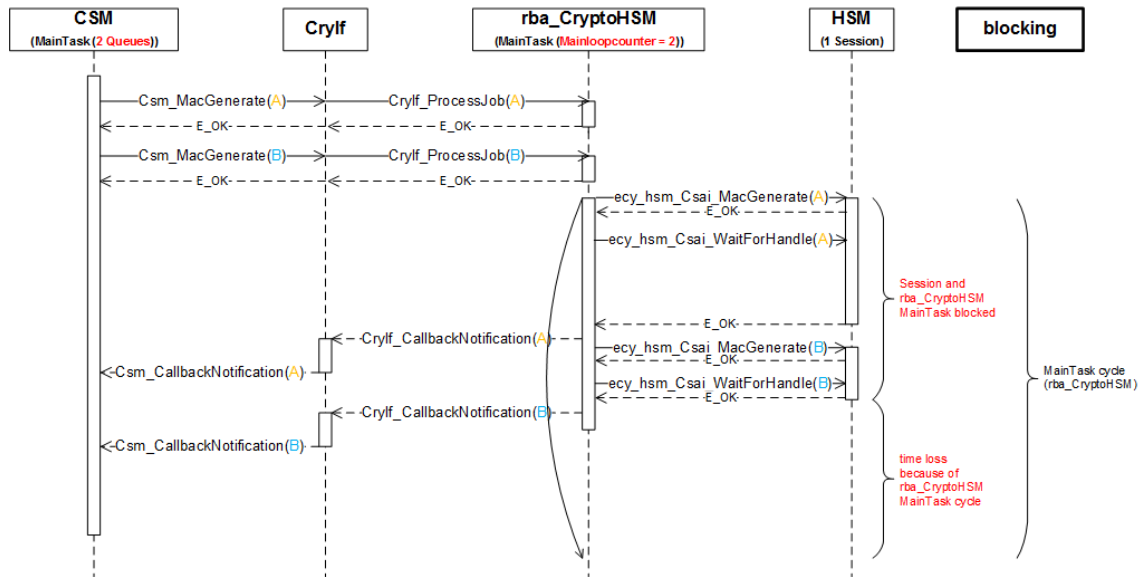


Figure 3.11.Configuration 4

The best performance can be ensured by using several queues and a Mainloop-counter oriented to that number of queues. Make sure that the loopcounter is not set too high, otherwise the main task will block for too long, which could lead to watchdog problems.

3.8.3 External Callout Support

External callout support is available for the RandomGenerate and RandomSeed operations. In order to configure a CsmJob and CsmKey to make use of these callout features, the following configurations need to be present.

- CsmJobKeyRef: Must refer to a CsmKey where CsmKeyRef is empty. No auto configuration is present. This CsmKey will be treated as an external callout CsmKey.
- CsmJobPrimitiveRef: Must refer to a CsmPrimitive with the following configuration (in CsmRandomGenerate/CsmRandomGenerateConfig):

CsmRandomGenerateAlgorithmFamily	CRYPTO_ALGOFAM_RNG
CsmRandomGenerateAlgorithmMode	CRYPTO_ALGOMODE_EXTERNAL_CALLOUT
CsmRandomGenerateAlgorithmSecondaryFamily	CRYPTO_ALGOFAM_NOT_SET
CsmRandomGenerateResultLength	Shall hold the length value of the key to be used together with RandomSeed (i.e. CsmKey configured in CsmJob/CsmJobKeyRef).

- CsmJobQueueRef: Must refer to a CsmQueue where CsmQueueRef is empty.

When making use of `RandomGenerate` (or `RandomSeed`) together with an external callout `CsmJob` (or external callout `CsmKey`), a callout function will be called instead of the typical `CryptoStack` function (i.e. `Csm_Rb_ProcessJob_ExternalCallout` instead of `Crylf_ProcessJob`, `Csm_Rb_CancelJob_ExternalCallout` instead of `Crylf_CancelJob`, and `Csm_Rb_RandomSeed_ExternalCallout` instead of `Crylf_RandomSeed`).

Note: External callout jobs are not protected against repeated job calls while the job is running or active. You need to ensure that an external callout job is not called again while it is still active or running.

3.8.4 MainFunction Partitioning

RTA-SEC supports `MainFunction` partitioning of resources. For `Csm`, this involves the generation of multiple `Csm_MainFunction` instances and mapping of `CsmQueues` to these instances where they should be handled.

The generation of multiple `Csm_MainFunction` instances is based on configuration via `CsmMainFunction` containers. The following parameters need to be configured for each `CsmMainFunction` container:

- **CsmMainFunctionPeriod**: The periodicity for this `MainFunction` instance.
- **CsmMainFunctionPartitionRef**: The `EcucPartition` (often interpreted as core) assignment for this `MainFunction` instance.

The default/legacy `MainFunction` (`Csm_MainFunction`) is still generated if there are `CsmQueues` that are not explicitly linked to a `CsmMainFunction` container, otherwise the `Csm_MainFunction` is no longer generated.

CsmQueue Partitioning

A `CsmQueue` can be explicitly assigned to a specific `MainFunction` container (via `CsmQueueMainFunctionRef`), and will be processed only by that specific `MainFunction` instance.

`CsmQueues` that are not mapped to a `MainFunction` container will be processed by the default `Csm_MainFunction` function.

MainFunction Forwarding

If automatic container filling is enabled, for `CsmQueues` linked to `MainFunction` containers, analogous `MainFunction` containers will be forwarded in `Crypto Drivers`, and these will be automatically linked to referenced `CryptoDriverObjects`.

Some `MainFunction` containers can be forwarded in `Csm` from `SecOC` if the corresponding `PDU`s are also linked to analogous `MainFunction` containers. If there are already `CsmMainFunction` containers configured with the same parameters, they are instead linked by the `SecOC` forwarder.

Callback buffering for rba_CryptoHSM

`rba_CryptoHSM` has a special implementation of the `MainFunction Partitioning` feature, which involves having only one `MainFunction` instance handling the job requests from all partitions. All upper layer callbacks are called from this unique `MainFunction` instance, regardless of their originating partition.

As this violates the MainFunction Partitioning design principle (all processing should be performed in the context associated to the partition), all callbacks from rba_CryptoHSM for asynchronous jobs that are explicitly assigned to MainFunction Partitions are buffered at the Csm level in dedicated queues. Csm will later forward them to upper layers in their corresponding MainFunction tasks in order to provide the callback in the same context the job came in.

The callback buffering is automatically enabled for all asynchronous CsmJobs executed on HSM which have CsmQueues explicitly mapped to CsmMainFunction containers. This cannot be altered via configuration.

Callbacks for asynchronous HSM CsmJobs for which the referenced CsmQueues are not linked to a MainFunction container will be processed in the rba_CryptoHSM MainFunction and will not be buffered in Csm.

3.8.5 VKMS Functionalities

Csm does not provide dedicated function calls to access VKMS functionalities within rba_CryptoHSM, so a specific configuration must be done from Csm to rba_CryptoHSM. The following VKMS-specific elements need to be configured in order to provide the interfaces for the purpose of accessing VKMS functions inside rba_CryptoHSM.

3.8.5.1 VKMS Csm Job related functionalities

In order to access a VKMS functionality within rba_CryptoHSM, CsmPrimitive configuration associated with a Csm Job and CryptoPrimitive associated to CryptoDriverObject referred by the Csm Job, need to comply with the following configuration for each VKMS function.

Handle DLC Start

Csm configuration (in CsmPrimitivesCsmDecrypt)

CsmDecryptAlgorithmFamily	CRYPTO_ALGOFAM_AES
CsmDecryptAlgorithmKeyLength	16
CsmDecryptAlgorithmMode	CRYPTO_ALGOMODE_CBC
CsmDecryptAlgorithmSecondaryFamily	CRYPTO_ALGOFAM_NOT_SET

rba_CryptoHSM configuration (in CryptoCryptoPrimitives)

CryptoPrimitiveAlgorithmFamily	CRYPTO_ALGOFAM_VKMS_HANDLE_DLC_START
CryptoPrimitiveAlgorithmMode	CRYPTO_ALGOMODE_NOT_SET
CryptoPrimitiveAlgorithmSecondary-Family	CRYPTO_ALGOFAM_NOT_SET
CryptoPrimitiveService	DECRYPT

Handle DLC Finish

Csm configuration (in CsmPrimitivesCsmEncrypt)

CsmDecryptAlgorithmFamily	CRYPTO_ALGOFAM_AES
CsmDecryptAlgorithmKeyLength	16
CsmDecryptAlgorithmMode	CRYPTO_ALGOMODE_CBC
CsmDecryptAlgorithmSecondaryFamily	CRYPTO_ALGOFAM_NOT_SET

rba_CryptoHSM configuration (in CryptoCryptoPrimitives)

CryptoPrimitiveAlgorithmFamily	CRYPTO_ALGOFAM_VKMS_HANDLE_DLC_FINISH
CryptoPrimitiveAlgorithmMode	CRYPTO_ALGOMODE_NOT_SET
CryptoPrimitiveAlgorithmSecondary-Family	CRYPTO_ALGOFAM_NOT_SET
CryptoPrimitiveService	ENCRYPT

Get Verification Hash**Csm configuration (in CsmPrimitivesCsmMacGenerate)**

CsmDecryptAlgorithmFamily	CRYPTO_ALGOFAM_AES
CsmDecryptAlgorithmKeyLength	16
CsmDecryptAlgorithmMode	CRYPTO_ALGOMODE_CMAC
CsmDecryptAlgorithmSecondaryFamily	CRYPTO_ALGOFAM_NOT_SET
CsmMacGenerateResultLength	17

rba_CryptoHSM configuration (in CryptoCryptoPrimitives)

CryptoPrimitiveAlgorithmFamily	CRYPTO_ALGOFAM_VKMS_GET_VERIFICATION_HASH
CryptoPrimitiveAlgorithmMode	CRYPTO_ALGOMODE_NOT_SET
CryptoPrimitiveAlgorithmSecondary-Family	CRYPTO_ALGOFAM_NOT_SET
CryptoPrimitiveService	MAC_GENERATE

3.8.5.2 Get PSS Hash**Csm configuration (in CsmPrimitivesCsmMacGenerate)**

CsmDecryptAlgorithmFamily	CRYPTO_ALGOFAM_AES
CsmDecryptAlgorithmKeyLength	16
CsmDecryptAlgorithmMode	CRYPTO_ALGOMODE_CMAC
CsmDecryptAlgorithmSecondaryFamily	CRYPTO_ALGOFAM_NOT_SET
CsmMacGenerateResultLength	17

rba_CryptoHSM configuration (in CryptoCryptoPrimitives)

CryptoPrimitiveAlgorithmFamily	CRYPTO_ALGOFAM_VKMS_GET_PSS_HASH
CryptoPrimitiveAlgorithmMode	CRYPTO_ALGOMODE_NOT_SET
CryptoPrimitiveAlgorithmSecondaryFamily	CRYPTO_ALGOFAM_NOT_SET
CryptoPrimitiveService	MAC_GENERATE

3.8.5.3 Get Identity Hash

Csm configuration (in CsmPrimitivesCsmMacGenerate)

CsmDecryptAlgorithmFamily	CRYPTO_ALGOFAM_AES
CsmDecryptAlgorithmKeyLength	16
CsmDecryptAlgorithmMode	CRYPTO_ALGOMODE_CMAC
CsmDecryptAlgorithmSecondaryFamily	CRYPTO_ALGOFAM_NOT_SET
CsmMacGenerateResultLength	17

rba_CryptoHSM configuration (in CryptoCryptoPrimitives)

CryptoPrimitiveAlgorithmFamily	CRYPTO_ALGOFAM_VKMS_GET_IDENTITY_HASH
CryptoPrimitiveAlgorithmMode	CRYPTO_ALGOMODE_NOT_SET
CryptoPrimitiveAlgorithmSecondaryFamily	CRYPTO_ALGOFAM_NOT_SET
CryptoPrimitiveService	MAC_GENERATE

3.8.5.4 VKMS Csm Key related functionalities

In order to access a VKMS functionality within rba_CryptoHSM, the CryptoKey referred by a Csm Key must comply with the configuration of rba_CryptoHSM shown below. No special configuration of Csm is required.

As you can see, KeyElement configuration is used to map what HSM_VKMS functionality is to be executed within rba_CryptoHSM. Other than the parameters listed below, all other parameters have no influence over functional behaviour because this is managed by the VKMS Core application within rba_CryptoHSM. It is therefore recommended that all other parameters are configured as empty or as "false".

Get Status

CryptoKeyElementId	0xF0010003
CryptoKeyElementSize	8
CryptoRbKeyElementType	CRYPTO_ELEMENT_TYPE_VKMS_GET_STATUS

Get Training Counter

CryptoKeyElementId	0xF0010002
--------------------	------------

CryptoKeyElementSize	3
CryptoRbKeyElementType	CRYPTO_ELEMENT_TYPE_VKMS_GET_TRAININGCOUNTER

Get VKMS VIN

CryptoKeyElementId	0xF0010001
CryptoKeyElementSize	17
CryptoRbKeyElementType	CRYPTO_ELEMENT_TYPE_VKMS_GET_VIN

Get Key

CryptoRbHsmVkmsKeyTypeld	VKMS Key Type ID
CryptoKeyElementId	0xF0010001
CryptoKeyElementSize	KeySize + 1
CryptoRbKeyElementType	CRYPTO_ELEMENT_TYPE_VKMS_GET_KEY

Get Metadata

CryptoRbHsmVkmsKeyTypeld	VKMS Key Type ID
CryptoKeyElementId	0xF0010002
CryptoKeyElementSize	5
CryptoRbKeyElementType	CRYPTO_ELEMENT_TYPE_VKMS_GET_METADATA

Note: If VKMS Key Type ID is configured to 0, the output buffer will contain a concatenated list of key meta data for each configured and provisioned key as follows:

key type ID (2 bytes) | type training counter (2 bytes) | key flags (1 byte) | key genus (1 byte)

CryptoKeyElementSize will be configured taking into the consideration the maximum number of possible configured keys and applying a formula: $(MAX_NUMBER_KEYS * 6) + 1$

3.8.6 Algorithm modes or Padding schemes

Csm supports algorithm modes CRYPTO_ALGOMODE_CBC_NONE, CRYPTO_ALGOMODE_CBC_PKCS7_V1.5 and CRYPTO_ALGOMODE_CBC_NIST_SP800_38A only with the AES algorithm family.

3.8.7 SHE Load Key/SHE Load Key Slot

Csm provides a CsmPrimitive CsmRbSheLoadKey/CsmRbSheLoadKeySlot to load SHE keys, as described in Chapter 9 ("Memory Update protocol") from the SHE v1.1 specification.

In order to enable the SHE Load Key/SHE Load Key Slot functionality a CsmJob has to be created and a reference to CsmPrimitives containing CsmRbSheLoad-

Key/CsmRbSheLoadKeySlotconfiguration should be added in CsmJobPrimitiveRef.

The screenshot shows the configuration window for a CSM job named 'She_Key_Load'. The window is divided into three main sections: General, Attributes, and References.

- General:**
 - ShortName: She_Key_Load
 - LongName: FOR-ALL
- Attributes:**
 - CsmJobId: 3
 - CsmJobInterfaceUsePort: CRYPTO_USE_PORT_OPTIMIZED
 - CsmJobPrimitiveCalibration: false
 - CsmJobPriority: 0
 - CsmProcessingMode: CRYPTO_PROCESSING_ASYNC
 - CsmRbJobRteQLength: (empty)
- References:**
 - CsmInOutRedirectionRef: (empty)
 - CsmJobKeyRef: (empty)
 - CsmJobPrimitiveCallbackRef: CsmCallback_0
 - CsmJobPrimitiveRef: SHE_LoadKeySlot
 - CsmJobQueueRef: CsmQueue_Fwd_3

The CsmRbSheLoadKey/CsmRbSheLoadKeySlot primitive supports both synchronous and asynchronous processing. This is determined by the CsmProcessingMode parameter of the CsmJob that refers to the CsmRbSheLoadKey/CsmRbSheLoadKeySlot primitive, which should be configured as CRYPTO_PROCESSING_ASYNC to avoid what is usually a lengthy busy-waiting time until a key load operation is completed on the hardware.

Loading SHE keys is only supported when the hardware contains SHE or SHE emulation, so the CsmRbSheLoadKey/CsmRbSheLoadKeySlot primitive is only supported with Rba_CryptoHSM. When CsmRbSheLoadKey/CsmRbSheLoadKeySlot is configured, CsmRbAutoConfigCryptoSelect accepts only rba_CryptoHSM, and the necessary configuration dependencies will be automatically forwarded during code generation to rba_CryptoHSM.

3.8.8 SHE Get ID

Csm provides a CsmPrimitive CsmRbSheGetID to support the SHE Get Identity function (for retrieving the SHE UID) as described in the SHE v1.1 specification.

In order to enable the SHE Get ID functionality a CsmJob has to be created and a reference to CsmPrimitives containing CsmRbSheGetID configuration should be added in CsmJobPrimitiveRef.

3.8.9 SHE Export Ram Key

Csm provides a CsmPrimitive CsmRbSheExportRamKey to export the RAM key to a SHE wrapped key format, as described in the SHE functional specification v1.1.

In order to enable the SHE Export Ram Key functionality a CsmJob has to be created and a reference to CsmPrimitives containing CsmRbSheExportRamKey configuration should be added in CsmJobPrimitiveRef.

General	
ShortName*	SHE_Ram_Key_Export
LongName	FOR-ALL ▼ ✕
Attributes	
CsmJobId	1 ✕
CsmJobInterfaceUsePort*	CRYPTO_USE_PORT_OPTIMIZED ▼ ✕
CsmJobPrimitiveCallb...ation	false ▼ ✕
CsmJobPriority*	0 ✕
CsmProcessingMode*	CRYPTO_PROCESSING_ASYNC ▼ ✕
CsmRbJobRteQLength	✕
References	
CsmInOutRedirectionRef	▼ ✕
CsmJobKeyRef	▼ ✕
CsmJobPrimitiveCallbackRef	CsmCallback_2 ▼ ✕
CsmJobPrimitiveRef*	SHE_ExportRamKey ▼ ✕
CsmJobQueueRef	CsmQueue_Fwd_1 ▼ ✕

The CsmRbSheExportRamKey primitive supports both synchronous and asynchronous processing. This is determined by the parameter CsmProcessingMode of CsmJob refers to CsmRbSheExportRamKey primitive.

Exporting the SHE RAM key is only supported when the hardware contains SHE or SHE emulation, so the CsmRbSheExportRamKey primitive is only supported with Rba_CryptoHSM. When CsmRbSheExportRamKey is configured, CsmRbAuto-ConfigCryptoSelect accepts only rba_CryptoHSM, and the necessary configuration dependencies will be automatically forwarded during code generation to rba_CryptoHSM.

3.8.10 SHE Load Plain Key

Csm provides a CsmPrimitive CsmRbSheLoadPlainKey to load a plaintext key into the RAM_KEY slot without encryption or verification, as described in the SHE specification v1.1.

In order to enable the SHE Load Plain Key functionality a CsmJob has to be created and a reference to CsmPrimitives containing CsmRbSheLoadPlainKey configuration should be added in CsmJobPrimitiveRef.

General	
ShortName*	She_Plain_Key_Load
LongName	FOR-ALL ⓧ
Attributes	
CsmJobId	4 ⓧ
CsmJobInterfaceUsePort*	CRYPTO_USE_PORT_OPTIMIZED ⓧ
CsmJobPrimitiveCallb...ation	false ⓧ
CsmJobPriority*	0 ⓧ
CsmProcessingMode*	CRYPTO_PROCESSING_ASYNC ⓧ
CsmRbJobRteQLength	ⓧ
References	
CsmInOutRedirectionRef	ⓧ
CsmJobKeyRef	ⓧ
CsmJobPrimitiveCallbackRef	CsmCallback_1 ⓧ
CsmJobPrimitiveRef*	SHE_LoadPlainKey ⓧ
CsmJobQueueRef	CsmQueue_Fwd_4 ⓧ

Loading plain keys to the RAM_KEY slot in this manner is only supported when the hardware contains SHE or SHE emulation, so the CsmRbSheLoadPlainKey primitive is only supported with Rba_CryptoHSM. When CsmRbSheLoadPlainKey is configured, CsmRbAutoConfigCryptoSelect accepts only rba_CryptoHSM, and the necessary configuration dependencies will be automatically forwarded during code generation to rba_CryptoHSM.

3.9 Integration Advice

3.9.1 Init Functions

Csm is initialized via Csm_Init.

In order to use any RTA-SEC service, all the stack components (including rba_CryptoCCL or rba_CryptoHSM) must be initialised.

The very first initialisation of the stack may take longer to finish because initial values may be loaded into persistent storage. Software components which use the Csm module are responsible for checking global error and status information resulting from the Csm module startup. It is not possible to request any Csm services before the initialisation of the entire stack has completed (with no errors).

3.9.2 DeInit Functions

Csm_Rb_Deinit is used to disable the Csm module.

If persistent storage is used for key elements, the Csm_Rb_StorePermanentData function must be called during the shutdown procedure of the Cryptostack to save the updated NvM block with the new data before shutting the system down.

When calling Csm_Rb_Deinit, no other stack function must be running concurrently, and the underlying modules in the stack must also be shut down using their own de-initialisation function (if available).

The target ECU must be reset BEFORE reinitialising the Csm module in order to

allow the correct handling of persistent key data.

3.9.3 Scheduled Functions

Csm_Init and all other initialization functions (e.g. from CryptoDrivers or SecOC) have to be called first before calling any Maintask or any other Csm service. All Maintasks involved have to be called periodically according to all dependent services and timings.

To avoid Maintask cycles being wasted, Maintasks should be called in top-down order (e.g. SecOC → Csm → CryptoDriver).

When scheduling Csm_MainFunction, bear in mind that the main functions of other modules in the RTA-SEC stack cannot be run concurrently. In order to prevent data race conditions, all main functions within the stack must be run as part of the same task.

The Crypto Driver main function can handle only one job at a time (queue size of 1). Therefore Csm_MainFunction has to be called with the same frequency as the Crypto Driver main function.

3.9.4 Module Compatibility

The modules Csm, Crylf, Crypto, and rba_CryptoCCL or rba_CryptoHSM, form a functional composition. Only modules from the same package version should be used together.

The integration of Crylf, Crypto and either rba_CryptoCCL or rba_CryptoHSM is a prerequisite for using Csm.

3.9.5 SHE Get ID

The interface Csm_Rb_SheGetId can be used by the application software to get the SHE ID. This will also return a verification message in accordance with the SHE specification, consisting of a MAC generated with the MASTER_KEY from a supplied challenge, the SHE Id and status register as the input data. If no MASTER_KEY has been loaded, the verification message shall consist only of the value zero.

3.9.6 SHE Load Key Slot

A provided port could be generated by Csm if CsmJobInterfaceUsePort is set as CRYPTO_USE_PORT or CRYPTO_USE_PORT_OPTIMIZED. The generated provided port could be used for client server communication via RTE to invoke the Rb_LoadKeySlot operation.

The interface Csm_Rb_SheLoadKeySlot can be used by the application software to load up to 90 SHE keys in addition to the MASTER_ECU_KEY. This is supported through the combination of the KeySlot and M1 fields of the input parameter KeySlotM1M2M3Ptr.

- If SHE key slot 2 needs to be updated, the application software should set KeySlot to 0x02 and the ID nibble of M1 to 0x05.
- If SHE key slot 12 needs to be updated, the application software should set KeySlot to 0x0C and the ID nibble of M1 to 0x05.

- If MASTER_ECU_KEY needs to be updated, the application software should set KeySlot to 0xFF and the ID nibble of M1 to 0x01.

3.9.7 SHE Export Ram Key

A provided port could be generated by Csm if CsmJobInterfaceUsePort is set as CRYPTO_USE_PORT or CRYPTO_USE_PORT_OPTIMIZED. The generated provided port could be used for client server communication via RTE to invoke the Rb_ExportRamKey operation.

The interface Csm_Rb_SheExportRamKey can be used by the application software to export the RAM key into a SHE wrapped key format protected with the secret key. The exported RAM key is stored in the output parameter field M1M2M3M4M5Ptr.

3.9.8 SHE Load Plain Key

A provided port could be generated by Csm if CsmJobInterfaceUsePort is set as CRYPTO_USE_PORT or CRYPTO_USE_PORT_OPTIMIZED. The generated provided port could be used for client server communication via RTE to invoke the Rb_LoadPlainKey operation.

The interface Csm_Rb_SheLoadPlainKey can be used by the application software to load a plaintext key into the RAM_KEY slot without encryption or verification.

3.9.9 Using secure storage keys

Persistent keys in HSM secure storage, such as SHE keys, could be used for cryptographic jobs such as MAC generation and verification.

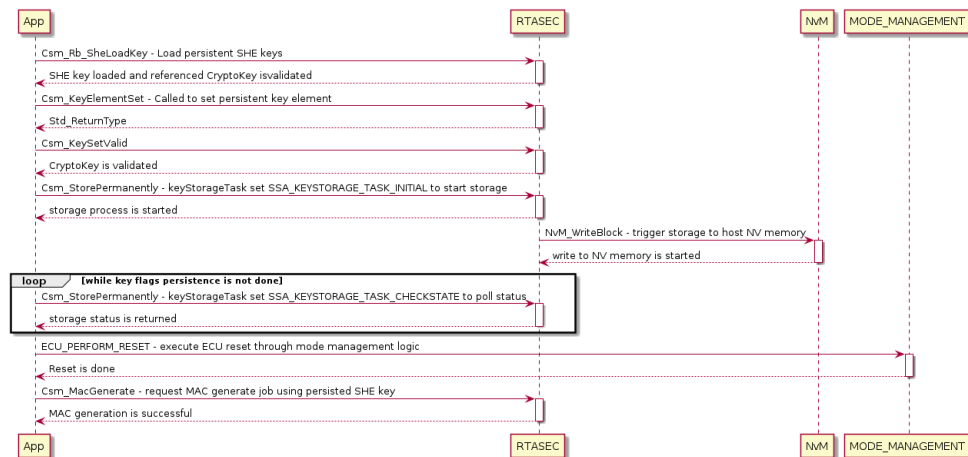
In order for secure storage keys to be used for cryptographic jobs across ECU power cycles, their validity flags have to be persisted in the host non-volatile memory. To persist the validity flags across ECU power cycles, RTA-SEC requires the NvM module to be configured and selected for code generation in the RTA-BSW project. rba_CryptoHSM will automatically forward its configuration dependencies for secure storage keys to the NvM module during code generation.

During run-time, the application software responsible for secure storage keys provisioning should be also made responsible for requesting persistence of the keys' validity flags. This is achieved by calling the Csm interface Csm_StorePermanently from the application software. A provided port is available through which client server communication via RTE to invoke the permanent storage operation is possible.

Csm_StorePermanently should be called once with the parameter to start the persistence of the keys' validity flags in the host non-volatile memory. This will trigger the persistence of all the secure storage keys that have been updated in the current active power cycle. For this reason, Csm_StorePermanently doesn't need to be called after every single secure storage key storage request done through Csm_KeyElementSet/Csm_KeySetValid or Csm_Rb_SheLoadKey.

The following is an example workflow for using secure storage keys for a MAC

generation job.



3.9.10 Diffie-Hellman key exchange handling

A key exchange according to the Diffie-Hellman algorithm can be performed in the following variants:

- Synchronous call via key management services: Csm_KeyExchangeCalcPubVal and Csm_KeyExchangeCalcSecret (rba_CryptoCCL and rba_CryptoHSM [Curve25519, Curve448, secp224r1, secp256r1, secp384r1]).
- Synchronous job: Csm_JobKeyExchangeCalcPubVal and Csm_JobKeyExchangeCalcSecret (rba_CryptoCCL and rba_CryptoHSM [Curve25519, Curve448, secp224r1, secp256r1, secp384r1]).
- Asynchronous job: Csm_JobKeyExchangeCalcPubVal and Csm_JobKeyExchangeCalcSecret (rba_CryptoCCL / rba_CryptoHSM [Curve25519, Curve448, secp224r1, secp256r1, secp384r1]).
- Synchronous call via key management services with existing key pair: Csm_KeyExchangeCalcSecret (rba_CryptoHSM [secp384r1]).
- Synchronous job with existing key pair: Csm_JobKeyExchangeCalcSecret (rba_CryptoHSM [secp384r1]).
- Asynchronous job with existing key pair: Csm_JobKeyExchangeCalcSecret (rba_CryptoHSM [secp384r1]).

If one of the first 3 variants is used (no existing key pair) via rba_CryptoHSM, you must ensure that the calculation of the public value (Csm_KeyExchangeCalcPubVal, Csm_JobKeyExchangeCalcPubVal) is always followed by the corresponding calculation of the shared secret (Csm_KeyExchangeCalcSecret, Csm_JobKeyExchangeCalcSecret) and that the calculation is finished before starting the next key exchange job. The execution order cannot be changed (e.g. by queuing several key exchange jobs).

Examples:

Key exchange with partner A: Csm_JobKeyExchangeCalcPubVal(jobA1, keyA, pubValA, pubValLengthA), Csm_JobKeyExchangeCalcSecret(jobA2, keyA, part-

nerPubValA,partnerPubValLengthA)

Key exchange with partner B: Csm_JobKeyExchangeCalcPubVal(jobB1, keyB, pubValB, pubValLengthB), Csm_JobKeyExchangeCalcSecret(jobB2, keyB, partnerPubValB, partnerPubValLengthB)

Allowed order:

1. Csm_JobKeyExchangeCalcPubVal(jobA1, keyA, pubValA, pubValLengthA)
2. Csm_JobKeyExchangeCalcSecret(jobA2, keyA, partnerPubValA, partnerPubValLengthA)
3. Csm_JobKeyExchangeCalcPubVal(jobB1, keyB, pubValB, pubValLengthB)
4. Csm_JobKeyExchangeCalcSecret(jobB2, keyB, partnerPubValB, partnerPubValLengthB)

Not allowed order:

1. Csm_JobKeyExchangeCalcPubVal(jobA1, keyA, pubValA, pubValLengthA)
2. Csm_JobKeyExchangeCalcPubVal(jobB1, keyB, pubValB, pubValLengthB)
3. Csm_JobKeyExchangeCalcSecret(jobA2, keyA, partnerPubValA, partnerPubValLengthA)
4. Csm_JobKeyExchangeCalcSecret(jobB2, keyB, partnerPubValB, partnerPubValLengthB)

For details about the handling of key exchange with an existing key pair, see the Crypto driver section.

3.9.11 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The Csm module does not use any exclusive OS resources.

3.10 Hardware Requirements

Table 3.117.Csm Hardware Requirements

Support	Usage
FLASH	6386 bytes
RAM	5401 bytes

Usage calculated based on a sample configuration.

4 Key Manager

Module name	KeyM
Package	RTA-SEC
AUTOSAR Module ID	109
AUTOSAR Version	22-11

4.1 Introduction

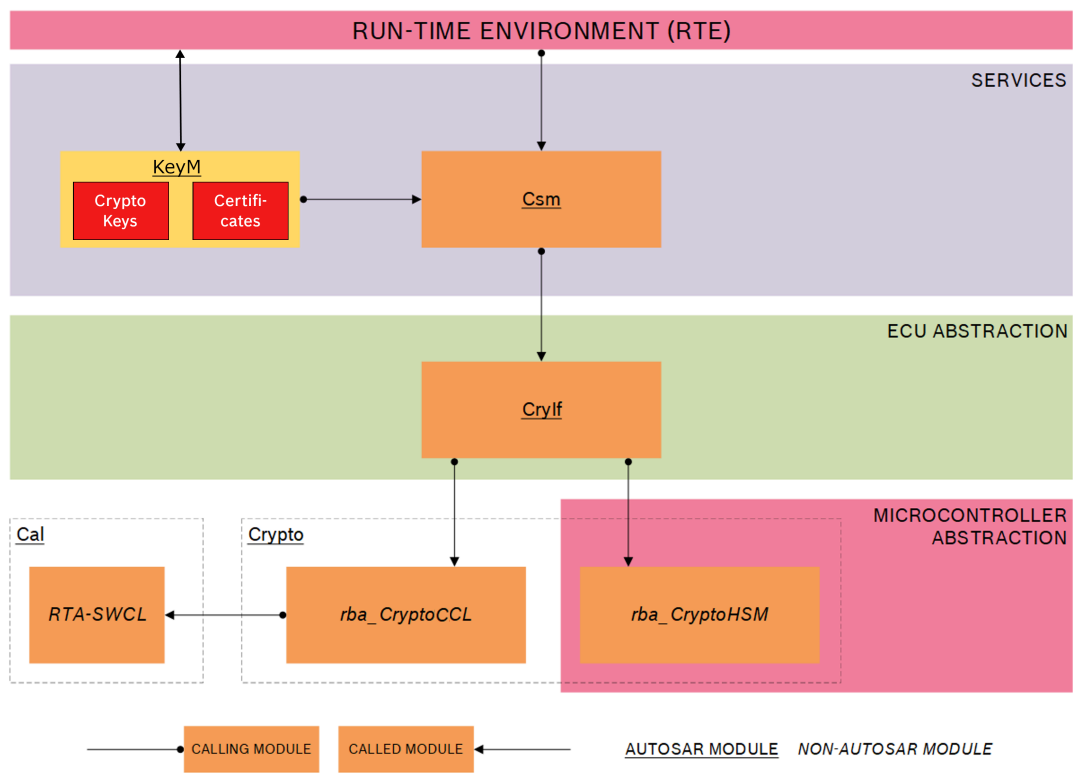


Figure 4.1.KeyManager Module

The AUTOSAR KeyM module consists of two sub modules, the crypto key submodule and the certificate submodule.

The crypto key submodule provides an API and configuration items to introduce or update pre-defined cryptographic key material. It acts as a key client to interpret the provided data from a key server and to create respective key materials. These keys are provided to the crypto service manager. After successful installation of the key material, the application is able to utilize the key for crypto operations.

The certificate submodule provides APIs and configuration to operate on certificates. This allows it to define certificate slots and associate them in a hierarchy as it is used in a PKI. Certificates can be permanently stored like a Root or intermediate certificate(s) so that they can be used to verify a given certificate

against a certificate chain. A certificate submodule will also support OCSP(Online Certificate Status Protocol) staple and CRL(Certificate Revocation List) to check if the certificate under validation is part of the revoked one. Furthermore, the certificate submodule allows to access certificate elements or to verify its contents. Currently KeyM supports x509 and CVC format certificates. Certificate submodule also provides feature to create a certificate signing request called as CSR for user.

Module Configuration

This module is not currently supported by the RTA-BSW Configuration Generator (ConfGen) and must be manually configured.

4.2 Supported Features

4.2.1 KeyM Certificate Handling

- Service certificate(KeyM receives requests for certificate signing, storing of root or intermediate certificates, update root or intermediate certificates, storing of signed certificates through the Update CSR feature.)
- Setting a certificate
- Getting a certificate, including first and next certificate elements and also certificate element by index and count.
- Verifying certificate, verifying one certificate against the other and verifying a certificate chain.
- Verifying OCSP staple for checking the revocation status of x509 certificate.
- Checking the revocation status of x509 certificate through CRL.
- Getting the certificate status.
- Getting the path of a certificate to the root.
- Completely extracting the particular certificate structure(complete ASN1 stream. i.e. Subject or issuer etc.).
- Extracting different elements of a certificate as per configuration. This feature can be used to extract x509, CRL or CVC certificate elements.
- Setting the individual elements for CSR creation.
- KeyM_Rb_CancelCertificateJob Api for cancellation of ongoing Certificate Job.
- KeyM sets the public key of certificates configured after verification is successful. This is done for KeyMCertAlgorithmType [RSA/ECC[ECDSA/ED-DSA]] certificates.
- Security Events reporting to IdsM.

4.2.2 KeyM Crypto Key Operations

- Key Start
- Key Prepare

- Key Update
- Key Finalize
- Key Verify
- Cancel ongoing asynchronous key update and verify job.

4.2.3 KeyM Module Certificates

The KeyM module currently only supports X.509 and CVC certificates.

4.2.4 Persistent Key Storage Handling

For each certificate that is configured to be stored persistently in NvM, a corresponding NvMBlockDescriptor needs to be configured that has sufficient size and the reference of this is to be configured in KeyMNvMBlock container.

For each certificate that is configured to be stored persistently in Csm, a corresponding CsmKey needs to be configured that has sufficient size and the reference of this is to be configured in KeyMCertificateCsmKeyTargetRef in the KeyM-CertificateContainer.

4.2.5 Supported Security Events

KeyM supports following Security Events:

- KEYM_SEV_INST_ROOT_CERT_OP: Attempt to install a root certificate.
- KEYM_SEV_UPD_ROOT_CERT_OP: Attempt to update an existing root certificate.
- KEYM_SEV_INST_INTERMEDIATE_CERT_OP: Attempt to install an intermediate certificate.
- KEYM_SEV_UPD_INTERMEDIATE_CERT_OP: Attempt to update an intermediate certificate.
- KEYM_SEV_CERT_VERIF_FAILED: A request to verify a certificate against a certificate chain was not successful.

Note: It is recommended that until and unless intentional IdsMReportingModeFilter of KeyM relevant IdsMEvent container in IdsM shall have values: DETAILED or DETAILED_BYPASSING_FILTERS. Otherwise even if KeyM sends the Context data, it will be ignored by IdsM for further processing.

4.3 Extensions

4.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

4.3.1.1 KeyM/KeyMCertificate/KeyMCertEncodingType

Short Name	KeyMCertEncodingType
------------	----------------------

Description	Specify in which encoding the certificate will be accepted. If this parameter is not present, then the KeyM will try to detect the encoding type in a vendor specific way from the received certificate itself, e.g., if vendor supports only DER encoding and the certificate is in a different encoding type then KeyM will reject it; if a vendor supports two encoding types, they may decide which one it is, depending on the certificate data.
Multiplicity	0..1
Default Value	-
Enumeration Literals	CUSTOM, DER, PEM
Type	ECUC-ENUMERATION-PARAM-DEF

4.3.1.2 KeyM/KeyMCertificate/KeyMCertEncodingTypeCustom

Short Name	KeyMCertEncodingTypeCustom
Description	Customer specific encoding types. This parameter defines the function name for Custom encoding type to DER converter. Signature of this Api : void KeyM_Priv_ExportCustomToDer(uint8 in_au8[], uint32 inLength_u32, uint8 out_au8[], uint32 *outLength_pu32).
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

4.3.1.3 KeyM/KeyMCertificate/KeyMCertificateElement/KeyMRbCertificateElementCustomIteration

Short Name	KeyMRbCertificateElementCustomIteration
Description	This parameter is to enable custom iterator feature for Api KeyM_CertElementGetFirst/KeyM_CertElementGetNext.
Multiplicity	0..1
Default Value	-
Enumeration Literals	FORD_DAT3_ELEMENT_ITERATOR
Type	ECUC-ENUMERATION-PARAM-DEF

4.3.1.4 KeyM/KeyMCertificate/KeyMCertificateElement/KeyMRbCertificateElementIsCritical

Short Name	KeyMRbCertificateElementIsCritical
------------	------------------------------------

Description	For CSR usecase KeyM sets the isCritical bit to true if this parameter is configured with value true and corresponding CSR Element is set. For Certificate usecase if this parameter is not configured or configured TRUE and actual certificate does not have this extension, in that case KeyM parsing will fail with negative result. User has to explicitly configure this parameter to FALSE for optional certificate elements.
Multiplicity	0..1
Default Value	true
Type	ECUC-BOOLEAN-PARAM-DEF

4.3.1.5 KeyM/KeyMCertificate/KeyMCertificateElement/KeyMRbCertificateElementMaxIterations

Short Name	KeyMRbCertificateElementMaxIterations
Description	Maximum number of Iterative Items which a certificate element can have. This parameter is applicable only when KeyMCertificateElementHasIteration is configured with value true. If not configured it will have size as 1. This parameter affects the internal buffer needs for Certificate elements with iterations..
Multiplicity	0..1
Value Range	1..65535
Default Value	1
Type	ECUC-INTEGER-PARAM-DEF

4.3.1.6 KeyM/KeyMCertificate/KeyMRbCertOcspRef

Short Name	KeyMRbCertOcspRef
Description	This is a reference to OCSP which will be used while validating this certificate.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

4.3.1.7 KeyM/KeyMCertificate/KeyMRbCertificateCallbackUsePort

Short Name	KeyMRbCertificateCallbackUsePort
------------	----------------------------------

Description	If this parameter is set to true, the KeyM uses a port requiring a PortInterface CertificateCallbackServices_{CertificateCallbackName}. If this is false, the KeyM expects to find the names of the functions to be used in KeyMCertificateVerifyCallbackNotificationFunc or KeyMServiceCertificateCallbackNotificationFunc.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

4.3.1.8 KeyM/KeyMCertificate/KeyMRbCertificateGroupId

Short Name	KeyMRbCertificateGroupId
Description	This parameter defines the group identifier from where upper certificate will be searched in case statically configured upper certificate subject is not matching with issuer of this certificate.
Multiplicity	0..1
Value Range	0..255
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

4.3.1.9 KeyM/KeyMCertificateElementVerification/KeyMCertificateElementCondition/KeyMCertificateElementConditionValue/KeyMRbCertificateElementConditionCallOut

Short Name	KeyMRbCertificateElementConditionCallOut
Description	This container contains the configuration of dedicated callouts
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

4.3.1.10 KeyM/KeyMCertificateElementVerification/KeyMCertificateElementCondition/KeyMCertificateElementConditionValue/KeyMRbCertificateElementConditionCallOut/KeyMRbCertificateElementConditionCallOut

Short Name	KeyMRbCertificateElementConditionCallOut
Description	This parameter contains the dedicated API which are called out to fetch the compare value.
Multiplicity	1..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

4.3.1.11 KeyM/KeyMGeneral/KeyMRbCclIterationCount

Short Name	KeyMRbCclIterationCount
Description	This parameter defines the number of calls of ccl iterative functions in background per activation. This will reduce certificate verification interval.
Multiplicity	0..1
Value Range	1..65535
Default Value	1
Type	ECUC-INTEGER-PARAM-DEF

4.3.1.12 KeyM/KeyMGeneral/KeyMRbCertCsmSignatureVerifyJobRef

Short Name	KeyMRbCertCsmSignatureVerifyJobRef
Description	Reference to the CSM job that is used to verify the signature
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

4.3.1.13 KeyM/KeyMGeneral/KeyMRbCertElementLockEnabled

Short Name	KeyMRbCertElementLockEnabled
Description	The lock can be enabled or disabled for certificate element get API's (KeyM_CertElementGet, KeyM_CertElementGetByIndex, KeyM_CertElementGetFirst, KeyM_CertElementGetNext, KeyM_CertificateElementGetCount) with the help of this parameter
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

4.3.1.14 KeyM/KeyMGeneral/KeyMRbCryptoKeyCallbackUsePort

Short Name	KeyMRbCryptoKeyCallbackUsePort
Description	If this parameter is set to true, the KeyM uses a port requiring a PortInterface KeyMCryptoKeyNotification. If this is false, the KeyM will use the Crypto key callback functions.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

4.3.1.15 KeyM/KeyMGeneral/KeyMRbCsrKeyGenerationDisabled

Short Name	KeyMRbCsrKeyGenerationDisabled
Description	KeyM generates the Key pair each time KeyM_ServiceCertificate() service is called with KEYM_SERVICE_CERT_REQUEST_CSR. This parameter is to disable this behaviour. If configured True, KeyM will assume that the key pair is already generated by application and is already set valid for use.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

4.3.1.16 KeyM/KeyMGeneral/KeyMRbQueueAddOn

Short Name	KeyMRbQueueAddOn
Description	KeyM internal Queue size add on for increased queueing capacity of service requests. If not configured KeyM will have queue size as 2.
Multiplicity	0..1
Value Range	1..252
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

4.3.1.17 KeyM/KeyMGeneral/KeyMRbRequestDataMaxLength

Short Name	KeyMRbRequestDataMaxLength
Description	Maximum length in bytes of the request data. This parameter is introduced to provide the maximum size of request data and a provision for giving array size via configuration (e.g. for arrays like KeyM_ServiceCertificate_ArrayDataType) for APIs such as KeyM_Start, KeyM_Prepere, KeyM_Verify, KeyM_Finalize and KeyM_ServiceCertificate. If this parameter is not configured then the default value of 1024 shall be used. If other than 1024 is required, it can be configured as needed by the user.
Multiplicity	0..1
Value Range	1..65535
Default Value	1024
Type	ECUC-INTEGER-PARAM-DEF

4.3.1.18 KeyM/KeyMGeneral/KeyMRbResponseDataMaxLength

Short Name	KeyMRbResponseDataMaxLength
Description	Maximum length in bytes of the response data. This parameter is introduced to provide the maximum size of response data and a provision for giving array size via configuration (e.g. for arrays like KeyM_KeyM-CryptoKeyNotification_CryptoKeyFinalizeCallbackNotification) for APIs like KeyM_Start, KeyM_Prepare, KeyM_Verify, KeyM_Finalize. If this parameter is not configured then the default value of 1024 shall be used. If other than 1024 is required, it can be configured as needed by the user.
Multiplicity	0..1
Value Range	1..65535
Default Value	1024
Type	ECUC-INTEGER-PARAM-DEF

4.3.1.19 KeyM/KeyMGeneral/KeyMRbServerTimeSourceExtObjectId

Short Name	KeyMRbServerTimeSourceExtObjectId
Description	This is the object identifier (OID) that is used to identify the server time source certificate extension.
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

4.3.1.20 KeyM/KeyMGeneral/KeyMRbVersionInfoApi

Short Name	KeyMRbVersionInfoApi
Description	Enables (TRUE) or disables (FALSE) the version Info Api of the key manager. If set to true, the KeyM_VersionInfo() function is available.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

4.3.1.21 KeyM/KeyMGeneral/KeyMRbX509CsmSignatureVerifyEnabled

Short Name	KeyMRbX509CsmSignatureVerifyEnabled
Description	Enables (TRUE) or disables (FALSE) the Signature Verification via csm for x509 certificate.
Multiplicity	0..1

Default Value	true
Type	ECUC-BOOLEAN-PARAM-DEF

4.3.1.22 KeyM/KeyMGeneral/KeyMSecurityEventRefs/KeyMRbCertCsmHashJobRef

Short Name	KeyMRbCertCsmHashJobRef
Description	Reference to a CSM job to calculate a Hash. Note: This parameter is only required if Security event reporting is enabled by configuring at least one of security event.
Multiplicity	1..1
Default Value	-
Type	ECUC-REFERENCE-DEF

4.3.1.23 KeyM/KeyMGeneral/KeyMSecurityEventRefs/SEV_CERT_INTERMEDIATE_INST_REQ

Short Name	SEV_CERT_INTERMEDIATE_INST_REQ
Description	Attempt to install an intermediate certificate. Tags: atp.Status=draft
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

4.3.1.24 KeyM/KeyMGeneral/KeyMSecurityEventRefs/SEV_CERT_INTERMEDIATE_UPD_REQ

Short Name	SEV_CERT_INTERMEDIATE_UPD_REQ
Description	Attempt to update an existing intermediate certificate. Tags: atp.Status=draft
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

4.3.1.25 KeyM/KeyMGeneral/KeyMSecurityEventRefs/SEV_CERT_ROOT_INST_REQ

Short Name	SEV_CERT_ROOT_INST_REQ
Description	Attempt to install a root certificate. Tags: atp.Status=draft
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

4.3.1.26 KeyM/KeyMGeneral/KeyMSecurityEventRefs/SEV_CERT_ROOT_UPD_REQ

Short Name	SEV_CERT_ROOT_UPD_REQ
Description	Attempt to update an existing root certificate. Tags: atp.Status=draft
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

4.3.1.27 KeyM/KeyMGeneral/KeyMSecurityEventRefs/SEV_CERT_VERIF_FAILED

Short Name	SEV_CERT_VERIF_FAILED
Description	A request to verify a certificate against a certificate chain was not successful. Tags: atp.Status=draft
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

4.3.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

4.3.2.1 KeyM_CsrElementSet

Syntax	Std_ReturnType KeyM_CsrElementSet(KeyM_CertificateIdType CertId, KeyM_CertElementIdType CertElementId, KeyM_CsrEncodingType EncodingType, const uint8 *ElementData, uint32 ElementDataLength)
Parameters In	CertId: The identifier of the certificate.
	CertElementId: The identifier of the certificate element.
	EncodingType: The encoding type. If it is encoded then it means it is Extension.
	ElementData: Individual element corresponding to CertElementId of the configuration.
	ElementDataLength: Element data Length.
Parameter Out	ElementDataLength: Element data Length.
Return Value	E_OK: Certificate Element accepted.
	E_NOT_OK: Certificate Element not accepted. Api has to be called again for all Elements.

	KEYM_E_PARAMETER_MISMATCH: Parameters do not match with expected value.
	KEYM_E_KEY_CERT_SIZE_MISMATCH: Parameter size doesn't match.
Description	Verifies the certificate stored in the key referenced by verifyKeyId with the certificate stored in the key referenced by KeyId.

4.3.2.2 KeyM_CertificateElementGetByIndex

Syntax	Std_ReturnType KeyM_CertificateElementGetByIndex(KeyM_CertificateIdType CertId, KeyM_CertElementIdType CertElementId, uint32 Index, uint8* CertElementDataPtr, uint32* CertElementDataLengthPtr)
Parameters In	CertId: Identifier of the certificate where the element shall be read from.
	CertElementId: Specifies the ElementId where the data shall be read from.
	Index: Specifies the index to the element that shall be read (0..N).
Parameter Out	CertElementDataPtr: Pointer to a data buffer allocated by the caller of this function. If the function returns E_OK element data is copied into this buffer.
Parameter InOut	CertElementDataLengthPtr: In: Pointer to a value that contains the maximum data length of the CertElementData buffer.
	CertElementDataLengthPtr: Out: The data length will be overwritten with the actual length of data placed to the buffer if the function returns E_OK.
Return Value	E_OK: Element found and data provided in the buffer.
	E_NOT_OK: Unable to read the element data.
	KEYM_E_PARAMETER_MISMATCH: Invalid certificate ID, element ID invalid or index out of range.
	KEYM_E_KEY_CERT_SIZE_MISMATCH: Provided buffer for the certificate element too small.
	KEYM_E_KEY_CERT_EMPTY: No certificate data available, the certificate is empty.
	KEYM_E_CERT_INVALID: Certificate is not valid or not verified successfully
Description	Function to provide the element data of a certificate. The function is used if an element type can have more than one parameter. The index specifies which element shall be read.

4.3.2.3 KeyM_CertificateElementGetCount

Syntax	Std_ReturnType KeyM_CertificateElementGetCount(KeyM_CertificateIdType CertId, KeyM_CertElementIdType CertElementId, uint16* CountPtr)
Parameters In	CertId: Holds the identifier of the certificate.
	CertElementId: Specifies the certificate element.
Parameter Out	CountPtr: Pointer to the buffer where the number of available data elements for this certificate element shall be copied to.
Return Value	E_OK: Count value has been provided
	E_NOT_OK: Unable to provide the count value.
	KEYM_E_PARAMETER_MISMATCH: Certificate ID or certificate element ID invalid resp. out of range.
Description	This function provides the total number of data elements that are available for the specified certificate element. This function retrieves the total number of elements.

4.3.2.4 KeyM_CertStructureGet

Syntax	Std_ReturnType KeyM_CertStructureGet(KeyM_CertificateIdType CertId, KeyM_CertificateStructureType CertStructure, uint8 *CertStructureData, uint32 *CertStructureDataLength)
Parameters In	CertId: Holds the identifier of the certificate.
	CertStructure: Holds the certificate structure type.
Parameter Out	CertStructureData: Data returned by the function.
Parameter InOut	CertStructureDataLength: In: Max number of bytes available CertStructureData. Out: Actual number.
Return Value	E_OK: Certificate structure was retrieved successfully.
	E_NOT_OK: Due to internal error, the certificate structure could not be retrieved.
	KEYM_E_PARAMETER_MISMATCH : Parameter does not match with expected value.
	KEYM_E_KEY_CERT_SIZE_MISMATCH: Parameter size doesn't match.
	KEYM_E_KEY_CERT_INVALID: The certificate is not valid or has not yet been verified.
Description	Function to provide certificate structure.

4.3.2.5 KeyM_Rb_CancelCertificateJob

Syntax	Std_ReturnType KeyM_Rb_CancelCertificateJob(KeyM_CertificateIdType CertId)
--------	---

Parameter In	CertId: Index of Certificates handled in KeyM.
Return Value	E_OK: Certificate verification cancellation accepted.
	E_NOT_OK: Operation not accepted due to an internal error.
	CRYPTO_E_JOB_CANCELED: Request accepted but will be done in background. Notification is given when done.
Description	Function to cancel the operation which was requested by previous service

4.3.2.6 KeyM_Rb_CsmCustomServiceCallback

Syntax	void KeyM_Rb_CsmCustomServiceCallback(uint32 jobId, Crypto_ResponseType result)
Parameters In	jobId: Job Id for which Signature verify was called.
	result: Result of operation.
Return Value	None
Description	Function is to be configured in CSM job callback for Custom service job.

4.3.2.7 KeyM_Rb_CsmSignatureGenerateCallback

Syntax	void KeyM_Rb_CsmSignatureGenerateCallback(uint32 jobId, Crypto_ResponseType result)
Parameters In	jobId: Job Id for which Signature generate was called.
	result: Result of operation.
Return Value	None
Description	Function is to be configured in CSM job call back for signature job if it is of async type.

4.3.2.8 KeyM_Rb_CsmSignatureVerifyCallback

Syntax	void KeyM_Rb_CsmSignatureVerifyCallback(uint32 jobId, Std_ReturnType result)
Parameters In	jobId: Job Id for which Signature verify was called.
	result: Result of operation.
Return Value	None
Description	Function is to be configured in CSM job call back for signature job if is of Async type.

4.4 Limitations

- NvInitBlockCallback with respect to

KeyMCryptoKeyNvmBlockRef->KeyMNvmBlockDescriptorRef->NvmBlockDescriptor for the initial value *Read* is not supported for the KeyM_Update feature.

- If no subject/attribute elements are configured, then the Certificate Signing Request only generates with the public key, and the signature by the corresponding private key.
- Any item in an extension with an iteration *True* cannot have an individual value > 8 bytes. If they are OID, then its DER-encoded value cannot be > 8 bytes.
- KeyM_MainBackgroundFunction scheduling only in the chain task type background task is supported.
- Currently, the maximum signature length for the key and signature for the CSR usecase is limited to 144. (It will be made generic at a later date.)
- Certificate Signing Request (CSR) for CVC Certificates is not supported.
- KeyM_Init should be called *only* after Csm and its lower layer is fully initialized. Otherwise, KeyM will initialize without the ROOT certificate, even if it is stored in HSM.
- If set root is not successful at the Nvm write stage, then KeyM will give a callback with the result *KEYM_RT_KEY_CERT_WRITE_FAIL/KEYM_RT_KEY_CERT_UPDATE_FAIL*. Internally, however, the certificate status may be *KEYM_CERTIFICATE_VALID*. If so, a reset is required before again trying to call KeyM_ServiceCertificate or KeyM_ServiceCertificateByCertId i.e. for the *KEYM_SERVICE_CERT_SET_ROOT* operation.
- If the public key is set to Csm fail, KeyM will give a callback with the result *KEYM_RT_NOT_OK*. Internally, however, the certificate status may be *KEYM_CERTIFICATE_PARSED_NOT_VALIDATED*. If so, a reset is required before again trying to call KeyM_ServiceCertificate or KeyM_ServiceCertificateByCertId i.e. for the *KEYM_SERVICE_CERT_SET_ROOT* operation.
- If KeyMRbCertificateElementCustomIteration is configured, then the *currentindex_u8* datatype in the KeyM_CertElementIteratorType structure must be initialized as *zero* before the first call of the KeyM_CertElementGetFirst API.
- KeyM provides an RSA Public Key in the *ASN.1 Encoded (DER)* format. Additionally, the CryptoKeyElementFormat parameter must be configured with the value *CRYPTO_KE_FORMAT_BIN_RSA_PUBLICKEY*.
- *Dynamic Grouping* is not supported for the *CRL certificate*.
- *Dynamic Grouping* is not considered while performing a background activity for KeyM_ServiceCertificate{ByCertId} requests i.e. for a CRL update or, while validating a certificate, upper certificate selection using Dynamic Grouping is supported only for the temporary leaf certificate.
- KeyM will support only one responder certificate for each OCSP.
- Identification of a Certificate Revocation Status (CRL) in an OCSP response is based only on the certificate serial number.

- KeyM will verify only the standard certificate elements as per the AUTOSAR specification. If the user wants to do an additional check, it can be done via mode rules.
- KeyM will not consider any default value if it is not present as part of the certificate element e.g. if the *BasicConstraints* extension is present, but the *cA flag* is omitted, then the *cA flag* shall be considered to have the default value FALSE (as per RFC 5280).

4.5 Exclusions

The following features from AR v22-11 are not supported in version v12.7.0.

API functions not currently implemented:

- KeyM_CertElementGetByIndex
- KeyM_CertElementGetCount

The following configuration parameters are not supported in version v12.7.0.

- KeyM/KeyMCertificateElementVerification/KeyMCertificateElementRule/KeyMArgumentRef
- KeyM/KeyMGeneral/KeyMSecurityEventRefs/KEYM_SEV_CERT_VERIF_-FAILED
- KeyM/KeyMGeneral/KeyMSecurityEventRefs/KEYM_SEV_INST_INTERMEDIATE_CERT_OP
- KeyM/KeyMGeneral/KeyMSecurityEventRefs/KEYM_SEV_INST_ROOT_CERT_OP
- KeyM/KeyMGeneral/KeyMSecurityEventRefs/KEYM_SEV_UPD_INTERMEDIATE_CERT_OP
- KeyM/KeyMGeneral/KeyMSecurityEventRefs/KEYM_SEV_UPD_ROOT_CERT_OP

4.6 Deviations

4.6.1 AUTOSAR Deviations

The following deviations should be noted with respect to the stated AUTOSAR requirements.

4.6.1.1 KeyM_CertDataType

[SWS_KeyM_00041] KeyM_CertDataType, with reference to the certData structure type, is (uint8*) instead of the stated AUTOSAR type (void*).

4.6.1.2 Security Event Reporting to IdsM

- If a HASH calculation fails, then HashedID8 of a certificate (8 Byte) will have a dummy value of all 0s.
- In place of certificateIssuerName, KeyMCertificateName

[ECUC_KeyM_00024] is used (first 32 Byte). If the length is not 32 bytes, then the remaining bytes are padded with 0s.

- While Security Event Reporting is in progress, if certificate data is changed then the corresponding HASH value may be inconsistent.

4.6.1.3 OCSF Response Handling

This is only supported for *Leaf/User/End Entity Certificates*.

Rationale: When two intermediate certificates are issued by one root certificate, if the certificates are misused and so revoked, they can still be portrayed as *not* revoked. This is in case one of them is used as a *responder certificate* inside an OCSF response.

4.6.1.4 KeyM_CertElementGet

The following certificate elements are not supported for the KeyM_CertElementGet operation:

- CertificateIssuerUniqueIdentifier
- CertificateSubjectUniqueIdentifier
- CertificateSignatureAlgorithm

4.6.1.5 KeyM_ServiceCertificate

KeyM_ServiceCertificate must have a callback function parameter configured or KeyMRbCertificateCallbackUsePort set to TRUE.

4.6.1.6 KeyM_ServiceCertificate Callback Notification

[SWS_KeyM_00147] The KeyM_ServiceCertificate callback notification, as specified by the AUTOSAR documentation, does not provide a return value. Instead, the return value shall have a Std_ReturnType with *E_OK*:

- To harmonize all callbacks to be of the same types i.e. to make them inline with the KeyM_VerifyCertificate callback notification type.
- RTE will always provide a callback with *E_OK*.

4.6.1.7 Verification of CVC Certificate

Module verification for the CVC certificate is not currently supported for the signature algorithm.

4.6.1.8 KeyMCertificateElementObjectType

The KeyMCertificateElementObjectType parameter floating point data type is not currently supported.

Configuration parameters which differ from the AR v22-11 specification are listed below.

KeyM

[RTA-BSW] Description	Configuration of the KeyM (Key Manager) module.
[AUTOSAR] Description	Configuration of the Mcu (Microcontroller Unit) module.

KeyM/KeyMCertificate/KeyMCertCsmSignatureGenerateJobRef

[RTA-BSW] Description	Reference to a CSM job to calculate a signature. Note: This parameter is only required if for this certificate CSR is to be generated. For CSR usecase it is to be made sure that KeyMCertPrivateKeyStorageCryptoKeyRef parameter is configured in such a way that corresponding key in Crypto driver can hold the key pair. (private and public key part in same key. i.e. CRYPTO_ELEMENT_TYPE_ECDSA_EDD25519_KEYPAIR) Currently for other types it is not supported.
[AUTOSAR] Description	Reference to a CSM job to calculate a signature

KeyM/KeyMCertificate/KeyMCertCsmSignatureVerifyKeyRef

[RTA-BSW] Description	Reference to the CSM key associated to the CSM signature verify job. The Public Key of this certificate shall be set to the key element (CRYPTO_KE_SIGNATURE_KEY) where the key references to.
[AUTOSAR] Description	Reference to the CSM key associated to the CSM signature verify job.

KeyM/KeyMCertificate/KeyMCertFormatType

[RTA-BSW] Enumeration Literals	CRL, OCSP, CVC, X509
[AUTOSAR] Enumeration Literals	CRL, CVC, X509

KeyM/KeyMCertificate/KeyMCertPrivateKeyStorageCryptoKeyRef

[RTA-BSW] Description	Defines a storage location of the private key of a certificate. This parameter is only required for CSR usecase.
[AUTOSAR] Description	Defines a storage location of the private key of a certificate.

KeyM/KeyMCertificate/KeyMCertPublicKeyAlgorithmType

[REMOVED] Introduction	If this parameter is omitted, KeyMCertAlgorithmType shall be used as default value.
------------------------	---

[RTA-BSW] Enumeration Literals	ECC, RSA, ECC_COMPRESS_INFO_FROM_KEY, ECC_COMPRESS_INFO_FROM_KEY_EVEN
[AUTOSAR] Enumeration Literals	ECC, ECC_COMPRESS_INFO_FROM_KEY, RSA

KeyM/KeyMCertificate/KeyMCertUpperHierarchicalCertRef

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

KeyM/KeyMCertificate/KeyMCertificateCustomService

[RTA-BSW] Parent Container	KeyM/KeyMCertificate/KeyMCertificateCustomService
[AUTOSAR] Parent Container	KeyM/KeyMCertificate/KeyMCertificateElement/KeyMCertificateCustomService
[RTA-BSW] Description	The presence of this container defines the custom processing for this certificate. Currently this is disabled.
[AUTOSAR] Description	The presence of this container defines the custom processing for this certificate.
[RTA-BSW] Upper Multiplicity	0
[AUTOSAR] Upper Multiplicity	1

KeyM/KeyMCertificate/KeyMCertificateCustomService/KeyM-CertCsmCustomJobRef

[RTA-BSW] Parent Container	KeyM/KeyMCertificate/KeyMCertificateCustomService/KeyM-CertCsmCustomJobRef
[AUTOSAR] Parent Container	KeyM/KeyMCertificate/KeyMCertificateElement/KeyMCertificateCustomService/KeyM-CertCsmCustomJobRef

KeyM/KeyMCertificate/KeyMCertificateElement/KeyMCertificateElementObjectType

[RTA-BSW] Description	Certificate elements are stored in ASN.1 format. In this item the type of ASN.1 TLV can be specified (e.g. INTEGER has the value '2'). This can be used to identify only such certificate elements. If the type is different, the element is not included in the search. This parameter also helps identifying suitable logical operation in the KeyMCertElementConditionType parameter w.r.t. corresponding KeyMCertificateElementRef.
[AUTOSAR] Description	Certificate elements are stored in ASN.1 format. In this item the type of ASN.1 TLV can be specified (e.g. INTEGER has the value '2'). This can be used to identify only such certificate elements. If the type is different, the element is not included in the search.

KeyM/KeyMCertificate/KeyMCertificateElement/KeyMCertificateElementOfStructure

[RTA-BSW] Enumeration Literals: CertificateAttributeName, CertificateExtension, CertificateIssuerName, CertificateIssuerUniqueIdentifier, CertificateSerialNumber, CertificateSignature, CertificateSignatureAlgorithm, CertificateSignatureAlgorithmID, CertificateSubjectAuthorization, CertificateSubjectName, CertificateSubjectPublicKeyInfo_PublicKeyAlgorithm, CertificateSubjectPublicKeyInfo_SubjectPublicKey, CertificateSubjectUniqueIdentifier, CertificateValidityPeriodNotAfter, CertificateValidityPeriodNotBefore, CertificateVersionNumber, RevokedCertificates

[AUTOSAR] Enumeration Literals: CertificateExtension, CertificateIssuerName, CertificateIssuerUniqueIdentifier, CertificateSerialNumber, CertificateSignature, CertificateSignatureAlgorithm, CertificateSignatureAlgorithmID, CertificateSubjectAuthorization, CertificateSubjectName, CertificateSubjectPublicKeyInfo_PublicKeyAlgorithm, CertificateSubjectPublicKeyInfo_SubjectPublicKey, CertificateSubjectUniqueIdentifier, CertificateValidityPeriodNotAfter, CertificateValidityPeriodNotBefore, CertificateVersionNumber, RevokedCertificates

KeyM/KeyMCertificateElementVerification/KeyMCertificateElementCondition

[RTA-BSW] Introduction	One KeyMCertificateElementCondition shall contain either one KeyMCertificateElementConditionArray or one KeyMCertificateElementConditionCertificateElement or one KeyMCertificateElementConditionPrimitive or one KeyMCertificateElementConditionSenderReceiver.
[AUTOSAR] Introduction	One KeyMCertificateElementCondition shall contain either one KeyMCertificateElementSwcCallback or one KeyMCertificateElementSwcSRDataElementRef or one KeyMCertificateElementSwcSRDataElementValueRef.

[RTA-BSW] Upper Multiplicity	65535
[AUTOSAR] Upper Multiplicity	*

KeyM/KeyMCertificateElementVerification/KeyMCertificateElementCondition/KeyMCertificateElementConditionValue/KeyMCertificateElementConditionArray/KeyMCertificateElementConditionArrayElement

[RTA-BSW] Upper Multiplicity	65535
[AUTOSAR] Upper Multiplicity	*

KeyM/KeyMCertificateElementVerification/KeyMCertificateElementCondition/KeyMCertificateElementRef

[RTA-BSW] Description	Reference to a certificate element used for the condition. According to Autosar this parameter has multiplicity 0-1. But 1 multiplicity only makes sense.
[AUTOSAR] Description	Reference to a certificate element used for the condition.
[RTA-BSW] Lower Multiplicity	1
[AUTOSAR] Lower Multiplicity	0

KeyM/KeyMCertificateElementVerification/KeyMCertificateElementRule

[RTA-BSW] Upper Multiplicity	65535
[AUTOSAR] Upper Multiplicity	*

KeyM/KeyMGeneral/KeyMEnableSecurityEventReporting

[RTA-BSW] Description	Switches the reporting of security events to the IdsM: - true: reporting is enabled. - false: reporting is disabled.
[AUTOSAR] Description	Switches the reporting of security events to the IdsM: Tags: atp.Status=draft
[REMOVED] Introduction	- true: reporting is enabled. - false: reporting is disabled.

KeyM/KeyMGeneral/KeyMSecurityEventRefs

[RTA-BSW] Description	Container for the references to IdsMEvent elements representing the security events that the KeyM module shall report to the IdsM in case the corresponding security related event occurs. The standardized security events in this container can be extended by vendor-specific security events. Tags: atp.Status=draft
[AUTOSAR] Description	Container for the references to IdsMEvent elements representing the security events that the KeyM module shall report to the IdsM in case the corresponding security related event occurs (and if KeyMEnableSecurityEventReporting is set to "true"). The standardized security events in this container can be extended by vendor-specific security events. Tags: atp.Status=draft

KeyM/KeyMGeneral/KeyMServiceCertificateFunctionEnabled

[RTA-BSW] Description	Enables (TRUE) or disables (FALSE) the certificate service function of the key manager. If set to true, the KeyM_ServiceCertificate() function has to be called accordingly.
[AUTOSAR] Description	Enables (TRUE) or disables (FALSE) the certificate service function of the key manager. If set to true, the KeyM_ServiceCertificate[ByCertId] () function has to be called accordingly.

4.7 API Definition

List of supported API functions.

KeyM_Init	Initializes the KeyM management module.
KeyM_Deinit	This function resets the key management module to the uninitialized state.
KeyM_GetVersionInfo	Returns the version information of this module.
KeyM_Start	Optional function and only used if the configuration item KeyMCryptoKeyStartFinalizeFunctionEnabled is set to true.
KeyM_Prepare	Used to prepare a key update operation with the main intention to provide information for the key operation to the key server.
KeyM_Update	Initiates the key generation or update process.
KeyM_Finalize	Finalizes key update operations.
KeyM_Verify	Sends requests to the key server to verify the provided keys.
KeyM_ServiceCertificate	Used when the key server requests an operation from the key client.

KeyM_SetCertificate	Provides the certificate data to the key management module to temporarily store the certificate.
KeyM_GetCertificate	Provides the certificate data.
KeyM_VerifyCertificates	Verifies two certificates that are stored and parsed internally against each other.
KeyM_VerifyCertificate	Verifies a certificate that was previously provided with KeyM_SetCertificate against already stored and provided certificates stored with other certificate IDs.
KeyM_VerifyCertificateChain	Performs a certificate verification against a list of certificates.
KeyM_CertElementGet	Provides the content of a specific certificate element.
KeyM_CertElementGetFirst	Initializes the interactive extraction of a certificate data element.
KeyM_CertElementGetNext	Provides further data from a certificate element.
KeyM_CertGetStatus	Provides the status of a certificate.
KeyM_MainFunction	Called periodically according the specified time interval.
KeyM_MainBackgroundFunction	Called from a pre-emptive operating system when no other task operation is needed.

For details on using these calls, please refer to AUTOSAR_SWS_KeyManager.pdf (version v22-11).

4.8 Configuration Advice

4.8.1 Module Integration

The integration of the following libraries rba_CCL, StbM and Csm is the prerequisite for using KeyM.

4.9 Integration Advice

4.9.1 Init Functions

KeyM is initialized via KeyM_Init function.

KeyM_Init should only be called after the CSM (Crypto Service Manager) and its lower layer modules are fully initialized. Otherwise KeyM will initialize with unavailable root certificates if said certificates are stored as CsmKeys.

KeyM_Init should only be called after the CSM (Crypto Service Manager) and its lower layer are fully initialized. Otherwise KeyM will initialize with the ROOT certificate which is not available even if it is stored in the Hardware Security Module (HSM).

During initialization, the certificate submodule will retrieve the permanently stored certificates, these will be prepared for parsing and made available on demand after validating, e.g. for certificate element extraction or verification against other certificates.

Before KeyM is initialized all the CryptoStack components as well as NvM which have Init functions have to be called, then NvM_ReadAll also needs to be performed before KeyM_Init so persistent data can be retrieved from non-volatile memory.

**NOTE**

Note that it is not possible to request any of the KeyM services before the module is completely initialized (i.e. with no errors). KeyM initialization takes time, with the first part of the Initialization taking place in KeyM_Init then the remaining process is completed in further successive calls of the KeyM_MainBackgroundFunction and KeyM_MainFunction. Once the initial parsing and validation of certificates is complete followed by the loading of permanent keys, KeyM moves its state to Init finished.

4.9.2 Delnit Functions

KeyM is de-initialized via KeyM_Deinit function.

A shutdown phase is not required for the KeyM module which also improves the security.

**NOTE**

It is strictly prohibited to run a Deinit() → Init cycle without a reset.

4.9.3 Scheduling Aspects

KeyM has 4 functions which needs Integrators attention when scheduling.

KeyM_Init and all other initialization functions e.g. from Csm or CryptoDrivers have to be called first before calling any KeyM specific service.

- KeyM_Init shall only be called after Csm and NvM initialization in the EcuM.
- KeyM_MainFunction should be scheduled in a fixed periodic task of the OS. i.e. 10ms.
- The timing value of the KeyMMainFunctionPeriod of KeyMGeneral container has to be considered in configuration, see Autosar Specifications and Design Documents for further information.
- KeyM_MainBackgroundFunction should be scheduled in a background task or any other periodic but repetitive task of OS. It must be called as frequently as possible so that the service request from KeyM is more responsive.
- KeyM_Deinit should be called by EcuM during Ecu Shutdown phase.

4.9.4 KEYM_SERVICE_CERT_SET_ROOT Operation

If the set root is not successful at NvM write stage then KeyM will give callback with the result KEYM_RT_KEY_CERT_WRITE_FAIL or KEYM_RT_KEY_CERT_UP-

DATE_FAIL. Internally the certificate status maybe KEYM_CERTIFICATE_VALID, so a reset should be performed and retry calling KeyM_ServiceCertificate or KeyM_ServiceCertificateByCertId Api.

If public key set to Csm fail. KeyM will give callback with result KEYM_RT_NOT_OK, but internally the certificate status may be KEYM_CERTIFICATE_PARSED_NOT_VALIDATED, therefore a reset is required to retry calling KeyM_ServiceCertificate or KeyM_ServiceCertificateByCertId Api. i.e. for KEYM_SERVICE_CERT_SET_ROOT Operation.

4.9.5 KeyMRbCertificateElementIsCritical

If the configuration parameter KeyMRbCertificateElementIsCritical is not configured or configured TRUE and the actual certificate does not have this extension, then the KeyM parsing will fail with negative result. This parameter should be explicitly configured to FALSE for the optional certificate elements.

4.9.6 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The KeyM module uses the following OS resources:

- KeyM
- KeyM

4.10 Hardware Requirements

Table 4.55. KeyM Hardware Requirements

Support	Usage
FLASH	23224 bytes
RAM	30610 bytes

Usage calculated based on a sample configuration.

5 Intrusion Detection System Manager

Module name	IdsM
Package	RTA-SEC
AUTOSAR Module ID	108
AUTOSAR Version	22-11

5.1 Introduction

In the AUTOSAR Classic platform (ARC), Intrusion detection system manager (IdsM) is a distributed software component that serves as an on-board manager of security events (SEVs). Besides collecting, the IdsM has the capability of qualifying SEVs according to configurable filtering and forwarding rules. IdsM is housed in the security service package (SecServices) and further into the crypto stack (CryptoStack) as shown in the figure.

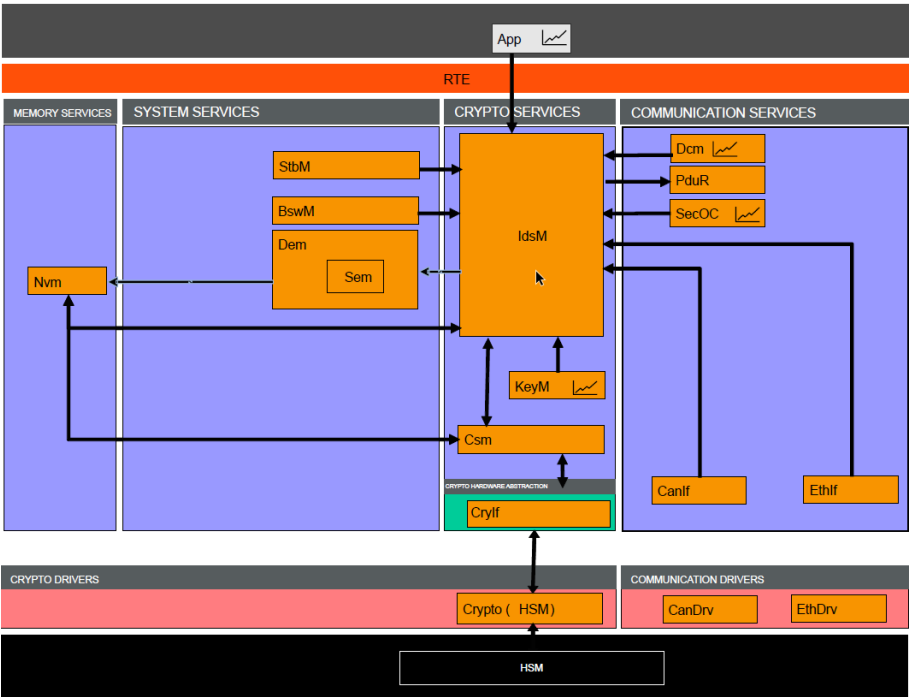


Figure 5.1. Interaction of the IdsM with other stack modules

IdsM characterizes a security event by the following:

- External event ID - An ID derived from the SecXT that helps in identifying the security event. The AUTOSAR standardized external event IDs range between 0x0000 and 0x7FFF (inclusive). Generic user defined security events can have IDs in the range 0x8000 - 0xFFFFE (inclusive). The external event ID in combination with the sensor instance ID can be used to uniquely identify a security event.
- Sensor instance ID - An ID assigned to a sensor generating the SEv to iden-

tify its source.

- Reporting mode - The reporting mode of the security event determines if the reported event has to be processed and if so with/without context data or filter chain.
- Internal event ID - An ID generated in the software that uniquely identifies every security event for internal handling and for symbolic name generation purposes. The symbolic names shall be used by BSW SWCs to report the security events.
- Filter chain - A reference to a filter chain that is to be applied on the security event before labelling it a qualified security event. Chapter 4 outlines filter chains more in detail.
- Sink IdsR - A boolean parameter indicating if the qualified security event has to be transmitted to the intrusion detection system reporter (IdsR).
- Sink Dem - A boolean parameter indicating if the qualified security event has to be locally persisted in the diagnostic event memory (Dem), a.k.a, Security event memory (Sem).
- Additional reporting parameter option - AUTOSAR uses this parameter to purely generate the service interfaces with count and timestamp thereby restricting the usage only to user defined security events (service interfaces are defined only for events with external event ID in range 0x8000 - 0xFFFFE). The configuration parameter in this SW implementation is used for all security events, i.e., if IdsMAdditionalParameterOption is CountTimestamp and the event is reported without a timestamp then IdsM will request the configured IdsMTimestampOption (StbM or Custom) and add it to the reported event. This addition to AUTOSAR requirements unifies the handling of security events inside of IdsM.
- Maximum context data size - AUTOSAR uses this parameter to purely generate generate the service interfaces with context data thereby restricting the usage only to user defined security events (service interfaces are defined only for events with external event ID in range 0x8000 - 0xFFFFE). The configuration parameter in this SW implementation is used for validating the context data size reported. SWS_IdSM_02107 is modified to validate the reported contextDataSize is compared against configured IdsMEventMaxContextDataSize and not directly with the maximum available buffer size (IdsMContextDataBufferSize). The configuration validator scripts make this comparison and throw an error when IdsMEventMaxContextDataSize is greater than the largest available context data buffer (IdsMContextDataBufferSize).

For software components in BSW and ASW to report a security event, ARC IdsM provides the following 6 public APIs for different granularity of data.

API	Context Data	Count	Timestamp
IdsM_SetSecurityEvent	-	-	-
IdsM_SetSecurityEventWithContextData	Yes	-	-
IdsM_SetSecurityEventWithCount	-	Yes	-
IdsM_SetSecurityEventWithCountContextData	Yes	Yes	-
IdsM_SetSecurityEventWithTimestampCount	-	Yes	Yes
IdsM_SetSecurityEventWithTimestampCountContext-Data	Yes	Yes	Yes

Module Configuration

This module can be generated by the RTA-BSW Configuration Generator.

For any unsupported automatic configuration, the module can be manually configured by the user.

5.2 Supported Features

5.2.1 Reporting and Buffering of Security Events

IdsM supports reporting of security events for BSW and ASW SWCs with the following reporting modes.

- OFF - Reported security event is dropped.
- BRIEF - Security event is processed without context data. Any reported context data is discarded. The configured filter chain is applied and then qualification is determined.
- BRIEF_BYPASSING_FILTERS - Security event is processed without context data. Any reported context data is discarded. The configured filter chain is ignored and the security event is directly qualified.
- DETAILED - Security event is processed with reported context data. The configured filter chain is applied and then qualification is determined. If no context data is reported, the security event is handed over to the filter chain without a context data.
- DETAILED_BYPASSING_FILTERS - Security event is processed with reported context data. The configured filter chain is ignored and the security event is directly qualified. If no context data is reported, the security event is directly qualified.
- Buffering of security events - IdsM buffers the received security events in the event buffers and the respective context data in the context data buffers. IdsM creates a link between the Event Buffer and the Context Data

Buffer to associate the security events to their context data.

- Event buffers - A Security Event exists in the context data for as long as the ldsM Main function period. The number of security event buffers required is determined by the peak count of security events generated by all the configured security sensors between two consecutive ldsM Main functions. This peak count represents the maximum possible load on the system and provides a worst-case scenario estimate for buffer requirements.

However, such a peak scenario is unlikely in most cases, as security events typically occur sporadically. Therefore, it is recommended to find a compromise with memory resource availability when configuring event buffers. This involves assessing the average event generation rate, historical data, and expected sensor activity patterns to allocate buffers in a manner that optimizes performance without overburdening the system's memory resources. Properly balancing these factors ensures that the system remains efficient and responsive, even during unexpected surges in security event generation.

- Context data buffers - Unlike configuring the number of event buffers, setting up context data buffers is more complex. This complexity arises from the fact that the lifespan of the context data in the context data buffers is longer than the period of an ldsM Main function due to the aggregation buffer. Additionally, context data buffers can be configured in different sizes, adding to the complexity.

To configure the context data buffer count, it is recommended that group events are configured to have the same or similar-sized context data. Determine the various sizes of the context data buffers required for each group. Once this grouping is done, calculate the amount of context data buffers necessary for each group of security events based on the peak event generation rate and the possible aggregation duration. Follow these steps to ensure proper configuration:

1. Group Similar Events - Rationale - efficient buffer management. Identify events that share the same or similar context data sizes.
 2. Determine Buffer Sizes - Rationale - This ensures that each buffer can adequately store the context data without wasting memory. For each group, determine the required sizes of the context data buffers.
 3. Calculate Buffer Count - Rationale - Ensure that enough buffers are allocated to handle the peak load without causing data loss or memory overflow. For each group of security events, calculate the number of context data buffers needed. Use the peak event generation rate and the expected aggregation duration to determine the buffer count.
- Qualified security event buffers - The qualified events generated by ldsM are buffered in the qualified event buffers until they are transferred to either the transmission sink or the security event memory sink. The number of qualified event buffers required is determined by the ratio between the peak count of security events qualified by ldsM and the rate at which the sinks can drain

these buffers.

Calculating the peak number of security events that IdsM could qualify in a single Main function is not recommended, due to its theoretical nature and unpredictability caused by the filter chains between reception and qualification.

Instead, users should assume the peak qualification count to be equal to the event buffer count and determine the buffer needs based on the sinking delays, such as the delay between the transmission trigger and transmission confirmation. This approach ensures that the buffers are adequately sized to handle the qualified events without causing data loss or system inefficiency.

5.2.2 Application of Filters

The following application of filters on reported security events are supported:

Block State Filter

The block state filter represents a list of states in which security events shall be blocked from qualification. If the ECU state reported by BswM over IdsM_BswM_StateChanged, matches at least one of the block states in the block state filter of the filter chain, then all security events utilizing that filter chain are dropped when reported.

Sampling Filter

The sampling filter reduces the throughput of qualified security events by a configurable factor of "N". Every "N" reported security event of a particular ID is qualified.

Aggregation Filter

The aggregation filter reduces the throughput of qualified security events from reported security events within a configurable interval of time "t". Within a time interval of "t", also known as the aggregation time interval. A security event of a particular ID is qualified at most once (if it was reported during the interval).

The aggregation filter will add the counts of the unique reported security events and generates a qualified security event at the end of the aggregation interval with the sum of the counts aggregated. The context data of this aggregated event can be configured to belong to either the first or the last event from the interval. The timestamp always belongs to the same event as the context data. The aggregation filter does not forward any event during the aggregation interval.

- Enhancement 1: IdsM is capable of aggregating context data of security events in addition to the AR specified count. The maximum number of context data to be aggregated is specified over the configuration parameter IdsMRbNumberOfContextDataAggregated.

With the aggregation of the context data, comes the variance for handling dropping of context data when aggregating threshold (IdsMRbNumberOfContextDataAggregated) is reached. The configuration IdsMContextDataSelector is extended for this purpose.

- If IdsMContextDataSelector is *FIRST*, then the aggregation filter will be

aggregated with the context data belonging to the first N events reported within the aggregation interval, where $N = \text{IdsMRbNumberOfContextDataAggregated}$.

- If `IdsMContextDataSelector` is *LAST*, then the aggregation filter will be aggregated with the context data belonging to the last N events reported within the aggregation interval, where $N = \text{IdsMRbNumberOfContextDataAggregated}$.
- If `IdsMContextDataSelector` is *LAST_RANDOM_REPLACE*, the the aggregation filter will be aggregated with the context data belonging to N events reported within the aggregation interval by replacing an older stored context data at random, where $N = \text{IdsMRbNumberOfContextDataAggregated}$. `IdsM` interfaces with `Csm` to generate random numbers to achieve this behavior.
- Enhancement 2: `IdsM` is capable of aggregating over the power-on cycles with configuration parameter `IdsMRbAggregateOverPowerOnCycles` set to *True*, i.e., the aggregation interval can span over multiple power-on on cycles.

Enabling this feature would require an additional process to be scheduled, i.e., a clean-up process `IdsM_Rb_CleanUpService` has to be scheduled during system shut down before `NvM_WriteAll` is called or during system init after `IdsM_Init`. This process prevents orphan data to occupy the context data buffers. The below sequence diagrams (`IdsM_Rb_CleanUpService_Init` and `IdsM_Rb_CleanUpService`) show the same.

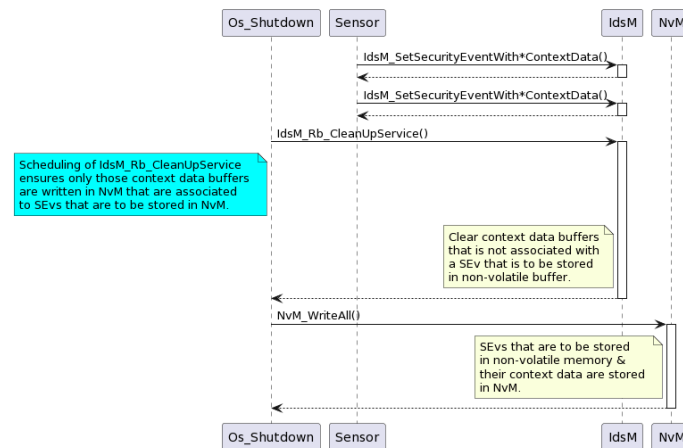


Figure 5.2: Sequence Diagram for `IdsM_Rb_CleanUpService` During OS Shutdown

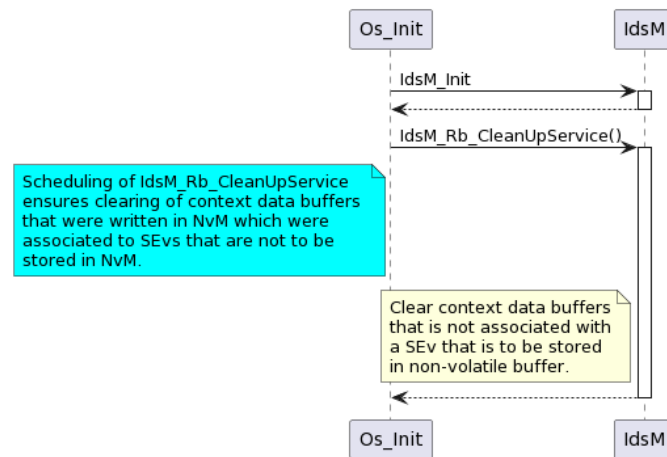


Figure 5.3. Sequence Diagram for IdsM_Rb_CleanUpService During OS Init

Threshold Filter

The threshold filter reduces the throughput of the qualified security events from the reported security events within a configurable interval of time “t” by a count of “N”. Within the time interval “t”, also known as the threshold time interval. A security event of a particular ID has to be reported at least “N” times in order to generate a qualified security event. If it is reported “M” times where $M > N$, then there are $(M - N + 1)$ qualified security events generated.

- Enhancement 1: The directions of the thresholding done by the event threshold filter is made configurable over the parameter **IdsMRbEvent-ThresholdDirection**. This parameter allows the filter to forward from-N events (AR behavior) or upto-N events.

Custom Filter

IdsM offers a possibility to insert a custom filter to allow projects to implement their own custom filtering logic, abiding by a basic set of rules. The definition of the function lays in the responsibility of the project configuring it. The function prototype is available over **IdsM.h**. The function prototype is generated based on the configured custom filter name in **IdsMRbCustomFilterFunction**. There can be up to 100 custom filters configured. Documenting the custom filter brings out SW details that are not required to be a part of this document however this is an exceptional case therefore mentions to type definitions and function names are inevitable. The custom filter shall satisfy the following

- Return value shall be a boolean. The return value shall be **TRUE** if the processed event is to be dropped. **FALSE** if processed event is to be passed to the next filter or to be qualified. This satisfies the high level AUTOSAR requirements to filters (**SWS_IdsM_00904** and **SWS_IdsM_00905**).
- First argument of the function prototype is **IdsM_Event_Config_tst**. This shall NOT be modified by the user. It is only for reading purposes.

Currently this variable is in the RAM however this would be moved to the ROM in the future. This type definition can be included over `IdsM.h`.

- Second argument of the function prototype is `IdsM_EventBuffer_tst`. Modification of elements of this structure is allowed as a part of the implemented filtering algorithm. However failures arising in `IdsM` due to modification of this structure shall be the responsibility of the project. This type definition can be included over `IdsM.h`.
- Calling of `IdsM_ClearLinkedContextDataBuffer` to clear associated context data during dropping is in the responsibility of the function, e.g., `(void) IdsM_ClearLinkedContextDataBuffer(eventBuffer_pst)`.

5.2.3 Filter Chains

Filter chains can be built with the aforementioned filters and these filter chains can then be associated to security events. A filter chain must contain at least one filter and at most 104 filters (this section will later clarify how the number 104 is calculated).

The order of the filters in the filter chain is standardized by AUTOSAR (SWS_IdsM_01004) to be a block state filter, sampling filter, aggregation filter and then the threshold filter.

With the introduction of the custom filters and with the fixed ordering from AUTOSAR this is very restrictive and does not provide room to realize the full flexibility. To overcome this, additional configuration parameters have been added to re-sequence the filter chain (`IdsMRbFilterChainSequence`).

A sequence number can be assigned to every filter (`IdsMRbBlockStateFilterSequenceNumber`, `IdsMRbForwardEveryNthFilterSequenceNumber`, `IdsMRbEventAggregationFilterSequenceNumber`, `IdsMRbEventThresholdFilterSequenceNumber` and `IdsMRbCustomFilterSequenceNumber`). This sequence number is valid for the entire `IdsM` instance. Since there can be up to 100 custom filters configured, the sequence numbers of the filters can be in the range of 1 to 104 (4 AUTOSAR filter + 100 custom filters).

5.2.4 Calibration of Filters

`IdsM` provides the ability to calibrate filter chains and global filters over calibration parameters visible in the A2L. The calibration feature is optional that can be enabled over an array of parameters under `IdsMRbCalibrationSupport` and also individually for each filter type in the filter chain.

The calibration allows the user to enable/disable filters in a filter chain, enable/disable global filters and calibrate the filter parameters of each filter.

5.2.5 Calibration Parameters

The filter parameters inside a filter chain are made conditionally calibratable that can be enabled or disabled by configuration. When calibration is enabled, the generated calibration variables for a filter chain are initialised with configured filter parameters from corresponding `IdsMFilterChain` (see table).

Config Parameter to Enable Calibration	Calibration Label	Description	Configurational Equivalent
IdsMRbBlockStateFilter-CalibrationSupport	IdsM_<FilterChainShortName>_isBlockStateFilterEnabled_C	Enable or disable block state filter in this filter chain	Configuration of IdsMBlock-StateFilter
	IdsM_<FilterChainShortName>_maskBlockStateActivation_C	The block state filter's activation mask for this filter chain	A bitwise combination of IdsM-BlockStateReference
IdsMRbSamplingFilterCalibrationSupport	IdsM_<FilterChainShortName>_isForwardEveryNthFilterEnabled_C	Enable or disable sampling filter in this filter chain	Configuration of IdsMForwardEveryNthFilter
	IdsM_<FilterChainShortName>_nrNthParameter_C	The sampling factor IdsM-NthParameter value for this filter chain	Value of IdsMNthParameter
IdsMRbThresholdFilter-CalibrationSupport	IdsM_<IdsMFilterChain.shortName>_isEventThresholdFilterEnabled_C	Enable or disable event threshold filter in this filter chain	Configuration of IdsMEvent-ThresholdFilter
	IdsM_<IdsMFilterChain.shortName>_nrEventThreshold_C	Number of events threshold for event threshold filter in this filter chain	Value of IdsMEventThreshold-Number
	IdsM_<IdsMFilterChain.shortName>_tiEventThresholdInterval_C	Time interval in seconds for event threshold filter in this filter chain	Value of IdsMEventThreshold-TimeInterval
	IdsM_<IdsMFilterChain.shortName>_swtEventThresholdDirection_C	The filtering direction for event threshold filtering for this filter chain	Value of IdsMRbEventThreshold-Direction

IdsMRbAggregationFilter-CalibrationSupport	IdsM_<IdsMFilterChain.shortName>_isEventAggregationFilterEnabled_C	Enable or disabled event aggregation filter in this filter chain	Configuration of IdsMAggregationFilter
	IdsM_<IdsMFilterChain.shortName>_nrContextDataAggregated_C	Number of context data to be aggregated	Value of IdsMRbNumberOfContextDataAggregated
	IdsM_<IdsMFilterChain.shortName>_tiEventAggregationInterval_C	Time interval in seconds for event aggregation filter in this filter chain	Value of IdsMEventAggregationTimeInterval
	IdsM_<IdsMFilterChain.shortName>_swtEventAggregationContextDataSourceSelector_C	The context data source selector for this filter chain	Value of IdsMContextDataSourceSelector
IdsMRbCustomFilterCalibrationSupport	IdsM_<IdsMFilterChain.ShortName>_is<IdsMRbCustomFilter.shortName>FilterEnabled_C	Enable or disable this custom filter in this filter chain	Configuration of IdsMRbCustomFilters
IdsMRbReportingMode-CalibrationSupport	IdsM_<ShortNameofIdsMEvent>stReportingMode_C	Enable or disable reporting mode in this event	Value of IdsMReportingModeFilter
IdsMRbRateLimitationFilterCalibrationSupport	IdsM_isEventRateLimitationFilterEnabled_C	Enable or disable event rate limitation filter	Configuration of IdsMFilterEventRateLimitation
	IdsM_nrMaxEventRateLimitation_C	The maximum number of SEVs that are passed on by event rate limitation filter	Value of IdsMRateLimitationMaximumEvents
	IdsM_tiEventRateLimitationInterval_C	The time interval in seconds for event rate limitation filter	Value of IdsMRateLimitationTimeInterval

IdsMRbTrafficLimitationFilterCalibrationSupport	IdsM_isTrafficLimitationFilterEnabled_C	Enable or disable Traffic limitation filter	Configuration of IdsMFilterTrafficLimitation
	IdsM_nrMaxBytesTrafficLimitation_C	The maximum number of bytes transmitted	Value of IdsMTrafficLimitation-MaximumBytes
	IdsM_tiTrafficLimitationInterval_C	The time interval for the traffic limitation filter	Value of IdsMTrafficLimitation-TimeInterval

5.2.6 Qualified Security Events

When a reported security event is not dropped by a filter in the associated filter chain or the event does not have a filter chain associated, the event is considered qualified. The qualified security event is destined towards the configured sinks (IdsMSinkIdsR and IdsMSinkDem). Before arriving at the sinks, the qualified security event is packed into a standard structure defined in SWS_IdsM_01200. With a packed qualified security event referred to as the QSEV datagram.

5.2.7 Buffering Qualified Security Events

The packed QSEv datagram is forwarded to the sinks such as PduR and Dem. These interfaces are not necessarily synchronous, e.g., a transmission requested is placed to PduR over the API PduR_IdsMTransmit and upon successful or unsuccessful transmission there is a confirmation provided over IdsM_TxConfirmation.

Additionally a QSEv can be configured to include a signature which could be obtained over an asynchronous Csm job. To better utilize resource and to parallelize these actions, a QSEv buffering is introduced. A generated QSEv datagram can be buffered until all the pending actions such as signature, Dem storage and PduR transmissions are concluded either successfully or unsuccessfully. The QSEv buffer has a depth configured over IdsMNumberOfQualifiedEventBuffers.

The number of qualified event buffers required is determined by the ratio between the peak count of security events qualified by IdsM and the rate at which the sinks can drain these buffers. Calculating the peak number of security events that IdsM could qualify in a single main function is not recommended due to its theoretical nature and unpredictability caused by the filter chains between reception and qualification. Instead, users should assume the peak qualification count to be equal to the event buffer count and determine the buffer needs based on the sinking delays, such as the delay between the transmission trigger and transmission confirmation. This approach ensures that the buffers are adequately sized to handle the qualified events without causing data loss or system inefficiency.

5.2.8 Global Rate Limitation Filters

The packed QSEv datagram has to pass the global rate limitation filters before it is handed over to PduR for transmission. If the global filters decide to drop a QSEv, no PduR relevant actions are performed for the QSEv however the other configured sinks are still actively addressed.

If the internal event is raised when current traffic exceeds a configured traffic limitation, the data will remain in the QSEv buffer until the IdsMTrafficLimitationTimeInterval is reached.

5.2.9 Transmission of QSEv

The QSEv datagram is an IDS message specified in AUTOSAR by the Intrusion Detection System Manager Protocol Specification. The IDS message has an event frame, a timestamp frame, a context data frame and a signature frame, of which only the event frame is mandatory. With AR24-11, the standard was made

backwards incompatible (IDS protocol version 1 > 2) to introduce versioning of context data. In short, a security sensor is expected to version its context data to introduce a lifecycle for security events. The context data version is reported to the IdsM, along with the security event that later gets included in the IDS message (see image below). The implementation allows the security sensors to use interfaces from the AR24-11 specification, as well as the older APIs (e.g. IdsM_SetSecurityEvent). The IdsM additionally provides the possibility to transmit IDS messages in protocol versions 1 and 2, based on the configuration parameter IdsMRbIdsProtocolVersionCompatibility, for easier system integration.

FieldName	Length	Description of the data
Protocol Version	4 Bit	The version of the IdsM protocol
Protocol Header	4 Bit	IdsM protocol header information: Bit[0]: 0 - No Context Data Frame included, 1 - Context Data Frame included Bit[1]: 0 - No Timestamp included, 1 - Timestamp included Bit[2]: 0 - No Signature Frame included, 1 - Signature Frame included Bit[3]: reserved
IdsM Instance Id	10 Bit	Unique identifier of the sending IdsM instance 0-1023
Module Instance Id	6 Bit	Identifier to differ between multiple instances of modules
Event Id	16 Bit	Unique identifier of a Security Event: Range of AUTOSAR internal IDs: 0...0x7FFF Range of Customer specific IDs: 0x8000...0xFFFF
Count	16 Bit	Number of IdsM calls which result in the current event after processing the configured filter, e.g. <i>EventAggregation</i>
Reserve	8 Bit	Reserved for future use
Timestamp	8 Bytes	Timestamp / Tickstamp when event was detected: (optional) Byte[0] Bit[7]=0: AUTOSAR Standard, Byte[0] Bit[6]: reserved Byte[0] Bit[7]=1: OEM Specific / Custom Timestamp Resolution in ms. Maybe not necessary for every event type. If not set, field is filled by IdsR. If not authentic time, IdsR might recalculate the time and insert a new value
Context Data Version	2 Bytes	Version of the IDS Context Data. To distinguish different versions (optional) Context Data Version Byte[0], Bit [7]=0: Original reported Context Data Context Data Version Byte[0], Bit [7]=1: Context Data provided by user callout
Context Data Length	1 or 4 Bytes	Length information of Context Data. Only available if Context Data exists (optional) Most Significant Bit of first byte Context Data signals if Context Data Length is encoded in 7 Bit or 31 Bit: Context Data Length Byte[0] Bit[7]=0: Length is encoded in 7 Bits - Context Data Length Byte[0] Bit[0..6] - Valid values: 1..127 Bytes Context Data Length Byte[0] Bit[7]=1: Length is encoded in 31 Bits - Context Data Length Byte[0] Bit[0..6], Byte[1..3] Bit[0..7] - Valid values: 1..(2 ³¹) - 1 Bytes Note: The 2 Bytes for Context Data is not included in the Context Data Length)
Context Data	1...(2 ³¹) - 1 Bytes	Binary blob attached by the sensor: (optional) Modelled context data provided by corresponding sensor components
Signature Length	2 Bytes	Length information of Signature. Only available if Signature exists. (optional) Signature Byte[0..1]: Signature Length 1..65535 Bytes
Signature	1..65535 Bytes	Signature for authentication of security event: (option) Signature calculated with Eventframe + Optional Timestamp + Optional Context Data Signature Byte[2..n]: Signature Data - configurable via MetaModel

Figure 5.4. Intrusion Detection System Protocol Overview

A QSEv datagram is transmitted over IdsMI fTxPdu if the underlying security event has IdsMSinkIdsR configured and the size of the datagram fits within the referenced Pdu (IdsMI fTxPduRef). If the size of the generated datagram is larger than

the IfPdu, a Tp transmission is triggered based on the configured IdsMTpTxPdu. If IdsMRbSinkIdsRFixedIfPduLength is set to TRUE, the transmitted Pdu will always have the size of the Pdu referenced by IdsMIIfTxPduRef. In this case, additional bytes added to reach the Pdu size from the computed QSEv datagram size are filled with zeros.

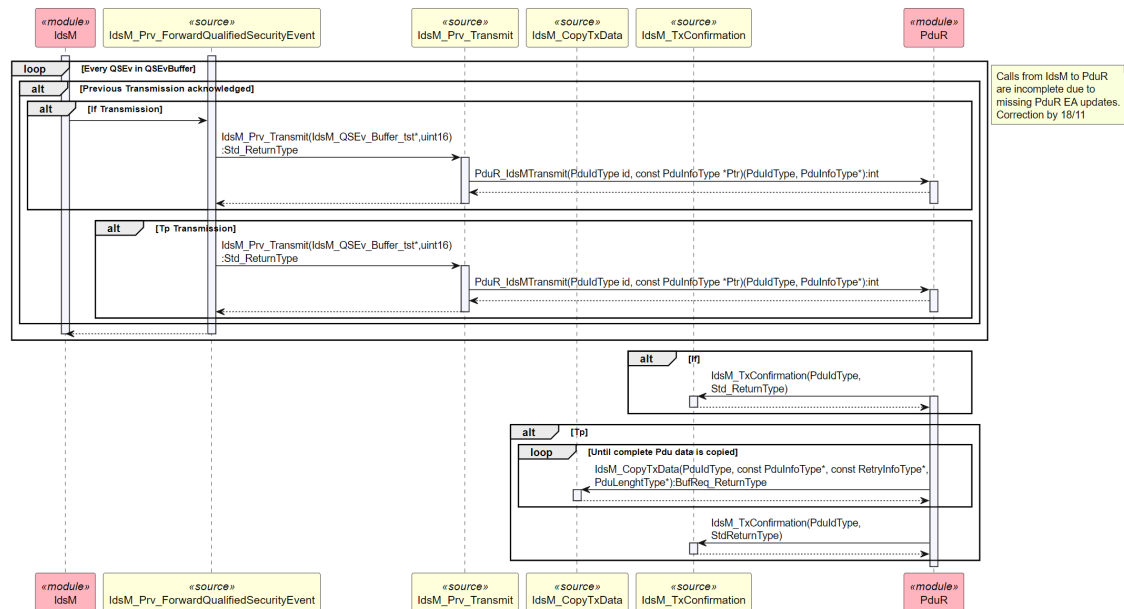


Figure 5.5. Transmission of a QSEv datagram

5.2.10 Diagnostic Reconfiguration of Reporting Mode

IdsM allows the reporting mode of events to be diagnostically reconfigurable over AUTOSAR, standardized interfaces IdsM_Dcm_SetReportingMode_Start and read back over IdsM_Dcm_GetReportingMode_Start and IdsM_Dcm_GetReportingMode_RequestResults.

5.2.11 Context Data Modification

IdsM supports a configurable way to remove or modify an SEv's reported context data. This is achieved over AR standardized callouts known as the context data modifier callout. The following variations are possible:

- Global Context Data Modification- IdsM supports configuring context data modifier callout at the IdsM instance level called the global context data modifier. The global context data modification callout will be called with the security event ID, pointer to the reported context data array, reported context data size, maximum allowed size of modified context data and pointer to a buffer to store the modified context data.

This API will be called for events (external and internal events) with reporting mode set to DETAILED or DETAILED_BYPASSING_FILTERS and does not have an event specific context data modifier callout configured. The callout can be

configured at 2 stages in the SEv processing,

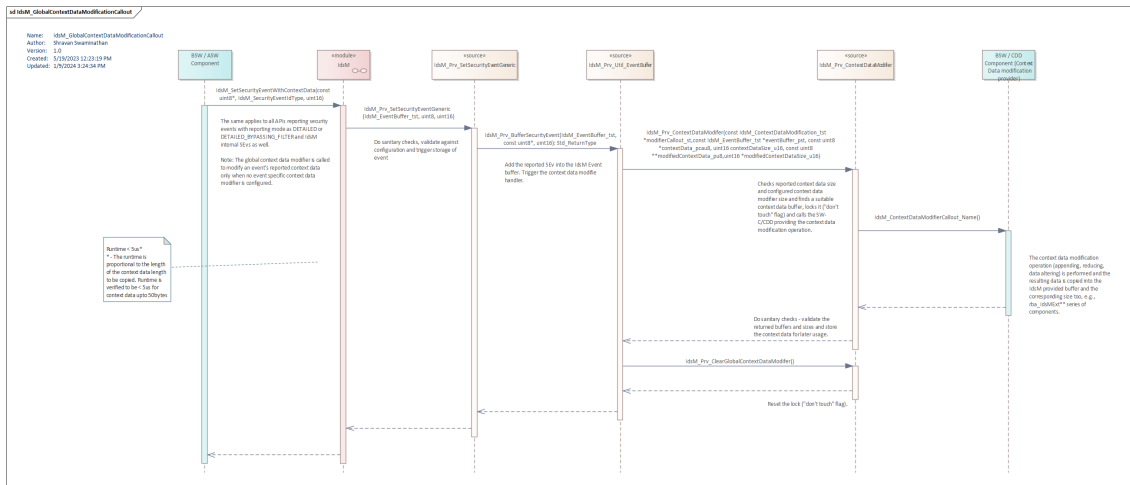


Figure 5.6.Reception of SEv - Configured Over IdsMGlobalContextDataModifierCalloutFct

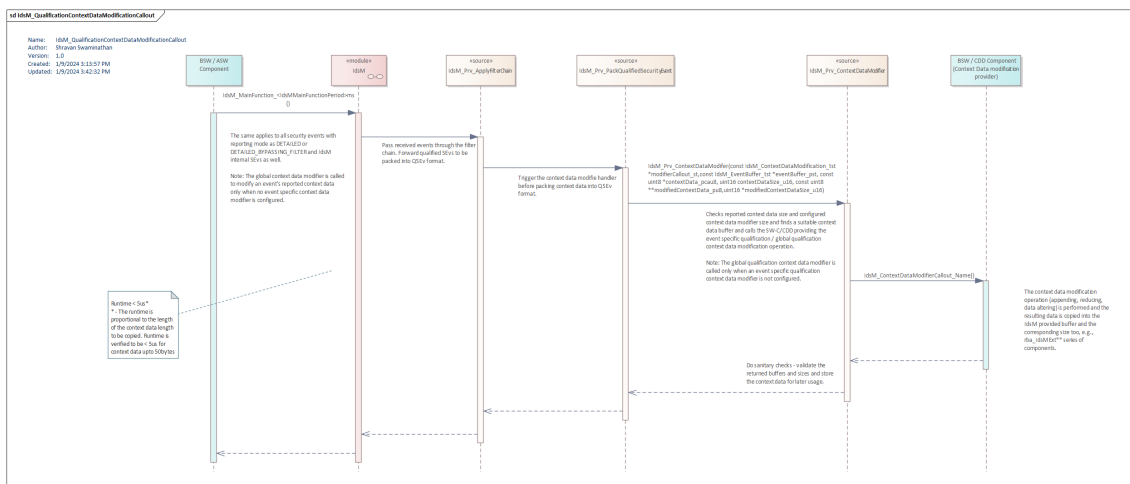


Figure 5.7.Qualification of SEv - Configured Over IdsMRbGlobalQualificationContextDataModifierCalloutFct

If both `IdsMGlobalContextDataModifierCalloutFct` and `IdsMRbGlobalQualificationContextDataModifierCalloutFct` are configured, each callout will be called at its respective processing phase.

- **Event Specific Context Data Modification - IdsM** supports configuring context data modifier callout at the IdsM event level called the event specific context data modifier. The event specific context data modification callout will be called with the security event ID, pointer to reported context data array, reported context data size, maximum allowed size of modified context data and pointer to a buffer to store the modified context data.

This API will be called for events (external and internal events) with reporting

If both `IdsMContextDataModifierCalloutRef` and `IdsMRbQualificationContextDataModifierCalloutRef` are configured, both of the callouts will be called at their respective processing phases.

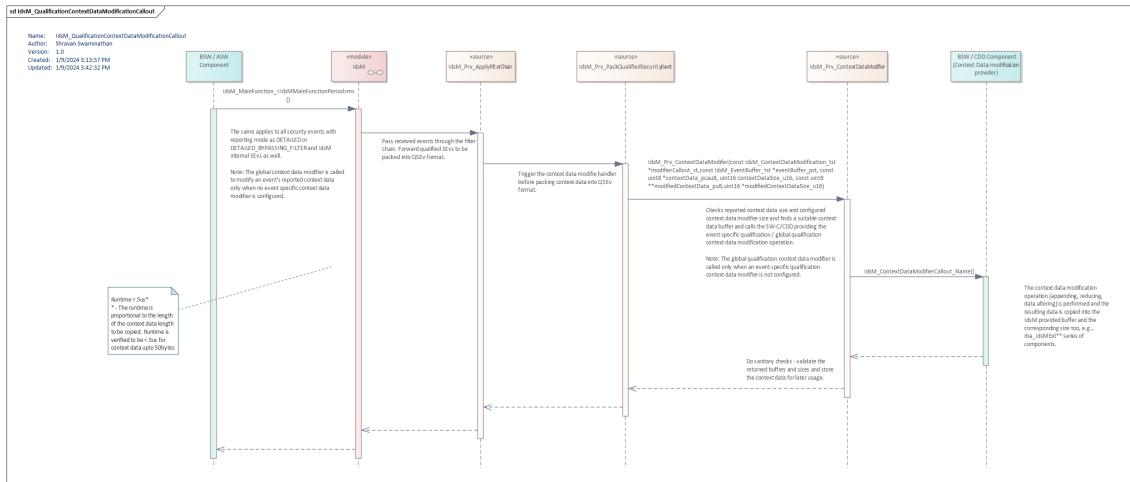


Figure 5.9. Qualification of SEv - Configured Over IdsMRbQualificationContextDataModifierCalloutRef

If `IdsMContextDataModifierCalloutRef` or `IdsMRbQualificationContextDataModifierCalloutRef` is configured, the global context data modifier callout from that corresponding processing stage of the SEv will be skipped, e.g., if an event has `IdsMContextDataModifierCalloutRef` configured, only the callout behind `IdsMContextDataModifierCalloutRef` will be called the `IdsMGlobalContextDataModifiedCalloutFct` will not be called and similarly also for `IdsMRbQualificationContextDataModifierCalloutRef` and `IdsMRbGlobalQualificationContextDataModifierCalloutFct`.

5.2.12 Signature Support

IdsM protects the authenticity and integrity of QSEvs using signature generated by CSM. Signature is generated by csm job IdsMCsmJobReference in same data format to each QSEv in two different types of processing.

- Synchronous signature generation - In this case when the function Csm_SignatureGenerate returns, the signature is immediately available.
- Asynchronous signature generation - In this case IdsM gets notified by Csm notification callback function IdsM_CsmNotification about the generation of the signature.

Configuration of CsmJob in IdsM calls Csm_SignatureGenerate with either a asynchronous or synchronous job referenced by IdsMCsmJobReference. The length of the signature depends on the configuration of algorithm and the same length has to be requested by IdsM.

The length cannot be truncated as all the bytes are required for the signature verification. An appropriate key values of correct length as per the algorithm has to be configured in the job. Key values are configured in CsmRbAutoConfigKeyElements. If CsmRbAutoConfigKeyElements is already configured then CsmKeyRef shouldn't be configured.

5.2.13 Diagnostic Reconfiguration of Reporting Mode

IdsM allows the reporting mode of events to be diagnostically reconfigurable over AUTOSAR standardized interfaces IdsM_Dcm_SetReportingMode_Start, which can be read back over IdsM_Dcm_GetReportingMode_Start and IdsM_Dcm_GetReportingMode_RequestResults.

5.2.14 Timestamp

Timestamps are optional and can be provided to the IdsM in different ways and should be configured for all the QSEvs. The timestamp can be provided by smart sensor (while reporting a security event) or it shall be fetched from a chosen timestamp origin (IdsMTimestampOption).

Additionally an Rb configuration IdsMRbTimestampTimesliceSelector shall specifies when to add timestamp to recieved SEv. Setting IdsMRbTimestampTimesliceSelector to IDSM_TIMESTAMP_TIMESLICE_FILTERCHAIN_INPUT IdsM shall add the timestamp before security event is fed to filter chain and setting it to IDSM_TIMESTAMP_TIMESLICE_FILTERCHAIN_OUTPUT shall add timestamp while QSEv generation (i.e when SEv is qualified).

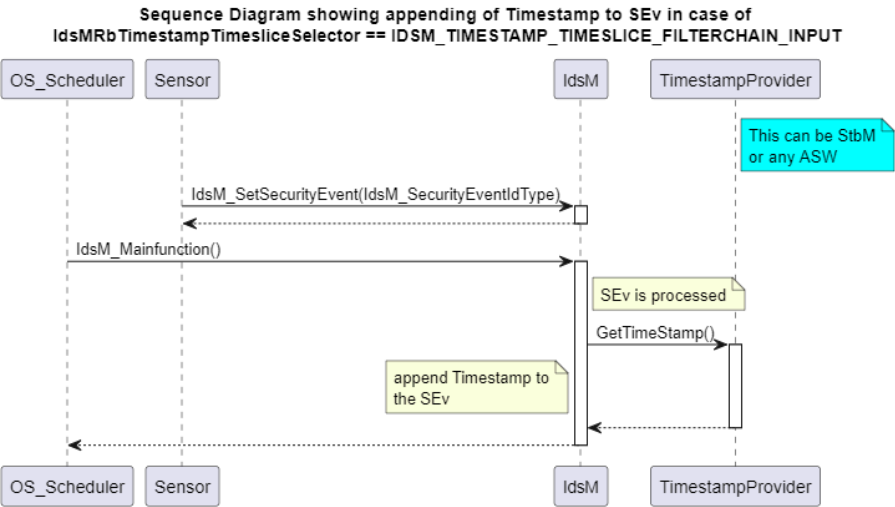


Figure 5.10.Timestamp Timeslice Input

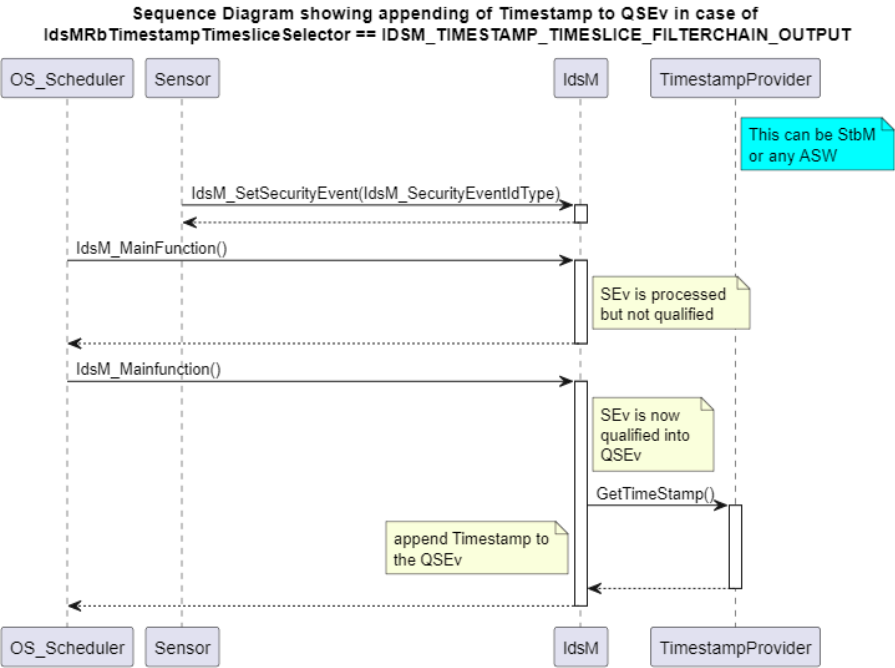


Figure 5.11.Timestamp Timeslice Output

Timestamping Usecases Supported by AUTOSAR

Usecase ID	Timestamp Provided by Caller	IdsMTimestamp Configured	AR IDS Message	AR Requirement
A	-	-	Doesn't have timestamp	SWS_IdsM_01106
B	-	X	IDS message has timestamp fetched over configured IdsM-TimestampOption	SWS_IdsM_01107 SWS_IdsM_01111
C	X	-	Doesn't have timestamp	SWS_IdsM_01106
D	X	X	IDS message has timestamp provided by security sensor	SWS_IdsM_01108

However AR doesn't address the usecase where the security sensor provides the timestamp to be used for the IDS message to IdsM (e.g., IdsM_SetSecurityEventWithTimestampCount) that doesn't have any IdsMTimestamp configured. To cover this missing usecase (E from below table), RTA-BSW provides an additional configuration parameter.

Usecase ID	Timestamp Provided by Caller	IdsMTimestamp Configured	IdsMTimestamp Support	AR IDS Message	Requirement for reference
A	-	-	-	Doesn't have timestamp	SWS_IdSM_01106
F	-	-	X	Doesn't have timestamp	SWS_Rb_IdSM_07782
G	-	X	-	Doesn't have timestamp	SWS_IdSM_01106 SWS_Rb_IdSM_07781
B	-	X	X	IDS message has timestamp fetched over configured IdsM-TimestampOption	SWS_IdSM_01107 SWS_IdSM_01111
C	X	-	-	Doesn't have timestamp	SWS_IdSM_01106
E	X	-	X	IDS message has timestamp provided by security sensor	SWS_Rb_IdSM_07780
H	X	X	-	Doesn't have timestamp	SWS_IdSM_01106
D	X	X	X	IDS message has timestamp provided by security sensor	SWS_IdSM_01108

5.2.15 Heartbeat

The IdsM can be configured to generate a heartbeat SEv over the IdsMSecurityEventRefs and the periodicity of the heartbeat over IdsMRbHeartbeatCycleTime. The heartbeat is merely an internal security event that conveys that IdsM is alive. There is currently no context data support for the heartbeat SEv.

5.2.16 Displacement Strategy

In scenarios where there are insufficient event and/or qualified event buffers, IdsM can be configured to buffer the events based on severity, or drop the incoming events by configuring the displacement strategy to *IDS_M_BUFFER_DISPLACEMENT_SEVERITY_BASED* or *IDS_M_BUFFER_DISPLACEMENT_DROP_LATEST*.

5.2.17 Severity-based Buffer Management

Each security event configured in IdsM has an attributed severity. When there are insufficient events and/or qualified event buffers, a severity-based displacement strategy is employed and *IDS_M_BUFFER_DISPLACEMENT_SEVERITY_BASED* can be chosen. This means that the least severe event is dropped when there are insufficient buffers available. This strategy ensures that higher severity events, which are more critical, are prioritized and retained in the buffers. By prioritizing higher severity events, IdsM maintains the relevance of the security events being processed and reported. This approach helps to manage limited buffer resources effectively, ensuring that the most important security events are not lost, even under heavy event load conditions. Consequently, the system remains robust and responsive to the most critical security threats.

5.2.18 Local Persistence of QSEv

Dem (Diagnostic Event Manager) is a sink for IdsM. IdsM provides the option to store qualified security events in a non-volatile manner using Dem. To enable subsequent storage of events in Dem, IdsM reports a PASSED Event once the FAILED Event has been stored in Dem. Dem triggers the callback configured under *DemDataElementReadFnc* when the Dem Event is reported with both FAILED and PASSED reports. However, the expected behavior is for Dem to trigger the callback function only for FAILED reports. To address this issue, Dem has created ALM-28 CR 656158. Until this is fixed, the callback function in IdsM returns *E_OK* for every passed event to avoid raising a Det error. The data buffer is ignored by Dem when the callback function returns *E_OK* following a PASSED report.

5.2.19 Internal Security Event

IdsM is capable of detecting security events arising within the component's functionality. The following table summarizes the security events:

Name	Description	Id
SEV_IDSMM_NO_EVENT_BUFFER_AVAILABLE	A received Security Event cannot be handled because there are no event buffers available to store the event.	46
SEV_IDSMM_NO_CONTEXT_DATA_BUFFER_AVAILABLE	The context data of a received Event cannot be stored because there are no context data buffers available.	47
SEV_IDSMM_TRAFFIC_LIMITATION_EXCEEDED	The transmission of a qualified Security Event would lead to a violation of configured traffic limitation thresholds.	48
SEV_IDSMM_COMMUNICATION_ERROR	An error occurred when attempting to transmit a qualified Security Event.	49
SEV_IDSMM_NO_QUALIFIED_EVENT_BUFFER_AVAILABLE	A qualified Security Event cannot be handled due to insufficient qualified event buffers available.	87

The global parameter `IdsMEnableSecurityEventReporting` enables/disables internal security events.

5.2.20 Measurement Points

IdsM has a chain of units that process a Security Event, reception/reporting, filter chains, QSEvprocessing and sink consumption of the QSEv. The measurement points aim to trace a Security Event from reception up to consumption by a sink. The following measurement points are available :

Event-specific Counts:

- Number of specific events received/reported to IdsM.
- Number of specific events qualified by IdsM.

Global Counts:

- Total number of events received/reported to IdsM.
- Total number of events qualified by IdsM.
- Total number of events transmitted successfully to IdsR.
- Total number of events failed to transmit to IdsR.
- Total number of events persisted successfully.

Buffer Related Counts:

- Peak fill status of Event buffer, QSEv buffer and Context data buffer.
- Fill distribution status of Event buffer, QSEv buffer and Context data buffer.

5.2.21 Available Sensor Modules

The following sensors modules with the listed security events are available and conform to AUTOSAR unless specified otherwise. Sensors and security events that are not listed below are in development and will be available with subsequent releases.

- SecOC
 - SECOC_SEV_MAC_VERIFICATION_FAILED
 - SECOC_SEV_FRESHNESS_NOT_AVAILABLE
- IdsM
 - IDSM_INTERNAL_EVENT_NO_EVENT_BUFFER_AVAILABLE
 - IDSM_INTERNAL_EVENT_NO_CONTEXT_DATA_BUFFER_AVAILABLE
 - IDSM_INTERNAL_EVENT_TRAFFIC_LIMITATION_EXCEEDED
 - IDSM_INTERNAL_EVENT_COMMUNICATION_ERROR
- Dcm
 - DIAG_SEV_REQUEST_OUT_OF_RANGE
 - DIAG_SEV_WRITE_INVALID_DATA
 - DIAG_SEV_SECURITY_ACCESS_DENIED
 - DIAG_SEV_COMMUNICATION_CONTROL_SWITCHED_OFF
 - DIAG_SEV_DTC_SETTING_SWITCHED_OFF
 - DIAG_SEV_SUBFUNCTION_NOT_SUPPORTED
 - DIAG_SEV_INCORRECT_MESSAGE_LENGTH_OR_FORMAT
 - DIAG_SEV_REQUEST_SEQUENCE_ERROR
 - DIAG_SEV_REQUEST_OUT_OF_RANGE
 - DIAG_SEV_REQUESTED_ACTIONS_REQUIRES_AUTHENTICATION
 - DIAG_SEV_SECURITY_ACCESS_NUMBER_OF_ATTEMPTS_EXCEEDED
 - DIAG_SEV_SECURITY_ACCESS_INVALID_KEY
 - DIAG_SEV_SECURITY_ACCESS_REQUIRED_TIME_DELAY_NOT_EXPIRED
 - DIAG_SEV_NUMBER_OF_FAILED_AUTHENTICATION_ATTEMPTS_EXCEEDED
 - DIAG_SEV_CERTIFICATE_FAILURE
 - DIAG_SEV_ECU_UNLOCK_SUCCESSFUL
 - DIAG_SEV_AUTHENTICATION_SUCCESSFUL
 - DIAG_SEV_CLEAR_DTC_SUCCESSFUL
 - DIAG_SEV_ECU_RESET
 - DIAG_SEV_WRITE_DATA
 - DIAG_SEV_REQUEST_DOWNLOAD
- KeyM
 - KEYM_SEV_INST_ROOT_CERT_OP
 - KEYM_SEV_UPD_ROOT_CERT_OP
 - KEYM_SEV_INST_INTERMEDIATE_CERT_OP
 - KEYM_SEV_UPD_INTERMEDIATE_CERT_OP
 - KEYM_SEV_CERT_VERIF_FAILED
- EthStack

- ETHIF_SEV_DROP_UNKNOWN_ETHERTYPE
- ETHIF_SEV_DROP_VLAN_DOUBLE_TAG
- ETHIF_SEV_DROP_INV_VLAN
- ETHIF_SEV_DROP_ETH_MAC_COLLISION
- SOAD_SEV_DROP_PDU_RX_TCP
- SOAD_SEV_DROP_PDU_RX_UDP
- SOAD_SEV_DROP_MSG_RX_UDP_LENGTH
- SOAD_SEV_DROP_MSG_RX_UDP_SOCKET
- SOAD_SEV_REJECTED_TCP_CONNECTION
- TCPIP_SEV_DROP_INV_PORT_TCP
- TCPIP_SEV_ARP_IP_ADDR_CONFLICT
- TCPIP_SEV_DROP_INV_IPV6_ADDR
- TCPIP_SEV_DROP_INV_IPV4_ADDR
- TCPIP_SEV_DROP_INV_PORT_UDP

5.3 Extensions

5.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

5.3.1.1 IdsM/IdsMConfiguration/IdsMBufferConfiguration/IdsMEventDisplacementStrategy

Short Name	IdsMEventDisplacementStrategy
Description	This parameter specifies the IdsM the displacement approach in the IdsMEventBuffer and IdsMQualifiedEventBuffers.
Multiplicity	1..1
Default Value	-
Enumeration Literals	IDS_M_BUFFER_DISPLACEMENT_DROP_LATEST, IDS_M_BUFFER_DISPLACEMENT_SEVERITY_BASED
Type	ECUC-ENUMERATION-PARAM-DEF

5.3.1.2 IdsM/IdsMConfiguration/IdsMBufferConfiguration/IdsMQualifiedEventBuffers

Short Name	IdsMQualifiedEventBuffers
Description	Buffers used to store the QSEvs.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

5.3.1.3 IdsM/IdsMConfiguration/IdsMBufferConfiguration/IdsMQualifiedEventBuffers/IdsMNumberOfQualifiedEventBuffers

Short Name	IdsMNumberOfQualifiedEventBuffers
Description	The number of qualified event buffers used to store the SEvs. Depending on the rate at which QSEvs are generated and the rate at which the sinks can consume them successfully, the number of QSEv buffers can be determined, e.g., assuming the sink SinkIdsR has a worst-case TxConfirmation latency of "k" milliseconds, if for every "n" millisecond period where "n" is considerably larger than "k", at the maximum "m" QSEv can be generated, IdsM would need at least "m" Ids MNumberOfQualifiedEventBuffers to prevent dropping QSEvs. The window "n" can be chosen as any interval considerably larger than "k" and where the peak QSEv generation rate can be estimated. Therefore $\text{IdsMNumberOfQualifiedEventBuffers} \geq n/k$.
Multiplicity	1..1
Value Range	1..65535
Default Value	1
Type	ECUC-INTEGER-PARAM-DEF

5.3.1.4 IdsM/IdsMConfiguration/IdsMContextDataModification/IdsMRbGlobalQualificationContextDataModifierCalloutFct

Short Name	IdsMRbGlobalQualificationContextDataModifierCalloutFct
Description	Global context data modifier callout function to be called during the Packing of QSEv . This callout can be overruled by event-specific context data modifier callouts.
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

5.3.1.5 IdsM/IdsMConfiguration/IdsMContextDataModification/IdsMRbGlobalQualificationContextDataSizeModifier

Short Name	IdsMRbGlobalQualificationContextDataSizeModifier
Description	Global context data size modifier used in combination with global context data size modifier callout function.
Multiplicity	0..1
Value Range	-1499..1499

Default Value	0
Type	ECUC-INTEGER-PARAM-DEF

5.3.1.6 IdsM/IdsMConfiguration/IdsMEvent/IdsMContextDataModifierOptions/IdsMRbQualificationContextDataModifierCalloutRef

Short Name	IdsMRbQualificationContextDataModifierCalloutRef
Description	Reference to Qualification specific context data modifier callout function.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

5.3.1.7 IdsM/IdsMConfiguration/IdsMEvent/IdsMEventSeverity

Short Name	IdsMEventSeverity
Description	The severity of a security event. The severity of the event is used for determining the event to be dropped when buffers are full. Severity 0 is interpreted as the lowest and 255 as the highest severity.
Multiplicity	0..1
Value Range	0..255
Default Value	0
Type	ECUC-INTEGER-PARAM-DEF

5.3.1.8 IdsM/IdsMConfiguration/IdsMEvent/IdsMEventStatistics

Short Name	IdsMEventStatistics
Description	A boolean configuration to enable and disable collecting event specific statistics. .
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.9 IdsM/IdsMConfiguration/IdsMEvent/IdsMRbDemEventReference

Short Name	IdsMRbDemEventReference
Description	This parameter references to the mapping to DemEventParameter via configuration parameter IdsMRbDemEventReference. This serves as a mapping between an IdsM security event and a Dem Event.
Multiplicity	0..1
Default Value	-

Type	ECUC-REFERENCE-DEF
------	--------------------

5.3.1.10 IdsM/IdsMConfiguration/IdsMEvent/IdsMServiceInterfaceOptions/IdsMEventEnableTimestampReporting

Short Name	IdsMEventEnableTimestampReporting
Description	Enables/disables reporting of timestamps over C/S interfaces. - true: reporting is enabled. - false: reporting is disabled. This parameter is used to configure the service interfaces for the C API IdsM_ReportSecurityEvent. Configuring this parameter instructs IdsM to generate a C/S interface compliant to the IdsM_ReportSecurityEvent
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.11 IdsM/IdsMConfiguration/IdsMFilterChain/IdsMEventAggregation-Filter/IdsMRbNumberOfContextDataAggregated

Short Name	IdsMRbNumberOfContextDataAggregated
Description	The maximum number of context data to be aggregated by the aggregation filter. AR aggregation filter behavior is achieved with IdsMRbNumberOfContextDataAggregated = 1 (default value).
Multiplicity	0..1
Value Range	1..65535
Default Value	1
Type	ECUC-INTEGER-PARAM-DEF

5.3.1.12 IdsM/IdsMConfiguration/IdsMFilterChain/IdsMEventThresholdFilter/IdsMRbEventThresholdDirection

Short Name	IdsMRbEventThresholdDirection
Description	This parameter configures the direction in which the thresholding is done. IDSM_EVENT_THRESHOLD_FILTER_FORWARD_FROM_N - (0) - Event threshold filter behaves as per AR by dropping security events until IdsMEventThresholdNumber. IDSM_EVENT_THRESHOLD_FILTER_FORWARD_UPTO_N - (1) - Event threshold filter forwards the first N events and drops after the N events.
Multiplicity	0..1

Default Value	IDSMEVENT_THRESHOLD_FILTER_FORWARD_FROM_N
Enumeration Literals	IDSMEVENT_THRESHOLD_FILTER_FORWARD_FROM_N, IDSMEVENT_THRESHOLD_FILTER_FORWARD_UPTO_N
Type	ECUC-ENUMERATION-PARAM-DEF

5.3.1.13 IdsM/IdsMConfiguration/IdsMFilterChain/IdsMRbCustomFilter

Short Name	IdsMRbCustomFilter
Description	Custom filter to be used over a callout
Multiplicity	0..100
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

5.3.1.14 IdsM/IdsMConfiguration/IdsMFilterChain/IdsMRbCustomFilter/IdsMRbCustomFilterReference

Short Name	IdsMRbCustomFilterReference
Description	Reference to IdsMRbCustomFilter to hook a custom filter to a filter chain
Multiplicity	1..1
Default Value	-
Type	ECUC-REFERENCE-DEF

5.3.1.15 IdsM/IdsMConfiguration/IdsMRbCustomFilter

Short Name	IdsMRbCustomFilter
Description	Custom filter to be used over a callout.
Multiplicity	0..100
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

5.3.1.16 IdsM/IdsMConfiguration/IdsMRbCustomFilter/IdsMRbCustomFilterFunction

Short Name	IdsMRbCustomFilterFunction
Multiplicity	1..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

Description: IdsM generates a callout to {IdsMRbCustomFilterFunction} at the position indicated by IdsMRbCustomFilterSequenceNumber. The definition of the

function lays in the responsibility of the project configuring it. Rules for the function - (1) Return value shall be a boolean. TRUE if processed event is filtered out. FALSE if process event is to be passed to the next filter or to be qualified The filter shall respect SWS_IdsM_00904, SWS_IdsM_00905. (2) First argument of the function prototype is IdsM_Event_Config_tst. This shall NOT be modified by the user. It is only for reading purposes. (3) Second argument of the function prototype is IdsM_EventBuffer_tst. Modification of elements of this structure is allowed as a part of the implemented filtering algorithm. However failures arising in IdsM due to modification of this structure shall be in the responsibility of the user. Consultation with RTA-BSW IdsM responsible is advised. (4) Calling of IdsM_Priv_ClearLinkedContextDataBuffer to clear associated context data during dropping is in the responsibility of the function, e.g., IdsM_Priv_Apply-BlockStateFilter.

5.3.1.17 IdsM/IdsMGeneral/IdsMEnableSecurityEventReporting

Short Name	IdsMEnableSecurityEventReporting
Description	Switches the reporting of internal security events.
Introduction	- true: reporting is enabled. - false: reporting is disabled.
Multiplicity	1..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.18 IdsM/IdsMGeneral/IdsMInstanceStatistics

Short Name	IdsMInstanceStatistics
Description	A boolean configuration to enable and disable collecting instance statistics. .
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.19 IdsM/IdsMGeneral/IdsMRbAggregateOverPowerOnCycles

Short Name	IdsMRbAggregateOverPowerOnCycles
Description	Enables or disables aggregation of events over multiple Power On cycles for aggregation filter. Enabling this configuration parameter stores the aggregation filter relevant variables in NVM.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.20 IdsM/IdsMGeneral/IdsMRbCalibrationSupport

Short Name	IdsMRbCalibrationSupport
Description	Container to configure the calibration support for IdsM.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

5.3.1.21 IdsM/IdsMGeneral/IdsMRbCalibrationSupport/IdsMRbAggregationFilterCalibrationSupport

Short Name	IdsMRbAggregationFilterCalibrationSupport
Description	Parameter to enable/disable calibration support for event aggregation filter in the filter chain
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.22 IdsM/IdsMGeneral/IdsMRbCalibrationSupport/IdsMRbBlockStateFilterCalibrationSupport

Short Name	IdsMRbBlockStateFilterCalibrationSupport
Description	Parameter to enable/disable calibration support for block state filter in the filter chain
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.23 IdsM/IdsMGeneral/IdsMRbCalibrationSupport/IdsMRbCustomFilterCalibrationSupport

Short Name	IdsMRbCustomFilterCalibrationSupport
Description	Parameter to enable/disable calibration support for custom filter
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.24 IdsM/IdsMGeneral/IdsMRbCalibrationSupport/IdsMRbDisplacementStrategyCalibrationSupport

Short Name	IdsMRbDisplacementStrategyCalibrationSupport
------------	--

Description	Parameter to enable/disable calibration support for IdsM Event displacement strategy
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.25 IdsM/IdsMGeneral/IdsMRbCalibrationSupport/IdsMRbEventSeverityCalibrationSupport

Short Name	IdsMRbEventSeverityCalibrationSupport
Description	Parameter to enable/disable calibration support for event severity
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.26 IdsM/IdsMGeneral/IdsMRbCalibrationSupport/IdsMRbRateLimitationFilterCalibrationSupport

Short Name	IdsMRbRateLimitationFilterCalibrationSupport
Description	Parameter to enable/disable calibration support for global rate limitation filter
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.27 IdsM/IdsMGeneral/IdsMRbCalibrationSupport/IdsMRbReportingModeCalibrationSupport

Short Name	IdsMRbReportingModeCalibrationSupport
Description	Parameter to enable/disable calibration support for event reporting mode
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.28 IdsM/IdsMGeneral/IdsMRbCalibrationSupport/IdsMRbSamplingFilterCalibrationSupport

Short Name	IdsMRbSamplingFilterCalibrationSupport
Description	Parameter to enable/disable calibration support for sampling filter in the filter chain
Multiplicity	0..1
Default Value	false

Type	ECUC-BOOLEAN-PARAM-DEF
------	------------------------

5.3.1.29 IdsM/IdsMGeneral/IdsMRbCalibrationSupport/IdsMRbSinkDemCalibrationSupport

Short Name	IdsMRbSinkDemCalibrationSupport
Description	Parameter to enable/disable calibration support for Sink Dem
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.30 IdsM/IdsMGeneral/IdsMRbCalibrationSupport/IdsMRbSinkIdsRCalibrationSupport

Short Name	IdsMRbSinkIdsRCalibrationSupport
Description	Parameter to enable/disable calibration support for Sink IdsR
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.31 IdsM/IdsMGeneral/IdsMRbCalibrationSupport/IdsMRbThresholdFilterCalibrationSupport

Short Name	IdsMRbThresholdFilterCalibrationSupport
Description	Parameter to enable/disable calibration support for event threshold filter in the filter chain
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.32 IdsM/IdsMGeneral/IdsMRbCalibrationSupport/IdsMRbTrafficLimitationFilterCalibrationSupport

Short Name	IdsMRbTrafficLimitationFilterCalibrationSupport
Description	Parameter to enable/disable calibration support for global traffic limitation filter
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.33 IdsM/IdsMGeneral/IdsMRbDefaultTransmission

Short Name	IdsMRbDefaultTransmission
Description	This parameter enable or disable transmission availability of the underlying network to forward QSEv to IdsR sink . This is a default value untill the actual state is provided by BswM via callback during runtime. true : transmission is enabled, forward QSEv to IdsR sink. false : transmission is not enabled, QSEv shall be buffered.
Multiplicity	0..1
Default Value	true
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.34 IdsM/IdsMGeneral/IdsMRbFilterChainSequence

Short Name	IdsMRbFilterChainSequence
Description	Sequence of the filters in the filter chain. No configuration needed if no custom filters are used and AUTOSAR filter order is required.
Multiplicity	1..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

5.3.1.35 IdsM/IdsMGeneral/IdsMRbFilterChainSequence/IdsMRbBlock-StateFilterSequenceNumber

Short Name	IdsMRbBlockStateFilterSequenceNumber
Description	Sequence number of the block state filter in the filter chain. The sequence number is allowed to be discontinuous. Default sequence number = 1, i.e., block state filter is the first filter applied in the filter chain.
Multiplicity	0..1
Value Range	1..104
Default Value	1
Type	ECUC-INTEGER-PARAM-DEF

5.3.1.36 IdsM/IdsMGeneral/IdsMRbFilterChainSequence/IdsMRbCustomFilterSequenceNumber

Short Name	IdsMRbCustomFilterSequenceNumber
Description	Custom filter to be used over a callout

Multiplicity	0..100
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

5.3.1.37 IdsM/IdsMGeneral/IdsMRbFilterChainSequence/IdsMRbCustomFilterSequenceNumber/IdsMRbCustomFilterRef

Short Name	IdsMRbCustomFilterRef
Description	This parameter serves as an association between sequence numbers and custom filters by having a reference to IdsMRbCustomFilter.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

5.3.1.38 IdsM/IdsMGeneral/IdsMRbFilterChainSequence/IdsMRbCustomFilterSequenceNumber

Short Name	IdsMRbCustomFilterSequenceNumber
Description	Sequence number of the custom filter in the filter chain. The sequence number is allowed to be discontinuous. No default sequence number, i.e., the sequence number of custom filter have to always be explicitly configured.
Multiplicity	0..1
Value Range	1..104
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

5.3.1.39 IdsM/IdsMGeneral/IdsMRbFilterChainSequence/IdsMRbEventAggregationFilterSequenceNumber

Short Name	IdsMRbEventAggregationFilterSequenceNumber
Description	Sequence number of the event aggregation filter in the filter chain. The sequence number is allowed to be discontinuous. Default sequence number = 3, i.e., event aggregation filter is the third filter applied in the filter chain.
Multiplicity	0..1
Value Range	1..104
Default Value	3
Type	ECUC-INTEGER-PARAM-DEF

5.3.1.40 IdsM/IdsMGeneral/IdsMRbFilterChainSequence/IdsMRbEventThresholdFilterSequenceNumber

Short Name	IdsMRbEventThresholdFilterSequenceNumber
Description	Sequence number of the event threshold filter in the filter chain. The sequence number is allowed to be discontinuous. Default sequence number = 4, i.e., event threshold filter is the first filter applied in the filter chain.
Multiplicity	0..1
Value Range	1..104
Default Value	4
Type	ECUC-INTEGER-PARAM-DEF

5.3.1.41 IdsM/IdsMGeneral/IdsMRbFilterChainSequence/IdsMRbForwardEveryNthFilterSequenceNumber

Short Name	IdsMRbForwardEveryNthFilterSequenceNumber
Description	Sequence number of the sampling filter (forward every Nth filter) in the filter chain. The sequence number is allowed to be discontinuous. Default sequence number = 2, i.e., forward every Nth filter is the first filter applied in the filter chain.
Multiplicity	0..1
Value Range	1..104
Default Value	2
Type	ECUC-INTEGER-PARAM-DEF

5.3.1.42 IdsM/IdsMGeneral/IdsMRbHeartbeatCycleTime

Short Name	IdsMRbHeartbeatCycleTime
Description	The period the heartbeat will be raised (as float in seconds)..
Multiplicity	0..1
Value Range	0..INF
Default Value	0.0
Type	ECUC-FLOAT-PARAM-DEF

5.3.1.43 IdsM/IdsMGeneral/IdsMRbIdsProtocolVersionCompatibility

Short Name	IdsMRbIdsProtocolVersionCompatibility
------------	---------------------------------------

Description	The context data of AR-defined security events are versioned starting from AR24-11. Security sensors are expected to provide the version of the context data when reporting a security event to IdsM. The generated IDS message contains both the context data and its version. Our implementation supports the older (pre-AR24-11) reporting APIs, IdsM_SetSecurityEvent*, alongside the new API, IdsM_ReportSecurityEvent, for a period of at least two years. A deprecation notice may be issued after this period based on customer demand. Security sensors are allowed to use either IdsM_SetSecurityEvent* or IdsM_ReportSecurityEvent to report security events; however, we strongly encourage customers to version their context data and report it using IdsM_ReportSecurityEvent. This compatibility parameter allows the IDS message to be generated in either a uniform mode or an intentionally mixed mode (between IDS protocol version 1 and version 2).
Multiplicity	0..1
Default Value	IDSM_IDS_PROTOCOL_VERSION_1
Enumeration Literals	IDSM_IDS_PROTOCOL_VERSION_1, IDSM_IDS_PROTOCOL_VERSION_2, IDSM_IDS_PROTOCOL_MIXED_MODE
Type	ECUC-ENUMERATION-PARAM-DEF

5.3.1.44 IdsM/IdsMGeneral/IdsMRbMainBackgroundFunctionSupport

Short Name	IdsMRbMainBackgroundFunctionSupport
Description	This container enables/disables the main background function. Configured: Main background function is enabled. Not configured: Main background function is disabled.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

5.3.1.45 IdsM/IdsMGeneral/IdsMRbMainBackgroundFunctionSupport/IdsMRbCopyCalibrationValues

Short Name	IdsMRbCopyCalibrationValues
Description	Parameter to enable/disable copying of calibration parameters into RAM over the IdsM_Rb_MainBackgroundFunction.
Multiplicity	0..1
Default Value	false

Type	ECUC-BOOLEAN-PARAM-DEF
------	------------------------

5.3.1.46 IdsM/IdsMGeneral/IdsMRbRandomGenerate

Short Name	IdsMRbRandomGenerate
Description	If this container exists IdsM generates random number over referred CsmJobs. The random number is primarily used for the randomized aggregation filter
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

5.3.1.47 IdsM/IdsMGeneral/IdsMRbRandomGenerate/IdsMRbRandomGenerateCsmJobReference

Short Name	IdsMRbRandomGenerateCsmJobReference
Description	This parameter references the Csm job that is used to generate random number by IdsM.
Multiplicity	1..1
Default Value	-
Type	ECUC-REFERENCE-DEF

5.3.1.48 IdsM/IdsMGeneral/IdsMRbRandomGenerate/IdsMRbRandomNumberLength

Short Name	IdsMRbRandomNumberLength
Description	This parameter defines the length of the random number in bytes calculated by the crypto service.
Multiplicity	1..1
Value Range	1..4294967295
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

5.3.1.49 IdsM/IdsMGeneral/IdsMRbRandomGenerate/IdsMRbRandomSeedLength

Short Name	IdsMRbRandomSeedLength
Description	This parameter defines the length of the random seed in bytes to be fed to the Csm_RandomSeed when seed entropy is exhausted.
Multiplicity	1..1
Value Range	1..65535

Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

5.3.1.50 IdsM/IdsMGeneral/IdsMRbSinkIdsRFixedIfPduLength

Short Name	IdsMRbSinkIdsRFixedIfPduLength
Description	This parameter enables/disables padding of zeros to the QSEv datagram to match the PduLength of the configured underlying If Pdu referred by IdsMIfTxPduRef. The QSEv datagram is computed by IdsM based on 5.1.1 IDS Protocol Overview from Specification of Intrusion Detection System Protocol. true: Zeros are padded to the end of the QSEv datagram to match the configured PduLength and transmission to PduR is triggered with size of data equivalent to configured PduLength. false: Transmission of QSEv datagram is requested based on IdsM computed QSEv datagram size.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.51 IdsM/IdsMGeneral/IdsMRbTimestampSupport

Short Name	IdsMRbTimestampSupport
Description	This parameter enables/disables the presense of timestamp in QSEvs. If True, - the sensor reporting the security event can provide a timestamp that will be used in QSEv. - If IdsMTimestamp is configured and sensor reporting security event does not provide a timestamp, IdsM will request timestamp from the configured source IdsMTimestampOption. If False, - the timestamp provided by the sensor reporting the security event will be dropped. QSEv will not have a timestamp. - IdsMTimestamp configuration is ignored. Default value: True. This allows the AR usage of IdsMTimestamp without additional configuration.
Multiplicity	0..1
Default Value	true
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.52 IdsM/IdsMGeneral/IdsMSecurityEventRefs

Short Name	IdsMSecurityEventRefs
------------	-----------------------

Description	If this container exists a heartbeat event will be linked.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

5.3.1.53 IdsM/IdsMGeneral/IdsMSecurityEventRefs/SEV_IDS_M_INTERNAL_EVENT_COMMUNICATION_ERROR

Short Name	SEV_IDS_M_INTERNAL_EVENT_COMMUNICATION_ERROR
Description	A security event is raised when a QSEv has to be dropped due to insufficient QSEv buffers available.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

5.3.1.54 IdsM/IdsMGeneral/IdsMSecurityEventRefs/SEV_IDS_M_INTERNAL_EVENT_HEARTBEAT

Short Name	SEV_IDS_M_INTERNAL_EVENT_HEARTBEAT
Description	This parameter references the of heartbeat events.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

5.3.1.55 IdsM/IdsMGeneral/IdsMSecurityEventRefs/SEV_IDS_M_INTERNAL_EVENT_NO_CONTEXT_DATA_BUFFER_AVAILABLE

Short Name	SEV_IDS_M_INTERNAL_EVENT_NO_CONTEXT_DATA_BUFFER_AVAILABLE
Description	A Sev cannot be handled because there are no more context data buffers available to process event.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

5.3.1.56 IdsM/IdsMGeneral/IdsMSecurityEventRefs/SEV_IDS_M_INTERNAL_EVENT_NO_EVENT_BUFFER_AVAILABLE

Short Name	SEV_IDS_M_INTERNAL_EVENT_NO_EVENT_BUFFER_AVAILABLE
Description	A Sev cannot be handled because there are no more event buffers available to process event.
Multiplicity	0..1
Default Value	-

Type	ECUC-REFERENCE-DEF
------	--------------------

5.3.1.57 IdsM/IdsMGeneral/IdsMSecurityEventRefs/SEV_IDS_M_INTERNAL_EVENT_NO_QUALIFIED_EVENT_BUFFER_AVAILABLE

Short Name	SEV_IDS_M_INTERNAL_EVENT_NO_QUALIFIED_EVENT_BUFFER_AVAILABLE
Description	A QSev cannot be handled due insufficient QSev available.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

5.3.1.58 IdsM/IdsMGeneral/IdsMSecurityEventRefs/SEV_IDS_M_INTERNAL_EVENT_TRAFFIC_LIMITATION_EXCEEDED

Short Name	SEV_IDS_M_INTERNAL_EVENT_TRAFFIC_LIMITATION_EXCEEDED
Description	A Sev cannot be handled because the traffic rate limitation event is reached.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

5.3.1.59 IdsM/IdsMGeneral/IdsMTimestamp/IdsMRbCustomTimestampFct

Short Name	IdsMRbCustomTimestampFct
Description	Custom Timestamp function for BSW module. With this function the IdsM will be able to fetch the timestamp from BSW Modules with a C function.
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

5.3.1.60 IdsM/IdsMGeneral/IdsMTimestamp/IdsMRbCustomTimestamp-HeaderFileIncludes

Short Name	IdsMRbCustomTimestampHeaderFileIncludes
Description	Header file which contains the prototype of the custom time-stamp callouts.
Multiplicity	0..1
Minimum String Length	0
Default Value	-
Type	ECUC-STRING-PARAM-DEF

5.3.1.61 IdsM/IdsMGeneral/IdsMTimestamp/IdsMRbTimestampTimesliceS-

electorIdsMRbTimestampTimesliceSelector

Short Name	IdsMRbTimestampTimesliceSelector
Description	This parameter specifies when to add timestamp to the received security event. IDSM_TIMESTAMP_TIMESLICE_FILTERCHAIN_INPUT - timestamp is added before security event is fed to filter chain (AR behaviour). IDSM_TIMESTAMP_TIMESLICE_FILTERCHAIN_OUTPUT - timestamp is added while QSEv generation.
Multiplicity	0..1
Default Value	IDSM_TIMESTAMP_TIMESLICE_FILTERCHAIN_INPUT
Enumeration Literals	IDSM_TIMESTAMP_TIMESLICE_FILTERCHAIN_INPUT, IDSM_TIMESTAMP_TIMESLICE_FILTERCHAIN_OUTPUT
Type	ECUC-ENUMERATION-PARAM-DEF

5.3.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

5.4 Limitations

5.5 Exclusions

The following features from AR v22-11 are not supported in version v12.7.0.

API functions not currently implemented:

IdsMContextDataModifierCallout_Name
IdsM_DemReadQSEv
IdsM_SetTransmissionState

The following configuration parameters are not supported in version v12.7.0.

- IdsM/IdsMConfiguration/IdsMFilterChain/IdsMBlockStateFilter/IdsMBlock-StateReference

5.6 Deviations

5.6.1 Autosar Deviations

The following deviations should be noted with respect to the stated Autosar requirements.

5.6.1.1 IdsM_SetSecurityEventWithTimestampCount and IdsM_SetSecurityEventWithTimestampCountContextData

SWS_IdsM_01112 - Timestamp not provided via event reporting interface If the timestamp feature is enabled and the SEv is reported without a timestamp parameter via the services IdsM_SetSecurityEventWithTimestampCount or

IdsM_SetSecurityEventWithTimestampCountContextData. Then IdsM shall set the option bit for the time stamp in protocol header to 0 before the transmission or storage of the QSEv.(RS_Ids_00503). SWS_IdsM_01112 is rejected, IdsM cannot determine if a timestamp was provided or not based on the value of the timestamp received on the interface IdsM_SetSecurityEventWithTimestampCount or IdsM_SetSecurityEventWithTimestampCountContextData.

Configuration parameters which differ from the AR v22-11 specification are listed below.

IdsM

[ADDED] Description	Configuration of the IdsM (Intrusion detection system manager) module.
---------------------	--

IdsM/IdsMConfiguration

[RTA-BSW] Description	Configuration parameters of the module IdsM.
[AUTOSAR] Description	Configuration parameters of the module IdsM. Tags: atp.Status=draft

IdsM/IdsMConfiguration/IdsMBlockState

[RTA-BSW] Description	Configuration of an IdsM blocking state used in the IdsMStateBlock-Filter to suspend the collection of security events. The active state is reported by the BswM via IdsM_BswM_StateChanged().
[AUTOSAR] Description	Configuration of an IdsM blocking state used in the IdsMStateBlock-Filter to suspend the collection of security events. Tags: atp.Status=draft
[REMOVED] Introduction	The active state is reported by the BswM via IdsM_BswM_StateChanged().

IdsM/IdsMConfiguration/IdsMBlockState/IdsMBlockStateID

[RTA-BSW] Description	This value specifies the identifier of this block state.
[AUTOSAR] Description	This value specifies the identifier of this block state. Tags: atp.Status=draft

IdsM/IdsMConfiguration/IdsMBufferConfiguration

[RTA-BSW] Description	Configuration of the event buffers and context data buffers used by IdsM.
[AUTOSAR] Description	Configuration of the event buffers and context data buffers used by IdsM. Tags: atp.Status=draft

IdsM/IdsMConfiguration/IdsMBufferConfiguration/IdsMContext-DataBuffer

[RTA-BSW] Description	Buffer that is reserved to store the context data of SEVs. Depending on the type of SEv that is processed, there can be significant differences in sizes of the context data. TRACE[SWS_IdsM_00704]
[AUTOSAR] Description	Buffer that is reserved to store the context data of SEVs. Tags: atp.Status=draft
[REMOVED] Introduction	Depending on the type of SEv that is processed, there can be significant differences in sizes of the context data.

IdsM/IdsMConfiguration/IdsMBufferConfiguration/IdsMContext-DataBuffer/IdsMContextDataBufferSize

[RTA-BSW] Description	Size of the context data buffer in bytes. It is recommended to configure buffers with an appropriate size depending on the configured SEVs.
[AUTOSAR] Description	Size of the context data buffer in bytes. Tags: atp.Status=draft
[REMOVED] Introduction	It is recommended to configure buffers with an appropriate size depending on the configured SEVs.

IdsM/IdsMConfiguration/IdsMBufferConfiguration/IdsMContext-DataBuffer/IdsMNumberOfContextDataBuffers

[RTA-BSW] Description	The number of buffers with the configured buffer size specified in IdsMContextDataBufferSize. It is recommended to configure an appropriate number of buffers depending on the configured SEVs.
[AUTOSAR] Description	The number of buffers with the configured buffer size specified in IdsMContextDataBufferSize. Tags: atp.Status=draft
[REMOVED] Introduction	It is recommended to configure an appropriate number of buffers depending on the configured SEVs.

IdsM/IdsMConfiguration/IdsMBufferConfiguration/IdsMEvent-Buffers

[RTA-BSW] Description	Buffers used to store the SEVs. TRACE[SWS_IdsM_00701]
[AUTOSAR] Description	Buffers used to store the SEVs. Tags: atp.Status=draft

IdsM/IdsMConfiguration/IdsMBufferConfiguration/IdsMEvent-

Buffers/IdsMNumberOfEventBuffers

[RTA-BSW] Description	The number of event buffers used to store the SEVs. The suggested number of buffers can be calculated as follows: $\text{IdsMNumberOfBuffers} = \text{Number of Aggregation Filter Instances} + \text{Upper bound of parallel processed SEVs}$. Number of Aggregation Filter Instances = The number of configured SEVs that use a filter chain that contains an aggregation filter. Upper bound of parallel processed SEVs = 10% of the number of configured SEVs.
[AUTOSAR] Description	The number of event buffers used to store the SEVs. Tags: atp.Status=draft
[REMOVED] Introduction	The suggested number of buffers can be calculated as follows: $\text{IdsMNumberOfBuffers} = \text{Number of Aggregation Filter Instances} + \text{Upper bound of parallel processed SEVs}$. Number of Aggregation Filter Instances = The number of configured SEVs that use a filter chain that contains an aggregation filter. Upper bound of parallel processed SEVs = 10% of the number of configured SEVs.

IdsM/IdsMConfiguration/IdsMEvent

[RTA-BSW] Description	Configuration of the IdsM Event unit which is reported by a sensor and its parameters
[AUTOSAR] Description	Configuration of the IdsM Event unit which is reported by a sensor and its parameters. Tags: atp.Status=draft

IdsM/IdsMConfiguration/IdsMEvent/IdsMContextDataModifierOptions/IdsMContextDataModifierCalloutRef

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

IdsM/IdsMConfiguration/IdsMEvent/IdsMExternalEventId

[RTA-BSW] Description	The external security event ID which is reported to the sink. There are two different value ranges depending on the referencing module: Standardized SEv ID is defined by the AUTOSAR specification. This ID is usually derived from the SecXT. Standard ID range: 0x0000 - 0x8000 Generic User Event ID is defined by the user. Used when a SW-C / Application references the SEv. Generic ID range: 0x8000 - 0xFFFF. 0xFFFF is considered an invalid ID. SWS_IdsM_00608 must be respected. TRACE[SWS_IdsM_00604].
[AUTOSAR] Description	The external security event ID which is reported to the sink. Tags: atp.Status=draft

[REMOVED] Introduction	There are two different value ranges depending on the referencing module: Standardized SEv ID is defined by the AUTOSAR specification. This ID is usually derived from the SecXT. Standard ID range: 0x0000 - 0x8000 Generic User Event ID is defined by the user. Used when a SW-C / Application references the SEv. Generic ID range: 0x8000 - 0xFFFF. 0xFFFF is considered an invalid ID
---------------------------	--

IdsM/IdsMConfiguration/IdsMEvent/IdsMFilterChainRef

[RTA-BSW] Description	Reference to a configured IdsM filter chain.
[AUTOSAR] Description	Reference to a configured IdsM filter chain. Tags: atp.Status=draft

IdsM/IdsMConfiguration/IdsMEvent/IdsMInternalEventId

[RTA-BSW] Description	Consecutive number used internally as an identifier by the IdsM module. This number is calculated internally and shall not be configured manually. This parameter is only available to publish the result of this calculation. Applications using IdsM APIs shall not rely on the value of this parameter. Instead, they shall use the symbolic name value.
[AUTOSAR] Description	Consecutive number used internally as an identifier by the IdsM module. Tags: atp.Status=draft
[REMOVED] Intro- duction	This number is calculated internally and shall not be configured manually. This parameter is only available to publish the result of this calculation. Applications using IdsM APIs shall not rely on the value of this parameter. Instead, they shall use the symbolic name value.

IdsM/IdsMConfiguration/IdsMEvent/IdsMReportingModeFilter

[RTA-BSW] Description	The reporting mode filter defines the level of detail of the reporting. Whether SEv should be dropped, forwarded with context data or forwarded without context data. The parameter determines if the SEv is either: - dropped (OFF) - sent without context data (BRIEF) - sent with context data (DETAILED) - sent without context data, ignoring the rest of the filter chain (BRIEF_BYPASSING_FILTERS) - sent with context data ignoring the rest of the filter chain (DETAILED_BYPASSING_FILTERS) TRACE[SWS_IdsM_01010]
[AUTOSAR] Description	The reporting mode filter defines the level of detail of the reporting. Whether SEv should be dropped, forwarded with context data or forwarded without context data. Tags: atp.Status=draft

[REMOVED] Introduction	The parameter determines if the SEv is either: - dropped (OFF) - sent without context data (BRIEF) - sent with context data (DETAILED) - sent without context data, ignoring the rest of the filter chain (BRIEF_BYPASSING_FILTERS) - sent with context data ignoring the rest of the filter chain (DETAILED_BYPASSING_FILTERS)
---------------------------	---

IdsM/IdsMConfiguration/IdsMEvent/IdsMSensorInstanceId

[RTA-BSW] Description	The instance ID of the sensor which reports security events to the IdsM. If there is only one instance of a sensor, the default ID is 0. TRACE[SWS_IdSM_00605]. SWS_IdSM_00608 must be respected. The range of the sensor instance ID shall be from 0 to 63. [SWS_Rb_IdSM_07779]
[AUTOSAR] Description	The instance ID of the sensor which reports security events to the IdsM. Tags: atp.Status=draft
[REMOVED] Intro- duction	If there is only one instance of a sensor, the default ID is 0.
[RTA-BSW] Maximum Value	63
[AUTOSAR] Maximum Value	65535

IdsM/IdsMConfiguration/IdsMEvent/IdsMServiceInterfaceOptions

[RTA-BSW] Description: Additional configuration parameters of a SEv. Addition to AR - This container, if configured, is used for application and basic software SEvs. AUTOSAR defines it only for application SEvs. This satisfies AUTOSAR and in addition to it the parameters in IdsMServiceInterfaceOptions are used for basis software SEv as well in the following way. (1) Configured IdsMEventMaxContextData-Size is used to validate the context data size provided while reporting an SEv over the APIs that support context data (IdsM_SetSecurityEventWith**ContextData). (2) If IdsMAdditionalParameterOption is configured to CountTimestamp and the SEv was reported without a timestamp, IdsM will obtain the timestamp from StbM and add it to the SEv regardless if the SEv is a ASW or a BSW SEv. If IdsMAdditionalParameterOption is not configured to CountTimestamp and the SEv was reported with a timestamp, IdsM will drop the provided timestamp and buffer the event without a timestamp. The AUTOSAR requirement is statisfied because the parameters of this container are used to generate the service interfaces for the SEv when configured IdsMExternalId is in range 0x8000 - 0xFFFE as specified in AR IdsM SWS Ch 8.7.

[AUTOSAR] Description: Adittional configuration parameters of a SEv when the sensor is a SW-C or application.

Tags: atp.Status=draft

IdsM/IdsMConfiguration/IdsMEvent/IdsMServiceInterfaceOptions/

IdsMAdditionalParameterOption

[RTA-BSW] Description	In addition to the optional context data the Service Port Interface can be extended by the following parameters: - None: No extensions - Count: Additionally a count can be passed. This value is used as an initialization counter for the corresponding SEv. - Count and Timestamp: Additionally to the count a timestamp can be passed. Note: The timestamp option depends on whether IdsMTimeStampOption is configured to enable timestamps. Configuring this parameter instructs IdsM to generate a C/S interface compliant to the IdsM_SetSecurityEvent*.
[AUTOSAR] Description	In addition to the optional context data the Service Port Interface can be extended by the following parameters: Tags: atp.Status=draft
[REMOVED] Introduction	- None: No extensions - Count: Additionally a count can be passed. This value is used as an initialization counter for the corresponding SEv. - Count and Timestamp: Additionally to the count a timestamp can be passed. Note: The timestamp option depends on whether IdsM-TimeStampOption is configured to enable timestamps.
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1
[REMOVED] Default Value	None

IdsM/IdsMConfiguration/IdsMEvent/IdsMServiceInterfaceOptions/ IdsMEventMaxContextDataSize

[RTA-BSW] Description	Maximum number of bytes of context data to be accepted by IdsM for this particular event. Additional hint for configuration - TRACE[SWS_IdsM_00501] The functions IdsM_SetSecurityEventWithContextData, IdsM_SetSecurityEventWithCountContextData and IdsM_SetSecurityEventWithTimestampCountContextData shall support a maximum length of 1500 bytes for the context data. To avoid overloading of the network, a maximum of 1500 bytes for the context data is recommended, especially when transmitting on CAN Bus. There might be cases in which this limit is insufficient to transmit all the sensor information, in that case it shall be evaluated that there are enough resources to avoid flooding of the communication channels. This parameter also configured the information needed for the C/S interface mapped to the API IdsM_ReportSecurityEvent
[AUTOSAR] Description	Maximum number of bytes used by the IdsM and the RTE when forwarding context data of the corresponding security event. Tags: atp.Status=draft
[REMOVED] Introduction	This parameter is only used for SW-C use cases. This is the maximum amount of bytes defined for transmission of the context data. In case this is a Basic Software Module SEv, the configuration of this parameter is not necessary and will be ignored.

IdsM/IdsMConfiguration/IdsMEvent/IdsMSinkDem

[RTA-BSW] Description	The QSEv will be sent to the Dem Module into a Security Event Memory (Sem) to persist it on the local ECU.
[AUTOSAR] Description	The QSEv will be sent to the Dem Module into a Security Event Memory (Sem) to persist it on the local ECU. Tags: atp.Status=draft

IdsM/IdsMConfiguration/IdsMEvent/IdsMSinkIdsR

[RTA-BSW] Description	The QSEv will be sent to the IDS Reporter
[AUTOSAR] Description	The QSEv will be sent to the IDS Reporter. Tags: atp.Status=draft

IdsM/IdsMConfiguration/IdsMFilterChain

[RTA-BSW] Description	A filter chain is a combination of filters that affects one or more SEvs. A filter receives a SEv, checks condition(s) and, e.g. - forwards SEv immediately/later - drops SEv - stores SEv - modifies SEv Consider that the filter order is defined as follows: - Reporting Mode Level (per SEv ID) - Block State (per SEv ID) - Forward Every nth (per SEv ID) - Event Aggregation (per SEv ID) - Event Threshold (per SEv ID) - Event Rate Limitation (per IdsM Instance) - Traffic Limitation (per IdsM Instance)
[AUTOSAR] Description	A filter chain is a combination of filters that affects one or more SEvs. Tags: atp.Status=draft
[REMOVED] Introduction	A filter receives a SEv, checks condition(s) and, e.g. - forwards SEv immediately/later - drops SEv - stores SEv - modifies SEv Consider that the filter order is defined as follows: - Reporting Mode Level (per SEv ID) - Block State (per SEv ID) - Forward Every nth (per SEv ID) - Event Aggregation (per SEv ID) - Event Threshold (per SEv ID) - Event Rate Limitation (per IdsM Instance) - Traffic Limitation (per IdsM Instance)

IdsM/IdsMConfiguration/IdsMFilterChain/IdsMBlockStateFilter

[RTA-BSW] Description	This state filter drops SEvs if the current State reported by the BswM is in this state filter list.
[AUTOSAR] Description	This state filter drops SEvs if the current State reported by the BswM is in this state filter list. Tags: atp.Status=draft

IdsM/IdsMConfiguration/IdsMFilterChain/IdsMEventAggregation-Filter

[RTA-BSW] Description	All received events of a certain event ID that are received by this filter during a single aggregation time interval are not forwarded immediately. Instead, only the last or the first received SEv is stored in an aggregation buffer, depending on the configuration of "IdsMContextDataSourceSelector". The counter field of the SEv is modified so that it contains the sum of the counter fields of all incoming SEvs during the current aggregation time interval. At the end of the aggregation time interval, the buffered SEv is sent out and the aggregation buffer is cleared. If there was no incoming SEv until the end of the aggregation time interval, no message will be sent.
-----------------------	--

[AUTOSAR] Description	All received events of a certain event ID that are received by this filter during a single aggregation time interval are not forwarded immediately. Tags: atp.Status=draft
[REMOVED] Introduction	Instead, only the last or the first received SEv is stored in an aggregation buffer, depending on the configuration of "IdsMContextDataSourceSelector". The counter field of the SEv is modified so that it contains the sum of the counter fields of all incoming SEvs during the current aggregation time interval. At the end of the aggregation time interval, the buffered SEv is sent out and the aggregation buffer is cleared. If there was no incoming SEv until the end of the aggregation time interval, no message will be sent.

IdsM/IdsMConfiguration/IdsMFilterChain/IdsMEventAggregation-Filter/IdsMContextDataSourceSelector

[REMOVED] Introduction	IDS_M_FILTERS_CTX_USE_FIRST = ContextData of first received SEv is used for resulting QSEv. IDS_M_FILTERS_CTX_USE_LAST = ContextData of last received SEv is used for resulting QSEv.
[RTA-BSW] Enumeration Literals	IDS_M_FILTERS_CTX_USE_FIRST, IDS_M_FILTERS_CTX_USE_LAST, IDS_RB_FILTERS_CTX_USE_LAST_RANDOM_REPLACE
[AUTOSAR] Enumeration Literals	IDS_M_FILTERS_CTX_USE_FIRST, IDS_M_FILTERS_CTX_USE_LAST

[RTA-BSW] Description: The resulting SEv from the aggregation filter contains the context data from one of the following two sources: IDS_M_FILTERS_CTX_USE_FIRST = ContextData of first IdsMRbNumberOfContextDataAggregated received SEv are used for resulting QSEv. If IdsMRbTimestampTimesliceSelector = IDS_M_TIMESTAMP_TIMESLICE_FILTERCHAIN_INPUT, timestamp corresponds to the first received SEv. IDS_M_FILTERS_CTX_USE_LAST = ContextData of last IdsMRbNumberOfContextDataAggregated received SEv are used for resulting QSEv. If IdsMRbTimestampTimesliceSelector = IDS_M_TIMESTAMP_TIMESLICE_FILTERCHAIN_INPUT, timestamp corresponds to the last received SEv. IDS_RB_FILTERS_CTX_USE_LAST_RANDOM_REPLACE = IdsM will start to aggregate the contextData of all the SEv until IdsMRbNumberOfContextDataAggregated many SEv have been received. When more than IdsMRbNumberOfContextDataAggregated are received, the context data from the newest SEv is stored whilst deleting an older one at random. If IdsMRbTimestampTimesliceSelector = IDS_M_TIMESTAMP_TIMESLICE_FILTERCHAIN_INPUT, timestamp corresponds to the last received SEv. If IdsMRbTimestampTimesliceSelector = IDS_M_TIMESTAMP_TIMESLICE_FILTERCHAIN_OUTPUT, timestamp corresponds to the QSEv generation.

[AUTOSAR] Description: The resulting SEv from the aggregation filter contains the context data from one of the following two sources:

Tags: atp.Status=draft

IdsM/IdsMConfiguration/IdsMFilterChain/IdsMEventAggregation-Filter/IdsMEventAggregationTimeInterval

[RTA-BSW] Description	Length of the aggregation time interval (as float in seconds). Note: Shall be configured as a multiple of the IdsM main function period.
[AUTOSAR] Description	Length of the aggregation time interval (as float in seconds). Tags: atp.Status=draft
[REMOVED] Introduction	Note: Shall be configured as a multiple of the IdsM main function period.

IdsM/IdsMConfiguration/IdsMFilterChain/IdsMEventThresholdFilter

[RTA-BSW] Description	During each time interval "IdsMEventThresholdTimeInterval", the filter drops the first "IdsMEventThresholdNumber - 1" SEvs and forwards all other incoming SEvs immediately until the end of the time interval.
[AUTOSAR] Description	During each time interval "IdsMEventThresholdTimeInterval", the filter drops the first "IdsMEventThresholdNumber - 1" SEvs and forwards all other incoming SEvs immediately until the end of the time interval. Tags: atp.Status=draft

IdsM/IdsMConfiguration/IdsMFilterChain/IdsMEventThresholdFilter/IdsMEventThresholdNumber

[RTA-BSW] Description	This parameter assigns the threshold p for each SEv ID affected by this threshold filter. All SEvs p-1 are dropped, SEvs equal or greater than p are forwarded.
[AUTOSAR] Description	This parameter assigns the threshold 'p' for each SEv ID affected by this threshold filter. Tags: atp.Status=draft
[REMOVED] Introduction	All SEvs 'p-1' are dropped, SEvs equal or greater than 'p' are forwarded.

IdsM/IdsMConfiguration/IdsMFilterChain/IdsMEventThresholdFilter/IdsMEventThresholdTimeInterval

[RTA-BSW] Description	Length of the threshold time interval (as float in seconds). Note: Shall be configured as a multiple of the IdsM main function period.
-----------------------	--

[AUTOSAR] Description	Length of the threshold time interval (as float in seconds). Tags: atp.Status=draft
[REMOVED] Introduction	Note: Shall be configured as a multiple of the IdsM main function period.

IdsM/IdsMConfiguration/IdsMFilterChain/IdsMForwardEveryNthFilter

[RTA-BSW] Description	Out of all incoming SEVs, drop all but every nth. Those will be forwarded without modification.
[AUTOSAR] Description	Out of all incoming SEVs, drop all but every nth. Those will be forwarded without modification. Tags: atp.Status=draft

IdsM/IdsMConfiguration/IdsMFilterChain/IdsMForwardEveryNthFilter/IdsMNthParameter

[RTA-BSW] Description	For each SEv ID for which this filter is configured, this parameter assigns the appropriate n. Only 1 from n SEVs will be forwarded.
[AUTOSAR] Description	For each SEv ID for which this filter is configured, this parameter assigns the appropriate n. Tags: atp.Status=draft
[REMOVED] Introduction	Only 1 from n SEVs will be forwarded.

IdsM/IdsMConfiguration/IdsMPdus

[RTA-BSW] Description	Configuration of the PDU references used to send the events data.
[AUTOSAR] Description	Configuration of the PDU references used to send the events data. Tags: atp.Status=draft

IdsM/IdsMConfiguration/IdsMPdus/IdsMIIfTxPdu

[RTA-BSW] Description	IF PDU used to transmit a QSEv via the PduR to the IdsR. If the total size of the QSEv data to be transmitted fits in a single frame of the underlying bus, the IF PDU is used.
[AUTOSAR] Description	IF PDU used to transmit a QSEv via the PduR to the IdsR. Tags: atp.Status=draft
[REMOVED] Introduction	If the total size of the QSEv's data to be transmitted fits in a single frame of the underlying bus, the IF PDU is used.

IdsM/IdsMConfiguration/IdsMPdus/IdsMIfTxPdu/IdsMIfTxPduHandleId

[RTA-BSW] Description	IdsM does not use this parameter, content will be ignored. The existence of this parameter is needed by PduR.
[AUTOSAR] Description	IdsM does not use this parameter, content will be ignored. Tags: atp.Status=draft
[REMOVED] Introduction	The existence of this parameter is needed by PduR.

IdsM/IdsMConfiguration/IdsMPdus/IdsMIfTxPdu/IdsMIfTxPduRef

[RTA-BSW] Description	Reference to the IF PDU used for transmission of the QSEvs. IdsM is not post build capable. Though the container referred to has post-build capable configurations (Pdu/PduLength), IdsM does not support. The burden lays on the configurator / integrator to maintain the consistency in the configuration.
[AUTOSAR] Description	Reference to the IF PDU used for transmission of the QSEvs. Tags: atp.Status=draft

IdsM/IdsMConfiguration/IdsMPdus/IdsMTpTxPdu

[RTA-BSW] Description	TP PDU used to transmit a QSEv via the PduR to the IdsR. If the total size of the QSEv data to be transmitted is bigger than the size of a single frame of the underlying bus, the TP PDU is used. IdsM is not post build capable. Though the container referred to has post-build capable configurations (Pdu/PduLength), IdsM does not support. The burden lays on the configurator / integrator to maintain the consistency in the configuration.
[AUTOSAR] Description	TP PDU used to transmit a QSEv via the PduR to the IdsR. Tags: atp.Status=draft
[REMOVED] Introduction	If the total size of the QSEv's data to be transmitted is bigger than the size of a single frame of the underlying bus, the TP PDU is used.

IdsM/IdsMConfiguration/IdsMPdus/IdsMTpTxPdu/IdsMTpTxPduHandleId

[RTA-BSW] Description	IdsM does not use this parameter, content will be ignored. The existence of this parameter is needed by PduR.
[AUTOSAR] Description	IdsM does not use this parameter, content will be ignored. Tags: atp.Status=draft
[REMOVED] Introduction	The existence of this parameter is needed by PduR.

IdsM/IdsMConfiguration/IdsMPdus/IdsMTpTxPdu/IdsMTpTxPduRef

[RTA-BSW] Description	Reference to the TP PDU used for transmission of the QSEvs.
[AUTOSAR] Description	Reference to the TP PDU used for transmission of the QSEvs. Tags: atp.Status=draft

IdsM/IdsMGeneral

[RTA-BSW] Description	General configuration parameters of IdsM.
[AUTOSAR] Description	General configuration parameters of IdsM. Tags: atp.Status=draft

IdsM/IdsMGeneral/IdsMDevErrorDetect

[RTA-BSW] Description	This parameter enables/disables the Development Error Detection and Notification. true: Development error detection is enabled. false: Development error detection is disabled.
[AUTOSAR] Description	This parameter enables/disables the Development Error Detection and Notification. Tags: atp.Status=draft
[REMOVED] Introduction	true: Development error detection is enabled. false: Development error detection is disabled. Note: In general, the development error detection is recommended during pre-test phase. It is not recommended to enable the development error detection in production code due to increased runtime and ROM needs.

IdsM/IdsMGeneral/IdsMDiagnosticSupport

[RTA-BSW] Description	Enables or disables the Dcm APIs which are used to read and write certain values of the IdsM module through the diagnostic communication manager.- TRACE[SWS_IdsM_01800] If this is TRUE, IdsM will forward the DcmDspRoutines IdsM_Dcm_SetReportingMode and IdsM_Dcm_GetReportingMode_Start. IdsM does not forward the DcmDspRoutineIdentifier for these DcmDspRoutines.
[AUTOSAR] Description	Enables or disables the Dcm APIs which are used to read and write certain values of the IdsM module through the diagnostic communication manager. Tags: atp.Status=draft
[REMOVED] Introduction	true: Dcm APIs are enabled false: Dcm APIs are disabled

IdsM/IdsMGeneral/IdsMGlobalRateLimitationFilters

[RTA-BSW] Description	Global rate limitation filters for all SEVs.
[AUTOSAR] Description	Global rate limitation filters for all SEVs. Tags: atp.Status=draft

IdsM/IdsMGeneral/IdsMGlobalRateLimitationFilters/IdsMFilterEventRateLimitation

[RTA-BSW] Description	For configurable time intervals of length "IdsMRateLimitationTimeInterval" this filter forwards all the SEVs until reaching the limit "IdsMRateLimitationMaximumEvents". The limit is measured in number of incoming SEVs. Until the end of the time interval, all subsequent SEVs are dropped. This is helpful to cap the load that the IdsM generates unto information sinks like the IdsR. This filter is not specific to a single SEv but it applies to all SEVs handled by the current IdsM instance.
[AUTOSAR] Description	For configurable time intervals of length "IdsMRateLimitationTimeInterval" this filter forwards all the SEVs until reaching the limit "IdsMRateLimitationMaximumEvents". Tags: atp.Status=draft
[REMOVED] Introduction	The limit is measured in number of incoming SEVs. Until the end of the time interval, all subsequent SEVs are dropped. This is helpful to cap the load that the IdsM generates unto information sinks like the IdsR. This filter is not specific to a single SEv but it applies to all SEVs handled by the current IdsM instance. Note: Each possible SEv counts as a single one, regardless of its counter value.

IdsM/IdsMGeneral/IdsMGlobalRateLimitationFilters/IdsMFilterEventRateLimitation/IdsMRateLimitationMaximumEvents

[RTA-BSW] Description	The maximum number of SEVs which are passed on by this filter in a single rate limitation time interval. TRACE[SWS_IdSM_01080]
[AUTOSAR] Description	The maximum number of SEVs which are passed on by this filter in a single rate limitation time interval. Tags: atp.Status=draft

IdsM/IdsMGeneral/IdsMGlobalRateLimitationFilters/IdsMFilterEventRateLimitation/IdsMRateLimitationTimeInterval

[RTA-BSW] Description	Time interval length of the event rate limitation filter (as float in seconds). Note: Shall be configured as a multiple of the IdsM main function period. TRACE[SWS_IdSM_01080]
-----------------------	---

[AUTOSAR] Description	Time interval length of the event rate limitation filter (as float in seconds). Tags: atp.Status=draft
[REMOVED] Introduction	Note: Shall be configured as a multiple of the IdsM main function period.

IdsM/IdsMGeneral/IdsMGlobalRateLimitationFilters/IdsMFilterTrafficLimitation

[RTA-BSW] Description	The traffic limitation filter forwards all the incoming SEVs until reaching the limit "IdsMTrafficLimitation-MaximumBytes". The limit is measured in incoming amount of bytes. This filter forwards SEVs only, if the accumulated sizes of all incoming SEVs in the current traffic limitation time interval up until the current SEV is smaller or equal than a configurable maximum number of bytes "IdsMTrafficLimitation-MaximumBytes". The length of the traffic limitation time interval is configurable in "IdsMTrafficLimitationTimeInterval". This filter is not specific to a single SEV but it applies to all SEVs handled by the current IdsM instance.
[AUTOSAR] Description	The traffic limitation filter forwards all the incoming SEVs until reaching the limit "IdsMTrafficLimitation-MaximumBytes". Tags: atp.Status=draft
[REMOVED] Introduction	The limit is measured in incoming amount of bytes. This filter forwards SEVs only, if the accumulated sizes of all incoming SEVs in the current traffic limitation time interval up until the current SEV is smaller or equal than a configurable maximum number of bytes "IdsMTrafficLimitationMaximum-Bytes". The length of the traffic limitation time interval is configurable in "IdsMTrafficLimitation-TimeInterval". This filter is not specific to a single SEV but it applies to all SEVs handled by the current IdsM instance.

IdsM/IdsMGeneral/IdsMGlobalRateLimitationFilters/IdsMFilterTrafficLimitation/IdsMTrafficLimitationMaximumBytes

[RTA-BSW] Description	The maximum number of bytes to be sent out by the IdsM in a single traffic limitation time interval. TRACE[SWS_IdSM_01090]
-----------------------	--

[AUTOSAR] Description	The maximum number of bytes to be sent out by the IdsM in a single traffic limitation time interval. Tags: atp.Status=draft
-----------------------	--

IdsM/IdsMGeneral/IdsMGlobalRateLimitationFilters/IdsMFilterTrafficLimitation/IdsMTrafficLimitationTimeInterval

[RTA-BSW] Description	Length of the traffic limitation time interval (as float in seconds). Note: Shall be configured as a multiple of the IdsM main function period. TRACE[SWS_IdsM_01090]
[AUTOSAR] Description	Length of the traffic limitation time interval (as float in seconds). Tags: atp.Status=draft
[REMOVED] Introduction	Note: Shall be configured as a multiple of the IdsM main function period.

IdsM/IdsMGeneral/IdsMInstanceld

[RTA-BSW] Description	The unique identifier of the sending IdsM instance. This ID helps identifying the origin of a SEv, together with the SEv configuration parameters: ExternalEventId and the IdsMSensorInstanceld..
[AUTOSAR] Description	The unique identifier of the sending IdsM instance. Tags: atp.Status=draft
[REMOVED] Introduction	This ID helps identifying the origin of a SEv, together with the SEv configuration parameters: ExternalEventId and the IdsMSensorInstanceld. Note: There is only one IdsM (from the AUTOSAR Classic Platform) instance per ECU.

IdsM/IdsMGeneral/IdsMMainFunctionPeriod

[RTA-BSW] Description	The period between successive calls to the IdsM main function (as float in seconds) .. Note: Shall be configured as a multiple of the IdsM main function period.
[AUTOSAR] Description	The period between successive calls to the IdsM main function (as float in seconds). Tags: atp.Status=draft

IdsM/IdsMGeneral/IdsMNvmBlockDescriptor

[RTA-BSW] Description	Choose a NvM block descriptor reference, that is used to load and store the non-volatile data of IdsM module. The supported NvM block names are: RAM: IdsM_NvMRamBlockData ROM: IdsM_NvMRomBlockData.
-----------------------	---

[AUTOSAR] Description	Choose a NvM block descriptor reference, that is used to load and store the non-volatile data of IdsM module. Tags: atp.Status=draft
[REMOVED] Introduction	The supported NvM block names are: RAM: IdsM_NvMRamBlockData ROM: IdsM_NvMRomBlockData

IdsM/IdsMGeneral/IdsMSignature

[RTA-BSW] Description	If this container exists all qualified security events are signed by the crypto service.
[AUTOSAR] Description	If this container exists all qualified security events are signed by the crypto service. Tags: atp.Status=draft

IdsM/IdsMGeneral/IdsMSignature/IdsMCsmJobReference

[RTA-BSW] Description	This parameter references the Csm job that is used to generate signatures when qualified security events must be signed.
[AUTOSAR] Description	This parameter references the Csm job that is used to generate signatures when qualified security events must be signed. Tags: atp.Status=draft

IdsM/IdsMGeneral/IdsMSignature/IdsMSignatureLength

[RTA-BSW] Description	This parameter defines the length of the signature in bytes calculated by the crypto service.
[AUTOSAR] Description	This parameter defines the length of the signature in bytes calculated by the crypto service. Tags: atp.Status=draft

IdsM/IdsMGeneral/IdsMSignatureSupport

[RTA-BSW] Description	This parameter enables/disables the functionality of sending messages to the network with a signature of encryption calculated by the crypto services.
[AUTOSAR] Description	This parameter enables/disables the functionality of sending messages to the network with a signature of encryption calculated by the crypto services. Tags: atp.Status=draft

IdsM/IdsMGeneral/IdsMTimestamp

[RTA-BSW] Description	If this container exists a timestamp field is added to all qualified security events.
[AUTOSAR] Description	If this container exists a timestamp field is added to all qualified security events. Tags: atp.Status=draft

IdsM/IdsMGeneral/IdsMTimestamp/IdsMStbmTimeBaseReference

[RTA-BSW] Description	This parameter references the time source when the origin of the timestamp is AUTOSAR.
[AUTOSAR] Description	This parameter references the time source when the origin of the timestamp is AUTOSAR. Tags: atp.Status=draft

IdsM/IdsMGeneral/IdsMTimestamp/IdsMTimestampOption

[RTA-BSW] Description	This parameter species if the origin of the timestamp is from the AUTOSAR stack, from the application (custom timestamp) or from a BSW Module. TRACE[SWS_IdsM_01100]
[AUTOSAR] Description	This parameter species if the origin of the timestamp is from the AUTOSAR stack or from the application (custom timestamp). Tags: atp.Status=draft
[RTA-BSW] Enumeration Literals	AUTOSAR, Custom, Custom_CFUNC
[AUTOSAR] Enumeration Literals	AUTOSAR, Custom

IdsM/IdsMGeneral/IdsMVersionInfoApi

[RTA-BSW] Description	This parameter enables/disables the function IdsM_GetVersionInfo() to get major, minor and patch version information of the module.
[AUTOSAR] Description	This parameter enables/disables the function IdsM_GetVersionInfo() to get major, minor and patch version information of the module. Tags: atp.Status=draft

5.7 API Definition

List of supported API functions.

IdsM_Init	Initializes the IdsM management module.
IdsM_GetVersionInfo	Returns the version information of this module.
IdsM_MainFunction	Called periodically according the specified time interval.
IdsM_CopyTxData	Called to acquire the transmit data of an I-PDU segment (N-PDU).
IdsM_ReportSecurityEvent	Application interface to report security events to the IdsM.
IdsM_SetSecurityEvent	Application interface to report security events to the IdsM.
IdsM_SetSecurityEventWithContextData	Application interface to report security events with context data to the IdsM.
IdsM_SetSecurityEventWithCount	Application interface for Smart Sensors to report security events with a count value to the IdsM.
IdsM_SetSecurityEventWithCountContextData	Application interface for Smart Sensors to report security events with a count value and context data to the IdsM.
IdsM_SetSecurityEventWithTimestampCount	Application interface for Smart Sensors to report security events with a timestamp and a count value to the IdsM.
IdsM_SetSecurityEventWithTimestampCountContextData	Application interface for Smart Sensors to report security events with a timestamp.
IdsM_BswM_StateChanged	This callback function is invoked by the BswM to indicate ECU state changes.
IdsM_TransmissionSetState	This API is used by BswM to enable/disable the forwarding of the qualified security events (QSEvs).
IdsM_TpTxConfirmation	Called after the I-PDU has been transmitted on its network with the result indicating whether the transmission was successful or not.

IdsM_TxConfirmation	Communication interface module confirms the transmission of a PDU or the failure to transmit a PDU.
IdsM_Dcm_GetReportingMode_RequestResults	Service called by Dcm module to provide the routine results and reporting mode for requested SEv.
IdsM_Dcm_GetReportingMode_Start	Service called by Dcm module to request the reporting mode of a specific SEv.
IdsM_Dcm_SetReportingMode_Start	Service called by Dcm module to modify the reporting mode of a specific SEv.
IdsM_CsmNotification	Indicates that generation of signature by Csm via this callback is obtained if the signature is generated by asynchronous Csm job.

For details on using these calls, please refer to AUTOSAR_SWS_IntrusionDetectionSystemManager.pdf (version v22-11).

5.8 Configuration Advice

5.8.1 IdsM Configuration

Configure IdsM according to IdsM_EcucParamDef.arxml. The following order is recommended to configure IdsM successfully.

- All required and mandatory configuration parameters in IdsMGeneral
- Configure any and all IdsMEvents required. It is strongly recommended to configure AUTOSAR defined events with their AUTOSAR short names.
- Configure any IdsMFilterChains, required IdsMBlockState and link to IdsMEvents. Configure IdsMRbCustomFilters if required, define the IdsMRbCustomFilterFunction in C. Link the configured IdsMRbCustomFilters to IdsMFilterChains.
- Configure IdsMEventBuffers under IdsMBufferConfiguration based on the project requirement.
- Configure IdsMContextDataBuffers under IdsMBufferConfiguration if IdsMEvents with non-zero context data is configured.
- Configure IdsMPdus if at least one IdsMEvent has IdsMSinkIdsR enabled.

5.8.2 Configuration of CsmJob in IdsM

IdsM calls Csm_SignatureGenerate with either a asynchronous or synchronous job referenced by IdsMCsmJobReference. The length of the signature depends on the configuration of algorithm and the same length has to be requested by

IdsM. The length cannot be truncated as all the bytes are required for the signature verification. Appropriate key values of correct length as per the algorithm has to be configured in the job, key values are configured in CsmRbAutoConfigKeyElements. If CsmRbAutoConfigKeyElements is already configured then CsmKeyRef shouldn't be configured.

5.8.3 IdsM Interface to Dcm

When the IdsM interface to Dcm parameter IdsMDiagnosticSupport is set to TRUE, then the user is required to configure the routine containers in DcmDspRoutine with the following fixed name:

- IdsM_Dcm_SetReportingMode
- IdsM_Dcm_GetReportingMode

In addition the parameter DcmDspRoutineIdentifier must be configured with a specific value for each container.

Optional IdsM Configuration

- Configure PduR and lower network layers if IdsMSinkIdsR is enabled for at least one IdsMEvent. IdsM_Validate action will validate this constraint.
- Configure a StbMSynchronizedTimeBase if IdsMTimestampOption is configured to AUTOSAR Timestamp. IdsM_Validate action will validate this constraint.
- Configure SecOC or Dcm as sensor modules by referring to their corresponding documentation.

5.9 Integration Advice

5.9.1 Init Functions

- IdsM_Init shall be placed after NvM_Init.

5.9.2 Delnit Functions

No Delnit is required.

5.9.3 Scheduled Functions

- When SinkIdsR is used, IdsM_MainFunction is recommended to be scheduled before the communication driver's transmission main functions.
- It would be beneficial to schedule the sensors module's SEv detection function before IdsM_MainFunction. This is only a recommendation to minimize delay in logging and forwarding QSEvs.

5.9.4 OS Resources

The number of the active OS resources, and their usage, depends on the user

configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The IdsM module uses the following OS resources:

- IdsM_StbM_GetCurrentTime
- IdsM_EventBuffer
- IdsM_GlobalContextDataModifier

5.10 Hardware Requirements

Table 5.129.IdsM Hardware Requirements

Support	Usage
FLASH	2480 bytes
RAM	302 bytes

Usage calculated based on a sample configuration.

6 rba_CryptoAuCSC

Module name	rba_CryptoAuCSC
Package	RTA-SEC
AUTOSAR Module ID	N/A
AUTOSAR Version	22-11

6.1 Introduction

The rba_CryptoAuCSC module is part of the micro controller abstraction layer which is the lower layer of the Crypto Interface and Crypto Service Manager modules. It implements a generic interface for synchronous and asynchronous cryptographic primitives and also supports key storage, key configuration, and key management for cryptographic services. The keys and the cryptographic primitives can be configured by using the standard AUTOSAR configuration mechanisms (provided through the Crypto component).

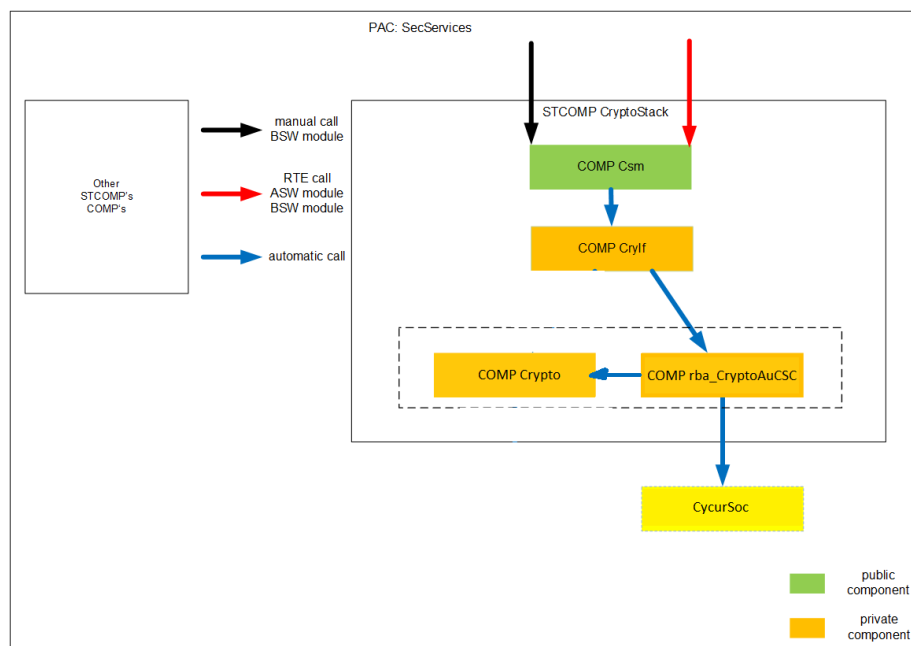


Figure 6.1.rba_CryptoAuCSC Interaction

The component rba_CryptoAuCSC is part of the following software stack:

- Csm (COMP: Csm), mapping the abstract AUTOSAR standardized user interfaces to the less abstracted interfaces of Crypto.
- Crylf (COMP: Crylf), acting as an additional mapping layer between Csm and rba_Crypto <variant> components.
- Crypto (COMP: Crypto), implement the configuration interface for all Crypto components, also provides libraries for common or AUTOSAR defined functionalities which are then used by rba_CryptoAu <variant> components

- rba_CryptoAuCSC (COMP: rba_CryptoAuCSC), implements a generic interface for synchronous and asynchronous cryptographic primitives by using the CycurSoc (HSM) hardware. With the CycurSoc (HSM) offering the underlying cryptographic functionality for rba_CryptoAuCSC.
- The CycurSoc itself is used by rba_CryptoAuCSC but is not a part of the structural component Crypto.
- SecServices also contains the components required to integrate this software stack, with the exception of CycurSoc.

Module Configuration

This module's configuration parameters are included in the configuration of another module. See the Configuration Advice section for more details.

6.2 Supported Features

6.2.1 General

rba_CryptoAuCSC is a component which provides AUTOSAR APIs and functionalities by using the Crypto library component. It also interfaces with the CycurSoc to perform the required cryptographic operations. The component can be configured via the Crypto module-provided parameter definition.

It has the following features:

- Support for exporting *SHE RAM* keys and loading of *SHE plaintext* keys.
- Streaming support for *AES CTR* encryption/decryption.
- Streaming support for *AES CBC* encryption/decryption.
- Streaming support for *ChaCha20-Poly1305 AEAD* encryption/decryption.
- Streaming support for *AES GCM AEAD* encryption/decryption.

6.3 Extensions

6.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

- No known extension parameters are defined in this version.

6.4 Limitations

Encrypt and Decrypt (AES, CTR and CBC) in streaming mode:

In *UPDATE* calls, output data will always be generated in AES blocks (16 bytes). Particularly if input buffering is enabled, output buffers should be ready to receive at least one additional block on top of the blocks expected based on the input data.

If input buffering is disabled, any input data provided with jobs that have the *UPDATE* mode flag set must be sized on 16-byte boundaries (CycurSoC can only

process full AES blocks).

All jobs with the *FINISH* mode flag set must have a valid output data buffer, even if no further output data is expected.

If PKCS padding is used, the ciphertext will be up to one full AES block (16 bytes) bigger than the provided plaintext. For decryption, this last ciphertext block *MUST BE* provided with the *FINISH* call. For encryption, this means that the *FINISH* call will always produce at least one more block of ciphertext.

Hash in streaming mode:

All input data must be provided via an *UPDATE* call; in the *FINISH* call, the input data is ignored.

MAC Generate and Verify in streaming mode:

All input data must be provided via an *UPDATE* call; in the *FINISH* call, the input data is ignored.

AEAD Encrypt and Decrypt in streaming mode:

Only *AES GCM* and *ChaCha20-Poly1305* are supported in streaming mode. *AES CCM* support is limited to single-call mode.

In *UPDATE* calls, output data will always be generated in 16-byte blocks. Particularly if input buffering is enabled, output buffers should be ready to receive at least one additional block on top of the blocks expected based on the input data.

The associated data (secondary input) has to be fully supplied upon the first *UPDATE* call, with real input data (plaintext / ciphertext). This means that there can be exactly one *UPDATE* call which has associated data and input data. If there is associated data included in an *UPDATE* call, after the first *UPDATE* call with input data, CycurSoC will reject the request and the job will fail accordingly.

If input buffering is disabled, any input data provided with jobs that have the *UPDATE* mode flag set must be sized on 16-byte boundaries.

If input buffering is disabled, or the plaintext is sized exactly on 16-byte boundaries, the last chunk of input data has to be provided in the *FINISH* call. Otherwise, CycurSoC will reject the request and the job will fail accordingly.

KeyElementSet:

If the *exportable* property shall be set for a key injected into CycurSoC using *KeyElementSet*, the corresponding Key Element's read access parameter (*CryptoKeyElementReadAccess*) has to be configured as *CRYPTO_RA_ALLOWED*.

It is not possible to directly set a key pair with *KeyElementSet* because CycurSoC expects asymmetric keys to be split into private and public key parts. Key pairs that are supposed to be brought into the system with *KeyElementSet* must be broken down into dedicated key elements for the private and public key part. The resulting key elements should point to the same CycurSoC key slot (i.e. the same *CryptoRbAuCscKeyId* value).

If provisioning is enabled (the *CryptoRbAuCscKeyProvisioning* parameter in *CryptoGeneral*), persistent keys will be erased (if needed) and then provisioned for every *KeyElementSet* request. If *KeyElementSet* shall only update persistent keys (which have already been provisioned), provisioning must be disabled.

KeyElementGet:

It is not possible to directly retrieve a key pair with KeyElementGet because CycurSoC expects asymmetric keys to be split into private and public key parts. The private and public parts of a key pair must therefore be put into dedicated key elements and requested individually with KeyElementGet.

TLS secrets can only be retrieved if the corresponding Key Element's size parameter (CryptoKeyElementSize) is configured to exactly 48 bytes. This takes precedence over other symmetric key retrievals. Therefore, it has to be ensured that all other symmetric keys - which need to be extracted with KeyElementGet - have a size configuration other than 48 bytes.

KeyElementCopy:

There is currently no internal copying mechanism implemented in CycurSoC, so KeyElementCopy will do a KeyElementGet + KeyElementSet sequence. Using the same key material in multiple key elements can be achieved by setting the same CycurSoC key slot for all of them (i.e. the same CryptoRbAuCscKeyId value).

Key Exchange:

Both KeyExchangeCalcPubVal and KeyExchangeCalcSecret require a valid key pair to already be present in the key elements used for key exchange. The passage in the Csm AUTOSAR specification stating that "if KeyExchangeCalcPubVal is called but no valid key material exists (key is not valid or essential key elements have length=0), the function shall generate the necessary key material and continue as normal" is neither implemented in Csm nor in rba_CryptoAuCSC.

SAFETY CONSIDERATIONS:*ASIL / QM separation:*

The parameters Crypto/CryptoGeneral/CryptoRbInputBuffering and Crypto/CryptoGeneral/CryptoRbOutputBuffering control if rba_CryptoAuCSC allocates extra buffers for the interactions with CycurSoC. If both parameters are configured as *TRUE*, all inputs and outputs will be buffered by rba_CryptoAuCSC. The memory region used for those buffers can be controlled with the MemMap defines that contain the *_SHARED_* keyword. If one (or both) parameter(s) is configured as *FALSE*, the application buffers in the particular direction (or both directions) are given directly to CycurSoC.

CycurSoC Job Priority Mapping

CryptoRbDriverObjectPriority	CycurSoC Job Priority
LOWEST	cycursoc_JobPriority_Low
LOW	cycursoc_JobPriority_Low
NORMAL	cycursoc_JobPriority_Normal
HIGH	cycursoc_JobPriority_High
HIGHEST	cycursoc_JobPriority_High

6.5 Exclusions

The following generic API functions listed in the Crypto Driver module specification have no functional equivalents in this module:

- Crypto_CustomSync
- Crypto_NvBlock_Init_<NvBlock>
- Crypto_NvBlock_ReadFrom_<NvBlock>
- Crypto_NvBlock_WriteTo_<NvBlock>
- Crypto_NvBlock_Callback_<NvBlock>

6.5.1 Configuration Exclusions

- There are no known exclusions in this version.

6.6 Deviations

- There are no known deviations in this version.

6.7 API Definition

List of supported API functions.

rba_CryptoAuCSC_Startup	Initializes the module.
rba_CryptoAuCSC_Persist	Performs persistent operations in module.
rba_CryptoAuCSC_Shutdown	Deinitializes the module.
rba_CryptoAuCSC_Process	Asynchronous job processing. Holds the identifier of the partition which related resources shall be processed.
rba_CryptoAuCSC_CheckJob	Checks the validity of the job requested for the given driver object.
rba_CryptoAuCSC_ProcessDriverObject	Starts the processing of the job registered in the given driver object.
rba_CryptoAuCSC_CleanupDriverObject	Cleans up variant specific driver object.
rba_CryptoAuCSC_ProcessJob	Performs the crypto primitive that is configured in the job parameter.
rba_CryptoAuCSC_CancelJob	Removes the provided job from the queue and cancels the processing of the job if possible.
rba_CryptoAuCSC_KeyCopy	Copies a key with all its elements to another key in the same crypto driver.
rba_CryptoAuCSC_KeyElementCopy	Copies a key element to another key element in the same crypto driver.

rba_CryptoAuCSC_KeyElementCopyPartial	Copies a key element to another key element in the same crypto driver. The keyElementSourceOffset and keyElementCopyLength allows to copy a part of the source key element into the destination.
rba_CryptoAuCSC_KeyElementGet	Used to get a key element of the key identified by the cryptoKeyld and store the key element in the memory location pointed by the result pointer.
rba_CryptoAuCSC_KeyElementIdsGet	Used to retrieve information which key elements are available in a given key.
rba_CryptoAuCSC_KeyDerive	Derives a new key by using the key elements in the given key identified by the cryptoKeyld.
rba_CryptoAuCSC_KeyElementSet	Sets the given key element bytes to the key identified by cryptoKeyld.
rba_CryptoAuCSC_KeySetValid	Sets the key state of the key identified by cryptoKeyld to valid.
rba_CryptoAuCSC_KeySetInvalid	Sets the key state to invalid for the status of the key identified by cryptoKeyld.
rba_CryptoAuCSC_KeyGetStatus	Returns the key state of the key identified by cryptoKeyld.
rba_CryptoAuCSC_KeyExchangeCalcPubVal	Calculates the public value for the key exchange and stores the public key in the memory location pointed by the public value pointer.
rba_CryptoAuCSC_KeyExchangeCalcSecret	Calculates the shared secret key for the key exchange with the key material of the key identified by the cryptoKeyld and the partner public key.
rba_CryptoAuCSC_KeyGenerate	Generates new key material stored in the key identified by cryptoKeyld.
rba_CryptoAuCSC_RandomSeed	Generates the internal seed state using the provided entropy source.

6.8 Configuration Advice



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

6.9 Integration Advice

6.9.1 General

The rba_CryptoAuCSC module cannot be used as a standalone module; it always requires the Crypto component to provide and implement AUTOSAR features. It is always bundled with the other Crypto stack modules from the same package delivery.

6.9.1.1 Exclusive Areas

A schedule manager in principle is not required as the components Csm, Crypto are libraries in a narrower sense and could be called directly by a user when the libraries and the user software are resident on the same core.

In all other cases, the call has to be managed by the RTE, for this there is a second component chain planned as the call of the cryptographic functions via RTE is specified in the AUTOSAR module Crylf. The AUTOSAR module Crylf is also specified for the realization of scheduled cryptographic functions.

6.9.1.2 Software Environment

As the Crypto component is an AUTOSAR component, the central AUTOSAR definitions (e.g. file compiler.h) and data types (e.g. file Std_Types.h) have to be present in the software environment.

Only the Csm may call cryptographic services. It is not allowed to call any Crypto provided function directly by the user application.

6.9.2 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The Rba_CryptoAuCSC module uses the following OS resources:

- rba_CryptoAuCSC_CycurSoc

6.10 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

7 rba_CryptoCCL

Module name	rba_CryptoCCL
Package	RTA-SEC
AUTOSAR Module ID	N/A
AUTOSAR Version	22-11

7.1 Introduction

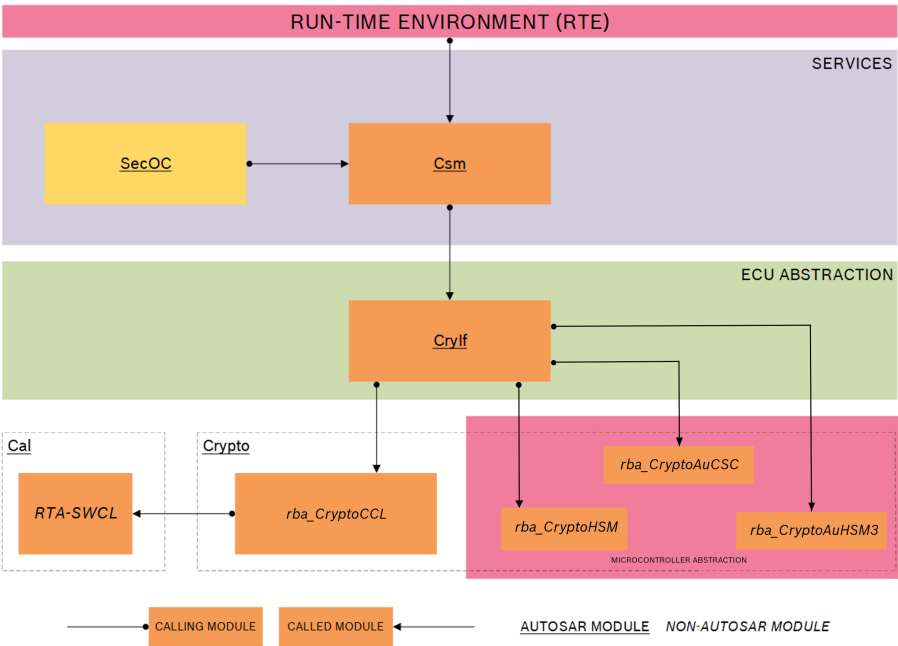


Figure 7.1.RTA-SEC Stack Architecture

rba_CryptoCCL is a driver module that is positioned outside of the AUTOSAR layered architecture. The diagram above shows the relationship between rba_CryptoCCL and the other components of the RTA-SEC stack.

rba_CryptoCCL is not defined separately in the AUTOSAR specification, but is a structural component of the generic AUTOSAR-specified Crypto Driver (Crypto) module. As such, its API functions closely follow those specified by AUTOSAR for the Crypto module, as they are a software-based implementation of some of those functions. Other functions specified for the Crypto module are provided by the rba_CryptoHSM, rba_CryptoAuCSC and rba_CryptoAuHSM3, which are drivers for hardware-based functions such as random number generation and signature verification.

- rba_CryptoCCL is a software-based driver, which is used by the Crypto module to configure specific instances of driver objects.
- rba_CryptoCCL is called from the Crypto Interface module (CryIf) using a specific configuration defined in Crypto, and it calls cryptographic primitives

defined in the external RTA-SWCL library module to perform specific cryptographic functions.

- The path from Csm, via CryIf and Crypto, through to a specific configured driver object is known as a channel. Multiple channels are defined based on each specific configured driver object required by the SWC(s) making the service requests through Csm.
- rba_CryptoCCL processes jobs requested by CryIf in either synchronous or asynchronous mode, and it provides the Key Management functions described by the AUTOSAR specification for the generic Crypto module.

Module Configuration

This module's configuration parameters are included in the configuration of another module. See the Configuration Advice section for more details.

7.2 Supported Features

7.2.1 General

The rba_CryptoCCL module provides the following functionality:

- Processing (and cancellation) of jobs.
- Key Management (Key) - copy, derive, set valid.
- Key Management (Key Element) - get, get Id, copy, set.
- *CryptoCCL and CryptoHSM KeyElementSet* support for *RSA ASN.1*-encoded Public Key information.
- Hash Calculation for empty string.
- Change CRYPTO_KE_CIPHER_PROOF key element ID to 6.

7.2.2 Cryptographic Services

Please refer to the Supported Features section of the Csm module for details of the cryptographic services supported by rba_CryptoCCL.

7.2.3 MainFunction Partitioning

RTA-SEC supports MainFunction partitioning of resources. For rba_CryptoCCL, this involves the generation of multiple Crypto_MainFunction instances and the mapping of CryptoDriverObjects to these instances where they should be handled.

7.2.4 Job Context Interface

The Job Context Interface allows saving/restoring the workspace's context data for a specific service from the Crypto driver. All dynamically created data is therefore stored within the driver, and can later be restored to continue the operation at the exact point where the context snapshot was taken. Key Element data are not affected. (This means that key elements are not part of the context data and must be set/read by the Key Element interface separately if necessary.)

Supported Services: Hash, Encrypt, AEADEncrypt, AEADDecrypt, Decrypt, MacGenerate, MacVerify, SignatureGenerate or SignatureVerify.

Context length in bytes for all context services:

Service	Context Length in bytes (approx)
HASH	424
ENCRYPT	548
DECRYPT	548
AEADENCRYPT	856
AEADDECRYPT	856
MACGENERATE	792
MACVERIFY	792
SIGNATUREGENERATE	424
SIGNATUREVERIFY	424



NOTE

The above values are an approximation. For primitives with algorithm mode *PURE*, nothing can be saved or restored. The Job Context Interface is only supported in CCL.

Please see Section 7.8.10 for configuration advice.

7.3 Extensions

7.3.1 Configuration Extensions

- No known extension parameters are defined in this version.

7.3.2 Additional Services

The following API are available in v12.7.0 in addition to those defined in AR v22-11

7.3.2.1 rba_CryptoCCL_DeInit

Syntax	rba_CryptoCCL_Deinit()
Parameters	NONE
Description	Deinitializes the Crypto Driver

7.3.2.2 rba_CryptoCCL_IsInitialized

Syntax	rba_CryptoCCL_IsInitialized()
Parameters	NONE
Return Value	TRUE - Module is initialised.

	FALSE - Module is not initialised
Description	Performs a check on the initialised status of the module.

7.3.2.3 rba_CryptoCCL_IsDeinitialized

Syntax	rba_CryptoCCL_IsDeinitialized(void)
Parameters	NONE
Return Value	TRUE - Driver is already deinitialized.
	FALSE - Driver is not deinitialized
Description	Check if the deinitialization is already done.

7.3.2.4 rba_CryptoCCL_Rb_StorePermanentData

Syntax	rba_CryptoCCL_Rb_StorePermanentData()
Parameters	NONE
Return Value	E_OK: Request successful.
	E_NOT_OK: Request failed.
	CRYPTO_E_PENDING: Request pending. Function has to be called again.
Description	Triggers persistent data store if needed and returns the save operation status.
	This function must be used before an ECU is reset or shut down if persistent data is required by the calling process.

7.3.2.5 rba_CryptoCCL_KeyGetStatus

Syntax	Std_ReturnType rba_CryptoCCL_KeyGetStatus(uint32 cryptoKeyId, Crypto_KeyStatusType* keyStatusPtr)
Parameter In	cryptoKeyId: The ID of the key for which the key state shall be returned.
Parameter Out	keyStatusPtr: Pointer to the data where the status of the key shall be stored.
Description	Returns the key state of the key identified by cryptoKeyId.

7.4 Limitations

7.4.1 Key Management Functions

- The functions rba_CryptoCCL_KeyExchangeCalcPubVal, rba_CryptoCCL_KeyExchangeCalcSecret and rba_CryptoCCL_KeyDerive run only in single call mode and do not support the streaming approach (start, update, finish).
- The wait while the Crypto driver is busy [SWS_Crypto_00120] has not been implemented because it requires extensive exclusive access protection of

variables. Also the active blocking of sync call shall be avoided, it is better to decline a sync job than to keep it waiting.

- The return of CRYPTO_E_RE_KEY_NOT_AVAILABLE [SWS_Crypto_00140] has not been implemented because it contradicts [SWS_Crypto_00086].
- To copy one key into another, the number and size of key elements (target and source key) must be equal, otherwise the function `rba_CryptoCCL_KeyCopy` will return with error `E_NOT_OK` (for a number mismatch) and `CRYPTO_E_KEY_SIZE_MISMATCH` (for a size mismatch). [SWS_Crypto_02305] [SWS_Crypto_02306]
- The length information of a key element is not stored permanently. The function `rba_CryptoCCL_KeyElementGet` always returns the whole element and not just a part of it. If the key element is written with data smaller than the size of the element, the remaining part is filled with 0x00. [SWS_Crypto_02311]

7.5 Exclusions

The following generic API functions listed in the Crypto Driver module specification have no functional equivalents in this module:

- `Crypto_CustomSync`
- `Crypto_NvBlock_Init_<NvBlock>`
- `Crypto_NvBlock_ReadFrom_<NvBlock>`
- `Crypto_NvBlock_WriteTo_<NvBlock>`
- `Crypto_NvBlock_Callback_<NvBlock>`

7.5.1 Configuration Exclusions

- There are no known exclusions in this version.

7.6 Deviations

- There are no known deviations in this version.

7.7 API Definition

List of supported API functions.

<code>rba_CryptoCCL_Init</code>	Initializes the module.
<code>rba_CryptoCCL_CancelJob</code>	Removes the provided job from the queue and cancels the processing of the job if possible.

rba_CryptoCCL_CertificateParse	Parses the certificate data stored in the key element CRYPTO_KE_CERT_DATA and fills the key elements CRYPTO_KE_CERT_SIGNEDDATA and CRYPTO_KE_CERT_PARSEDPUBLICKEY and CRYPTO_KE_CERT_SIGNATURE.
rba_CryptoCCL_CertificateVerify	Verifies the certificate stored in the key referenced by cryptoValidateKeyld with the certificate stored in the key referenced by cryptoKeyld.
rba_CryptoCCL_GetVersionInfo	Returns the version information of this module.
rba_CryptoCCL_KeyCopy	Copies a key with all its elements to another key in the same Crypto driver object.
rba_CryptoCCL_KeyDerive	Derives a new key by using the key elements in the given key identified by the cryptoKeyld.
rba_CryptoCCL_KeyGenerate	Holds the identifier of the key which is to be updated with the generated value.
rba_CryptoCCL_KeyElementCopy	Copies a key element to another key element in the same Crypto driver object.
rba_CryptoCCL_KeyElementCopyPartial	Uses keyElementSourceOffset and keyElementCopyLength to copy partial key element to another key element in the same crypto driver.
rba_CryptoCCL_KeyElementGet	Retrieves a key element of the key identified by the cryptoKeyld and stores the key element in the memory location pointed by the result pointer.
rba_CryptoCCL_KeyElementIdsGet	Retrieves information on which key elements are available for a given key.
rba_CryptoCCL_KeyElementSet	Sets the given key element bytes to the key identified by cryptoKeyld.
rba_CryptoCCL_KeyExchangeCalcPubVal	Calculates the public value for the key exchange and stores the public key in the memory location pointed to by the public value pointer.
rba_CryptoCCL_KeyExchangeCalcSecret	Calculates the shared secret key for the key exchange with the key material of the key identified by the cryptoKeyld and the partner public key. The shared secret key is stored as a key element in the same key.
rba_CryptoCCL_KeySetValid	Sets the key state of the key identified by cryptoKeyld to valid.
rba_CryptoCCL_MainFunction	The module main function which provides cyclic processing of asynchronous jobs.
rba_CryptoCCL_ProcessJob	Performs the crypto operation that is configured in the job parameters.
rba_CryptoCCL_KeyGenerate	Generates new key material and store it in the key identified by cryptoKeyld.

rba_CryptoCCL_RandomSeed	This function generates the internal seed state using the provided entropy source. This function can also be used to update the seed state with new entropy.
rba_CryptoCCL_KeyGetStatus	Returns the key state of the key identified by cryptoKeyld.
rba_CryptoCCL_KeySetInvalid	Sets the key state of the key identified by cryptoKeyld to invalid.

For details on using these calls, please refer to AUTOSAR_SWS_CryptoDriver.pdf (version v22-11).

7.8 Configuration Advice



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

7.8.1 General Advice

The Crypto module provides the AUTOSAR-defined configuration parameters required to define unique driver object instances for cryptographic primitives defined in rba_CryptoCCL (and the other Crypto stack modules).

Configuration advice relating to the standard AUTOSAR parameters can be found in the Crypto section of this Reference Guide. Please also refer to the AUTOSAR documentation on the Crypto driver (AUTOSAR_SWS_CryptoDriver.pdf) for full details of these standard configuration parameters.

The configuration advice below is specific to rba_CryptoCCL.

7.8.2 Job Processing

The Process Job function (rba_CryptoCCL_ProcessJob) can return a value of RBA_CRYPTTO_E_JOB_PENDING if the key exchange calculation has not yet finished. The call must be repeated until the function returns with E_OK or an error (depending on the setting of CryptoRbCclKeyExchangeLoopCounter, see below). If the loop counter is set to a very high value, the execution of the Process Job function is not divided into several parts and will just return with the final result instead of RBA_CRYPTTO_E_JOB_PENDING.

7.8.3 MainFunction Partitioning

RTA-SEC supports MainFunction partitioning of resources. For rba_CryptoCCL, this involves the generation of multiple Crypto_MainFunction instances and the mapping of CryptoDriverObjects to these instances where they should be

handled.

The generation of multiple `Crypto_MainFunction` instances is based on configuration via `CryptoMainFunction` containers. The following parameters need to be configured for each `CryptoMainFunction` container:

- **CryptoMainFunctionPeriod**: The periodicity for this `MainFunction` instance.
- **CryptoMainFunctionPartitionRef**: The `EcucPartition` (often interpreted as core) assignment for this `MainFunction` instance.
- **CryptoRbMainFunctionLoopCounter**: Holds the number of iterations to be performed internally by the `MainFunction` instance. This parameter can be used to increase or reduce the runtime of a single `MainFunction` call by processing multiple jobs at once (same usage as for the default/legacy `MainFunction`).

The default/legacy `MainFunction` (`rba_CryptoCCL_MainFunction`) is still generated, and it performs two distinct roles:

- It updates NVM blocks for modified persistent keys.
- It processes `CryptoDriverObjects` which are not explicitly linked to a `CryptoMainFunction` container.

If all `CryptoDriverObjects` are explicitly mapped to a `CryptoMainFunction` container, then the default/legacy `rba_CryptoCCL_MainFunction` will handle only NVM related operations.

CryptoDriverObjects Partitioning

`CryptoDriverObjects` can be explicitly assigned to a specific `MainFunction` container, and will be processed only by that specific `MainFunction` instance.

`CryptoDriverObjects` that are not mapped to a `MainFunction` container will be processed by the default `Crypto_MainFunction` function.

MainFunction Forwarding

If automatic container filling is enabled, for `CsmQueues` linked to `MainFunction` containers, analogous `MainFunction` containers will be forwarded in `Crypto Drivers` and these will be automatically linked to referenced `CryptoDriverObjects`.

If there are already `CryptoMainFunction` containers configured with the same parameters, they are linked by the `Csm` forwarder instead.

7.8.4 CryptoRbCclKeyExchangeLoopCounter

The Key Exchange function will be called several times by the main function in the background to get the correct result. The parameter `CryptoRbCclKeyExchangeLoopCounter` can be used to reduce the run time of a single main function cycle, but the effect will be an increase in the total run time.

7.8.5 CryptoRbCclSignatureLoopCounter

The Signature function will be called several times by the main function in the background to get the correct result. The parameter `CryptoRbCclSignatureLoop-`

Counter can be used to reduce the run time of a single main function cycle, but the effect will be an increase in the total run time.

7.8.6 CryptoKeyElement

When configuring `CryptoKeyElement` for `RandomGenerate` primitive, the following values must be set at configuration time for the `CryptoKeyElement` with `CryptoKeyElementId = 3`:

- `CryptoKeyElementInitValue = 0x01`
- `CryptoKeyElementSize = 1`

7.8.7 CryptoKeyNvBlockRef

If `CryptoKeyNvBlockRef` not configured, then the key is mapped to a default NvM block (the first one found in Crypto Driver configuration).

7.8.8 Non-volatile Memory Blocks

One or more NvM blocks are needed to store the key status flags and key element data. The number of blocks required is equal to the number of unique `CryptoNvBlock` configured in the `CryptoKeys`.

The NvM blocks used by the Crypto Driver can be configured in the NvM Mappings configurations. The NvM block can be partially configured, in which case the Crypto Driver will automatically complete the mandatory fields.

If no NvM blocks are manually configured and/or linked through NvM Mappings then the Crypto Driver will automatically forward blocks with necessary size to the NvM component. The data is stored in an array of bytes (`uint8[]`).

7.8.9 NvmBlockDescriptor

The `NvmBlockDescriptor` container must be pre-configured by the integrator along with all project specific parameters except for the following parameters which are always forwarded by the Crypto forwarder action:

- `NvMNvBlockLength`
- `NvMBlockUseSyncMechanism`
- `NvMRamBlockDataAddress`
- `NvMRomBlockDataAddress`

Note: The Crypto forwarder action does not check if any of the above parameters are already configured inside the referenced `NvMBlockDescriptor` container. Manual configuration of any of the above parameters inside the `NvMBlockDescriptor` container will result in upper multiplicity build errors.

7.8.10 Job Context Interface

The parameter `CryptoPrimitiveSupportContext` can be configured if the Crypto primitive supports storing/restoring the workspace's context data. Since this option is vulnerable to security, it shall only set to *TRUE* if absolutely needed.

7.9 Integration Advice

7.9.1 General

The rba_CryptoCCL module cannot be used as a stand-alone entity. It is reliant on the software library RTA-SWCL, and cannot be used without the Crylf, Csm and Crypto modules.

7.9.2 Init Functions

rba_CryptoCCL_Init initializes the rba_CryptoCCL module. As this initialization process can take some time to complete, the rba_CryptoCCL_IsInitialized function can be used to check when initialization has been completed.

7.9.3 DeInit Functions

rba_CryptoCCL_Deinit shuts down the rba_CryptoCCL module in case of a shutdown of the ECU. This function ensures that resources that are no longer required are released or deleted.

7.9.4 Scheduled Functions

rba_CryptoCCL_MainFunction provides cyclic processing of asynchronous jobs.

To avoid Maintask cycles being wasted, Maintasks should be called in top-down order (e.g. SecOC → Csm → CryptoDriver).

7.9.5 Job Processing

The synchronous calling of jobs can be done as single call or via a streaming approach (start, update, finish). For single calls, the job is called and returns with the result. For the streaming approach, the job must be called several times, and the result is only available and returned after the finish operation has been successfully completed.

The asynchronous calling of jobs can be done as single call or via the streaming approach. The initial rba_CryptoCCL_ProcessJob call will register the job internally and prepare it for processing. The cyclic main function checks the internal queue and processes the job in the background, based on the priority. If the final result is available, a callback function in Crylf (Crylf_CallbackNotification) is called to report the final result back to Csm.

7.9.6 Persistent Storage

The contents of all keys and their key elements are stored in application RAM during processing. This applies to both the actual data and the status of the individual keys. However, there are differences between volatile key elements and persistent key elements:

- Key elements are considered persistent if they are configured with CryptoKeyElementPersist = TRUE, otherwise they are volatile.
- The initialisation data of volatile key elements is loaded from the ECU flash memory into application RAM each time an ECU is started. However, the

validity of the keys must always be recreated, as it is lost after shutdown of the ECU.

- The initialisation data of persistent key elements is loaded from the ECU flash memory into a configured NvM storage block when the ECU is first started. If the ECU is subsequently restarted, the data is loaded from the NvM storage block into application RAM. The key validity is also stored persistently, and is available again after each restart. In contrast to the volatile key elements, this data need not be recreated when the ECU is restarted.
- An additional function `rba_CryptoCCL_Rb_StorePermanentData` is provided to explicitly trigger the storage of persistent key elements into NvM storage. This function checks if a store operation is required (i.e. some data has been changed) before performing the request, and it only updates the data if necessary. This function should be called to store persistent key elements (if required) before shutting down or restarting the target ECU. Note that this function depends on the cyclic operation of `rba_CryptoCCL_MainFunction`.

7.9.7 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The `Rba_CryptoCCL` module does not use any exclusive OS resources.

7.10 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

8 rba_CryptoHSM

Module name	rba_CryptoHSM
Package	RTA-SEC
AUTOSAR Module ID	N/A
AUTOSAR Version	22-11

8.1 Introduction

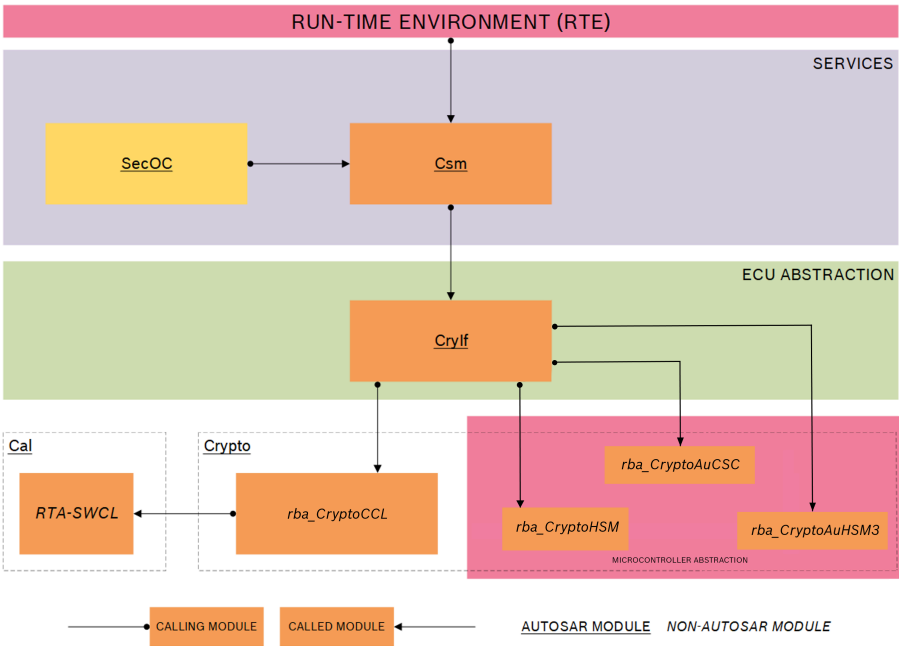


Figure 8.1.RTA-SEC Stack Architecture

rba_CryptoHSM is a driver module which is positioned in the Microcontroller Abstraction layer of the AUTOSAR layered architecture. The diagram above shows the relationship between rba_CryptoHSM and the other components of the RTA-SEC stack.

rba_CryptoHSM is not defined separately in the AUTOSAR specification, but is a structural component of the generic AUTOSAR-specified Crypto Driver module (Crypto). As such, its API functions closely follow those specified by AUTOSAR for the Crypto module, as they are a hardware-based implementation of some of those functions. Other hardware-based functions are provided by rba_CryptoAuCSC and rba_CryptoAuHSM3, while rba_CryptoCCL is a driver for software-based cryptographic functions.

- rba_CryptoHSM is a hardware driver, which is used by the Crypto module to configure specific instances of driver objects.
- rba_CryptoHSM is called from the Crypto Interface module (CryIf) using a specific configuration defined in Crypto, and it calls cryptographic primitives

supported in the ECU hardware to perform specific cryptographic functions.

- The path from Csm, via Crylf and Crypto, through to a specific configured driver object is known as a channel. Multiple channels are defined based on each specific configured driver object required by the SWC(s) making the service requests through Csm.

rba_CryptoHSM processes jobs requested by Crylf in either synchronous or asynchronous mode, and it provides the Key Management functions described by the AUTOSAR specification for the generic Crypto module.

Module Configuration

This module's configuration parameters are included in the configuration of another module. See the Configuration Advice section for more details.

8.2 Supported Features

8.2.1 General

The rba_CryptoHSM module provides support for the following AUTOSAR-defined functionality.

- Processing (and cancellation) of jobs.
- Key Management (Key) - copy, set valid.
- Key Management (Key Element) - get, get Id, copy, set.
- Certificate parsing and validation (for X.509 certificate)

8.2.2 Cryptographic Services

Please refer to the Supported Features section of the Csm module for details of the cryptographic services supported by rba_CryptoHSM.

8.2.3 Bulk Interface

rba_CryptoHSM provides a Bulk Interface to send multiple concurrent mac generate/verify jobs to the HSM in a single request. This reduces the communication overhead and polling times between the driver and the HSM hardware module.

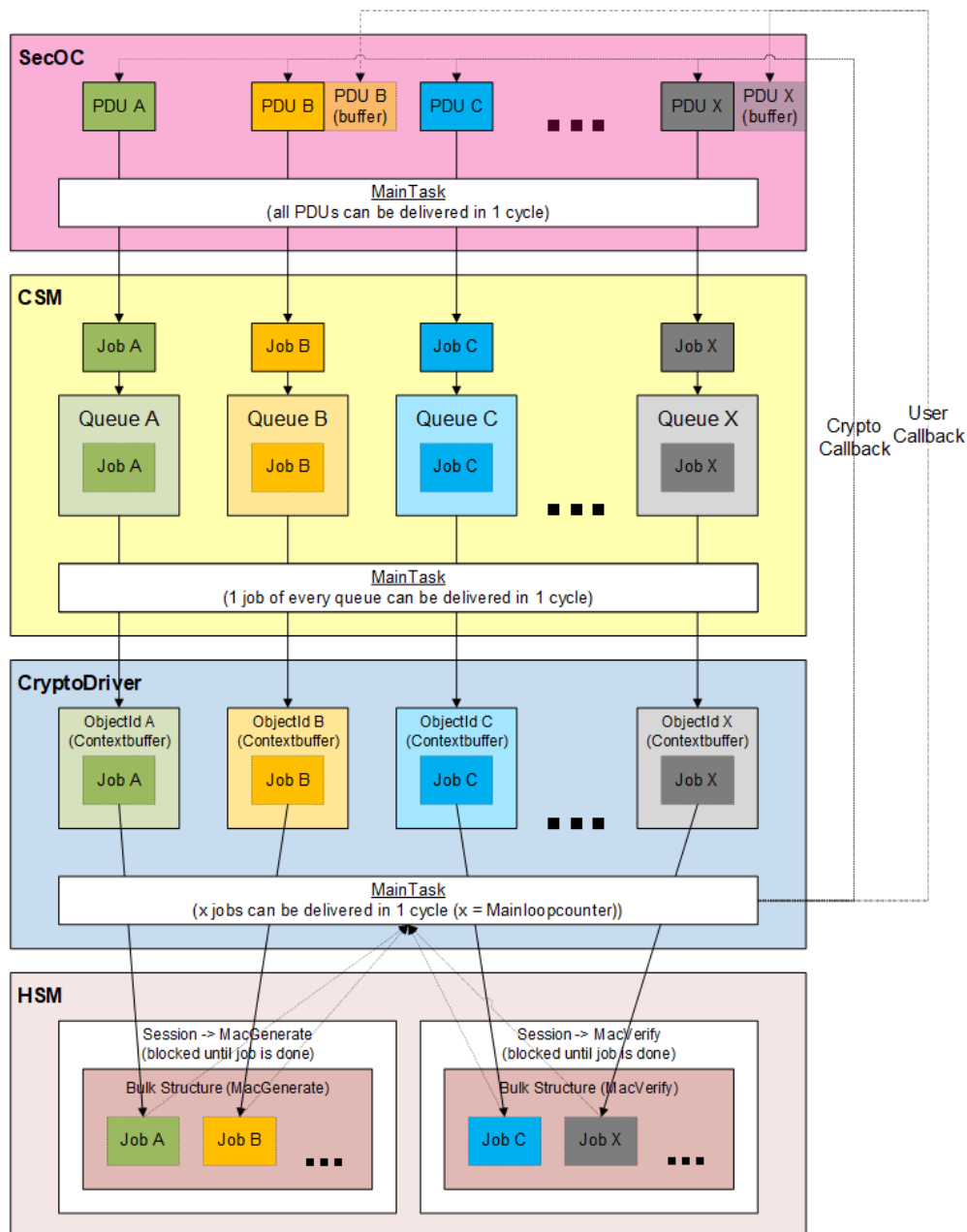


Figure 8.2.Bulk Interface

If the Bulk Interface feature is disabled (the default), then the standard Autosar procedure is used for processing the mac generate/verify jobs. If the Bulk Interface is enabled, then all mac jobs will be processed using the interface. A mixed operation, where some mac jobs should be processed without bulk and some mac jobs with bulk, is not possible.

It is also not possible to run mac jobs in synchronous mode or streaming mode (start/update/finish) with the Bulk Interface enabled. Only the asynchronous mode (singlecall) is supported for all mac jobs. All other primitives are unaffected by the bulk interface.

The maximum number of mac generate and mac verify jobs that can be sent to HSM by one single bulk request can be optionally limited, as can the runtime of the maintask. See the Configuration Advice for more details.

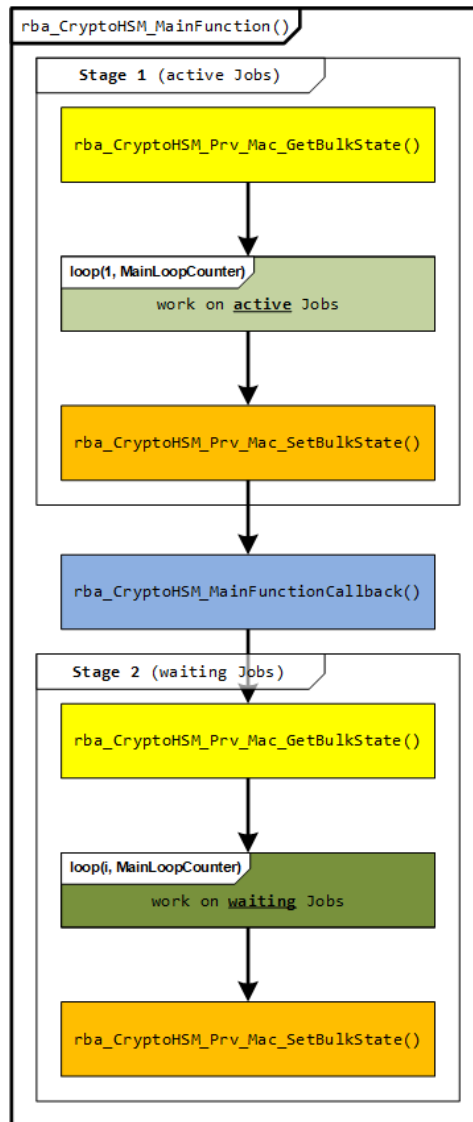


Figure 8.3.Bulk Main Task

In the first stage, all active jobs (i.e. already transferred to the HSM) are processed. If an error occurs while sending the bulk request to HSM, or something else is wrong with the HSM, all related jobs are cancelled. If a previous bulk request has already been completed by the HSM, the corresponding callbacks are triggered.

In a second stage, a similar procedure is done for inactive (i.e. new) jobs. If anything new is found, the bulk structure is filled with these jobs and the bulk request is sent to the HSM.

In between these two stages, additional code can be executed through

rba_CryptoHSM_MainFunctionCallback.

Reentrancy is not guaranteed for all MainFunctions of the CryptoStack and SecOC!

8.2.4 rba_CryptoHSM: MainFunction_ProcessDriverObjects

The asynchronous job processing is implemented using 2 main functions: "rba_CryptoHSM_MainFunction_ProcessDriverObjects_Request" and "rba_CryptoHSM_MainFunction_ProcessDriverObjects_Result". These handle all communication to HSM for asynchronous jobs, control the access to all HSM sessions and use session lock only if conflict with synchronous jobs/interfaces is detected.

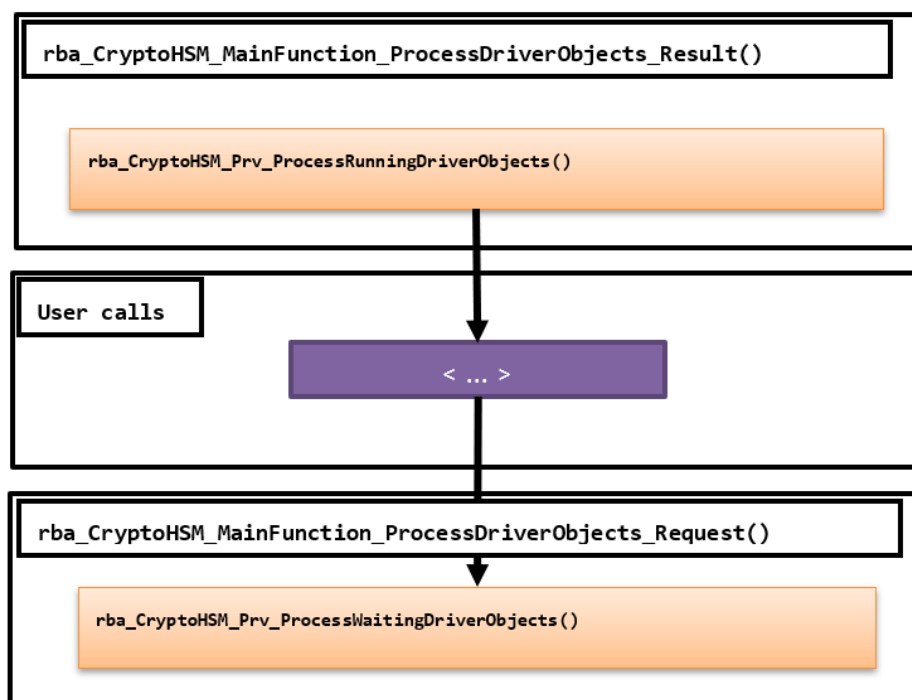


Figure 8.4. Main Function for Asynchronous Job

The `rba_CryptoHSM_MainFunction_ProcessDriverObjects_Result` function performs the following action: poll running jobs on HSM in all sessions. If a job is finished then it performs upper layer callback, clear resources and free the session context.

The `rba_CryptoHSM_MainFunction_ProcessDriverObjects_Request` function performs the following actions: read next job from all driver object queues and starts them on HSM; a HSM session can only execute jobs of a specific primitive type (e.g.: Hash, MAC, Random, etc.). Each driver object queue has multiple FIFOs - equal to the number of known producer contexts. When the next job is dequeued, it is always extracted from the highest priority FIFO with waiting jobs.

In between these two main functions other code can be executed in order to increase the processing performance.

See the Integration Advice for more information.

8.2.5 CSAI Error Propagation

Reporting of CSAI error codes is supported for the following services:

- Mac_Generate
- Mac_Verify
- Cipher_Encrypt
- Cipher_Decrypt
- RandomSeed
- RandomGenerate
- SHE_LoadKey/LoadKeySlot
- SHE_LoadPlainKey
- SHE_ExportRamKey

Please refer to the rba_CryptoHSM "Integration Advice" for information on configuring the CSAI error propagation callback.

8.2.6 Timeout Monitoring for Synchronous/Asynchronous Processing of Primitives

Timeout monitoring automatically cancels a HSM job if the processing time exceeds a configured timeout value. This feature is only supported for rba_CryptoHSM jobs with synchronous or asynchronous processing for the following services:

- Mac_Generate
- Mac_Verify
- Cipher_Encrypt
- Cipher_Decrypt
- RandomGenerate
- SHE_LoadKey/LoadKeySlot
- SHE_LoadPlainKey
- SHE_ExportRamKey

Please refer to the rba_CryptoHSM configuration/integration advice for information on enabling timeout monitoring for the services listed above.

8.2.7 VKMS Operations

rba_CryptoHSM supports the following VKMS Operations:

- HSM_VKMS_handleDLC_start
- HSM_VKMS_handleDLC_finish
- HSM_VKMS_getVerificationHash

- HSM_VKMS_getPssHash
- HSM_VKMS_getIdentityHash
- HSM_VKMS_getStatus
- HSM_VKMS_getTrainingCounter
- HSM_VKMS_getVkmsVin
- HSM_VKMS_getKey
- HSM_VKMS_getMetaData

8.2.8 Job input/output buffer redirection

rba_CryptoHSM supports the AUTOSAR defined Job Input/Output Buffer Redirection feature.

- Only the maximum size of Crypto Key Elements buffers can be used.
- Redirection is supported only in SINGLECALL job mode of operation.
- Crypto Key Elements with ID=1 cannot be used for job buffer redirection (as they are stored in HSM).
- The internal output buffering feature is also applicable when job buffer redirection is used.

The job buffer redirection feature is supported only by the following primitives:

- Hash
- MAC Generate/Verify
- Signature Generate/Verify
- Encrypt/Decrypt
- JobKeyExchangeCalcPubVal/JobKeyExchangeCalcSecret
- Job Random Seed/Random Generate
- SHE GetID
- SHE LoadKey/LoadKeySlot
- SHE LoadPlainKey
- SHE ExportRamKey

8.2.9 MainFunction Partitioning

RTA-SEC supports MainFunction partitioning of resources. For rba_CryptoHSM, this involves the mapping of CryptoDriverObjects to specific partitions in the context in which they should be handled. See the Configuration Advice for more details.

8.3 Extensions

8.3.1 Configuration Extensions

- No known extension parameters are defined in this version.

8.3.2 Additional Services

The following API are available in v12.7.0 in addition to those defined in AR v22-11

8.3.2.1 rba_CryptoHSM_DeInit

Syntax	rba_CryptoHSM_Deinit()
Parameters	NONE
Description	Deinitializes the Crypto Driver

8.3.2.2 rba_CryptoHSM_IsInitialized

Syntax	rba_CryptoHSM_IsInitialized()
Parameters	NONE
Return Value	TRUE - Module is initialised.
	FALSE - Module is not initialised
Description	Performs a check on the initialised status of the module.

8.3.2.3 rba_CryptoHSM_IsDeinitialized

Syntax	rba_CryptoHSM_IsDeinitialized(void)
Parameters	NONE
Return Value	TRUE - Driver is already deinitialized.
	FALSE - Driver is not deinitialized
Description	Check if the deinitialization is already done.

8.3.2.4 rba_CryptoHSM_Rb_StorePermanentData

Syntax	rba_CryptoHSM_Rb_StorePermanentData()
Parameters	NONE
Return Value	E_OK: Request successful.
	E_NOT_OK: Request failed.
	CRYPTO_E_PENDING: Request pending. Function has to be called again.
Description	Triggers persistent data store if needed and returns the save operation status.

	This function must be used before an ECU is reset or shut down if persistent data is required by the calling process.
--	---

8.3.2.5 rba_CryptoHSM_KeyGetStatus

Syntax	Std_ReturnType rba_CryptoHSM_KeyGetStatus(uint32 cryptoKeyId, Crypto_KeyStatusType* keyStatusPtr)
Parameter In	cryptoKeyId: The ID of the key for which the key state shall be returned.
Parameter Out	keyStatusPtr: Pointer to the data where the status of the key shall be stored.
Description	Returns the key state of the key identified by cryptoKeyId.

8.3.2.6 rba_CryptoHSM_MainFunction_ProcessDriverObjects

Syntax	rba_CryptoHSM_MainFunction_ProcessDriverObjects(void)
Description	Static runnable which handles all communication to HSM for asynchronous jobs.

8.4 Limitations

8.4.1 Key Management Functions

- The functions rba_CryptoHSM_KeyExchangeCalcPubVal, rba_CryptoHSM_KeyExchangeCalcSecret, rba_CryptoHSM_KeyGenerate and rba_CryptoHSM_RandomSeed run only in single call mode and do not support the streaming approach (start, update, finish).
- The return value CRYPTO_E_ENTROPY_EXHAUSTED [SWS_Crypto_00141] has not been implemented because rba_CryptoHSM has an internal TRNG, and the PRNG is reseeded automatically by the TRNG.
- The functionality for waiting while the Crypto driver is busy [SWS_Crypto_00120] has not been implemented because it requires extensive exclusive access protection of variables. Also the active blocking of sync call shall be avoided, it is better to decline a sync job than to keep it waiting.
- The return to the DET of CRYPTO_E_RE_KEY_NOT_AVAILABLE [SWS_Crypto_00140] has not been implemented because it contradicts [SWS_Crypto_00086].
- To copy one Key into another, the number and size of Key Elements (target and source key) must be equal, otherwise the function rba_CryptoHSM_KeyCopy will return with error E_NOT_OK (for a number mismatch) and CRYPTO_E_KEY_SIZE_MISMATCH (for a size mismatch). [SWS_Crypto_02305] [SWS_Crypto_02306]

- For function `rba_CryptoHSM_RandomSeed`, the internal state of the random generator is not stored in the Key Element `CRYPTO_KE_RANDOM_SEED`. This is due to the fact that variables cannot be stored within an HSM. [SWS_Crypto_02310]
- For HOST side Key Elements, the length is stored permanently (this is not the case for HSM side Key Elements). The function `rba_CryptoHSM_KeyElementGet` always returns the whole Element for HSM side Key Elements and the actual size for HOST side Key Elements. If the Key Element is written with data smaller than the size of the Element, the remaining part is filled with `0x00`. [SWS_Crypto_02311]
- For certificate Key Elements (id 0), when the whole certificate is read using `KeyElementGet`, the actual length of the certificate will be read out from the DER formatted certificate data and the actual size of the certificate will be used as the resulting length. [SWS_Crypto_00092]
- Key Elements stored on HSM can be copied in the case of symmetric keys with the constraint that, if both target and source Key Elements are volatile, then the key is referenced twice and it should not be modified after the copy operation. However, if one of the keys is modified, then the copy operation must be repeated.
- [SWS_Crypto_00144] is not implemented because it is incompatible with [SWS_Crypto_00122]. `E_OK` is used instead of `CRYPTO_E_JOB_CANCELED`.
- The primitives `SHE_GetID`, `SHE_LoadKey`, `SHE_LoadPlainKey` and `SHE_ExportRamKey` are available as alternative key management interfaces based on the SHE Specifications v1.1. These two primitives run only synchronously or asynchronously in `Singlecall` mode and do not support the streaming approach.
- Certificate Key Elements are not supported via the job interface and can only be set via the function interface `rba_CryptoHsm_KeyElementSet` (synchronously).

8.4.2 Certificate Handling

- The init value internal setup feature for certificates is not supported. If an intermediate certificate needs to be initialised then it has to be performed by an external triggered `KeyElementSet` operation. If it's a root certificate, then it can only be setup by directly injecting the certificate data into the HSM.
- Certificates are implicitly verified when they are stored in the HSM. The `rba_CryptoHSM_CertificateVerify` function won't trigger any verification, instead it just returns the result of the last verification. If a certificate is added, then its signer certificate needs to be already present in the HSM.

8.4.3 RSA Signature Keys

Due to the size of RSA keys, there is not enough room in RAM key handle space to fit 2 RSA signature keys. This can cause a problem when injecting the key into RAM memory.

8.5 Exclusions

The following generic API functions listed in the Crypto Driver module specification have no functional equivalents in this module:

- Crypto_KeyDerive
- Crypto_CustomSync
- Crypto_NvBlock_Init_<NvBlock>
- Crypto_NvBlock_ReadFrom_<NvBlock>
- Crypto_NvBlock_WriteTo_<NvBlock>
- Crypto_NvBlock_Callback_<NvBlock>

8.5.1 Configuration Exclusions

- There are no known exclusions in this version.

8.6 Deviations

- There are no known deviations in this version.

8.7 API Definition

List of supported API functions

rba_CryptoHSM_Init	Initialises the module.
rba_CryptoHSM_CancelJob	Removes the requested job from the queue and cancels it if possible.
rba_CryptoHSM_GetVersionInfo	Returns the version information for the module.
rba_CryptoHSM_KeyCopy	Copies a key with all its elements to another key in the same driver.
rba_CryptoHSM_KeyElementCopy	Copies a key element to another key element in the same driver.
rba_CryptoHSM_KeyElementCopyPartial	Uses keyElementSourceOffset and keyElementCopyLength to copy partial key element to another key element in the same crypto driver.
rba_CryptoHSM_KeyElementGet	Gets a key element of the key identified by the cryptoKeyld and stores the key element in the memory location pointed by the result pointer.

rba_CryptoHSM_KeyElementIdsGet	Retrieves information about which key elements are available in a given key.
rba_CryptoHSM_KeyElementSet	Sets the key element data for the key identified by cryptoKeyld.
rba_CryptoHSM_KeyExchangeCalcPubVal	Calculates the public value for the key exchange and stores the public key in the memory location pointed to by the public value pointer.
rba_CryptoHSM_KeyExchangeCalcSecret	Calculates the shared secret key for the key exchange with the key material of the key identified by the cryptoKeyld and the partner public key. The shared secret key is stored as a key element in the same key.
rba_CryptoHSM_KeySetValid	Sets the key state of the key identified by cryptoKeyld to valid.
rba_CryptoHSM_KeySetInvalid	Holds the identifier of the key which shall be set to invalid.
rba_CryptoHSM_ProcessJob	Performs the processing of the job as configured in the job parameters for the given driver object.
rba_CryptoHSM_RandomSeed	Generates the internal seed state using the provided entropy source - can be used to update the seed state with new entropy.
rba_CryptoHSM_KeyGenerate	Generates new key material and stores it in the key identified by cryptoKeyld.
rba_CryptoHSM_CertificateParse	Parses the certificate data stored in the key element CRYPTO_KE_CERT_DATA and fills the key elements CRYPTO_KE_CERT_SIGNEDDATA and CRYPTO_KE_CERT_PARSEDPUBLICKEY and CRYPTO_KE_CERT_SIGNATURE.
rba_CryptoHSM_CertificateVerify	Verifies the certificate stored in the key referenced by cryptoValidateKeyld with the certificate stored in the key referenced by cryptoKeyld.
rba_CryptoHSM_MainFunction	The module main function which provides cyclic processing of asynchronous jobs.
rba_CryptoHSM_KeyGetStatus	Returns the key state of the key identified by cryptoKeyld.

For details on using these calls, please refer to AUTOSAR_SWS_CryptoDriver.pdf (version v22-11).

8.8 Configuration Advice



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

8.8.1 General Advice

The Crypto module provides the AUTOSAR-defined configuration parameters required to define unique driver object instances for cryptographic primitives defined in rba_CryptoHSM (and the other Crypto stack modules).

Configuration advice relating to the standard AUTOSAR parameters can be found in the Crypto section of this Reference Guide. Please also refer to the AUTOSAR documentation on the Crypto driver (AUTOSAR_SWS_CryptoDriver.pdf) for full details of these standard configuration parameters.

The configuration advice below is specific to rba_CryptoHSM.

8.8.2 MainFunction Partitioning

RTA-SEC supports MainFunction partitioning of resources. For rba_CryptoHSM, this involves the mapping of CryptoDriverObjects to specific partitions in the context in which they should be handled.

rba_CryptoHSM provides one unique MainFunction instance for handling requests to HSM, even though the system is logically divided into multiple MainFunction partitions, so the following tasks are fixed and must always be scheduled:

- **rba_CryptoHSM_MainFunction**: Updates NvM blocks for modified persistent keys.
- **rba_CryptoHSM_MainFunction_ProcessDriverObjects**: Handles async communication to all HSM sessions and all CryptoDriverObjects.

rba_CryptoHSM can have multiple Crypto_MainFunction instances based on the configuration of CryptoMainFunction containers. The following parameters need to be configured for each CryptoMainFunction container:

- **CryptoMainFunctionPeriod**: The periodicity for this MainFunction instance.
- **CryptoMainFunctionPartitionRef**: The EcucPartition (often interpreted as core) assignment for this MainFunction instance.
- **CryptoRbMainFunctionLoopCounter**: Relevant only to rba_CryptoCCL.

rba_CryptoHSM_MainFunction_JobQueue instances are generated in order to

handle the extraction of the next job waiting in the CryptoDriverObject's job queue and schedule it in the HSM queue of CryptoDriverObject IDs. This intermediary step is required for handling waiting jobs in job queues so that they are scheduled in their assigned MainFunction partition context.

CryptoDriverObjects with a job queue configured, but no CryptoMainFunction reference, are handled in the default rba_CryptoHSM_MainFunction_JobQueues function. If all CryptoDriverObjects are explicitly mapped to CryptoMainFunction containers, rba_CryptoHSM_MainFunction_JobQueues is not generated.

CryptoDriverObjects Partitioning

CryptoDriverObjects can be explicitly assigned to a MainFunction container, and will be processed only by that specific MainFunction instance. The assignment is only reflected in the generation of main tasks for handling job queues.

Regardless of the MainFunction assignment and the enabling/disabling of individual job queues, all HSM requests are handled by the rba_CryptoHSM_MainFunction_ProcessDriverObjects task.

MainFunction forwarding

If automatic container filling is enabled, for CsmQueues linked to MainFunction containers, analogous MainFunction containers will be forwarded in Crypto Drivers, and these will be automatically linked to referenced CryptoDriverObjects.

If there are already CryptoMainFunction containers configured with the same parameters, these ones are linked by the Csm forwarder instead.

8.8.3 Configuration of ecy_hsm

The ecy_hsm module must be configured with at least following configurations in order to avoid CodeGen errors:

- ecy_hsm/ecy_hsm_AES_KEY
- ecy_hsm/ecy_hsm_AES_KEY/DYNAMIC
- ecy_hsm/ecy_hsm_ECC_KEY
- ecy_hsm/ecy_hsm_ECC_KEY/DYNAMIC

If CryptoKeyElementPersist is set to True, ecy_hsm_RESOURCE_OBJECTS should be configured properly to provide KeyId mapping to the predefined ecy_hsm_Csai_KeyIdT

8.8.4 Algorithm modes or Padding schemes

To ensure that it functions without error, when configuring rba_CryptoHSM, the CryptoKeyElementId parameter must be set to the predefined value, as specified by SWS_Csm_01022.

8.8.5 CryptoRbKeyElementType

The following changes should be made to CryptoRbKeyElementType values:

- CRYPTO_ELEMENT_TYPE_SIGGEN must be changed to CRYPTO_ELEMENT_TYPE_EDDSA_ED25519_PRIV_KEY

- CRYPTO_ELEMENT_TYPE_SIGVER must be changed to CRYPTO_ELEMENT_TYPE_EDDSA_ED25519_PUB_KEY
- CRYPTO_ELEMENT_TYPE_DEFAULT must be changed to CRYPTO_ELEMENT_TYPE_SYMMETRIC_128

8.8.6 CryptoRbHsmTimeout

If not specifically set, the default for the following timeout parameters will be obtained from CryptoRbHsmTimeout:

- CryptoRbHsmMacTimeout
- CryptoRbHsmSignatureTimeout
- CryptoRbHsmKeyExchangeTimeout
- CryptoRbHsmKeyGenerateTimeout
- CryptoRbHsmRandomTimeout
- CryptoRbHsmSheTimeout
- CryptoRbHsmCipherTimeout
- CryptoRbHsmCancelTimeout
- CryptoRbHsmVkmsTimeout

8.8.7 Key Exchange

For the synchronous KeyExchange interface (rba_CryptoHSM_KeyExchangeCalcPubVal and rba_CryptoHSM_KeyExchangeCalcSecret), the algorithm has to be configured via the key element CRYPTO_KE_KEYEXCHANGE_ALGORITHM (12):

- ECDH with Curve 25519. Init value of key element with Id 12 must be configured with CRYPTO_ALGOFAM_EC_CURVE25519 (0x84).
- ECDH with Curve 448. Init value of key element with Id 12 must be configured with CRYPTO_ALGOFAM_EC_CURVE448 (0x93).
- ECDH with Curve SECP224R1. Init value of key element with Id 12 must be configured with CRYPTO_ALGOFAM_EC_SECP224R1 (0x8A).
- ECDH with SECP256R1. Init value of key element with Id 12 must be configured with CRYPTO_ALGOFAM_EC_SECP256R1 (0x85).
- ECDH with SECP384R1. Init value of key element with Id 12 must be configured with CRYPTO_ALGOFAM_EC_SECP384R1 (0x86).

For the synchronous/asynchronous KeyExchange interface (rba_CryptoHSM_JobKeyExchangeCalcPubVal and rba_CryptoHSM_JobKeyExchangeCalcSecret), the algorithm has to be configured in the primitives:

- CryptoPrimitiveAlgorithmFamily has to be configured with CRYPTO_ALGOFAM_ECDH.
- CryptoPrimitiveAlgorithmSecondaryFamily has to be configured with CRYPTO_ALGOFAM_CUSTOM

- CryptoPrimitiveAlgorithmSecondaryFamilyCustomRef has to be configured with CRYPTO_ALGOFAM_EC_CURVE25519, CRYPTO_ALGOFAM_EC_SECP384R1 or CRYPTO_ALGOFAM_EC_SECP256R1.

8.8.7.1 Key Exchange with an existing key pair

Key exchange can be done by consecutive calls of `rba_CryptoHSM_KeyExchangeCalcPubVal` and `rba_KeyExchangeCalcSecret`. In this case, in the `rba_Crypto_KeyExchangeCalcPubValue` function, the HSM will generate a new (random) key pair. The public key is passed to the user and the private key is used to calculate a shared secret with the partner's public key in the `rba_KeyExchangeCalcSecret` function.

A second option for a key exchange is the usage of an existing key pair. In this case, for ECDH, the user needs to configure a `CalcSecret` job without any reference to any `CalcPubVal` job. It means that the key used for `CalcSecret` cannot be used by any `CalcPubVal` job. However when performing key exchange with the ECBD algorithm, the default ECBD sequence remains the same. Please refer to ECBD algorithm documentation below.

Then the `rba_CryptoHSM_KeyExchangeCalcSecret` function is called. The shared secret is calculated with the private key of the key pair and the partner's public key. You have to ensure that the key pair is populated with key data (e.g. by calling the `rba_CryptoHSM_KeyElementSet` function).

You have to configure the HSM key slot ID for the key pair. Only the handle of the key pair is linked to the key element `CRYPTO_KE_KEYEXCHANGE_PRIVKEY`. The key pair data is not physically copied into the private key element. Nevertheless, you have to ensure that the key element size of the element `CRYPTO_KE_KEYEXCHANGE_PRIVKEY` is sufficient to link the handle of the key pair to it (e.g. 144 bytes for SECP384R1).

Only the key pair type `ECDSA_SECP384R1` is supported for key exchange with existing key pair.

The configuration parameter `CryptoRbHsmKeyExchangeWithHandle` has to be set to `TRUE` to enable key exchange with existing key pair.

8.8.7.2 Key Exchange using ECBD Algorithm

ECBD (Elliptic Curve Burmester-Desmedt), is a public Key Exchange Protocol based on Elliptic Curve Cryptography (ECC). It allows three or more participants to agree on a common shared secret key that can be used for different applications. So that this can be achieved, all the participants must use a Key Negotiation Protocol and be placed in a ring order.

There are four main states for the key negotiations defined as Round1, Round2, Update Second public keys, and Calculate Shared Secret. The current solution can handle the Key Negotiation of ECBD Key Exchange Protocol. The aforementioned APIs interact with CSMs Jobs APIs so it provides the Shared Secret Key to the requester. The five CycurHSMs APIs are as follows:

- Round 1: Generates the first public key.

- Round 2: Generate a second public key.
- Update Second public keys from each participant: An UPDATE mode need to be called for each participant.
- Calculate shared secret. Call the FINISH mode.

Round 1: In Round 1 it is currently possible to generate or inject a new key pair, where the output from this Round will be the public key part of the key pair.

The JobKeyExchangeCalcPubVal function will initialize the ECBD operation for the curve SECP224 and storing all given parameters internally. The result will be an public key extracted from a key pair (generated by Round 1) and has to be broadcasted to all communication partners.

Round 2: The second Round performs the second step of the Burmester-Desmedt Key Exchange operation using the JobKeyExchangeCalcPubVal API. It creates the second public key based on first public keys from left and right neighbor in the "ring". This output should be used in the second broadcast.

The next phase is to Update Second public keys from each participant using the JobKeyExchangeCalcSecret API. Must be called repeatedly after receiving a public key during the second broadcast phase. It applies the Update state of Key Negotiation process by storing internally all participants second key.

Finally, the Calculate shared secret phase performs the last step of the Key Exchange operation, and must be called only when all second public keys of the other participants and the own second public key have been called in the phase before.

This function returns a key handle to the ECBD shared secret that shall be used to derive a new shared secret. Be aware that this key handle is only allocated in the HSM RAM. This functions applies the Finish state of the Key Negotiation process.

A KeyElement Id 1 must be configured for ECBD, instead of a KeyElement Id 0x8000 0001 which was configured in the ECDH algorithm.

In ECBD sequence it is mandatory to release the source key handle during KeyDerivation. For this the Key Element CRYPTO_KE_KEYEXCHANGE_ECBD_RELEASE_SOURCE_ELEMENT with Id 1007U, must be configured with content 1.

Configuration Steps for ECBD

Round 1

- Set the Assignee Key Element (TESTCD_HSM_KEYEX_CALC_PUB_ASSIGNEE_LENGTH) with the own assigned participant number in the ring.
- Set the Participants Key Element (CRYPTO_KE_KEYEXCHANGE_ECBD_PARTICIPANTS) with the number of participants.
- Call JobKeyExchangeCalcPubVal to execute ECBD's Round 1 and compute 1st Public Key.

Round 2

- Set the Left Key Element (CRYPTO_KE_KEYEXCHANGE_ECBD_PUB-KEY_LEFT) with the first public key received from the neighbor at your left in the ring.
- Set the Right Key Element (CRYPTO_KE_KEYEXCHANGE_ECBD_PUB-KEY_RIGHT) with the first public key received from the neighbor at your right in the ring.
- Call JobKeyExchangeCalcPubVal to Execute ECBD's Round 2 and compute 2nd Public Key.

Start

Call JobKeyExchangeCalcSecret in the START MODE to execute ECBD's Start step.

Update

This update step must be executed many times as the number of participants in the ring (including itself described as own participant):

- Set the Assignee Key Element (TESTCD_HSM_KEYEX_CALC_PUB_ASSIGNEE_LENGTH) with the participant number which will be updated.
- Call JobKeyExchangeCalcSecret to execute ECBD Update step with the computed 2nd Public Key, generated in Round 2 (for the own participant) or received from the other participants.

Finish

- Call JobKeyExchangeCalcSecret to execute ECBD Finish step and compute the Shared Secret Key.

Key Derive

- After generate the shared secret, the KeyDerive should be called to derive a new shared secret. Then, the ECBD KeyExchange process is completed.

The Release Source Element must exist and have content 1 so the keyhandle is released on this phase.

8.8.8 Configuration of Driver Objects

According to AUTOSAR, multiple independent Crypto driver objects of the same primitive type can be configured. While it is theoretically possible to do this, configuring multiple driver objects of the same primitive type is not recommended as only one HSM session is reserved per primitive.

This implies that even though jobs on different driver objects can run asynchronously, the corresponding jobs inside the HSM will block each other because the HSM performs only one job at a time per session. Therefore it is strongly recommended that you only use one crypto driver object per primitive because having multiple objects will not provide any benefits.

In the case of Signature Generation/Verification primitives using the underlying algorithm family CRYPTO_ALGOFAM_ECDSA_SECP384R1, there MUST NOT be

multiple driver objects configured, as doing so would lead to unexpected behaviour due to a shared internal buffer which could get overwritten. As described above, having multiple driver objects for effectively the same primitives has no benefits and therefore should be avoided.

8.8.9 Certificate Parsing

Only AUTOSAR-defined certificate elements can be parsed. To support parsing and reading out the complete certificate from HSM, an extra vendor specific key element needs to be created in the configuration.

If the complete certificate needs to be read out, you need to create a key element with id 1000 and correct buffer size. Using key element id 1000 or 0, the complete certificate can be read out after performing the CertificateParse operation.

8.8.10 Bulk Interface

As described in the Supported Features, rba_CryptoHSM provides a Bulk Interface to send multiple concurrent mac generate/verify jobs to the HSM in a single request. This feature is controlled by the CryptoRbHsmBulkProcessingModeEnabled configuration parameter.

The maximum number of mac generate and mac verify jobs that can be sent to HSM by one single bulk request can be limited by the CryptoRbHsmMacBulkMaxGenerateJobs and CryptoRbHsmMacBulkMaxVerifyJobs configuration parameters.

If nothing is specified for CryptoRbHsmMacBulkMaxGenerateJobs or CryptoRbHsmMacBulkMaxVerifyJob, then the bulk structures will be created according to the configured number of mac generate and verify jobs. It is possible to select smaller bulk structures to save memory and runtime.

Each mac job needs to be configured to have its own queue of at least size 1. Normally the Csm will forward such configuration automatically if the bulk interface is enabled, but you can do it manually as well.

The runtime of the maintask can be limited by using the mainloop counter configuration parameter CryptoRbMainFunctionLoopCounter. This value can be used to specify the maximum number of jobs to be processed per maintask call. You must select this value to ensure that all jobs to be processed can, at some point, be processed.

Utilization of the Bulk Interface also influences the configuration of the I-PDUs in SecOC. See the Configuration Advice for SecOc for more details.

8.8.11 SHE Get ID

rba_CryptoHSM provides a CryptoPrimitiveService SHE_GETID to support the SHE Get Identity function (for retrieving the SHE UID) as described in the SHE v1.1 specification.

General	
ShortName*	CryptoPrimitive_She_GetID
LongName	FOR-ALL ▼ ⊗ ▼

Attributes	
CryptoPrimitive...thmFamily*	CRYPTO_ALGOFAM_NOT_SET ▼ ⊗ ▼
CryptoPrimitiv...orithmMode*	CRYPTO_ALGOMODE_NOT_SET ▼ ⊗ ▼
CryptoPrimitiveAlgor...mily*	CRYPTO_ALGOFAM_NOT_SET ▼ ⊗ ▼
CryptoPrimitiveService*	SHE_GETID ▼ ⊗ ▼

Any other parameter configuration values in the CryptoPrimitive container will be rejected during code generation if CryptoPrimitiveService is set to SHE_GETID.

A CryptoDriverObject needs to be created and a reference to the CryptoPrimitive has to be in CryptoPrimitiveRef. The manual configuration of CryptoDriverObject can be avoided if the Csm parameter CsmRbAutoConfigCryptoSelect is set as rba_CryptoHSM, in which case the Csm module will automatically forward the necessary CryptoDriverObject configurations during code generation.

8.8.12 SHE Load Key Slot

rba_CryptoHSM provides a CryptoPrimitiveService SHE_LOADKEYSLOT to load SHE keys as described in Chapter 9 - Memory Update protocol from the SHE specification v1.1.

In order to enable the SHE Load Key Slot functionality a Crypto primitive has to be configured as follows.

📁 ▶ 📁 ▶ 📁 ▶ 📁 ▶ [CryptoPrimitives \[3\]](#) ▶ [CryptoPrimitive "CryptoPrimitive_She_KeyLoad"](#) ▶

General	
ShortName*	CryptoPrimitive_She_KeyLoad
LongName	FOR-ALL ▼ ⊗ ▼

Attributes	
CryptoPrimitive...thmFamily	CRYPTO_ALGOFAM_NOT_SET ▼ ⊗ ▼
CryptoPrimitiv...orithmMod	CRYPTO_ALGOMODE_NOT_SET ▼ ⊗ ▼
CryptoPrimitiveAlgor...mily*	CRYPTO_ALGOFAM_NOT_SET ▼ ⊗ ▼
CryptoPrimitiveService*	SHE_LOADKEYSLOT ▼ ⊗ ▼

Any other parameter configuration values in the CryptoPrimitive container will be rejected during code generation if CryptoPrimitiveService is set to SHE_LOADKEYSLOT.

A CryptoDriverObject needs to be created and a reference to the CryptoPrimitive has to be in CryptoPrimitiveRef. The manual configuration of CryptoDriverObject can be avoided if the Csm parameter CsmRbAutoConfigCryptoSelect is set as

rba_CryptoHSM, in which case the Csm module will automatically forward the necessary CryptoDriverObject configurations during code generation.

It is possible to configure a CryptoKeyElement that maps to a SHE key slot. An example use case in which such configuration is useful is when a specific SHE key is required for MAC authentication.

In order to configure a CryptoKeyElement to map to a SHE key slot a CryptoKeyElement configuration should be created and CryptoKeyElementFormat should be set to CRYPTO_KE_FORMAT_BIN_SHEKEYS. The remaining configuration parameters in the container CryptoKeyElement should be set as follows.

The screenshot shows the configuration interface for a CryptoKeyElement. The breadcrumb path is: CryptoKeyElements "CryptoKeyElements" > CryptoKeyElements [1] > CryptoKeyElement "CryptoKeyElement SHE".

General Tab:

- ShortName***: CryptoKeyElement_SHE
- LongName**: FOR-ALL

Attributes Tab:

- CryptoKeyElementAccess**: false
- CryptoKeyElementFormat**: CRYPTO_KE_FORMAT_BIN_SHEKEYS
- CryptoKeyElementId***: 1
- CryptoKeyElementInitValue**: (empty)
- CryptoKeyElementPersist***: true
- CryptoKeyElementReadAccess**: CRYPTO_RA_DENIED
- CryptoKeyElementSize***: 16
- CryptoKeyElementWriteAccess**: CRYPTO_WA_ENCRYPTED
- CryptoRbKeyElementType**: (empty)
- CryptoRbHsmKeySlotId**: DYNAMIC_BASIC_SHE_DFLASH_KEY_BASE

References Tab:

- CryptoKeyElementVirtualTargetRef**: (empty)

The configurations shown above of the CryptoKeyElement container are valid for all SHE general purpose keys and the MASTER_KEY slots. To use a specific key slot, only a change to the CryptoRbHsmKeySlotId parameter is required.

To configure the SHE RAM_KEY slot for usage, the required parameter settings in the CryptoKeyElement container are different. The parameter CryptoKeyElementPersist should be set to FALSE because of the volatile nature of the RAM_KEY, and the parameter CryptoRbHsmKeySlotId should be set to VOLATILE_BASIC_SHE_RAM_KEY or ecy_hsm_CSAI_KEYID_AES_KEY_VOLATILE_BASIC_SHE_RAM_KEY.

The configurations for the RAM_KEY in the CryptoKeyElementContainer should be set as follows.





[CryptoKeyElement "CryptoKeyElement_SHE_RAM_KEY"](#)

General

ShortName*

LongName

Attributes

CryptoKeyElementAccess

CryptoKeyElementFormat

CryptoKeyElementId*

CryptoKeyElementInitValue

CryptoKeyElementPersist*

CryptoKeyElementReadAccess

CryptoKeyElementSize*

CryptoKeyElementWriteAccess

CryptoRbKeyElementType

CryptoRbHsmKeySlotId

References

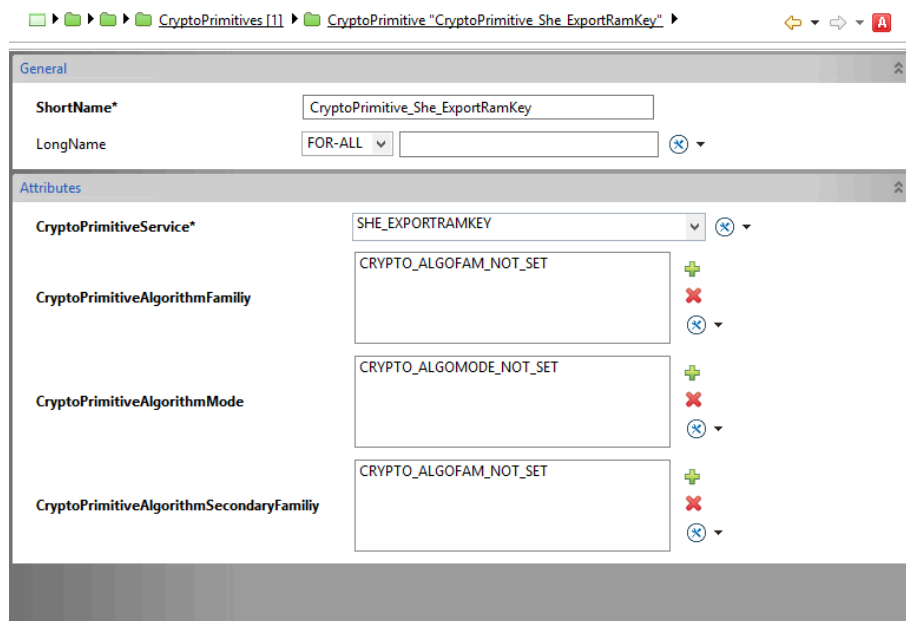
CryptoKeyElementVirtualTargetRef

Any other parameter configuration values in the `CryptoKeyElement` container will be rejected during code generation if `CryptoKeyElementFormat` is set to `CRYPTO_KEY_ELEMENT_FORMAT_BIN_SHEKEYS`.

8.8.13 SHE Export Ram Key

`rba_CryptoHSM` provides a `CryptoPrimitiveService SHE_EXPORTRAMKEY` to export the SHE RAM key as described in the SHE functional specification v1.1.

In order to enable the SHE Export Ram Key functionality a Crypto primitive has to be configured as follows:



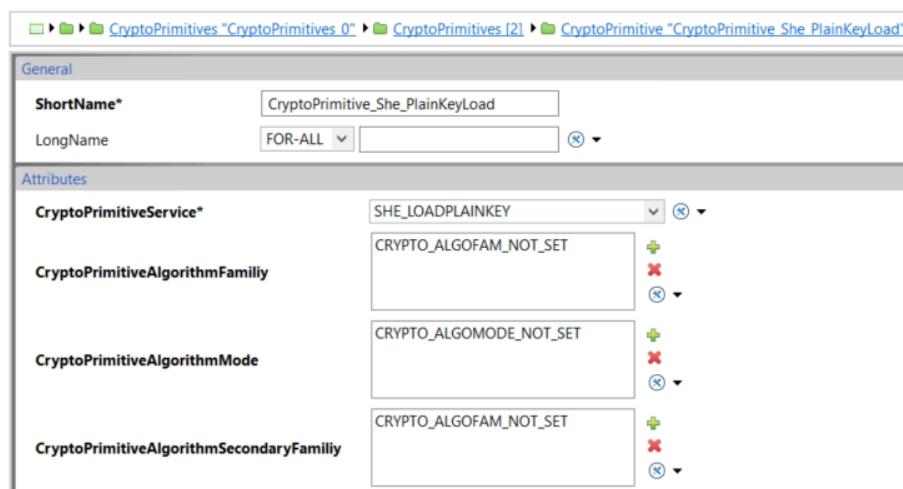
Any other parameter configuration values in the CryptoPrimitive container will be rejected during code generation if CryptoPrimitiveService is set to SHE_EXPORTRAMKEY.

A CryptoDriverObject needs to be created and a reference to the CryptoPrimitive has to be in CryptoPrimitiveRef. The manual configuration of CryptoDriverObject can be avoided if the Csm parameter CsmRbAutoConfigCryptoSelect is set as rba_CryptoHSM, in which case the Csm module will automatically forward the necessary CryptoDriverObject configurations during code generation.

8.8.14 SHE Load Plain Key

rba_CryptoHSM provides a CryptoPrimitiveService SHE_LOADPLAINKEY to load plaintext keys to the RAM_KEY slot as described in the SHE specification v1.1.

In order to enable the SHE Load Plain Key functionality a Crypto primitive has to be configured as follows.



Any other parameter configuration values in the `CryptoPrimitive` container will be rejected during code generation if `CryptoPrimitiveService` is set to `SHE_LOAD_PLAINKEY`.

A `CryptoDriverObject` needs to be created and a reference to the `CryptoPrimitive` has to be in `CryptoPrimitiveRef`. The manual configuration of `CryptoDriverObject` can be avoided if the `Csm` parameter `CsmRbAutoConfigCryptoSelect` is set as `rba_CryptoHSM`, in which case the `Csm` module will automatically forward the necessary `CryptoDriverObject` configurations during code generation.

8.8.15 CSAI Error Propagation

The CSAI Error Propagation feature involves the forwarding of internal error codes returned by the CSAI interfaces to another consuming module. This feature is enabled via the `CryptoRbHsmCsaiErrorPropagationEnabled` configuration parameter.

The error code propagation is performed only for the following primitives:

- MAC Generate
- MAC Verify
- Encrypt - ECB and CBC
- Decrypt - ECB and CBC
- Random Generate
- Random Seed
- SHE Load Key / Load Key Slot
- SHE Load Plain Key
- SHE Export RAM Key

Refer to the `rba_CryptoHSM` "Integration Advice" for information on configuring the CSAI error propagation callback.

8.8.16 Timeout Monitoring of Synchronous Jobs

The configuration parameter "`Crypto/CryptoGeneral/CryptoRbHsmTimeoutMonitoringEnabled`" can be configured "`TRUE`" to enable timeout monitoring for supported services (Mac, Cipher, Random, SHE Key Management) in synchronous mode. If timeout monitoring is enabled, a synchronous job is cancelled when the processing duration of a HSM job exceeds the configured timeout value.

After a cancellation request, `rba_CryptoHSM` waits for the cancellation request to be fulfilled by waiting for the HSM job to finish. The parameter `CryptoRbHsmCancelTimeout` configures the timeout duration to wait for a cancelled job to finish. If `CryptoRbHsmCancelTimeout` is not configured, then the default value is obtained from parameter `CryptoRbHsmTimeout`. Note: `CryptoRbHsmCancelTimeout` is only valid when `CryptoRbHsmTimeoutMonitoringEnabled` is "`TRUE`". If a HSM job fails to finish within the timeout duration `CryptoRbHsmCancelTimeout`, then the client is informed of a failed cancel request via an integration callout.

Please refer to the rba_CryptoHSM integration advice “Timeout monitoring for synchronous/asynchronous processing of primitives” for information on synchronous timeout monitoring fine-tuning/callouts.

8.8.17 Timeout Monitoring of Asynchronous Jobs

The configuration parameter “Crypto/CryptoGeneral/CryptoRbHsmTimeoutMonitoringEnabled” can be configured “TRUE” to enable timeout monitoring for supported services (Mac, Cipher, Random, SHE Key Management) in asynchronous mode. If timeout monitoring is enabled, an asynchronous job is cancelled when the number of main function cycles exceeds the configured timeout count.

The asynchronous job timeout count value is obtained from the integration callout function `rba_CryptoHSM_TimeoutMonitoring_GetTimeoutDuration_Callout` in the integration template `rba_CryptoHSM_Cfg_TimeoutMonitoring.h`. The default implementation of `rba_CryptoHSM_TimeoutMonitoring_GetElapsedCounterValue_Callout` returns `0xffffffff`.

If `CryptoRbHsmCancelTimeout` is not configured, then the default value is obtained from the parameter `CryptoRbHsmTimeout`. The job processes until job cancellation finishes, or the configured cancel timeout count is exceeded. If a HSM job fails to finish within the cancel timeout count `CryptoRbHsmCancelTimeout`, then the client is informed of a failed cancel request via an integration callout.

The time between each count depends upon the configured main function period `CryptoMainFunctionPeriod` in addition to any overhead for processing queued jobs. It is advised that only a single asynchronous job is queued or additional overhead counts are configured to manage the potential inaccuracies specified above.

Please refer to the rba_CryptoHSM integration advice “Timeout monitoring for synchronous/asynchronous processing of primitives” for information on asynchronous timeout monitoring fine-tuning/callouts.

8.8.18 VKMS Operations

In order to make use of the VKMS Operations listed in the Supported Features section, a proper rba_CryptoHSM configuration needs to be in place. Depending on the feature to be configured, you will need to configure either a `CryptoDriver-Object` (underlying `CryptoPrimitive` configuration) or a `CryptoKey` (underlying `CryptoKeyElement` configuration) or in some cases both. See the Csm Configuration Advice for more details.

8.8.19 CryptoRbInputBuffering/CryptoRbOutputBuffering

In normal mode, if either of the configuration parameters `CryptoRbInputBuffering` or `CryptoRbOutputBuffering` are disabled then, if memory access of the HSM is limited, the application should provide an input buffer which is mapped to the HSM SHARED memory segment.

8.8.20 CryptoRbHsmCopyWithoutNVStorage

If `CryptoRbHsmCopyWithoutNVStorage` is set to *true* (the default is *false*), then the default behaviour of the `rba_CryptoHSM_KeyElementCopy` function is changed because the persisting storage of the *Key Element* in `rba_CryptoHsm` is disabled.

8.8.21 CryptoKeyNvBlockRef

If `CryptoKeyNvBlockRef` not configured, then the key is mapped to a default NvM block (the first one found in Crypto Driver configuration).

8.8.22 Non-volatile Memory Blocks

One or more NvM blocks are needed to store the key status flags and key element data. The number of blocks required is equal to the number of unique `CryptoNvBlock` configured in the `CryptoKeys`.

The NvM blocks used by the Crypto Driver can be configured in the NvM Mappings configurations. The NvM block can be partially configured, in which case the Crypto Driver will automatically complete the mandatory fields.

If no NvM blocks are manually configured and/or linked through NvM Mappings then the Crypto Driver will automatically forward blocks with necessary size to the NvM component. The data is stored in an array of bytes (`uint8[]`).

8.8.23 NvMBlockDescriptor

The `NvMBlockDescriptor` container must be pre-configured by the integrator along with all project specific parameters except for the following parameters which are always forwarded by the Crypto forwarder action:

- `NvMNvBlockLength`
- `NvMBlockUseSyncMechanism`
- `NvMRamBlockDataAddress`
- `NvMRomBlockDataAddress`

Note: The Crypto forwarder action does not check if any of the above parameters are already configured inside the referenced `NvMBlockDescriptor` container.- Manual configuration of any of the above parameters inside the `NvMBlockDescriptor` container will result in upper multiplicity build errors.

8.8.24 HwCsp Interface

The `rba_CryptoHSM` provides a mechanism to use the hardware accelerator. This mechanism provided by HSM is named as Hardware Crypto Service Provider (`HwCsp`) and is only use for Mac operations. All processing is performed on the host side and the keys used are managed by the HSM resource manager.

Limitation :Every Crypto Driver Object which is configured to use the `HwCsp` is mapped to one AES Engine and no other Crypto Driver Object can access the same resource. Therefore, the number of Crypto Driver Objects using `HwCsp` is limited to the number of AES Engines available on hardware and defined by `Cryp-`

toRbHsmHwCspNumberOfAESEnginesparameter.

Configuration :To configure a Crypto Driver Object to use this mechanism, the parameter CryptoRbHsmUseHwCsp shall be true. Be aware that bulk interface and HwCsp cannot be enabled at same time.

Mac Operation :After setting the Key, the first Mac execution will load the key from the HSM Key Management to the host side and then perform the Mac operation. If the key remains unchanged, subsequent Mac calls will be faster by running the service itself. So, be aware that the first call will take longer to execute.

8.9 Integration Advice

8.9.1 General

The rba_CryptoHSM module cannot be used as a stand-alone entity. It is reliant on the software library CycurHSM, and cannot be used without the CryIf, Csm and Crypto modules.

The cryptographic processing in rba_CryptoHSM is performed by the underlying HSM hardware.



NOTE

It is mandatory to integrate CycurHSM V2.7.15 r1 first before using rba_CryptoHSM.

8.9.2 Init Functions

rba_CryptoHSM_Init initializes the rba_CryptoHSM module. As this initialization process can take some time to complete, the rba_CryptoHSM_IsInitialized function can be used to check if initialization has been completed.

8.9.3 Delnit Functions

rba_CryptoHSM_Deinit shuts down the rba_CryptoHSM module. This function ensures that resources that are no longer required are released or deleted.

8.9.4 Scheduled Functions

rba_CryptoHSM_MainFunction provides cyclic processing of asynchronous jobs.

To avoid Maintask cycles being wasted, Maintasks should be called in top-down order (e.g. SecOC → Csm → CryptoDriver).

8.9.5 Job Processing

The synchronous calling of jobs can be done as single call or via a streaming approach (start, update, finish). For single calls, the job is called and returns with the result. For the streaming approach, the job must be called several times, and the result is only available and returned after the finish operation has been successfully completed.

The asynchronous calling of jobs can be done as single call or via the streaming

approach. The initial `rba_CryptoHSM_ProcessJob` call will register the job internally and prepare it for processing. The cyclic MainFunctions check the internal queue and processes the job in the background, based on the priority. If the final result is available, a callback function in `Crylf` (`Crylf_CallbackNotification`) is called to report the final result back to Csm.

rba_CryptoHSM: MainFunction_ProcessDriverObjects

As described in the Supported Features, the asynchronous job processing is implemented using 2 main functions: "`rba_CryptoHSM_MainFunction_ProcessDriverObjects_Request`" and "`rba_CryptoHSM_MainFunction_ProcessDriverObjects_Result`".

These must be called cyclically to process all ACTIVE jobs and must be scheduled in a task with a frequency equal (or higher) to the fastest known producer context of asynchronous jobs in the system. The functions must be called in the following order:

1. `rba_CryptoHSM_MainFunction_ProcessDriverObjects_Result()`
2. `rba_CryptoHSM_MainFunction_ProcessDriverObjects_Request()`

8.9.6 Persistent Storage

The contents of all keys and their key elements are stored in application RAM during processing. This applies to both the actual data and the status of the individual keys. However, there are differences between volatile key elements and persistent key elements:

- Key elements are considered persistent if they are configured with `CryptoKeyElementPersist = TRUE`, otherwise they are volatile.
- Key elements stored on the HSM must always have their `keyId` set to 1. These key elements are always persistent and can only be written and not read.
- The initialisation data of volatile key elements is loaded from the ECU flash memory into application RAM each time an ECU is started. However, the validity of the keys must always be recreated, as it is lost after shutdown of the ECU.
- The initialisation data of persistent key elements is loaded from the ECU flash memory into a configured NvM storage block when the ECU is first started. If the ECU is subsequently restarted, the data is loaded from the NvM storage block into application RAM. The key validity is also stored persistently, and is available again after each restart. In contrast to the volatile key elements, this data need not be recreated when the ECU is restarted. The persistent AES keys are Supported up to 32 bytes in size (256 bits).
- An additional function (`rba_CryptoHSM_Rb_StorePermanentData`) is provided to explicitly trigger the storage of persistent key elements into NvM storage. This function checks if a store operation is required (i.e. some data has been changed) before performing the request, and it only updates the data if necessary. This function should be called to store persistent key

elements (if required) before shutting down or restarting the target ECU. Note that this function depends on the cyclic operation of `rba_CryptoHSM_MainFunction`.

During processing, key elements will be automatically loaded into the HSM RAM present on the ECU. If these are persistent key elements, they will be stored automatically in the non-volatile storage of the HSM. However, if any of these keys is updated during processing, the HSM will consider them as new keys, regardless of the content or key ID, and will thus require additional RAM slots to store the new keys. It is recommended that the `rba_CryptoHSM_Deinit` function is called after such an update by the application. The updated key elements will then be available (via `rba_CryptoHSM_Init`) after re-initialisation of the module. This will ensure that the HSM RAM storage limit is not exceeded, as the reinitialisation will free the HSM RAM used by the updated keys.

8.9.7 Integrating with ESCRYPT CycurHSM

- Integration of CycurHSM V2.7.12 is the prerequisite for using `rba_CryptoHSM`.
- `rba_CryptoHSM_Init` must be called after the HSM start-up has completed.
- `pFunctionGetCounterValue` and `pFunctionGetElapsedCounterValue` configured by `ecy_HSM_ConfigureCallouts` should be implemented using a 1ms Os counter.
- Your linker file should locate `".hsm_shared"` and `".hsm_shared_ro"` to a region of global RAM (not core local) to which the HSM is permitted access.
- BRIDGE registers symbols, which are the communications bridge between the HSM and the HOST, shall be assigned to correct memory address in the linker file.
- `__ghs_hsmstart` and `__ghs_hsmend` which is `rba_CryptoHSM_Cfg_OutputBuffering.h` should point to the start and end address of the `.hsm_shared` area. This is needed to ascertain at runtime whether output buffering is required or not.

8.9.8 CSAI Error Propagation Callback

For propagating the error to an external module, the following public interfaces are added:

- `rba_CryptoHSM_CsaiErrorCallback`
- `rba_CryptoHSM_CsaiErrorBulkMacGenCallback` - when Bulk MAC Generate optimization mode is enabled
- `rba_CryptoHSM_CsaiErrorBulkMacVerCallback` - when Bulk MAC Verify optimization mode is enabled

`rba_CryptoHSM` will not provide an implementation for these APIs. Instead, it will provide a integration template file `"rba_CryptoHSM_Cfg_CsaiErrorPropagation.h"` for the project integrator who will further define the function body as

needed.

To propagate the error to an external module:

- Enable CSAI error propagation "Crypto/CryptoGeneral/CryptoRbHsmCsaiErrorPropagationEnabled" (TRUE)
- Update definition for the CSAI error callback function within the integration template file "rba_CryptoHSM_Cfg_CsaiErrorPropagation.h"

8.9.9 Configuring Keys on the HSM

To allow the correct internal keys to be mapped to the HSM key resources of the right size, the configuring of keys on the HSM requires the addition of a special tag to the respective instance names.

ECDSA_SECP384_PUB_SIZE Public Keys have to be tagged with "SECP384".
ECDSA_SECP256_PUB_SIZE Public Keys have to be tagged with "SECP256".
EDDSA_PUB_SIZE_32 Public Keys have to be tagged with "EDDSA".

Example: CFG_CRYPT0_<instance-name>_SECP384.

The same naming scheme applies for ECC Private Key and ECC Key Pair resources.

RSA_PUB and RSA_PRIV keys have to be tagged with "2048", "3072" or "4096" in their instance name.

8.9.10 CryptoKeys

Although the SHE primitives do not need CryptoKeys, the SHE feature is not compatible with Virtual Key Element usage by other primitives which use SHE key slots from HSM. It is assumed that each CryptoKeyElement mapped to an HSM SHE key slot ID is referenced (via the CryptoKeyType container) by one CryptoKey container only.

8.9.11 Timeout monitoring for synchronous/asynchronous processing of primitives

rba_CryptoHSM provides integration templates for you to set the configurations and implement the callouts required for timeout monitoring of requests running on HSM. The integration callouts need only be implemented if CryptoRbHsmTimeoutMonitoringEnabled is set to TRUE.

The rba_CryptoHSM_Cfg_TimeoutMonitoring.h file contains a set of integration callout functions that you need to implement in order for timeout monitoring to work properly:

rba_CryptoHSM_TimeoutMonitoring_GetTimeoutDuration_Callout returns the timeout threshold in ticks that is used to check if an operation has exceeded its designated time budget.

rba_CryptoHSM_TimeoutMonitoring_GetCounterValue_Callout returns the starting value of the counter/timer used for timeout monitoring.

rba_CryptoHSM_TimeoutMonitoring_GetElapsedCounterValue_Callout returns the elapsed counter/timer value since the start of monitoring.

Important information regarding `rba_CryptoHSM_TimeoutMonitoring_GetElapsedCounterValue_Callout`:

1. The following parameter 'driverObjectId' can be used by the integrator to selectively early finish a pending timeout for a selected (KeyExchange) driverObjectId.
2. The integrator can get the crypto primitive and, if applicable, its associated crypto driver object via the following global arrays: `rba_CryptoHSM_Priv_DriverObjectCfg_acst[]` and `rba_CryptoHSM_Priv_DriverObjectContext_ast[]` as follows:
 - Use `rba_CryptoHSM_Priv_DriverObjectCfg_acst[objectId_u32].service_u8` to get the service id (e.g: CRYPTO_KEYEXCHANGE_CALCPUBVAL)
 - Use `rba_CryptoHSM_Priv_DriverObjectContext_ast[objectId_u32].primitiveContext_u → keyExchangeContext_cp → associatedKeyExchangeObjectId_u32` to get the associated driver object for a KeyExchange driver object.
3. In order to get access to `rba_CryptoHSM_Priv_DriverObjectCfg_acst[]` and `rba_CryptoHSM_Priv_DriverObjectContext_ast[]` global arrays, the following include: `#include "rba_CryptoHSM_Priv.h"` must be added to `rba_CryptoHSM_Cfg_TimeoutMonitoring.c`.

`rba_CryptoHSM_TimeoutMonitoring_RequestCancel_Failed_Callout` is called by `rba_CryptoHSM` when a service has exceeded its allocated timeout, and the attempt at cancelling the service has also timed out. This could indicate a serious issue with the HSM or incorrect configurations of the timeout values.

The following is an example of interactions occurring during timeout monitoring of a primitive undergoing synchronous processing.



Figure 8.5. Example sync timeout monitoring interactions

The following is an example of interactions occurring during timeout monitoring of a primitive undergoing asynchronous processing.

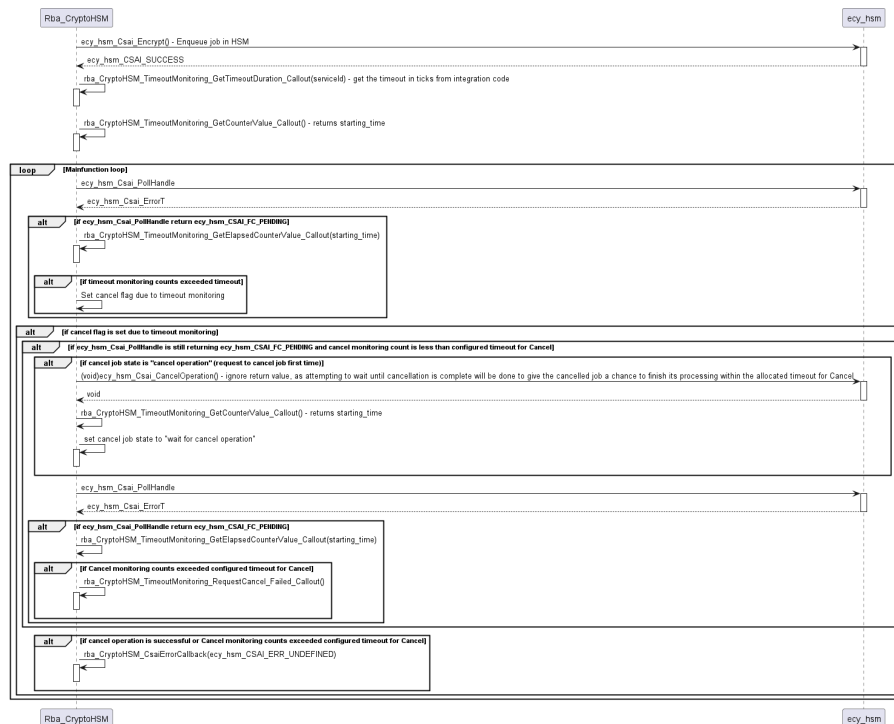


Figure 8.6.Example async timeout monitoring interactions

**WARNING**

The default implementation of `rba_CryptoHSM_TimeoutMonitoring_GetCounterValue_Callout` and `rba_CryptoHSM_TimeoutMonitoring_GetElapsedCounterValue_Callout` in the integration template `rba_CryptoHSM_Cfg_TimeoutMonitoring.h` returns `0x00000000` and `0xffffffff` respectively. If the callouts are not modified by the user, `ecy_hsm_Csai_PollHandle` will be called only once during synchronous processing. If that single call to `ecy_hsm_Csai_PollHandle` has indicated that processing was still ongoing, `Rba_CryptoHSM` will break its busy-wait loop and will attempt to cancel the ongoing operation in HSM.

When implementing the configurations for the timeout thresholds in `rba_CryptoHSM_TimeoutMonitoring_GetTimeoutDuration_Callout` in functions in `rba_CryptoHSM_Cfg_TimeoutMonitoring.h` and the other callouts in the same file, bear in mind that the value returned from `rba_CryptoHSM_TimeoutMonitoring_GetElapsedCounterValue_Callout` is considered as an elapsed counter value and compared to the value returned from `rba_CryptoHSM_TimeoutMonitoring_GetTimeoutDuration_Callout`.

It is up to the implementation in `rba_CryptoHSM_Cfg_TimeoutMonitoring.h` to decide the time value of each count/tick increment. For example, if `rba_CryptoHSM_TimeoutMonitoring_GetElapsedCounterValue_Callout` returns the elapsed value of a counter ticking every 10 microseconds, the values configured

in `rba_CryptoHSM_TimeoutMonitoring_GetElapsedCounterValue_Callout` will be the number of integer multiples of 10 microseconds.

8.9.12 AEAD Encrypt/Decrypt Primitives

If AEAD primitives are used in streaming mode, the secondary input data (additional data/associated data) must be provided before the primary input data (plain/cipher text), otherwise an error will be returned. Both the primary and secondary input data can be provided in a single UPDATE call.

- The length of Initialization Vector (IV) must be 12 bytes (HSM V2.6.0.1 restriction).
- The size of tag must be at least 16 bytes (HSM V2.6.0.1 restriction).

8.9.13 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The `Rba_CryptoHSM` module does not use any exclusive OS resources.

8.10 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

9 rba_CryptoAuHSM3

Module name	rba_CryptoAuHSM3
Package	RTA-SEC
AUTOSAR Module ID	N/A
AUTOSAR Version	22-11

9.1 Introduction

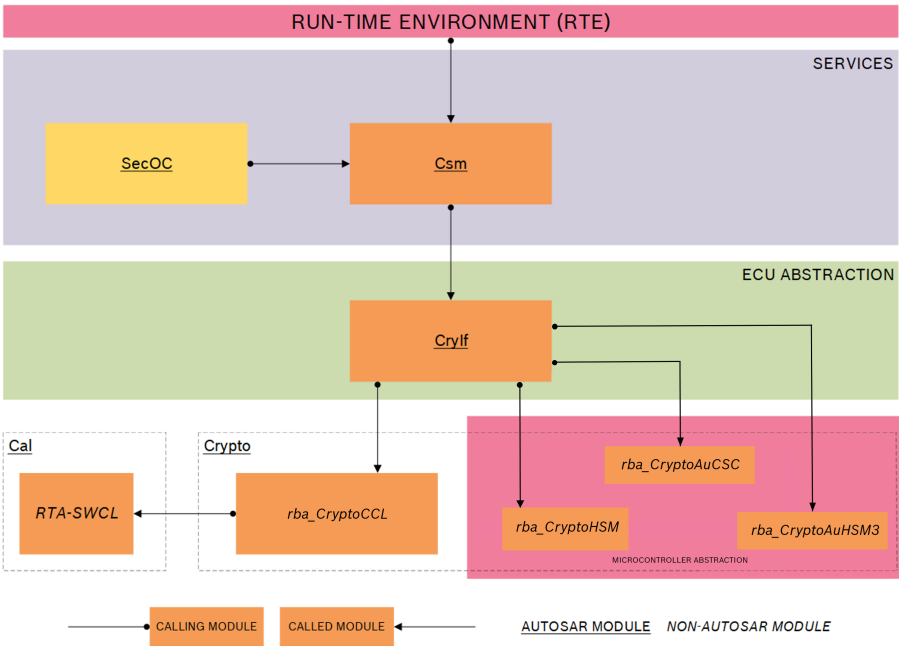


Figure 9.1.RTA-SEC Stack Architecture

The *rba_CryptoAuHSM3* module is part of the Microcontroller Abstraction Layer (MCAL) and sits beneath the Crypto Interface (CryIf) and Crypto Service Manager (Csm) modules, alongside the *rba_CryptoHSM* module as a component of the Crypto module. It implements a generic interface for synchronous and asynchronous cryptographic primitives and also supports key storage, configuration and management for cryptographic services. The keys and the cryptographic primitives can be configured by using the standard AUTOSAR configuration mechanisms (provided via Crypto).

rba_CryptoAuHSM3 is part of the following software stack (see diagram above):

- *Csm*: maps the abstract AUTOSAR-standardized user interfaces to the less abstracted interfaces of *Crypto*.
- *CryIf*: acts as an additional mapping layer between *Csm* and the *rba_Crypto*<variant> components.
- *Crypto*: implements the configuration interface for all *Crypto* components,

and provides libraries for common or AUTOSAR-defined functionalities which are then used by the rba_Crypto<variant> components.

- *rba_CryptoAuHSM3*: implements a generic interface for synchronous and asynchronous cryptographic primitives by using the CycurHSM3 (HSM) hardware. CycurHSM3 (HSM) offers the underlying cryptographic functionality for rba_CryptoAuHSM3.

CycurHSM3 is used by rba_CryptoAuHSM3 but is not a part of the Crypto structural component.

Module Configuration

This module's configuration parameters are included in the configuration of another module. See the Configuration Advice section for more details.

9.2 Supported Features

9.2.1 Supported Primitives

Primitive	Synchronous	Asynchronous	STREAMING	SINGLECALL
Hash (SHA2-224/256/384/512), truncation supported	x	x	x	x
Hash (SHA3-224/256/384/512), truncation supported	x	x	x	x
Mac (SipHash-2-4)	x	x	x	x
Mac (AES128/256 CMAC)	x	x	x	x
Cipher (AES128/256 CBC with padding: PKCS7 NIST SP800-38A NONE)	x	x	x*	x
Cipher (AES128/256 ECB with padding: PKCS7 NONE)	x	x	x*	x
Signature (ECDSA SECP256R1 with hash: SHA256 SHA512)	x	x	x	x
Signature (ECDSA SECP521R1 with hash: SHA512)	x	x	x	x
Signature (EDDSA ED25519 with hash: SHA512)	x	x	o	x
Signature (RSA PSS 2k,3k,4k with hash SHA256)	x	x	x	x
Signature (RSA PKCS1 v1_5 2k,4k with hash SHA256)	x	x	x	x
RandomGenerate (DRBG/TRNG)	x	x	o	x
RandomSeed	x	x	-	x

CertificateParse (X509) (**) (***)	x	o	-	x
CertificateVerify (X509) (**)	x	o	-	x
KeyElementGet (**)	x	o	-	x
KeyElementSet (**)	x	o	-	x

x = implemented, o = not implemented, - = not relevant

x* = Boundary conditions for the usage of Cipher Primitive STREAMING:

- **For all padded Encrypt calls:** Any chunk presented to *STREAMSTART*, *UPDATE* or *FINISH*, which is not the last chunk of the message, must be a multiple of 16 bytes. Otherwise, if the following call is not a *FINISH* call, an error is reported.
- **For all none padded Encrypt calls and all Decrypt calls:** Any chunk presented to *STREAMSTART*, *UPDATE* or *FINISH* must be a multiple of 16 bytes. Otherwise, if the following call is not a *FINISH* call, an error is reported.
- **For all padded Decrypt calls:** The last chunk must be provided in a *FINISH* call. Otherwise, an error is reported.
- The Decrypt input and output buffers must be the same size (clear text message + padding).



NOTE

For *SINGLECALL*, the Decrypt output buffer only needs the size of the clear text message (without padding).

(**) All certificate operations (CertificateVerify, injectCertificate via keyElementSet, readCertificate via keyElementGet) must use the same driver object.

(***) The public key needs to be injected again via *KeyElementSet* to ensure that all key properties are set. The public key is injected by the HSM3 GetPubkeyFromCert API, so it is possible that not all key properties are configured.

9.3 Extensions

9.3.1 Configuration Extensions

- No known extension parameters are defined in this version.

9.3.2 Additional Services

The following API are available in v12.7.0 in addition to those defined in AR v22-11

9.4 Limitations

9.5 Exclusions

The following generic API functions listed in the Crypto Driver module specification have no functional equivalents in this module:

- Crypto_GetVersionInfo
- Crypto_CustomSync
- Crypto_NvBlock_Init_<NvBlock>
- Crypto_NvBlock_ReadFrom_<NvBlock>
- Crypto_NvBlock_WriteTo_<NvBlock>
- Crypto_NvBlock_Callback_<NvBlock>

9.5.1 Configuration Exclusions

- There are no known exclusions in this version.

9.6 Deviations

- There are no known deviations in this version.

9.7 API Definition

List of supported API functions (all requests are routed through the corresponding Crypto API):

rba_CryptoAuHSM3_CancelJob
rba_CryptoAuHSM3_KeyCopy
rba_CryptoAuHSM3_KeyElementCopy
rba_CryptoAuHSM3_KeyElementCopyPartial
rba_CryptoAuHSM3_KeyElementGet
rba_CryptoAuHSM3_KeyElementIdsGet
rba_CryptoAuHSM3_KeyElementSet
rba_CryptoAuHSM3_KeyExchangeCalcPubVal
rba_CryptoAuHSM3_KeyExchangeCalcSecret
rba_CryptoAuHSM3_KeySetValid
rba_CryptoAuHSM3_KeySetInvalid
rba_CryptoAuHSM3_ProcessJob
rba_CryptoAuHSM3_RandomSeed
rba_CryptoAuHSM3_KeyGenerate
rba_CryptoAuHSM3_KeyGetStatus
rba_CryptoAuHSM3_KeyDerive
rba_CryptoAuHSM3_Rb_CertificateParse
rba_CryptoAuHSM3_Rb_CertificateVerify

For details on using these calls, please refer to the Crypto module's API list (Section 2.7) and AUTOSAR_SWS_CryptoDriver.pdf (version v22-11).

9.8 Configuration Advice

The Crypto module (Chapter 2) supports the configuration parameters required to define unique driver object instances for cryptographic primitives defined in rba_CryptoAuHSM3.



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

9.9 Integration Advice

9.9.1 General

The rba_CryptoAuHSM3 module cannot be used as a stand-alone entity. It is reliant on the software library CycurHSM3, and cannot be used without the CryIf, Csm and Crypto modules.

The cryptographic processing in rba_CryptoAuHSM3 is performed by the underlying HSM hardware.



NOTE

It is mandatory to integrate CycurHSM3 version first before using rba_CryptoHSM.

9.9.2 Init Functions

rba_CryptoAuHSM3_Startup initializes the rba_CryptoAuHSM3 module.

9.9.3 Delnit Functions

rba_CryptoAuHSM3_Shutdown shuts down the rba_CryptoAuHSM3 module.

9.9.4 Scheduled Functions

rba_CryptoAuHSM3_Process provides processing of asynchronous jobs.

9.9.5 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in

the OS with the correct task/ISR allocation.

The Rba_CryptoAuHSM3 module does not use any exclusive OS resources.

9.10 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

10 SecOC

Module name	SecOC
Package	RTA-SEC
AUTOSAR Module ID	150
AUTOSAR Version	22-11

10.1 Introduction

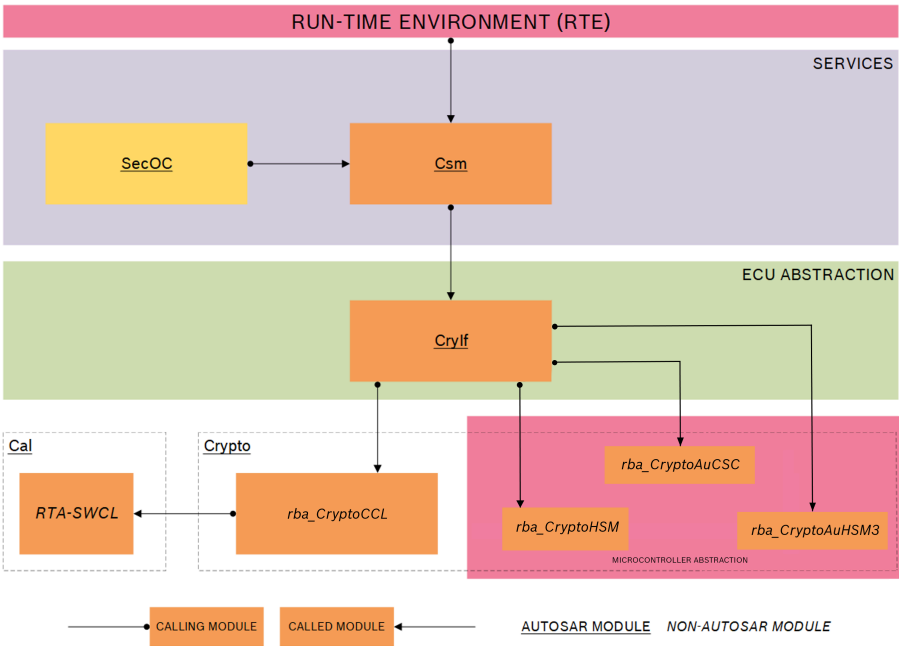


Figure 10.1.RTA-SEC Stack Architecture

The Secure Onboard Communication module (SecOC) is located in the communication services layer of the AUTOSAR layered architecture. It is considered a part of the RTA-SEC stack, and makes use of the cryptographic functions provided by other stack modules through the Crypto Service Manager (Csm). The diagram above shows the relationship between SecOC and the other stack modules.

SecOc provides services for the symmetric authentication and verification of data at I-PDU level using MAC (Message Authentication Code) cryptography, for use by the PDU Router (PduR).

Module Configuration

This module is not currently supported by the RTA-BSW Configuration Generator (ConfGen) and must be manually configured.

10.2 Supported Features

10.2.1 General

- SecOc supports authentication and verification using the CMAC_AES128 algorithm (with rba_CryptoHSM and rba_CryptoCCL) or the SIPHASH-2-4 algorithm (with rba_CryptoHSM).
- Cryptographic operations are supported for both software (rba_CryptoCCL) and hardware (rba_CryptoHSM) cryptographic driver modules via the Csm module.
- Synchronous and asynchronous communication with the cryptographic RTA-SEC stack modules is supported.
- Switch-off is supported for authentication and verification of both single and all I-PDUs through configuration.
- SecOc supports the internal (shared) buffer feature as described in the AUTOSAR specification.

10.2.2 PduR Interface

SecOc's PduR interface supports the following communication methods as specified by AUTOSAR.

- Authentication during direct transmission.
- Authentication during triggered transmission.
- Authentication during transport protocol transmission.
- Verification during direct reception.
- Verification during transport protocol reception.

10.2.3 MAC Truncation

Truncation of the message authenticator (MAC) is supported for the creation of Secured I-PDUs. This feature allows additional space to be reserved for the data payload of an Authentic I-PDU if required. See the Configuration Advice section for more details on the use of this feature.

10.2.4 MainFunction Partitioning

RTA-SEC supports MainFunction partitioning of resources. For SecOc, this involves the generation of multiple SecOC_MainFunctionTx/SecOC_MainFunctionRx instances and mapping of Tx/Rx PDUs to these instances where they should be handled. See the Configuration Advice for more details.

10.3 Extensions

10.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition

to those defined in AR v22-11

10.3.1.1 SecOC/SecOCGeneral/SecOCRbCryptDefaultInterface

Short Name	SecOCRbCryptDefaultInterface
Long Name	SecOC default cryptographic interface
Description	Selects the default cryptographic interface. It is set for all Rx and Tx pdus if the mac calculation is done via the cryptographic library (value Csm_rba_CryptoCCL), via hardware security module (value Csm_rba_CryptoHSM) or an external crypto driver (value Csm_Crypto_External). The value 'None' is only allowed for Tx PDUs in development environment (Det is enabled), no cryptographic interface is called. Nevertheless the PDUs are forwarded to the PduR for further routing. This configuration can be overwritten for each pdu in the pdu configuration containers. If the value 'Csm_Crypto_External' is selected here or in any PDU, SecOC forwarding to other components (e.g. Csm, Crylf, Crypto) is disabled.
Multiplicity	1..1
Default Value	Csm_rba_CryptoCCL
Enumeration Literals	Csm_Crypto_External, Csm_rba_CryptoCCL, Csm_rba_CryptoHSM, None
Type	ECUC-ENUMERATION-PARAM-DEF

10.3.1.2 SecOC/SecOCGeneral/SecOCRbCryptDefaultMode

Short Name	SecOCRbCryptDefaultMode
Long Name	SecOC cryptographic mode
Description	SecOC can call the cryptographic services synchronous or asynchronous. Both are done in the cyclic main functions but in case of asynchronous mode the result is received via callbacks. AR defines the asynchronous mode. This configuration is set for all Rx and Tx PDUs but it can be overwritten for each pdu in the pdu configuration containers.
Multiplicity	1..1
Default Value	synchronous
Enumeration Literals	asynchronous, synchronous
Type	ECUC-ENUMERATION-PARAM-DEF

10.3.1.3 SecOC/SecOCGeneral/SecOCRbCustomerFeat

Short Name	SecOCRbCustomerFeat
Long Name	SecOC Customer Features
Description	Contains the customer specific extensions. Do not configure anything in this container if you expect AR conformal implementation.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

10.3.1.4 SecOC/SecOCGeneral/SecOCRbCustomerFeat/SecOCRbExtKsnInitVal

Short Name	SecOCRbExtKsnInitVal
Long Name	SecOC external key serial number initial value
Description	This parameter defines a 16-Bit KSN initial value. It is used to initialize internal structures for dynamic received external KSNs. Value depends on SecOCRbNumExtKsn, therefore no default is pre-configured. If SecOCRbNumExtKsn is configured, then 0x0 taken as default value.
Multiplicity	0..1
Value Range	0..65535
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

10.3.1.5 SecOC/SecOCGeneral/SecOCRbCustomerFeat/SecOCRbExtendValueID

Short Name	SecOCRbExtendValueID
Long Name	SecOC Extended ValueID
Description	This parameter indicates a deviation of AR specification. The ValueIDs, SecOCDataId and SecOCFreshnessValueId, are extended to contain 5 bytes. FALSE: ValueID [2Bytes AR Spec] TRUE: ValueID [5Bytes Customer specific requirement] default: FALSE
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.6 SecOC/SecOCGeneral/SecOCRbCustomerFeat/SecOCRbExtendVerifyStatusCallout

Short Name	SecOCRbExtendVerifyStatusCallout
Long Name	SecOC extended verification status callout
Description	Name of the extended verification status callout. Active for a specific RxPdu, if SecOC/SecOCRxPduProcessing/SecOCRbExtendedVerifyStatusCalloutEnabled is set to true in addition.
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

10.3.1.7 SecOC/SecOCGeneral/SecOCRbCustomerFeat/SecOCRbKsn

Short Name	SecOCRbKsn
Long Name	SecOC key serial number
Description	This parameter defines the key serial number for Some/IP PDUs (only for customer specific extensions required)
Multiplicity	0..1
Value Range	0..65535
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

10.3.1.8 SecOC/SecOCGeneral/SecOCRbCustomerFeat/SecOCRbNumExtKsn

Short Name	SecOCRbNumExtKsn
Long Name	SecOC number of external key serial numbers
Description	This parameter defines the amount of external key serial numbers
Multiplicity	0..1
Value Range	1..65535
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

10.3.1.9 SecOC/SecOCGeneral/SecOCRbEnableCryptPduBypass

Short Name	SecOCRbEnableCryptPduBypass
Long Name	SecOC enable cryptographic pdu bypass

Description	When this configuration option ist set to TRUE the functionality inside the optional Interface SecOC_VerifyStatusOverride provide an additional overrideStatus = 43 (meaning bypass verification until notice)
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.10 SecOC/SecOCGeneral/SecOCRbEnableMeasurementPoint

Short Name	SecOCRbEnableMeasurementPoint
Long Name	Enable SecOC measurement points
Description	If this parameter is set to TRUE, the measurement points are enabled for all Tx and Rx PDUs. For details see user documentation.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.11 SecOC/SecOCGeneral/SecOCRbEnablePduCollection

Short Name	SecOCRbEnablePduCollection
Long Name	SecOC enable pdu collection
Description	If this parameter ist set to TRUE, pdu collection can be used
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.12 SecOC/SecOCGeneral/SecOCRbForwarder

Short Name	SecOCRbForwarder
Long Name	SecOC Forwarder configuration
Description	SecOC configuration that will be forwarded to CryptoStack.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

10.3.1.13 SecOC/SecOCGeneral/SecOCRbForwarder/SecOCRbDefault-CryptAlgoFamily

Short Name	SecOCRbDefaultCryptAlgoFamily
Long Name	SecOC cryptographic algorithm family

Description	Selects the algorithm family used for generation and verification of MAC.*At least the following AlgoFam, Mode, 2ndFam combinations are support:*AES, CMAC, AES*POLY1305, NOT_SET, POLY1305 (Poly1305 algorithm hasn't got different modes)*SIPHASH, SIPHASH_2_4, SPIHASH
Multiplicity	0..1
Default Value	AES
Enumeration Literals	AES, POLY1305, SIPHASH
Type	ECUC-ENUMERATION-PARAM-DEF

10.3.1.14 SecOC/SecOCGeneral/SecOCRbForwarder/SecOCRbDefault-CryptAlgoMode

Short Name	SecOCRbDefaultCryptAlgoMode
Long Name	SecOC cryptographic algorithm mode
Description	Selects the algorithm mode used for generation and verification of MAC.*At least the following AlgoFam, Mode, 2ndFam combinations are support:*AES, CMAC, AES*POLY1305, NOT_SET, POLY1305 (Poly1305 algorithm hasn't got different modes)*SIPHASH, SIPHASH_2_4, SPIHASH
Multiplicity	0..1
Default Value	CMAC
Enumeration Literals	CMAC, NOT_SET, SIPHASH_2_4
Type	ECUC-ENUMERATION-PARAM-DEF

10.3.1.15 SecOC/SecOCGeneral/SecOCRbForwarder/SecOCRbDefault-CryptAlgoSecondFamily

Short Name	SecOCRbDefaultCryptAlgoSecondFamily
Long Name	SecOC cryptographic algorithm secondary family
Description	Selects the algorithm secondary family used for generation and verification of MAC.*At least the following AlgoFam, Mode, 2ndFam combinations are support-*AES, CMAC, AES*POLY1305, NOT_SET, POLY1305 (Poly1305 algorithm hasn't got different modes)*SIPHASH, SIPHASH_2_4, SPIHASH
Multiplicity	0..1
Default Value	AES
Enumeration Literals	AES, POLY1305, SIPHASH
Type	ECUC-ENUMERATION-PARAM-DEF

10.3.1.16 SecOC/SecOCGeneral/SecOCRbForwarder/SecOCRbDefaultUseBulkProcessing

Short Name	SecOCRbDefaultUseBulkProcessing
Long Name	SecOC Default use Bulk Processing
Description	This parameter indicates if the MAC calculation should be processed in Bulk mode or in single shot mode only for those PDUs which do not have the parameter SecOCRb[Rx/Tx]UseBulkProcessing configured. The SecOCRbDefaultUseBulkProcessing parameter will be ignored by those PDUs that have SecOCRb[Rx/Tx]UseBulkProcessing configured. The MAC calculation will be processed according to the value of SecOCRb[Rx/Tx]UseBulkProcessing parameter. Note: Bulk mode is available only for PDUs with cryptographic interface set to HSM and cryptographic mode set to asynchronous. The default value is FALSE.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.17 SecOC/SecOCGeneral/SecOCRbForwarder/SecOCRbMaxLengthRx

Short Name	SecOCRbMaxLengthRx
Long Name	SecOC max length of verification data
Description	This parameter specifies the maximum length in bits of the data for mac verification. This info is forwarded to the csm primitives.
Multiplicity	0..1
Value Range	0..4294967295
Default Value	128
Type	ECUC-INTEGER-PARAM-DEF

10.3.1.18 SecOC/SecOCGeneral/SecOCRbForwarder/SecOCRbMaxLengthTx

Short Name	SecOCRbMaxLengthTx
Long Name	SecOC max length of authentication data
Description	This parameter specifies the maximum length in bits of the data for mac authentication. This info is forwarded to the csm primitives.
Multiplicity	0..1

Value Range	0..4294967295
Default Value	128
Type	ECUC-INTEGER-PARAM-DEF

10.3.1.19 SecOC/SecOCGeneral/SecOCRbMainFunctionPeriodRx

Short Name	SecOCRbMainFunctionPeriodRx
Long Name	Period of the default main function SecOC_Main-FunctionRx
Description	Specifies the period in seconds of the default main function (SecOC_MainFunctionRx) which handles all Rx PDUs with no reference to a SecOCMainFunctionRx container. If there is no such Rx PDU configured, this parameter has no usage (each SecOC_MainFunctionRx container has its own periodicity parameter).
Multiplicity	0..1
Value Range	0..INF
Default Value	0.1
Type	ECUC-FLOAT-PARAM-DEF

10.3.1.20 SecOC/SecOCGeneral/SecOCRbMainFunctionPeriodTx

Short Name	SecOCRbMainFunctionPeriodTx
Long Name	Period of the default main function SecOC_Main-FunctionTx
Description	Specifies the period in seconds of the default main function (SecOC_MainFunctionTx) which handles all Tx PDUs with no reference to a SecOCMainFunctionTx container. If there is no such Tx PDU configured, this parameter has no usage (each SecOC_MainFunctionTx container has its own periodicity parameter).
Multiplicity	0..1
Value Range	0..INF
Default Value	0.1
Type	ECUC-FLOAT-PARAM-DEF

10.3.1.21 SecOC/SecOCGeneral/SecOCRbPDUsWithSawtoothBitcounting

Short Name	SecOCRbPDUsWithSawtoothBitcounting
Long Name	PDUs use sawtooth bit counting

Description	When this parameter is set to TRUE, the SecOCAuthDataFreshnessStartPosition of a PDU and SecOCMessageLinkPos of a secure PDU (if a collection PDU is used) are interpreted following sawtooth bit counting. Otherwise monotone bit counting is assumed. According to AUTOSAR, sawtooth bit counting should be used, monotone bit counting is used as default to keep backwards compatibility.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.22 SecOC/SecOCGeneral/SecOCRbRetryVerifyImmediately

Short Name	SecOCRbRetryVerifyImmediately
Long Name	SecOC retry verification immediately
Description	If this parameter is set to true, SecOC retries the verification immediately in the same main function cycle if mac verification fails (see section 9.4 in AR specification). This feature is only possible for synchronous jobs.
Multiplicity	1..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.23 SecOC/SecOCGeneral/SecOCRbRteInUse

Short Name	SecOCRbRteInUse
Long Name	SecOC Rte in use
Description	The module local variable SecOCRbRteInUse and the global variable EcuCRbRteInUse are combined using OR logic. If SecOCRbRteInUse is FALSE, the RTE usage is determined by EcuCRbRteInUse. If SecOCRbRteInUse is TRUE, the condition matches that of EcuCRbRteInUse being TRUE on module scale.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.24 SecOC/SecOCGeneral/SecOCRbRteQLenVerifyStateOverride

Short Name	SecOCRbRteQLenVerifyStateOverride
Long Name	SecOC Rte queue length for VerifyStatusOverride

Description	This parameter is the depth of the RTE q-length, the history buffer, generated and provided by the Rte. Usage might be needed (IF): Rte is used (AND) Provided PORT and Required PORT are asynchron (AND) running on different OsApplications e.g. 1ms 10ms (AND) a history of all values is needed on the requesting side.
Multiplicity	0..1
Minimum Value	1
Default Value	1
Type	ECUC-INTEGER-PARAM-DEF

10.3.1.25 SecOC/SecOCGeneral/SecOCRbSafetyCallout

Short Name	SecOCRbSafetyCallout
Long Name	SecOC safety callout
Description	If the safety callout is configured, it is called in API SecOC_VerifyStatusOverride. Depending upon the result of the callout, the execution of API SecOC_VerifyStatusOverride is blocked or not. TRUE: VerifyStatusOverride is blocked, FALSE: VerifyStatusOverride is executed.
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

10.3.1.26 SecOC/SecOCGeneral/SecOCRbSecuredAreaDeviation

Short Name	SecOCRbSecuredAreaDeviation
Long Name	SecOC secured area Autosar deviation
Description	If this parameter is set to false, Range of secOCSe- curedTxPduLength and secOCSecuredRxPduLength is 1 .. 4294967295 (AR compliant), if the parameter is true, Range of secOCSecuredTxPduLength and secOCSecuredRxPduLength is 0 .. 4294967295 (AR deviation). If secOCSecuredTxPduLength or secOC- SecuredRxPduLength is set to 0, the PDU payload is excluded for MAC calculation. If secOCSecured TxPduLength or secOCSecuredRxPduLength is not configured (NULL_PTR), the complete PDU payload is included for MAC calculation.
Multiplicity	0..1
Default Value	false

Type	ECUC-BOOLEAN-PARAM-DEF
------	------------------------

10.3.1.27 SecOC/SecOCGeneral/SecOCRbUseGranularLocks

Short Name	SecOCRbUseGranularLocks
Long Name	SecOC use granular locks
Description	If granular locks are enabled, locks specific to each PDU are created, allowing for parallel processing of different PDUs. Conversely, if this feature is disabled, legacy lock functions are utilized, which serialize access to the entire context buffer array. These legacy lock functions are equivalent to global locks, resulting in slower PDU processing due to the serialization of access. However, the trade-off is reduced memory consumption.
Multiplicity	0..1
Default Value	true
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.28 SecOC/SecOCGeneral/SecOCRbUseSingleBuffer

Short Name	SecOCRbUseSingleBuffer
Long Name	SecOC use single buffer
Description	If single buffer is enabled, inputbuffer and outputbuffer are disabled and only the working buffer is used. The single buffer can be enabled either for every single pdu or for all pdus. If this parameter is set to TRUE, the single buffer solution is enabled for ALL pdus. To enable the single buffer for just ONE pdu, this parameter should be disabled and the parameter in the pdu should be enabled.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.29 SecOC/SecOCGeneral/SecOCRbVerificationStatusIndication-SyncMode

Short Name	SecOCRbVerificationStatusIndicationSyncMode
Long Name	SecOC VerificationStatus Indication Synchronous Mode
Description	Legacy parameter to provide old required port behaviour of the client/server-interface.
Multiplicity	0..1

Default Value	synchronous
Enumeration Literals	asynchronous, synchronous
Type	ECUC-ENUMERATION-PARAM-DEF

10.3.1.30 SecOC/SecOCGeneral/SecOCSecurityEventRefs/SEV_SEC-OC_FRESHNESS_NOT_AVAILABLE

Short Name	SEV_SECOC_FRESHNESS_NOT_AVAILABLE
Long Name	Security Event SEV_SECOC_FRESHNESS_NOT_AVAILABLE
Description	Failed to get freshness value from freshness manager
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

10.3.1.31 SecOC/SecOCGeneral/SecOCSecurityEventRefs/SEV_SECOC_MAC_VERIFICATION_FAILED

Short Name	SEV_SECOC_MAC_VERIFICATION_FAILED
Long Name	Security Event SEV_SECOC_MAC_VERIFICATION_FAILED
Description	MAC verification of a received PDU failed.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

10.3.1.32 SecOC/SecOCRxPduProcessing/SecOCRbExtendVerifyStatus-CalloutEnabled

Short Name	SecOCRbExtendVerifyStatusCalloutEnabled
Long Name	SecOC extended verification status callout enabled
Description	If parameter is set to true callout function configured in SecOC/SecOCGeneral/SecOCRbExtendVerifyStatusCallout is activated.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.33 SecOC/SecOCRxPduProcessing/SecOCRbRxCryptMode

Short Name	SecOCRbRxCryptMode
Long Name	SecOC cryptographic mode

Description	SecOC can call the cryptographic services synchronous or asynchronous. Both are done in the cyclic main functions but in case of asynchronous mode the result is received via callbacks. AR defines the asynchronous mode. This pdu specific configuration overwrites the global setting in container SecOCGeneral.
Multiplicity	0..1
Default Value	-
Enumeration Literals	asynchronous, synchronous
Type	ECUC-ENUMERATION-PARAM-DEF

10.3.1.34 SecOC/SecOCRxPduProcessing/SecOCRbRxDirectModeEnabled

Short Name	SecOCRbRxDirectModeEnabled
Long Name	SecOC Rx direct mode
Description	If set to TRUE, for explicitly partitioned Rx PDUs (SecOCRxPduMainFunctionRef reference parameter is configured), it performs at runtime an immediate call of the corresponding SecOC_MainFunctionRx_{instance} in the context of SecOC_[Tp]RxIndication.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.35 SecOC/SecOCRxPduProcessing/SecOCRbRxForwarder

Short Name	SecOCRbRxForwarder
Long Name	SecOC Forwarder configuration
Description	SecOC configuration of received PDUs that will be forwarded to CryptoStack. If nothing is configured here, the default value from SecOCGeneral/SecOCRb-Forwarder will be used.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

10.3.1.36 SecOC/SecOCRxPduProcessing/SecOCRbRxForwarder/SecOCRbAutoConfigHsmKeySlotId

Short Name	SecOCRbAutoConfigHsmKeySlotId
Long Name	SecOC cryptographic HSM key slot

Description	This parameter explicitly maps the secure cryptokey element into a specified key slot inside the HSM secure flash. If left empty, then an available key slot automatically assigned to it. The name (e.g. DYNAMIC_CFG_CRYPT_AES_KEY_BASE_0) of the key slot must be provided. This autoconfiguration parameter will only apply to key element which are stored in HSM.
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

10.3.1.37 SecOC/SecOCRxPduProcessing/SecOCRbRxForwarder/SecOCRbRxCryptAlgoFamily

Short Name	SecOCRbRxCryptAlgoFamily
Long Name	SecOC cryptographic algorithm family
Description	Selects the algorithm family used for generation and verification of MAC.*At least the following AlgoFam, Mode, 2ndFam combinations are support:*AES, CMAC, AES*POLY1305, NOT_SET, POLY1305 (Poly1305 algorithm hasn't got different modes)*SIPHASH, SIPHASH_2_4, SPIHASH
Multiplicity	0..1
Default Value	-
Enumeration Literals	AES, POLY1305, SIPHASH
Type	ECUC-ENUMERATION-PARAM-DEF

10.3.1.38 SecOC/SecOCRxPduProcessing/SecOCRbRxForwarder/SecOCRbRxCryptAlgoMode

Short Name	SecOCRbRxCryptAlgoMode
Long Name	SecOC cryptographic algorithm mode
Description	Selects the algorithm mode used for generation and verification of MAC.*At least the following AlgoFam, Mode, 2ndFam combinations are support:*AES, CMAC, AES*POLY1305, NOT_SET, POLY1305 (Poly1305 algorithm hasn't got different modes)*SIPHASH, SIPHASH_2_4, SPIHASH
Multiplicity	0..1
Default Value	-
Enumeration Literals	CMAC, NOT_SET, SIPHASH_2_4
Type	ECUC-ENUMERATION-PARAM-DEF

10.3.1.39 SecOC/SecOCRxPduProcessing/SecOCRbRxForwarder/SecOCRbRxCryptAlgoSecondFamily

Short Name	SecOCRbRxCryptAlgoSecondFamily
Long Name	SecOC cryptographic algorithm secondary family
Description	Selects the algorithm secondary family used for generation and verification of MAC.*At least the following AlgoFam, Mode, 2ndFam combinations are supported: *AES, CMAC, AES*POLY1305, NOT_SET, POLY1305 (Poly1305 algorithm hasn't got different modes)*SIPHASH, SIPHASH_2_4, SPIHASH
Multiplicity	0..1
Default Value	-
Enumeration Literals	AES, POLY1305, SIPHASH
Type	ECUC-ENUMERATION-PARAM-DEF

10.3.1.40 SecOC/SecOCRxPduProcessing/SecOCRbRxForwarder/SecOCRbRxUseBulkProcessing

Short Name	SecOCRbRxUseBulkProcessing
Long Name	SecOC use Bulk Processing
Description	This parameter indicates if the MAC verification for Rx PDUs should be processed in Bulk mode or in single shot mode. Possible values: 1. TRUE: the MAC job will be processed in Bulk mode, 2. FALSE: the MAC job will be processed in single shot mode. Note: If SecOCRbRxUseBulkProcessing is not configured, then it will use the configured value of SecOCGeneral/SecOCRbDefaultUseBulkProcessing as default. Bulk mode is available only for PDUs with cryptographic interface set to HSM and cryptographic mode set to asynchronous.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.41 SecOC/SecOCRxPduProcessing/SecOCRbRxPduCryptIF

Short Name	SecOCRbRxPduCryptIF
Long Name	SecOC cryptographic interface

Description	Selects the default cryptographic interface. It is set if the mac calculation is done via the cryptographic library (value Csm_rba_CryptoCCL), via hardware security module (value Csm_rba_CryptoHSM) or an external crypto driver (value Csm_Crypto_External). If value 'None' is selected no cryptographic interface is called. Nevertheless the PDUs are forwarded to the PduR for further routing. This pdu specific configuration overwrites the global setting in container SecOCGeneral. If the value 'Csm_Crypto_External' is selected here or in any PDU, SecOC forwarding to other components (e.g. Csm, CryIf, Crypto) is disabled.
Multiplicity	0..1
Default Value	-
Enumeration Literals	Csm_Crypto_External, Csm_rba_CryptoCCL, Csm_rba_CryptoHSM
Type	ECUC-ENUMERATION-PARAM-DEF

10.3.1.42 SecOC/SecOCRxPduProcessing/SecOCRbRxUseSingleBuffer

Short Name	SecOCRbRxUseSingleBuffer
Long Name	SecOC Rx use single buffer
Description	If single buffer is enabled, separate in/output buffers are disabled and only one working buffer is used. Ring buffers and Queuing for Receive Overflow Strategy are disabled. If this parameter ist set to TRUE, single buffer solution is enabled for the rx pdu.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.43 SecOC/SecOCTxPduProcessing/SecOCRbClearBufferAfterTransmit

Short Name	SecOCRbClearBufferAfterTransmit
Long Name	SecOCRb Clear buffers after transmit
Description	If this parameter is set to true, SecOC doesn't wait for TxConfirmation to clear the buffers and instead clears the buffers and resets the PDU state to idle directly after passing the secured PDU to the PduR.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.44 SecOC/SecOCTxPduProcessing/SecOCRbDynamicRuntimeLengthHandling

Short Name	SecOCRbDynamicRuntimeLengthHandling
Long Name	SecOC handling of dynamic Tx PDUs
Description	This parameter defines whether the Tx PDU is sent with the configured or dynamic length. If it is set to TRUE the calculated length during runtime is used. If it is set to FALSE, the configured length is used, unused area are filled with the configured default pattern (SecOCTxPduUnusedAreasDefault).
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.45 SecOC/SecOCTxPduProcessing/SecOCRbTxCryptMode

Short Name	SecOCRbTxCryptMode
Long Name	SecOC cryptographic mode
Description	SecOC can call the cryptographic services synchronous or asynchronous. Both are done in the cyclic main functions but in case of asynchronous mode the result is received via callbacks. AR defines the asynchronous mode. This pdu specific configuration overwrites the global setting in container SecOCGeneral.
Multiplicity	0..1
Default Value	-
Enumeration Literals	asynchronous, synchronous
Type	ECUC-ENUMERATION-PARAM-DEF

10.3.1.46 SecOC/SecOCTxPduProcessing/SecOCRbTxDirectModeEnabled

Short Name	SecOCRbTxDirectModeEnabled
Long Name	SecOC Tx direct mode
Description	If set to TRUE, for explicitly partitioned Tx PDUs (SecOCTxPduMainFunctionRef reference parameter is configured), it performs at runtime an immediate call of the corresponding SecOC_MainFunctionTx_{instance} in the context of SecOC_{IfItP}Transmit.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.47 SecOC/SecOCTxPduProcessing/SecOCRbTxForwarder

Short Name	SecOCRbTxForwarder
Long Name	SecOC Tx Forwarder configuration
Description	SecOC configuration of TxPdus that will be forwarded to CryptoStack. If nothing is configured here, the default value from SecOCGeneral/SecOCRb-Forwarder will be used.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

10.3.1.48 SecOC/SecOCTxPduProcessing/SecOCRbTxForwarder/SecOCRbAutoConfigHsmKeySlotId

Short Name	SecOCRbAutoConfigHsmKeySlotId
Long Name	SecOC cryptographic HSM key slot
Description	This parameter explicitly maps the secure cryptokey element into a specified key slot inside the HSM secure flash. If left empty, then an available key slot automatically assigned to it. The name (e.g. DYNAMIC_CFG_CRYPT_AES_KEY_BASE_0) of the key slot must be provided. This autoconfiguration parameter will only apply to key element which are stored in HSM.
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

10.3.1.49 SecOC/SecOCTxPduProcessing/SecOCRbTxForwarder/SecOCRbTxCryptAlgoFamily

Short Name	SecOCRbTxCryptAlgoFamily
Long Name	SecOC cryptographic algorithm family
Description	Selects the algorithm family used for generation and verification of MAC.*At least the following AlgoFam, Mode, 2ndFam combinations are support:*AES, CMAC, AES*POLY1305, NOT_SET, POLY1305 (Poly1305 algorithm hasn't got different modes)*SIPHASH, SIPHASH_2_4, SPIHASH
Multiplicity	0..1
Default Value	-

Enumeration Literals	AES, POLY1305, SIPHASH
Type	ECUC-ENUMERATION-PARAM-DEF

10.3.1.50 SecOC/SecOCTxPduProcessing/SecOCRbTxForwarder/SecOCRbTxCryptAlgoMode

Short Name	SecOCRbTxCryptAlgoMode
Long Name	SecOC cryptographic algorithm mode
Description	Selects the algorithm mode used for generation and verification of MAC.*At least the following AlgoFam, Mode, 2ndFam combinations are support:*AES, CMAC, AES*POLY1305, NOT_SET, POLY1305 (Poly1305 algorithm hasn't got different modes)*SIPHASH, SIPHASH_2_4, SPIHASH
Multiplicity	0..1
Default Value	-
Enumeration Literals	CMAC, NOT_SET, SIPHASH_2_4
Type	ECUC-ENUMERATION-PARAM-DEF

10.3.1.51 SecOC/SecOCTxPduProcessing/SecOCRbTxForwarder/SecOCRbTxCryptAlgoSecondFamily

Short Name	SecOCRbTxCryptAlgoSecondFamily
Long Name	SecOC cryptographic algorithm secondary family
Description	Selects the algorithm secondary family used for generation and verification of MAC.*At least the following AlgoFam, Mode, 2ndFam combinations are support-*AES, CMAC, AES*POLY1305, NOT_SET, POLY1305 (Poly1305 algorithm hasn't got different modes)*SIPHASH, SIPHASH_2_4, SPIHASH
Multiplicity	0..1
Default Value	-
Enumeration Literals	AES, POLY1305, SIPHASH
Type	ECUC-ENUMERATION-PARAM-DEF

10.3.1.52 SecOC/SecOCTxPduProcessing/SecOCRbTxForwarder/SecOCRbTxUseBulkProcessing

Short Name	SecOCRbTxUseBulkProcessing
Long Name	SecOC use Bulk Processing

Description	This parameter indicates if the MAC generation for Tx PDUs should be processed in Bulk mode or in single shot mode. Possible values: 1. TRUE: the MAC job will be processed in Bulk mode, 2. FALSE: the MAC job will be processed in single shot mode. Note: If SecOCRbTxUseBulkProcessing is not configured, then it will use the configured value of SecOCGeneral/SecOCRbDefaultUseBulkProcessing as default. Bulk mode is available only for PDUs with cryptographic interface set to HSM and cryptographic mode set to asynchronous.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.53 SecOC/SecOCTxPduProcessing/SecOCRbTxPduCryptIf

Short Name	SecOCRbTxPduCryptIf
Long Name	SecOC cryptographic interface
Description	Selects the default cryptographic interface. It is set if the mac calculation is done via the cryptographic library (value Csm_rba_CryptoCCL), via hardware security module (value Csm_rba_CryptoHSM) or an external crypto driver (value Csm_Crypto_External). If value 'None' is selected no cryptographic interface is called (only if Det is enabled). Nevertheless the PDUs are forwarded to the PduR for further routing. This pdu specific configuration overwrites the global setting in container SecOCGeneral. If the value 'Csm_Crypto_External' is selected here or in any PDU, SecOC forwarding to other components (e.g. Csm, Crylf, Crypto) is disabled.
Multiplicity	0..1
Default Value	-
Enumeration Literals	Csm_Crypto_External, Csm_rba_CryptoCCL, Csm_rba_CryptoHSM, None
Type	ECUC-ENUMERATION-PARAM-DEF

10.3.1.54 SecOC/SecOCTxPduProcessing/SecOCRbTxUseSingleBuffer

Short Name	SecOCRbTxUseSingleBuffer
Long Name	SecOC Tx use single buffer

Description	If single buffer is enabled, inputbuffer and outputbuffer are disabled and only the working buffer is used. If this parameter ist set to TRUE, single buffer solution is enabled for the tx pdu.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

10.4 Limitations

10.4.1 Asymmetric Key Authentication

Asymmetric key authentication operations using signatures are not currently supported.

10.4.2 PduR Interface

The interfaces to the PDU Router (PduR) are implemented in the private sub module SecOC_Priv_PduRIF. However, The AUTOSAR-specified Gateway functionality is not currently supported.

10.4.3 Check PDU Processing

- An I-PDU is processed either as an Authentic or Secured I-PDU. The same I-PDU cannot be referenced as an Authentic I-PDU in one location and as a Secured I-PDU in another part of the configuration.
- An I-PDU can only be processed once (either as an Authentic or Secured I-PDU, and only once) on reception.
- An I-PDU can only be processed once (either as an Authentic or Secured I-PDU, and only once) on transmission (no multi-cast).

10.4.4 Check PDU Content Length

- For each I-PDU, the freshness value must be greater than or equal to the truncated freshness value.
- A Secured I-PDU must be greater than or equal to the sum of Authentic I-PDU plus the truncated freshness value plus the authenticator.
- The authentic data freshness length must be less than or equal to the authentic PDU length.
- The start bit position of the authentic data freshness value must be within the boundary of authentic PDU data.

10.4.5 Freshness Value Management

Freshness value management is processed by an external freshness module and is not within the scope of the SecOC module. However, an interface is provided in the private sub module SecOC_Priv_FreshIF. This interface supports freshness values based on timestamp only, freshness counters are not supported.

The configuration of the complete freshness value length can be done in bits. If it is non-byte aligned, it is assumed that the freshness manager sets the unused bits to a padding value (probably 0x00). These padding bits will be used for mac generation.

10.4.6 Overflow strategy QUEUE for Rx PDUs

In previous versions older than RTA-CAR 7.0.1, the overflow strategy QUEUE was handled as 'LastIsBest' means a received PDU was replaced by a new received one until the next cycle of the main function. For this, the queue size was restricted to 1. Now the queues and PDUs will be queued as long as there are free queue entries.

The 'LastIsBest' can now be configured by selecting the overflow strategy REPLACE. The first PDU is processed and all other PDUs will overwrite a second buffer. The overflow strategy REJECT can be chosen for 'FirstIsBest' behaviour. The first PDU is processed, the second PDU is saved in a second buffer and all other PDUs will be discarded.

10.5 Exclusions

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- SecOC/SecOCGeneral/SecOCMaxAlignScalarType
- SecOC/SecOCGeneral/SecOCSecurityEventRefs/SECOC_SEV_FRESHNESS_NOT_AVAILABLE
- SecOC/SecOCGeneral/SecOCSecurityEventRefs/SECOC_SEV_MAC_VERIFICATION_FAILED
- SecOC/SecOCRxPduProcessing/SecOCSameBufferPduRef

10.6 Deviations

10.6.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

10.6.1.1 SecOC

[RTA-BSW] Description	Configuration of the SecOC (Secure Onboard Communication) module.
-----------------------	---

[AUTOSAR] Description	Configuration of the SecOC (SecureOnboardCommunication) module.
-----------------------	---

10.6.1.2 SecOC/SecOCGeneral/SecOCDefaultAuthenticationInformation-Pattern

[REMOVED] Introduction	If the configuration parameter is not present, SecOC module shall remove the Authentic I-PDU from its internal buffer and cancel the transmission request. If the configuration parameter is present, SecOC will use this value for each byte of Freshness Value and Authenticator when building the Authentication Information, and will not cancel the transmission request.
------------------------	--

[RTA-BSW] Description: The parameter describes the behaviour of SecOC when authentication build counter has reached the configuration value SecOCAuthenticationBuildAttempts, or the query of the freshness function returns E_NOT_OK or the calculation of the authenticator has returned a non-recoverable error such as returning E_NOT_OK or KEY_FAILURE. If the configuration parameter is not present, SecOC module shall remove the Authentic I-PDU from its internal buffer and cancel the transmission request. If the configuration parameter is present, SecOC will use this value for each byte of Freshness Value and Authenticator when building the Authentication Information, and will not cancel the transmission request.

[AUTOSAR] Description: The parameter describes the behaviour of SecOC when authentication build counter has reached the configuration value SecOCAuthenticationBuildAttempts, or the query of the freshness function returns E_NOT_OK or the calculation of the authenticator has returned a non-recoverable error such as returning E_NOT_OK or KEY_FAILURE.

10.6.1.3 SecOC/SecOCGeneral/SecOCDevErrorDetect

[RTA-BSW] Description	Switches the default error tracer (Det) detection and notification ON or OFF.
[AUTOSAR] Description	Switches the development error detection and notification on or off.
[REMOVED] Introduction	* true: detection and notification is enabled. * false: detection and notification is disabled.

10.6.1.4 SecOC/SecOCGeneral/SecOCEnableForcedPassOverride

[RTA-BSW] Description	When this configuration option is set to TRUE then the functionality inside the function SecOC_VerifyStatusOverride to forcibly override the VerifyStatus to "Pass" is enabled.
[AUTOSAR] Description	When this configuration option is set to TRUE then the functionality inside the function SecOC_VerifyStatusOverride to send I-PDUs to upper layer independent of the verification result is enabled.

10.6.1.5 SecOC/SecOCGeneral/SecOCEnableSecurityEventReporting

[RTA-BSW] Description	Switches the reporting of security events to the IdsM ON or OFF. If the switch is set to true the reporting is enabled and you have to configure the security events in sub-container SecOCSecurityEventRefs. If the switch is set to false the reporting is disabled.
[AUTOSAR] Description	Switches the reporting of security events to the IdsM: Tags: atp.Status=draft
[REMOVED] Introduction	- true: reporting is enabled. - false: reporting is disabled.

10.6.1.6 SecOC/SecOCGeneral/SecOCIgnoreVerificationResult

[RTA-BSW] Description	Switches the handling of verification result ON or OFF. If the switch is set to true the verification is performed but the verification result is ignored. So if the verification fails the PDU is forwarded to the PduR nevertheless and an error event is set in the Logging Module.
[AUTOSAR] Description	The result of the authentication process (e.g. MAC Verify) is ignored after the first try and the SecOC proceeds like the result was a success. The calculation of the authenticator is still done, only its result will be ignored.
[REMOVED] Intro- duction	* true: enabled (verification result is ignored). * false: disabled (verification result is NOT ignored).

10.6.1.7 SecOC/SecOCGeneral/SecOCOverrideStatusWithDataId

[RTA-BSW] Description	This option defines if the parameter "ValueId" of the function SecOC_VerifyStatusOverride() accepts the freshness value (as a collection of one or more Secured I-PDUs to freshness) or the dataId for individual Secured I-PDUs. True: Function SecOC_VerifyStatusOverride accepts SecOCDataId. False: Function SecOC_VerifyStatusOverride accepts SecOCFreshnessValueId.
[AUTOSAR] Description	This option defines if the parameter "ValueId" of the function SecOC_VerifyStatusOverride() accepts the freshness value (as a collection of one or more Secured I-PDUs to freshness) or the dataId for individual Secured I-PDUs.
[REMOVED] Introduction	* true: Function SecOC_VerifyStatusOverride accepts SecOCDataId as parameter. * false: Function SecOC_VerifyStatusOverride accepts SecOCFreshnessValueId as parameter.

10.6.1.8 SecOC/SecOCGeneral/SecOCPropagateOnlyFinalVerificationStatus

[RTA-BSW] Description	Specify if the verification status shall be reported only after the final determination of the verification status (TRUE) or on every verification attempt (FALSE). If the switch is set to true verification status shall be reported only after the final determination of the verification status and if the switch is set to false verification status shall be reported on every verification attempt.
[AUTOSAR] Description	This parameter is used to specify if the verification status shall be reported only after the final determination of the verification status (TRUE) or on every verification attempt (FALSE).

10.6.1.9 SecOC/SecOCGeneral/SecOCQueryFreshnessValue

[RTA-BSW] Description	Specifies if the freshness value shall be determined through a C-function (CD) or a software component (SW-C).
[AUTOSAR] Description	This parameter specifies if the freshness value shall be determined through a C-function (CD) or a software component (SW-C).

10.6.1.10 SecOC/SecOCGeneral/SecOCSecurityEventRefs

[RTA-BSW] Description	Container for the references to IdsMEvent elements representing the security events that the SecOC module shall report to IdsM. To enable the reporting to IdsM set SecOCEnableSecurityEventReporting in container SecOCGeneral to true.
[AUTOSAR] Description	Container for the references to IdsMEvent elements representing the security events that the SecOC module shall report to the IdsM in case the corresponding security related event occurs (and if SecOCEnableSecurityEventReporting is set to "true"). The standardized security events in this container can be extended by vendor-specific security events. Tags: atp.Status=draft

10.6.1.11 SecOC/SecOCGeneral/SecOCVerificationStatusCallout

[RTA-BSW] Description	Name of the verification status callout
[AUTOSAR] Description	Entry address of the customer specific call out routine which shall be invoked in case of a verification attempt.

10.6.1.12 SecOC/SecOCGeneral/SecOCVersionInfoApi

[RTA-BSW] Description	Switches the availability of the SecOC_GetVersionInfo ON or OFF.
-----------------------	--

[AUTOSAR] Description	If true the SecOC_GetVersionInfo API is available.
[AUTOSAR] Default Value	false
[RTA-BSW] Default Value	true

10.6.1.13 SecOC/SecOCRxPduProcessing/SecOCAuthDataFreshnessLen

[RTA-BSW] Description	This parameter specifies the length of the external authentic PDU data in bits.
[AUTOSAR] Description	The length of the external authentic PDU data in bits (uint16).
[RTA-BSW] Maximum Value	64
[AUTOSAR] Maximum Value	65535
[ADDED] Default Value	0

10.6.1.14 SecOC/SecOCRxPduProcessing/SecOCAuthDataFreshnessStart-Position

[RTA-BSW] Description	This parameter determines the start position in bits (uint16) of the Authentic PDU that shall be passed on to the Freshness SWC. The bit position starts counting from the MSB of the first byte of the PDU.
[AUTOSAR] Description	This value determines the start position in bits (uint16) of the Authentic PDU that shall be passed on to the Freshness SWC. The bit counting is done according to TPS_SYST_01068 and the bit ordering is done according to TPS_SYST_01069.
[ADDED] Default Value	0

10.6.1.15 SecOC/SecOCRxPduProcessing/SecOCAuthInfoTruncLength

[ADDED] Default Value	4
-----------------------	---

10.6.1.16 SecOC/SecOCRxPduProcessing/SecOCAuthenticationBuildAttempts

[ADDED] Default Value	0
-----------------------	---

10.6.1.17 SecOC/SecOCRxPduProcessing/SecOCAuthenticationVerifyAttempts

[RTA-BSW] Description	This parameter specifies the number of authentication verify attempts that are to be carried out when the verification of the authentication information failed for a given Secured I_PDU. If zero is set then only one authentication verification attempt is done.
-----------------------	--

[AUTOSAR] Description	This parameter specifies the number of authentication verify attempts that are to be carried out when the verification of the authentication information failed for a given Secured I-PDU. If zero is set, then only one authentication verification attempt is done.
--------------------------	---

10.6.1.18 SecOC/SecOCRxPduProcessing/SecOCClientServerVerification-StatusPropagationMode

[RTA-BSW] Description	This parameter is used to describe the propagation of the status of each verification attempt from SecOC module through the client/server interface to SWCs.
[AUTOSAR] Description	This parameter is used to determine the propagation of the verification status through the client/server interface to an SW-C.

10.6.1.19 SecOC/SecOCRxPduProcessing/SecOCDataId

[RTA-BSW] Maximum Value	1099511627775
[AUTOSAR] Maximum Value	65535

10.6.1.20 SecOC/SecOCRxPduProcessing/SecOCDynamicRuntimeLength-Handling

[RTA-BSW] Description	This parameter defines whether the length information for handling the received Rx Pdu is taken from the configuration or from the actually provided length information during runtime. True: SecuredIPdu length information is taken from the actually provided length information during runtime. False: SecuredIPdu length information is taken from parameter PduLength of the Pdu.
[AUTOSAR] Description	Defines whether the length information for handling this received Pdu is taken from the configuration or from the actually provided length information during runtime.
[REMOVED] Introduction	true: SecuredIPdu length information is taken from the actually provided length information during runtime. false: SecuredIPdu length information is taken from parameter PduLength of the Pdu.

10.6.1.21 SecOC/SecOCRxPduProcessing/SecOCFreshnessValueId

[RTA-BSW] Description	This parameter specifies the local freshness identifier used to generate the freshness value used to generate or verify a MAC.
[AUTOSAR] Description	This parameter defines the Id of the Freshness Value.
[REMOVED] Introduction	The Freshness Value might be a normal counter or a time value.
[RTA-BSW] Maximum Value	1099511627775

[AUTOSAR] Maximum Value	65535
-------------------------	-------

10.6.1.22 SecOC/SecOCRxPduProcessing/SecOCFreshnessValueLength

[RTA-BSW] Description	This parameter defines the length in bits of the Freshness Value.
[AUTOSAR] Description	This parameter defines the complete length in bits of the Freshness Value. As long as the key doesn't change the counter shall not overflow. The length of the counter shall be determined based on the expected life time of the corresponding key and frequency of usage of the counter.
[ADDED] Default Value	0

10.6.1.23 SecOC/SecOCRxPduProcessing/SecOCFreshnessValueTruncLength

[RTA-BSW] Description	This parameter defines the length in bits of the Freshness Value to be included in the payload of the Secured I-PDU. This length is specific to the least significant bits of the complete Freshness Counter. If the parameter is 0 no Freshness Value is included in the Secured I-PDU.
[AUTOSAR] Description	This parameter defines the length in bits of the Freshness Value to be included in the payload of the Secured I-PDU. This length is specific to the least significant bits of the complete Freshness Counter. If the parameter is 0 no Freshness Value is included in the Secured I-PDU.
[ADDED] Default Value	0

10.6.1.24 SecOC/SecOCRxPduProcessing/SecOCReceptionOverflowStrategy

[RTA-BSW] Description	This parameter defines the overflow strategy for receiving PDUs. QUEUE REJECT REPLACE
[AUTOSAR] Description	This parameter defines the overflow strategy for receiving PDUs

10.6.1.25 SecOC/SecOCRxPduProcessing/SecOCReceptionQueueSize

[RTA-BSW] Maximum Value	254
[AUTOSAR] Maximum Value	65535

[ADDED] Default Value	2
-----------------------	---

10.6.1.26 SecOC/SecOCRxPduProcessing/SecOCRxAuthServiceConfigRef

[REMOVED] Introduction	If PDUs with a dynamic length are used (e.g. CanTP or Dynamic Length PDUs) a MAC algorithm has to be chosen, that is not vulnerable to length extension attack (e.g. CMAC/HMAC).
[RTA-BSW] Multiplicity	0..*
[AUTOSAR] Multiplicity	1..1

10.6.1.27 SecOC/SecOCRxPduProcessing/SecOCRxAuthenticPduLayer/SecOCPduType

[RTA-BSW] Description	This parameter defines the API Type to use for communication with PduR. SECOC_IFPDU SECOC_TPPDU
[AUTOSAR] Description	This parameter defines API Type to use for communication with PduR.
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1
[ADDED] Default Value	SECOC_IFPDU

10.6.1.28 SecOC/SecOCRxPduProcessing/SecOCRxAuthenticPduLayer/SecOCRxAuthenticLayerPduId

[RTA-BSW] Description	SecOC identifier of authentic PDU, used by PduR for SecOC_TpCancelReceive. Do not set it manually, it will be set by SecOC forwarder.
[AUTOSAR] Description	PDU identifier assigned by SecOC module. Used by PduR for SecOC_TpCancelReceive.

10.6.1.29 SecOC/SecOCRxPduProcessing/SecOCRxPduMainFunctionRef

[REMOVED] Introduction	Mandatory, if multiple main functions are defined.
------------------------	--

10.6.1.30 SecOC/SecOCRxPduProcessing/SecOCRxPduSecuredArea/SecOCSecuredRxPduLength

[RTA-BSW] Description	This parameter defines the length (in bytes) of the area within the Pdu which is secured.
-----------------------	---

[AUTOSAR] Description	This parameter defines the length (in bytes) of the area within the Pdu which is secured
-----------------------	--

10.6.1.31 SecOC/SecOCRxPduProcessing/SecOCRxPduSecuredArea/SecOCSecuredRxPduOffset

[RTA-BSW] Description	This parameter defines the start position (offset in bytes) of the area within the Pdu which is secured.
[AUTOSAR] Description	This parameter defines the start position (offset in bytes) of the area within the Pdu which is secured

10.6.1.32 SecOC/SecOCRxPduProcessing/SecOCRxSecuredPduLayer

[RTA-BSW] Description	This is a choice container
[AUTOSAR] Description	This container specifies the Pdu that is received by the SecOC module from the PduR. For this Pdu the Mac verification is provided.

10.6.1.33 SecOC/SecOCRxPduProcessing/SecOCRxSecuredPduLayer/SecOCRxSecuredPdu/SecOCAuthPduHeaderLength

[RTA-BSW] Description	This parameter indicates the length in bytes of the Secured I-PDU Header in the Secured I-PDU. The length of zero means there is no header in the secured PDU.
[AUTOSAR] Description	This parameter indicates the length (in bytes) of the Secured I-PDU Header in the Secured I-PDU. The length of zero means there's no header in the PDU.

10.6.1.34 SecOC/SecOCRxPduProcessing/SecOCRxSecuredPduLayer/SecOCRxSecuredPdu/SecOCRxSecuredLayerPduld

[RTA-BSW] Description	PDU identifier assigned by SecOC module, Used by PduR for SecOC_PduRRxIndication. Do not set it manually, it will be set by SecOC forwarder.
[AUTOSAR] Description	PDU identifier assigned by SecOC module. Used by PduR for <nowiki>SecOC_[IfITp]RxIndication</nowiki>.

10.6.1.35 SecOC/SecOCRxPduProcessing/SecOCRxSecuredPduLayer/SecOCRxSecuredPdu/SecOCSecuredRxPduVerification

[RTA-BSW] Description	This parameter defines whether the signature authentication or MAC verification shall be performed on this Secured I-PDU. If set to false, the SecOC module extracts the Authentic PDU from the Secured PDU without verification. Default value (false) will disable signature authentication or MAC verification on this Secured I-PDU.
-----------------------	--

[AUTOSAR] Description	This parameter defines whether the signature authentication or MAC verification shall be performed on this Secured I-PDU. If set to false, the SecOC module extracts the Authentic I-PDU from the Secured I-PDU without verification.
--------------------------	---

10.6.1.36 SecOC/SecOCRxPduProcessing/SecOCRxSecuredPduLayer/SecOCRxSecuredPduCollection

[REMOVED] Introduction	SecOCRxAuthenticPdu contains the original Authentic I-PDU, i.e. the secured data, and the SecOCRxCryptographicPdu contains the Authenticator, i.e. the actual Authentication Information.
---------------------------	---

10.6.1.37 SecOC/SecOCRxPduProcessing/SecOCRxSecuredPduLayer/SecOCRxSecuredPduCollection/SecOCRxAuthenticPdu

[RTA-BSW] Description	This container specifies the Pdu that is transmitted by the SecOC module to the PduR after the Mac was verified.
[AUTOSAR] Description	This container specifies the PDU (that is received by the SecOC module from the PduR) which contains the Secured I-PDU Header and the Authentic I-PDU.

10.6.1.38 SecOC/SecOCRxPduProcessing/SecOCRxSecuredPduLayer/SecOCRxSecuredPduCollection/SecOCRxAuthenticPdu/SecOCAuthPduHeaderLength

[RTA-BSW] Description	This parameter indicates the length in bytes of the Secured I-PDU Header in the Authentic I-PDU. The length of zero means there is no header in the Authentic PDU.
[AUTOSAR] Description	This parameter indicates the length (in bytes) of the Secured I-PDU Header in the Secured I-PDU. The length of zero means there's no header in the PDU.

10.6.1.39 SecOC/SecOCRxPduProcessing/SecOCRxSecuredPduLayer/SecOCRxSecuredPduCollection/SecOCRxAuthenticPdu/SecOCRxAuthenticPduld

[RTA-BSW] Description	PDU identifier of the Authentic I-PDU assigned by SecOC module. Used by PduR for SecOC_PduRRxIndication. Do not set it manually, it will be set by SecOC forwarder.
[AUTOSAR] Description	PDU identifier of the Authentic I-PDU assigned by SecOC module. Used by PduR for SecOC_IfRxIndication.

10.6.1.40 SecOC/SecOCRxPduProcessing/SecOCRxSecuredPduLayer/SecOCRxSecuredPduCollection/SecOCRxCryptographicPdu/SecOCRxCryptographicPduld

[RTA-BSW] Description	PDU identifier of the Cryptographic I-PDU assigned by SecOC module. Used by PduR for SecOC_PduRRxIndication. Do not set it manually, it will be set by SecOC forwarder.
[AUTOSAR] Description	PDU identifier of the Cryptographic I-PDU assigned by SecOC module. Used by PduR for SecOC_IfRxIndication.

10.6.1.41 SecOC/SecOCRxPduProcessing/SecOCRxSecuredPduLayer/SecOCRxSecuredPduCollection/SecOCSecuredRxPduVerification

[RTA-BSW] Description	This parameter defines whether the signature authentication or MAC verification shall be performed on this Secured I-PDU. If set to false, the SecOC module extracts the Authentic PDU from the Secured PDU without verification. . Default value (false) will disable signature authentication or MAC verification on this Secured I-PDU.
[AUTOSAR] Description	This parameter defines whether the signature authentication or MAC verification shall be performed on this Secured I-PDU. If set to false, the SecOC module extracts the Authentic I-PDU from the Secured I-PDU without verification.

10.6.1.42 SecOC/SecOCRxPduProcessing/SecOCRxSecuredPduLayer/SecOCRxSecuredPduCollection/SecOCUseMessageLink/SecOCMessageLinkLen

[RTA-BSW] Description	Length of the message linker inside the Authentic PDU in bits.
[AUTOSAR] Description	Length of the Message Linker inside the Authentic I-PDU in bits.

10.6.1.43 SecOC/SecOCRxPduProcessing/SecOCRxSecuredPduLayer/SecOCRxSecuredPduCollection/SecOCUseMessageLink/SecOCMessageLinkPos

[RTA-BSW] Description	The position of message linker inside the Authentic PDU in bits.
[AUTOSAR] Description	The position of the Message Linker inside the Authentic I-PDU in bits.
[REMOVED] Introduction	The bit counting is done according to 01068 and the bit ordering is done according to TPS_SYST_01069.

10.6.1.44 SecOC/SecOCRxPduProcessing/SecOCUseAuthDataFreshness

[RTA-BSW] Description	This parameter indicates if a part of the Authentic-PDU shall be passed on to the SWC that verifies and generates the Freshness. If it is set to TRUE, the values SecOCAuthDataFreshnessStartPosition and SecOCAuthDataFreshnessLen must be set to specify the bit position and length within the Authentic-PDU.
[AUTOSAR] Description	A Boolean value that indicates if a part of the Authentic-PDU shall be passed on to the SWC that verifies and generates the Freshness. If it is set to TRUE, the values SecOCAuthDataFreshnessStartPosition and SecOCAuthDataFreshnessLen must be set to specify the bit position and length within the Authentic-PDU.

10.6.1.45 SecOC/SecOCRxPduProcessing/SecOCVerificationStatusPropagationMode

[RTA-BSW] Description	This parameter is used to describe the propagation of the status of each verification attempt from SecOC module to SWCs.
[AUTOSAR] Description	This parameter is used to describe the propagation of the status of each verification attempt from the SecOC module to SWCs.

10.6.1.46 SecOC/SecOCSameBufferPduCollection

[RTA-BSW] Description	SecOCBuffer configuration that may be used by a collection of Pdus. It is not supported for Rx PDUs.
[AUTOSAR] Description	SecOCBuffer configuration that may be used by a collection of Pdus.

10.6.1.47 SecOC/SecOCSameBufferPduCollection/SecOCBufferLength

[RTA-BSW] Description	This parameter defines the length of the buffer for Pdu collection in bytes.
[AUTOSAR] Description	This parameter defines the Buffer in bytes that is used by the SecOC module.
[ADDED] Default Value	0

10.6.1.48 SecOC/SecOCTxPduProcessing/SecOCAuthInfoTruncLength

[ADDED] Default Value	4
-----------------------	---

10.6.1.49 SecOC/SecOCTxPduProcessing/SecOCAuthenticationBuildAttempts

[ADDED] Default Value	0
-----------------------	---

10.6.1.50 SecOC/SecOCTxPduProcessing/SecOCDataId

[RTA-BSW] Maximum Value	1099511627775
[AUTOSAR] Maximum Value	65535

10.6.1.51 SecOC/SecOCTxPduProcessing/SecOCFreshnessValueId

[RTA-BSW] Description	This parameter specifies the local freshness identifier used to generate the freshness value used to generate or verify a MAC.
[AUTOSAR] Description	This parameter defines the Id of the Freshness Value.
[REMOVED] Introduction	The Freshness Value might be a normal counter or a time value.
[RTA-BSW] Maximum Value	1099511627775
[AUTOSAR] Maximum Value	65535

10.6.1.52 SecOC/SecOCTxPduProcessing/SecOCFreshnessValueLength

[RTA-BSW] Description	This parameter defines the length in bits of the Freshness Value. If it is non-byte aligned, padding bits from freshness manager are used for mac generation.
[AUTOSAR] Description	This parameter defines the complete length in bits of the Freshness Value. As long as the key doesn't change the counter shall not overflow. The length of the counter shall be determined based on the expected life time of the corresponding key and frequency of usage of the counter.
[ADDED] Default Value	0

10.6.1.53 SecOC/SecOCTxPduProcessing/SecOCFreshnessValueTruncLength

[RTA-BSW] Description	This parameter defines the length in bits of the Freshness Value to be included in the payload of the Secured I-PDU. This length is specific to the least significant bits of the complete Freshness Counter. If the parameter is 0 no Fresh-ness Value is included in the Secured I-PDU.
[AUTOSAR] Description	This parameter defines the length in bits of the Freshness Value to be included in the payload of the Secured I-PDU. This length is specific to the least significant bits of the complete Freshness Counter. If the parameter is 0 no Freshness Value is included in the Secured I-PDU.
[ADDED] Default Value	0

10.6.1.54 SecOC/SecOCTxPduProcessing/SecOCProvideTxTruncatedFreshnessValue

[RTA-BSW] Description	This parameter specifies if the Tx query freshness function provides the truncated freshness info instead of generating this by SecOC. In this case, SecOC shall add this data to the Authentic PDU instead of truncating the freshness value.
[AUTOSAR] Description	This parameter specifies if the Tx query freshness function provides the truncated freshness info instead of generating this by SecOC. In this case, SecOC shall add this data to the Authentic PDU instead of truncating the freshness value.

10.6.1.55 SecOC/SecOCTxPduProcessing/SecOCReAuthenticateAfterTriggerTransmit

[RTA-BSW] Description	This parameter specifies if the authentication information of the Secured PDU is updated after the successful transmission of a triggered transmission was confirmed. TRUE if the authentication information shall be updated after triggered transmission. FALSE if the authentication information shall not be updated after triggered transmission. Note: This parameter should only be set to FALSE if the upper layer SecOC_IfTransmit have the same or higher frequency than the SecOC_TriggerTransmit calls.
[AUTOSAR] Description	This parameter specifies if the authentication information of the Secured PDU is updated after
[REMOVED] Introduction	the successful transmission of a triggered transmission was confirmed. TRUE if the authentication information shall be updated after triggered transmission. FALSE if the authentication information shall not be updated after triggered transmission. Note: This parameter should only be set to FALSE if the upper layer SecOC_IfTransmit have the same or a higher frequency than the SecOC_TriggerTransmit calls.

10.6.1.56 SecOC/SecOCTxPduProcessing/SecOCTxAuthServiceConfigRef

[REMOVED] Introduction	If PDUs with a dynamic length are used (e.g. CanTP or Dynamic Length PDUs) a MAC algorithm has to be chosen, that is not vulnerable to length extension attack (e.g. CMAC/HMAC).
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

10.6.1.57 SecOC/SecOCTxPduProcessing/SecOCTxAutenticPduLayer/SecOCPduType

[RTA-BSW] Description	This parameter defines the API Type to use for communication with PduR. SECOC_IFPDU SECOC_TPPDU
[AUTOSAR] Description	This parameter defines API Type to use for communication with PduR.
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1
[ADDED] Default Value	SECOC_IFPDU

10.6.1.58 SecOC/SecOCTxPduProcessing/SecOCTxAutenticPduLayer/SecOCTxAutenticLayerPduId

[RTA-BSW] Description	PDU identifier assigned by SecOC module. Used by PduR for SecOC_PduRTransmit. Do not set it manually, it will be set by SecOC forwarder.
[AUTOSAR] Description	PDU identifier assigned by SecOC module. Used by PduR for <nowiki>SecOC_[IfITp]Transmit</nowiki>.

10.6.1.59 SecOC/SecOCTxPduProcessing/SecOCTxPduMainFunctionRef

[REMOVED] Introduction	Mandatory, if multiple main functions are defined.
------------------------	--

10.6.1.60 SecOC/SecOCTxPduProcessing/SecOCTxPduSecuredArea/SecOCSecuredTxPduLength

[RTA-BSW] Description	This parameter defines the length (in bytes) of the area within the Pdu which shall be secured.
[AUTOSAR] Description	This parameter defines the length (in bytes) of the area within the Pdu which shall be secured

10.6.1.61 SecOC/SecOCTxPduProcessing/SecOCTxPduSecuredArea/SecOCSecuredTxPduOffset

[RTA-BSW] Description	This parameter defines the start position (offset in bytes) of the area within the Pdu which shall be secured.
[AUTOSAR] Description	This parameter defines the start position (offset in bytes) of the area within the Pdu which shall be secured

10.6.1.62 SecOC/SecOCTxPduProcessing/SecOCTxPduUnusedAreasDe-

faultSecOCTxPduUnusedAreasDefault

[RTA-BSW] Description	The AUTOSAR SecOC module fills not used areas of a transmitted Secured Pdu or a transmitted Cryptographic Pdu with this byte pattern. This attribute is mandatory to avoid undefined behavior. The default pattern are only used if SecOCRbDynamicRuntimeLengthHandling is set to FALSE.
[AUTOSAR] Description	The AUTOSAR SecOC module fills not used areas of a transmitted Secured Pdu or a transmitted Cryptographic Pdu with this byte pattern. This attribute is mandatory to avoid undefined behavior.

10.6.1.63 SecOC/SecOCTxPduProcessing/SecOCTxSecuredPduLayer

[RTA-BSW] Description	This is a choice container
[AUTOSAR] Description	This container specifies the Pdu that is transmitted by the SecOC module to the PduR after the Mac was generated.

10.6.1.64 SecOC/SecOCTxPduProcessing/SecOCTxSecuredPduLayer/SecOCTxSecuredPdu/SecOCAuthPduHeaderLength

[RTA-BSW] Description	This parameter indicates the length in bytes of the Secured I-PDU Header in the Secured I-PDU. The length of zero means there is no header in the secured PDU.
[AUTOSAR] Description	This parameter indicates the length (in bytes) of the Secured I-PDU Header in the Secured I-PDU. The length of zero means there's no header in the PDU.

10.6.1.65 SecOC/SecOCTxPduProcessing/SecOCTxSecuredPduLayer/SecOCTxSecuredPdu/SecOCTxSecuredLayerPduld

[RTA-BSW] Description	PDU identifier assigned by SecOC module. Used by PduR for confirmation (SecOC_PduRTxConfirmation) and for TriggerTransmit. Do not set it manually, it will be set by SecOC forwarder.
[AUTOSAR] Description	PDU identifier assigned by SecOC module.
[REMOVED] Introduction	Used by PduR for confirmation (<code><nowiki>SecOC_[IfITp]TxConfirmation</nowiki></code>) and for TriggerTransmit.

10.6.1.66 SecOC/SecOCTxPduProcessing/SecOCTxSecuredPduLayer/SecOCTxSecuredPduCollection

[RTA-BSW] Description	This container specifies the Pdu that is transmitted by the SecOC module to the PduR after the Mac was generated. Two separate Pdus are transmitted to the PduR: Authentic I-PDU and Cryptographic I-PDU.
-----------------------	---

[AUTOSAR] Description	This container specifies the Pdu that is transmitted by the SecOC module to the PduR after the Mac was generated. Two separate Pdus are transmitted to the PduR:
[REMOVED] Introduction	Authentic I-PDU and Cryptographic I-PDU.

10.6.1.67 SecOC/SecOCTxPduProcessing/SecOCTxSecuredPduLayer/SecOCTxSecuredPduCollection/SecOCTxAuthenticPdu/SecOCAuthPduHeaderLength

[RTA-BSW] Description	This parameter indicates the length in bytes of the Secured I-PDU Header in the Authentic I-PDU. The length of zero means there is no header in the secured PDU.
[AUTOSAR] Description	This parameter indicates the length (in bytes) of the Secured I-PDU Header in the Secured I-PDU. The length of zero means there's no header in the PDU.

10.6.1.68 SecOC/SecOCTxPduProcessing/SecOCTxSecuredPduLayer/SecOCTxSecuredPduCollection/SecOCTxAuthenticPdu/SecOCTxAuthenticPduld

[RTA-BSW] Description	PDU identifier of the Authentic I-PDU assigned by SecOC module. Used by PduR for confirmation (SecOC_PduRTxConfirmation) and for TriggerTransmit. Do not set it manually, it will be set by SecOC forwarder.
[AUTOSAR] Description	PDU identifier of the Authentic I-PDU assigned by SecOC module. Used by PduR for confirmation (SecOC_IftxConfirmation) and for TriggerTransmit.

10.6.1.69 SecOC/SecOCTxPduProcessing/SecOCTxSecuredPduLayer/SecOCTxSecuredPduCollection/SecOCTxCryptographicPdu/SecOCTxCryptographicPduld

[RTA-BSW] Description	PDU identifier of the Cryptographic I-PDU assigned by SecOC module. Used by PduR for confirmation (SecOC_PduRTxConfirmation) and for TriggerTransmit. Do not set it manually, it will be set by SecOC forwarder.
[AUTOSAR] Description	PDU identifier of the Cryptographic I-PDU assigned by SecOC module. Used by PduR for confirmation (SecOC_IftxConfirmation) and for TriggerTransmit.

10.6.1.70 SecOC/SecOCTxPduProcessing/SecOCTxSecuredPduLayer/SecOCTxSecuredPduCollection/SecOCUseMessageLink/SecOCMessageLinkLen

[RTA-BSW] Description	Length of the message linker inside the Authentic PDU in bits.
[AUTOSAR] Description	Length of the Message Linker inside the Authentic I-PDU in bits.

10.6.1.71 SecOC/SecOCTxPduProcessing/SecOCTxSecuredPduLayer/SecOCTxSecuredPduCollection/SecOCUseMessageLink/SecOCMessageLinkPos

[RTA-BSW] Description	The position of message linker inside the Authentic PDU in bits.
[AUTOSAR] Description	The position of the Message Linker inside the Authentic I-PDU in bits.
[REMOVED] Introduction	The bit counting is done according to 01068 and the bit ordering is done according to TPS_SYST_01069.

10.6.1.72 SecOC/SecOCTxPduProcessing/SecOCUseTxConfirmation

[RTA-BSW] Description	If this parameter is set to TRUE, the function SecOC_SPduTxConfirmation shall be called to indicate that the Secured I-PDU has been initiated for transmission.
[AUTOSAR] Description	A Boolean value that indicates if the function SecOC_SPduTxConfirmation shall be called for this PDU.

10.7 API Definition

List of supported API functions

Table 10.128.SecOC Supported API functions

SecOC_Init	Initialises the module.
SecOC_DeInit	De-initialises the module.
SecOC_GetVersionInfo	Returns version information about the module.
SecOC_MainFunctionRx	Scheduled function to perform the processing of SecOC's authentication and verification processing for the Rx path.
SecOC_MainFunctionTx	Scheduled function to perform the processing of SecOC's authentication and verification processing for the Tx path.
SecOC_VerifyStatusOverride	Overrides verification status for all Secured I-PDUs with the given freshness value identifier.
SecOC_RxIndication	Called from PduR to trigger the receipt of a Secured I-PDU.
SecOC_TpRxIndication	Called after an I-PDU has been received - the result indicates whether the transmission was successful.
SecOC_TxConfirmation	Called from PduR to confirm the transmit of the Secured I-PDU given by the PDU identifier.

SecOC_TpTxConfirmation	Called after an I-PDU has been transmitted on the network - the result indicates whether the transmission was successful.
SecOC_TriggerTransmit	Checks whether the available data fits into the buffer size reported by PduInfoPtr → S-duLength.
SecOC_CopyRxData	Called to acquire the transmit data of an I-PDU segment (N-PDU).
SecOC_CopyTxData	Provides the received data of an I-PDU segment (N-PDU) to the upper layer.
SecOC_SendDefaultAuthenticationInformation	Provides the ability to enable the sending of un-authenticated PDU to lower layer.
SecOC_StartOfReception	This function is called at the start of receiving an I-PDU segment (N-SDU).
SecOC_GetRxFreshness	Callout interface used by SecOC to obtain the current freshness value.
SecOC_GetRxFreshnessAuthData	Callout interface used by SecOC to obtain the current freshness value.
SecOC_GetTxFreshness	Callout interface used by SecOC to obtain the current freshness value.
SecOC_GetTxFreshnessTruncData	Callout interface used by SecOC to obtain the current freshness value.
SecOC_SPduTxConfirmation	Callout interface used by SecOC to indicate that the Secured I-PDU has been initiated for transmission.
SecOC_IfTransmit	Requests transmission of a PDU.
SecOc_TpTransmit	Requests transmission of a PDU.
SecOc_IfCancelTransmit	Requests cancellation of an ongoing transmission of a PDU in a lower layer communication module.
SecOc_TpCancelTransmit	Requests cancellation of an ongoing transmission of a PDU in a lower layer communication module.
SecOc_TpCancelReceive	Requests cancellation of an ongoing reception of a PDU in a lower layer transport protocol module.

For further information please refer to AUTOSAR_SWS_SecureOnboardCommunication.pdf

provides the ability to enable the sending of un-authenticated PDU to lower layer.

10.8 Configuration Advice



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

10.8.1 MainFunction Partitioning

RTA-SEC supports MainFunction partitioning of resources. For SecOc, this involves the generation of multiple SecOC_MainFunctionTx/SecOC_MainFunctionRx instances and mapping of Tx/Rx PDUs to these instances where they should be handled.

The generation of multiple SecOC_MainFunctionTx/SecOC_MainFunctionRx instances is based on configuration via SecOCMainFunctionTx/SecOCMainFunctionRx containers. The following parameters need to be configured for each Tx/Rx MainFunction container:

- **SecOCRbMainFunctionPeriodTx/SecOCRbMainFunctionPeriodRx**: The periodicity for this MainFunction instance.
- **SecOCMainFunctionTxPartitionRef/SecOCMainFunctionRxPartitionRef**: The EcucPartition (often interpreted as core) assignment for this MainFunction.

If no periodicity is given in the main function containers, then the default periodicities **SecOCRbMainFunctionPeriodTx/SecOCRbMainFunctionPeriodRx** from the container SecOCGeneral will be applied. The default value of **SecOCRbMainFunctionPeriodTx/SecOCRbMainFunctionPeriodRx** is 10ms, this is also used if the default parameters are not configured.

The default/legacy main functions (SecOC_MainFunctionTx/SecOC_MainFunctionRx) are still generated if there are Tx/Rx PDUs which are not explicitly linked to a Tx/Rx MainFunction container, otherwise they are no longer generated.

PDU partitioning

Tx and Rx PDUs can be explicitly assigned to a specific MainFunction container, and will be processed only by that specific MainFunction instance.

Tx and Rx PDUs that are not mapped to a MainFunction container will be processed by the default SecOC_MainFunctionTx/SecOC_MainFunctionRx functions.

MainFunction forwarding

If automatic container filling is enabled, for PDUs linked to MainFunction containers, analogous MainFunction containers will be forwarded in CryptoStack, and these will be automatically linked to referenced CsmJobs and, implicitly, to the

referenced CryptoDriverObjects.

IMPORTANT: If SecOCMainFunctionTx/SecOCMainFunctionRx containers configured with identical parameters as CsmMainFunction containers, but the names are different, it is recommended that the SecOCMainFunctionTx/SecOCMainFunctionRx names are changed to be the same as CsmMainFunction containers. Otherwise, multiple CsmMainFunction for the same OS Task will be generated.

MainFunction Direct Mode

In case of PDUs which are explicitly assigned to a MainFunction container, the user has the possibility to enable the "direct" invocation of the corresponding generated SecOC_MainFunction[Tx/Rx] instance by using the following parameters:

- SecOCRbTxDirectModeEnabled: If set to TRUE, for explicitly partitioned Tx PDUs (SecOCTxPduMainFunctionRef reference parameter is configured), it performs at runtime an immediate call of the corresponding SecOC_MainFunctionTx_{instance} in the context of SecOC_{IfITp}Transmit.
- SecOCRbRxDirectModeEnabled: If set to TRUE, for explicitly partitioned Rx PDUs (SecOCRxPduMainFunctionRef reference parameter is configured), it performs at runtime an immediate call of the corresponding SecOC_MainFunctionRx_{instance} in the context of SecOC_{Tp}RxIndication.

The usage of the direct invocation of main function instance is exemplified in the sequence diagram below, which presents a gateway configuration routing a Synchronous Rx PDU to a Synchronous Tx PDU:

- Considering that the direct mode usage is clear in case of Synchronous PDUs, when used in conjunction with Asynchronous PDUs it is the user's responsibility to schedule the rest of the CryptoStack main function instances in order to finalize the Csm MAC job which was triggered in the context of SecOC_{IfITp}Transmit or SecOC_{Tp}RxIndication.
- The parameters **SecOCRbTxDirectModeEnabled** and **SecOCRbRxDirectModeEnabled**, for transmit and receive respectively are required for direct mode PDUs, this ensures that only these specifically triggered PDUs are processed, and prevent the processing of standard PDUs within the context of direct mode PDUs.

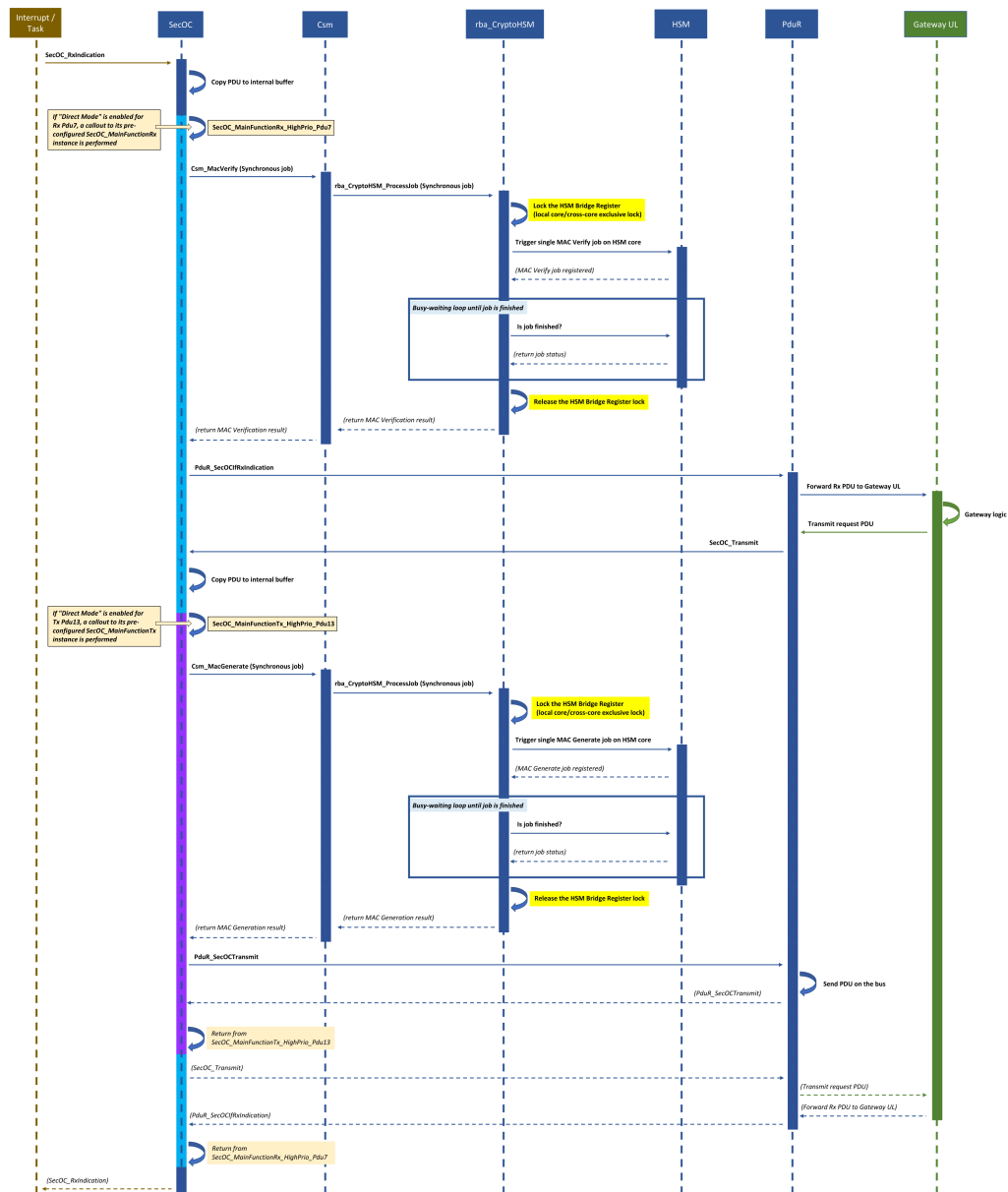


Figure 10.2.SecOC SecOC_MainFunction Split Sequence Diagram

10.8.2 Cryptographic Configuration

The AUTOSAR specification mandates the use of the SecOCRxAuthServiceConfigRef and SecOCTxAuthServiceConfigRef parameters to refer to a set of configured Csm job parameters for the configuration of authentication and verification operations within the SecOc module.

Within the RTA-BSW implementation of SecOc, new parameters have been added which avoid the need for the manual configuration of RTA-SEC stack objects.

- SecOCRbCryptDefaultInterface - sets the module which will be used as the default cryptographic interface (CCL, HSM, Crypto or None). This will be used for all I-PDUs (transmit and receive) unless overwritten by the specific PDU configuration parameters described below. Setting the value to 'None' will prevent any cryptographic function being called, i.e. authentication and verification is not performed.
- SecOCRbCryptDefaultMode - sets the default operation mode for cryptographic function calls to either synchronous (used by PduR) or asynchronous (used in the SecOC main functions).
- SecOCRbRxPduCryptIF - sets the module used for cryptographic function calls when verifying incoming I-PDUs. This will override the setting defined in SecOCCryptDefaultInterface.
- SecOCRbTxPduCryptIF - sets the module used for cryptographic function calls when authenticating outgoing I-PDUs. This will override the setting defined in SecOCCryptDefaultInterface.
- SecOCRbDefaultCryptAlgoMode - sets the algorithm used for generation and verification of MAC (CMAC or SIPHASH-2-4)

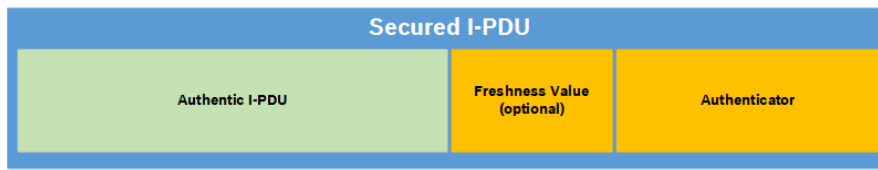
For more details regarding these parameters, please refer to the Extensions section.

10.8.3 SecServices Forwarding Scripts

It is possible to configure SecOc so that the forwarders don't need to automatically generate the configuration. In these cases, when the forwarder for SecOC detects that the configuration is not complete, it will attempt to automatically configure the key configuration. Due to a hard-coded assumption, the NvM is used, and hence the keys are generated to be persisted using the NvM. If the project does not include NvM, this will cause a failure. The solution is to manually configure the keys (so that the forwarder will not run).

10.8.4 Authentication with Secured I-PDU

The data for the calculation of the authenticator consists of the data identifier of the Secured I-PDU (parameter SecOCDataId), the payload of Authentic I-PDU data, and the (optional) complete freshness value. These individual elements are combined to construct the bit array that is passed into the authentication generation algorithm.



If generation is successful, the Secured I-PDU is created by adding the authenticator (and optionally the freshness value) to the payload of the Authentic I-PDU, as shown in the diagram above. The authenticator is mandatory and cannot be omitted. When using a MAC as the authenticator, the result length is 16 bytes. Normally it is not possible to add the complete MAC (and optionally the complete freshness value) to an I-PDU because of the limited number of bytes.

So the most significant bytes of the authenticator and optionally the least significant bytes of the freshness value are added to the Secured I-PDU. The length of the authenticator can be configured (parameter SecOCAuthInfoTxLength) but it must be at least 1 bit. The authenticator is added in the least significant bytes of the I-PDU. The optional freshness value is added between the authentic I-PDU and the authenticator when used.

10.8.5 Authentication with Cryptographic I-PDU

According to the AUTOSAR specification, SecOC should clear all buffers after the receipt of the confirmation SecOC_TxConfirmation. Unfortunately, it is not guaranteed that the confirmation is always sent, and without this confirmation, SecOC keeps the PDU in the waiting state and accepts no new PDU.

To avoid this situation, SecOc deviates from the AUTOSAR specification by allowing the wait for confirmation to be disabled (via the SecOCRbClearBuffer-AfterTransmit extension parameter) so that it clears all internal buffers immediately after forwarding the secured Cryptographic/Authentic I-PDUs to the PduR.

The following points should be noted:

- If SecOCRbClearBufferAfterTransmit is set to true, the authentication during triggered and transport protocol cannot work because the output buffer is cleared before the data are transmitted to the PduR.
- If the confirmation is not received for authentication during triggered and transport protocol, the output buffer is never cleared, and so the authenticator and the plain text are available for longer, making it possible to recalculate the key using this information.

10.8.5.1 SecOC_SendDefaultAuthenticationInformation

The optional SecOC_SendDefaultAuthenticationInformation API extension enables the sending of un-authenticated PDU to lower layer, for example when the authentication build counter has reached the configured value of SecOCAuthenticationBuildAttempts, or when the query of the freshness function returns E_NOT_OK, or if the calculation of the authenticator has returned a non-recoverable error.

This API extension is optional and is only available if `SecOCDefaultAuthenticationInformationPattern` is configured. If the service is not available, or if it is available but is called with `sendDefaultAuthenticationInformation` as `FALSE` for a given `FreshnessValueID`, SecOC will remove the Authentic I-PDU from its internal buffer and cancel the transmission request if the building of authentication information failed.

If the service is available, and if it is called with `sendDefaultAuthenticationInformation` as `TRUE` for a given `FreshnessValueID`, SecOC will use `SecOCDefaultAuthenticationInformationPattern` as authentication information and will not cancel the transmission request.

10.8.6 Bulk Interface (`rba_CryptoHSM`)

The `rba_CryptoHSM` module provides a Bulk Interface to send multiple MAC generate/verify jobs to the HSM at once (in a single request).

The Bulk Interface influences the configuration of the I-PDUs in SecOC. If the configuration is done manually, please ensure that each I-PDU is assigned to its own Csm queue, and that it has its own Csm job.

The Bulk Interface is only possible for I-PDUs with asynchronous mode, and with `rba_CryptoHSM` (i.e. not `rba_CryptoCCL`). All other configurations will cause a Csm validator error.

10.8.7 Bulk Configuration Parameters

The `SecOCRbRxUseBulkProcessing` parameter indicates if the MAC verification for Rx PDUs should be processed in Bulk mode or in single shot mode. Possible values: 1. `TRUE`: the MAC job will be processed in Bulk mode; 2. `FALSE`: the MAC job will be processed in single shot mode. Note: If `SecOCRbRxUseBulkProcessing` is not configured, then it will use the configured value of `SecOCGeneral/SecOCRbDefaultUseBulkProcessing` as default. Bulk mode is available only for PDUs with cryptographic interface set to HSM and cryptographic mode set to asynchronous.

The `SecOCRbTxUseBulkProcessing` parameter indicates if the MAC generation for Tx PDUs should be processed in Bulk mode or in single shot mode. Possible values: 1. `TRUE`: the MAC job will be processed in Bulk mode; 2. `FALSE`: the MAC job will be processed in single shot mode. Note: If `SecOCRbTxUseBulkProcessing` is not configured, then it will use the configured value of `SecOCGeneral/SecOCRbDefaultUseBulkProcessing` as default. Bulk mode is available only for PDUs with cryptographic interface set to HSM and cryptographic mode set to asynchronous.

The `SecOCRbDefaultUseBulkProcessing` parameter indicates if the MAC calculation should be processed in Bulk mode or in single shot mode only for those PDUs which do not have the parameter `SecOCRb[Rx/Tx]UseBulkProcessing` configured. The `SecOCRbDefaultUseBulkProcessing` parameter will be ignored by those PDUs that have `SecOCRb[Rx/Tx]UseBulkProcessing` configured. The MAC calculation will be processed according to the value of `SecOCRb[Rx/Tx]UseBulkProcessing` parameter. Note: Bulk mode is available only for PDUs with cryptographic interface set to HSM and cryptographic mode set to asynchronous. The default

value is FALSE.

10.8.8 SecOC Forwarder

The SecOC Forwarder recognises if the Bulk Interface feature has been enabled and configures the cryptostack objects accordingly. The SecOC Forwarder requires some configuration information additional to AUTOSAR. However the general configuration, valid for all Rx and Tx PDUs without individual configuration, can be done in subcontainer SecOCRbForwarder of SecOCGeneral.

- SecOCRbDefaultCryptAlgoFamily - selects the algorithm family used for generation and verification of MAC.
- SecOCRbDefaultCryptAlgoMode - selects the algorithm mode used for generation and verification of MAC.
- SecOCRbDefaultCryptAlgoSecondFamily - selects the algorithm secondary family used for generation and verification of MAC.
- SecOCRbMaxLengthTx - this parameter specifies the maximum length in bits of the data for MAC authentication.
- SecOCRbMaxLengthRx - this parameter specifies the maximum length in bits of the data for MAC verification.
- SecOCRbDefaultUseBulkProcessing - this parameter indicates if the MAC calculation should be processed in bulk mode or in single shot mode.

Most of these default configurations can be overwritten with individual values for each PDU. This can be done in the subcontainers **SecOCRbRxForwarder** and **SecOCRbTxForwarder** of the PDU configurations. The SecOC forwarder creates the PDU identifiers, the reference to a csm job and additional items in the crypto stack. The forwarding subcontainers are optional and can be empty if no forwarding is required. In this case, the SecOC forwarder creates only the PDU identifiers.

10.8.9 Dynamic PDUs

For Rx PDUs with a dynamic length the secured header is introduced, with the header including the current length of the payload. With AR4.5 it is possible to handle dynamic PDUs without a secured header, for this the flag **SecOCDynamicRuntimeLengthHandling** of the SecOC Rx Pdu is introduced.

Only for PDUs with **SecOCDynamicRuntimeLengthHandling** set to true SecOC accepts a pdu length different from the configured length. The minimum of the both length is used. PDUs with **SecOCDynamicRuntimeLengthHandling** set to false and a length different from the configured length are discarded.

It is possible to configure dynamic Tx PDUs with the parameter **SecOCRbDynamicRuntimeLengthHandling**. If it is set to TRUE the calculated length for secured PDU during runtime is used. If it is set to FALSE, the configured length of the secured PDU is used, unused areas are filled with the configured default pattern (SecOCTxPduUnusedAreasDefault).

The flag DynamicLength flag supported by EcuC for the global Pdu is ignored in SecOC.

10.8.10 Skip the authentic PDU during MAC generation/verification

In order to skip the authentic PDU when generating or verifying MAC authentication field, configure `SecOCGeneral/SecOCRbSecuredAreaDeviation` to `true` and configure `secOCTxPduSecuredArea/secOCSecuredTxPduLength` or `secOCRxPduSecuredArea/secOCSecuredRxPduLength` to `0`.

10.8.11 Sawtooth Bit Counting

If sawtooth bit counting is used to identify specific bits in a PDU, then the config parameter `SecOCRbPDUsWithSawtoothBitcounting` in `SecOCGeneral` has to be set to `TRUE`. Then the `SecOCAuthDataFreshnessStartPosition` of a PDU and `SecOCMessageLinkPos` of a secure PDU (if a collection PDU is used) are interpreted following sawtooth bit counting.

Otherwise monotone bit counting is assumed. According to AUTOSAR, sawtooth bit counting should be used, monotone bit counting is used as default to maintain backwards compatibility. With sawtooth bit counting enabled, the bit positions are converted internally to monotone bit counting to identify the correct bits in the bit stream of a PDU.

10.8.12 No Bit Representation in Secured Header

The optional parameter `SecOCRbSecuredHeaderRepresentation` has been removed and is no longer available. Therefore the length in the secured header is required to be given in bytes.

10.8.13 SecOC in Gateway ECU

By default, PDU data packets are configured in RTA-BSW so that, when they are routed between network clusters via a Gateway ECU, they are decrypted/encrypted using SecOC.

However, to eliminate the SecOC steps altogether during transmission (meaning that the Gateway ECU simply transmits an encrypted packet) set the `isIgnoreSecOCOnTxRouting` parameter to `True` in the Project's `algo.properties` file:

```
manprop_isIgnoreSecOCOnTxRouting:true
```

Now, when the Generic Importer imports the Gateway ECU's system description as part of the Configuration Generation process, the resulting ECUC values will reflect a configuration in which a transmitted PDU data packet is routed directly through the PduR layer to CanIf (thus omitting SecOC).



NOTE

For more details on setting parameters using the `algo.properties` file, please see section 3.2.1 *ConfGen Parameters* of the *RTA-CAR User Guide*.

10.9 Integration Advice

10.9.1 Init Functions

SecOC_Init initializes the SecOC module.

10.9.2 Delnit Functions

SecOC_Deinit de-initializes the SecOC module.

10.9.3 Scheduled Functions

The following functions are required to be scheduled for cyclic operation.

The MainFunctions have to be called periodically, each instance according to its own configured periodicity.

- SecOC_MainFunctionRx[_instance], Cyclic function to perform verification of Secured I-PDUs.
- SecOC_MainFunctionTx[_instance], Cyclic function to perform authentication of Authentic I-PDUs.

The main functions of the RTA-SEC stack should be called after the SecOC main functions in top-down order e.g. (SecOC → Csm → Crylf → Crypto).

10.9.4 Authentication and Verification

As noted above, authentication of PDUs is handled in the SecOC_MainFunctionTx[_instance] cyclic MainFunctions, while verification is performed by SecOC_MainFunctionRx[_instance].

Additional parameters are introduced in the SecOCGeneral container, and in the PDU container, to set the SecOC mode to synchronous or asynchronous for all PDUs or for PDU-specific operation.

If SecOC uses Csm asynchronously to calculate or verify the authenticator, one of the internal functions (SecOC_Priv_TxCallback or SecOC_Priv_RxCallback) is called by Csm to notify that authentication or verification is completed.

If Collection PDUs without Message Linker are used, strict time dependency of the Collection PDU parts must be regarded. The Upper Layer has to ensure that both parts are provided within one cycle of MainFunctionRx. The order of the parts of a matching pair (Auth#n/Crypto#n) or (Crypto#n/Auth#n) may swap, but only the last matching pair will be processed and verified.

10.9.5 Freshness

The freshness value management is processed by an (optional) external freshness module which is not part of the SecOC specification. However, the interfaces to this freshness module are provided in the private sub module SecOC_Priv_FreshIF.

10.9.6 Shared Buffers

Shared (internal) buffers are not protected against data races. The project has to ensure that I-PDUs sharing one buffer are not sent or received in parallel. A buffer may be overwritten if another I-PDU is received with the same Id.

10.9.7 PduR Configuration for SecOc

If the SecOC module is configured, then the PduRTransportProtocol parameter for SecOC's PduRBSwModules is required to be set to TRUE, otherwise the code generation will raise error.

10.9.8 Series Integration

It is recommended that the following configuration parameters are checked to ensure that authentication and verification are enabled:

- SecOCRbTxPduCryptIf (container SecOCTxPduProcessing) - The value 'None' disables the authentication for the Tx PDU. The secured PDU is sent with cleared authenticator and freshness value (equal 0x00).
- SecOCSecuredRxPduVerification (container SecOCRxPduProcessing) - When set to false, no verification will be done, the authentic PDU is sent nevertheless.
- SecOCIgnoreVerificationResult (container SecOCGeneral) - Verification will be done but the result of all verifications will be ignored, the authentic PDU is sent nevertheless.
- SecOCEnableForcedPassOverride (container SecOCGeneral) - When set to true, it is possible to pass or skip the verification of Rx PDUs with a specific freshness value or data identifier via the API SecOC_VerifyStatusOverride at runtime.

10.9.9 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The SecOC module does not use any exclusive OS resources.

10.10 Hardware Requirements

Table 10.129.SecOC Hardware Requirements

Support	Usage
FLASH	8708 bytes
RAM	538 bytes

Usage calculated based on a sample configuration.

11 Glossary

AEAD

Authenticated Encryption with Associated Data. A form of encryption which simultaneously assures the integrity, confidentiality and authenticity of data. Commonly used to encrypt network packet data in secure networks.

AES

Advanced Encryption Standard. A commonly used symmetric block cipher algorithm

Asymmetric Key

An algorithm used in cryptography which uses both a public and private key, one of which is used for encryption and the other for decryption

Asynchronous

A type of processing where the requested operation is not necessarily carried out immediately

Authentic I-PDU

An I-PDU which requires protection against unauthorised access

Cipher

A generic term for an encryption algorithm of some kind

Crylf

AUTOSAR Crypto Interface module

Crypto

AUTOSAR Crypto Driver module

CSAI

The CysurHSM Cryptographic Service Abstraction Interface for interaction with the HSM

Csm

AUTOSAR Crypto Service Manager module

Decryption

The process by which encrypted data is made readable

Encryption

The process by which data is made unreadable to all but a suitably authorised entity

Hash

A value produced from a hash function which uniquely represents the input data in a fixed-size format

Hash Function

A deterministic procedure which takes an arbitrary block of data and returns a fixed-size bit string (hash)

HSM

Hardware Security Module. A physical device which provides cryptographic functionality implemented in hardware

I-PDU

Interaction Layer Protocol Data Unit - a collection of messages for transfer between nodes in a network

Job

An instance of a configured cryptographic primitive which also contains a key reference and an assigned priority

Key

An artefact used in cryptography to encode, decode or authenticate information securely

MAC

Message Authentication Code. A method used for symmetric authentication of a transmitted message

N-PDU

Network Protocol PDU used in the network layer

N-SDU

Network service data unit (SDU) used in the network layer

OCSP

Online Certificate Status Protocol

Primitive

An instance of a configured cryptographic algorithm

rba_CryptoCCL

RTA-BSW module, a structural component of the generic AUTOSAR-specified Crypto Driver (Crypto) module

rba_CryptoHSM

RTA-BSW module, a structural component of the generic AUTOSAR-specified Crypto Driver (Crypto) module

RNG

Random Number Generation

RTE

AUTOSAR Run-Time Environment, implements the AUTOSAR Virtual Functional Bus interfaces and thereby realizes the communication between SWCs

SecOC

AUTOSAR Secure Onboard Communications module

Secured I-PDU

An Authentic I-PDU which has been processed by SecOC to add authenticator data

SHE

Security Hardware Extension

Symmetric Key

An algorithm used in cryptography which uses a single key shared between two parties for encryption and decryption

Synchronous

A type of processing where the requested operation is carried out immediately